TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA TOÁN - TIN HỌC

BÁO CÁO THỰC HÀNH

Môn học: Xử lý dữ liệu lớn

Tổng hợp các bài thực hành L01–L06

Sinh viên thực hiện: Nguyễn Thị Huyền Trang
Mã số sinh viên: 23280092
Lớp: 23KDL

TP. Hồ Chí Minh, Tháng 10 năm 2026

Mục lục

1 HDFS: tổ chức block, nhân bản và khả năng chịu lỗi DataNode **14**

Tóm tắt . . . 14

1.1 Giới thiệu . . . 14

1.2 Cơ sở lý thuyết . . . 15

1.2.1 Kiến trúc NameNode và DataNode . . . 15

1.2.2 Block và replica . . . 15

1.2.3 Heartbeat, node stale và node dead . . . 15

1.2.4 Under-replication và tái nhân bản . . . 16

1.2.5 Safe mode và fsimage . . . 16

1.3 Môi trường và phương pháp thực nghiệm . . . 16

1.3.1 Môi trường . . . 16

1.3.2 Dữ liệu . . . 17

1.3.3 Quy trình . . . 17

1.4 Kết quả thực nghiệm . . . 18

1.4.1 Khởi động cluster và bộ lệnh `hdfs dfs` . . . 18

1.4.2 Ảnh hưởng của kích thước block . . . 20

1.4.3 Thay đổi hệ số nhân bản . . . 22

1.4.4 Sự cố DataNode và tái nhân bản . . . 23

1.4.5 Safe mode và fsimage . . . 28

1.5 Thảo luận . . . 29

1.5.1 Tệp 300 MB với cấu hình mặc định chiếm bao nhiêu block, replica và dung lượng? . . . 29

1.5.2 Vì sao block cuối của tệp taxi nhỏ hơn 16 MB, và có tốn 16 MB đĩa không? . . . 29

1.5.3 Vị trí block được lưu ở đâu, và khi NameNode khởi động lại thì sao? . . . 29

1.5.4 Vì sao tệp hệ số 2 được phục hồi còn tệp hệ số 3 thì không? . . . 29

1.5.5 `fsck` báo gì khi có block under-replicated, và vì sao đó chưa phải lỗi? . . . 30

1.5.6 Nếu giữ `recheck-interval` mặc định 5 phút thì sao, và vì sao timeout dài lại tốt? . . . 30

1.5.7 Vì sao NameNode khởi động trong safe mode và phải chặn ghi? . . . 30

1.5.8 Vì sao 1 triệu tệp 1 KB là vấn đề, còn 1 tệp 1 KB thì không? 30
1.5.9 Hạn chế của thực nghiệm 31
1.6 Kết luận 31
Tài liệu tham khảo 31
Phụ lục A: Danh mục tệp nộp kèm 32
Phụ lục B: Trích đoạn mã nguồn 32

2 MapReduce với Hadoop Streaming, map-side join và combiner 34
Tóm tắt 34
2.1 Giới thiệu 34
2.2 Cơ sở lý thuyết 35
2.2.1 Mô hình MapReduce 35
2.2.2 Shuffle và sort 35
2.2.3 Hadoop Streaming 35
2.2.4 Combiner 35
2.2.5 Map-side join và distributed cache 36
2.2.6 YARN và JobHistory 36
2.3 Môi trường và phương pháp thực nghiệm 36
2.3.1 Môi trường 36
2.3.2 Dữ liệu 37
2.3.3 Quy trình 37
2.4 Kết quả thực nghiệm 38
2.4.1 Đếm từ, kiểm thử cục bộ 38
2.4.2 Đếm từ trên YARN 38
2.4.3 Dữ liệu taxi và map-side join cục bộ 40
2.4.4 Có và không có combiner 41
2.4.5 Giao diện YARN, JobHistory và CLI 43
2.4.6 Mở rộng: giá cước trung bình theo khu vực 47
2.5 Thảo luận 48
2.5.1 Trong đường ống cục bộ, lệnh nào làm việc của shuffle, và thiếu nó thì sao? 48
2.5.2 Vì sao reducer Streaming phải tự giữ `current_key`? 48
2.5.3 Job taxi có bao nhiêu map task, bao nhiêu reduce task, và vì sao? 48
2.5.4 Combiner giảm `Reduce shuffle bytes` bao nhiêu lần, và vì sao đầu ra combiner xấp xỉ số khu vực nhân số map task? 48
2.5.5 Vì sao combiner không làm job nhanh hơn ở đây, và khi nào nó có ích rõ? 49
2.5.6 Có dùng lại reducer "trung bình" làm combiner được không? 49
2.5.7 Map-side join trong bài hoạt động thế nào, và khi nào nó không còn hợp? 49
2.5.8 Counter `BAD_RECORDS` hiện ở đâu, và vì sao nên đếm dòng hỏng thay vì cho job dừng? 49
2.5.9 Hạn chế của thực nghiệm 50

2.6 Kết luận . . . 50
Tài liệu tham khảo . . . 50
Phụ lục A: Danh mục tệp nộp kèm . . . 51
Phụ lục B: Trích đoạn mã nguồn . . . 51

3 Hive: truy vấn SQL trên HDFS, phân vùng, định dạng lưu trữ cột và bucketing 53
Tóm tắt . . . 53
3.1 Giới thiệu . . . 53
3.2 Cơ sở lý thuyết . . . 54
3.2.1 Kiến trúc Hive . . . 54
3.2.2 Bảng ngoài, bảng quản lý và schema-on-read . . . 54
3.2.3 Định dạng cột ORC và Parquet . . . 54
3.2.4 Phân vùng và cắt tỉa phân vùng . . . 54
3.2.5 Bucketing . . . 55
3.2.6 Bảng giao dịch ACID . . . 55
3.3 Môi trường và phương pháp thực nghiệm . . . 55
3.3.1 Môi trường . . . 55
3.3.2 Dữ liệu . . . 56
3.3.3 Quy trình . . . 57
3.4 Kết quả thực nghiệm . . . 57
3.4.1 Bảng ngoài và nội dung Metastore . . . 57
3.4.2 Bảng phân vùng ORC và Parquet . . . 58
3.4.3 Kích thước lưu trữ theo định dạng . . . 61
3.4.4 So sánh thời gian truy vấn CSV, ORC và Parquet . . . 62
3.4.5 Cắt tỉa phân vùng . . . 64
3.4.6 Bucketing . . . 65
3.4.7 Phần mở rộng: thống kê và bảng ACID . . . 66
3.5 Thảo luận . . . 69
3.5.1 Metastore giữ gì, HDFS giữ gì, và `DROP TABLE` xóa những gì? . . . 69
3.5.2 Vì sao `zones` cần `OpenCSVSerde` còn `trips_csv` chỉ cần `ROW FORMAT DELIMITED`? . . . 69
3.5.3 Vì sao ORC và Parquet nhỏ hơn CSV 4–5 lần? . . . 69
3.5.4 Vì sao Q1 chạy nhanh hơn trên ORC và Parquet? . . . 69
3.5.5 Phân vùng tĩnh khác phân vùng động ra sao, và vì sao mặc định là `strict`? 70
3.5.6 Phân vùng theo `pulocationid` cộng ngày, hoặc theo `tpep_pickup_datetime`, thì sao? . . . 70
3.5.7 Khi nào dùng partitioning, khi nào dùng bucketing, và vì sao số tệp bucket cố định? . . . 70
3.5.8 Những dòng nào rơi vào phân vùng lạ, và pipeline thật nên xử lý ra sao? 71
3.5.9 Vì sao `count(*)` không nhanh hơn sau khi đã có thống kê? . . . 71
3.5.10 Hạn chế của thực nghiệm . . . 71

3.6 Kết luận . . . 72
Tài liệu tham khảo . . . 72
Phụ lục A: Danh mục tệp nộp kèm . . . 73
Phụ lục B: Trích đoạn mã nguồn . . . 73

4 Nạp dữ liệu từ cơ sở dữ liệu OLTP vào HDFS: Sqoop và Spark JDBC, nạp toàn phần và nạp gia tăng 75
Tóm tắt . . . 75
4.1 Giới thiệu . . . 75
4.2 Cơ sở lý thuyết . . . 76
4.2.1 Nạp toàn phần và nạp gia tăng . . . 76
4.2.2 Cách Sqoop chia split . . . 76
4.2.3 Đọc JDBC song song trong Spark . . . 76
4.2.4 Watermark, change log và snapshot . . . 77
4.2.5 Change Data Capture . . . 77
4.3 Môi trường và phương pháp thực nghiệm . . . 77
4.3.1 Môi trường . . . 77
4.3.2 Dữ liệu . . . 78
4.3.3 Quy trình . . . 79
4.4 Kết quả thực nghiệm . . . 79
4.4.1 Sqoop nạp toàn phần bảng `orders` . . . 79
4.4.2 Sqoop với cột chia bị lệch . . . 81
4.4.3 Sqoop nạp gia tăng, merge và export . . . 82
4.4.4 Spark JDBC: đọc song song và ghi Parquet . . . 85
4.4.5 So sánh Sqoop và Spark JDBC . . . 89
4.4.6 Spark nạp gia tăng bằng watermark . . . 90
4.5 Thảo luận . . . 93
4.5.1 Sqoop chạy câu SQL nào trước khi import, và từ đó ra 4 split bằng cách nào? . . . 93
4.5.2 Vì sao các file `customers` không đều, và nếu dùng `--split-by customer_id` thì sao? . . . 93
4.5.3 Với `lowerBound=1, upperBound=5003, numPartitions=4`, Spark có đọc dòng `order_seq` = 9,999 không? . . . 93
4.5.4 Vì sao phải chốt cận trên trước khi đọc, và chỉ lưu watermark sau khi ghi xong? . . . 93
4.5.5 Chế độ `append` hay `lastmodified` bắt được UPDATE, và chế độ nào bắt được DELETE? . . . 94
4.5.6 Watermark trên `updated_at` không giải quyết được gì, và CDC khác ở đâu? . . . 94
4.5.7 Điều gì xảy ra với cơ sở dữ liệu nguồn khi đặt `-m 50` hoặc `numPartitions=50`? . . . 94

4.5.8 Hạn chế của thực nghiệm . . . 95
4.6 Kết luận . . . 95
Tài liệu tham khảo . . . 95
Phụ lục A: Danh mục tệp nộp kèm . . . 96
Phụ lục B: Trích đoạn mã nguồn . . . 96

5 PySpark cơ bản: DataFrame, Spark SQL, kế hoạch thực thi và Spark UI 98
Tóm tắt . . . 98
5.1 Giới thiệu . . . 98
5.2 Cơ sở lý thuyết . . . 99
5.2.1 Driver, job, stage, task . . . 99
5.2.2 Transformation, action và lazy evaluation . . . 99
5.2.3 Catalyst, AQE và tổng hợp hai pha . . . 99
5.2.4 Chiến lược join . . . 99
5.2.5 Chế độ ANSI trong Spark 4 . . . 100
5.2.6 Parquet phân vùng và partition pruning . . . 100
5.3 Môi trường và phương pháp thực nghiệm . . . 100
5.3.1 Môi trường . . . 100
5.3.2 Dữ liệu . . . 101
5.3.3 Quy trình . . . 102
5.4 Kết quả thực nghiệm . . . 102
5.4.1 Đọc dữ liệu và đánh giá trễ . . . 102
5.4.2 Chất lượng dữ liệu và làm sạch . . . 103
5.4.3 Join với bảng khu vực và tổng hợp theo quận . . . 104
5.4.4 Hàm cửa sổ . . . 106
5.4.5 Spark SQL và kế hoạch vật lý . . . 108
5.4.6 Spark UI . . . 110
5.4.7 Ghi Parquet phân vùng và partition pruning . . . 118
5.5 Thảo luận . . . 120
5.5.1 Vì sao tệp 48 MB lại có 4 partition đầu vào? . . . 120
5.5.2 Dòng nào là transformation, dòng nào là action, và sao dựng plan nhanh vậy? . . . 121
5.5.3 `by_borough.show()` sinh bao nhiêu job, stage, và stage bị cắt ở đâu? . . . 121
5.5.4 Vì sao năm cột có cùng 140,162 NULL, và nên xử lý thế nào? . . . 121
5.5.5 `HashAggregate` partial trước `Exchange` làm gì, ứng với khái niệm nào ở L02? . . . 122
5.5.6 Vì sao bảng khu vực được broadcast, và khi nào Spark chuyển sang sort-merge join? . . . 122
5.5.7 `cache()` thay đổi gì trong plan và UI, và khi nào không nên cache? . . . 122
5.5.8 Vì sao cần `repartition("pickup_date")` trước `partitionBy("pickup_date")`? . . . 123

5.5.9 Hạn chế của thực nghiệm . . . 123
5.6 Kết luận . . . 123
Tài liệu tham khảo . . . 124
Phụ lục A: Danh mục tệp nộp kèm . . . 124
Phụ lục B: Trích đoạn mã nguồn . . . 125

6 Tối ưu hiệu năng Spark: shuffle, chiến lược join, data skew, AQE, phân vùng và cache 127
Tóm tắt . . . 127
6.1 Giới thiệu . . . 127
6.2 Cơ sở lý thuyết . . . 128
6.2.1 Shuffle và số partition shuffle . . . 128
6.2.2 Adaptive Query Execution . . . 128
6.2.3 Chiến lược join . . . 128
6.2.4 Data skew và salting . . . 129
6.2.5 repartition, coalesce và cache . . . 129
6.3 Môi trường và phương pháp thực nghiệm . . . 129
6.3.1 Môi trường . . . 129
6.3.2 Dữ liệu . . . 129
6.3.3 Quy trình . . . 130
6.4 Kết quả thực nghiệm . . . 131
6.4.1 Tổng quan hai lần chạy . . . 131
6.4.2 E1: số partition shuffle và AQE coalescing . . . 135
6.4.3 E2: broadcast hash join và sort-merge join . . . 136
6.4.4 E3: data skew, salting và AQE skew join . . . 139
6.4.5 E4: repartition và coalesce trước khi ghi . . . 144
6.4.6 E5: cache . . . 144
6.4.7 So sánh với kết quả tham chiếu . . . 145
6.5 Thảo luận . . . 146
6.5.1 Vì sao 200 partition chậm, và vì sao không nên đặt 1 partition dù nó chạy nhanh? . . . 146
6.5.2 `AQEShuffleRead coalesced` làm gì, và AQE biết gì mà planner tĩnh không biết? . . . 146
6.5.3 Con số nào cho thấy skew, vì sao salting không rút ngắn thời gian tổng trên 4 core, và khi nào salting thắng? . . . 146
6.5.4 AQE nhận ra partition lệch thế nào, xử lý ra sao, và kém salting ở đâu? 147
6.5.5 Khi nào không nên broadcast dù một phía "nhỏ"? . . . 147
6.5.6 `repartition(n)` hay `coalesce(n)`: cái nào shuffle, cái nào tạo file lệch, cái nào làm giảm song song stage trước? . . . 147
6.5.7 Vì sao cache có ích trong E5, còn cache một DataFrame rẻ thì thường không? . . . 148

6.5.8 Nên tinh chỉnh gì trước khi thêm phần cứng? 148
6.5.9 Hạn chế của thực nghiệm 148
6.6 Kết luận 148
Tài liệu tham khảo 149
Phụ lục A: Danh mục tệp nộp kèm 150
Phụ lục B: Trích đoạn mã nguồn 150

Danh mục hình

1.1 Đầu ra `dfsadmin -report` (3 DataNode sống, ánh xạ IP sang hostname), chuỗi lệnh `hdfs dfs` và kết quả `hdfs dfs -stat`. 18
1.2 Trang Overview của NameNode khi cluster khỏe mạnh: 3 Live Nodes, 0 Dead Nodes, 0 block under-replicated. 19
1.3 Trang Datanodes của NameNode khi cả ba DataNode đều ở trạng thái In service. 20
1.4 Trang Browse Directory hiển thị thư mục `/data/taxi` với hệ số nhân bản và block size của từng mục. 20
1.5 Kết quả `fsck -files -blocks -locations` của tệp Parquet với block mặc định (1 block) và block 16 MB (3 block). 21
1.6 Tệp block vật lý `blk_<id>` và `blk_<id>_<genstamp>.meta` trong thư mục dữ liệu của datanode1. 22
1.7 Kết quả `setrep -w 2`, `du -h` sau khi giảm hệ số và tệp được ghi với hệ số nhân bản 1. 23
1.8 `fsck` của tệp block 16 MB trước khi dừng datanode3: mỗi block có 2 replica trên datanode1 và datanode3. 23
1.9 Dòng thời gian thăm dò `dfsadmin -report` sau khi dừng datanode3 và dòng log `removeDeadDatanode` của NameNode. 24
1.10 Trang Overview của NameNode khi datanode3 đã bị tuyên bố chết. 25
1.11 Trang Datanodes khi datanode3 chết: datanode1 và datanode2 In service, datanode3 ở trạng thái Down. 26
1.12 `fsck` của tệp block 16 MB sau khi dừng datanode3: replica đã chuyển sang datanode2, tệp trở lại trạng thái OK. 26
1.13 `fsck /data` khi datanode3 chết: hai tệp hệ số 3 under-replicated nhưng HEALTHY; md5 của tệp tải về trùng bản gốc. 27
1.14 Trạng thái cluster sau khi datanode3 quay lại: không còn block under-replicated hay over-replicated. 27
1.15 Safe mode thủ công (ghi bị từ chối, đọc vẫn hoạt động), safe mode tự động khi khởi động lại và nội dung fsimage. 28

2.1 Kiểm thử cục bộ đường ống `cat | mapper | sort | reducer` trên bốn cuốn sách: top 5 từ và số dòng `the` khi bỏ `sort`. 38

2.2 Counter cuối job `wordcount` trên YARN: 4 map task, `Reduce input groups= 20967` và top 5 từ đọc từ HDFS. . . . 40
2.3 Chuyển Parquet sang CSV, 6 block trên HDFS, kiểm thử cục bộ map-side join trên 1,000 dòng và counter `BAD_RECORDS`. . . . 41
2.4 Trích các counter so sánh từ hai log và top 10 khu vực đón khách; `diff` giữa hai đầu ra cho kết quả IDENTICAL. . . . 42
2.5 YARN ResourceManager UI (cổng 8088): ba ứng dụng MapReduce đã hoàn thành và các chỉ số cluster. . . . 43
2.6 JobHistory UI (cổng 19888): danh sách các job đã hoàn thành. . . . 44
2.7 JobHistory UI: tổng quan job `zones_combiner` với 6 map task, 2 reduce task và thời gian từng pha. . . . 44
2.8 JobHistory UI: trang Counters của job không combiner. . . . 45
2.9 JobHistory UI: trang Counters của job có combiner. . . . 46
2.10 Kiểm tra các job đã hoàn thành bằng `yarn application -list` và `mapred job -list all`. . . . 47
2.11 Job tính giá cước trung bình theo khu vực với combiner `(sum, count)` và đối chiếu JFK với bản chạy không combiner. . . . 47

3.1 Khởi động cụm Hive (`docker compose ps`) và tải tệp CSV cùng bảng tra cứu zone lên HDFS. . . . 56
3.2 Giao diện web HiveServer2 (cổng 10002): danh sách truy vấn đã hoàn tất, tất cả chạy trên Tez. . . . 56
3.3 Kết quả truy vấn bảng ngoài và nội dung Metastore (PostgreSQL). . . . 58
3.4 `SHOW PARTITIONS` và số chuyến theo `pickup_month`. . . . 59
3.5 Thư mục warehouse `/user/hive/warehouse/taxi.db` trên giao diện NameNode. . . . 60
3.6 Năm thư mục phân vùng của bảng `trips_orc` trên giao diện NameNode. . . . 60
3.7 `DESCRIBE FORMATTED trips_orc` và 18 dòng nằm ngoài tháng 01/2024. . . . 61
3.8 Kích thước CSV, ORC và Parquet trên HDFS. . . . 62
3.9 Lần chạy thứ hai của `03_compare_formats.sql`. . . . 63
3.10 Kết quả `EXPLAIN DEPENDENCY` và `EXPLAIN EXTENDED`. . . . 65
3.11 Kết quả bucketing: mô tả bảng, lấy mẫu một bucket và vị trí các dòng JFK. . . . 66
3.12 Bốn tệp bucket của `trips_bucketed` trên giao diện NameNode. . . . 66
3.13 Phần mở rộng: thống kê cột và các thao tác ACID. . . . 68
3.14 Các thư mục `delta_*` và `delete_delta_*` của bảng `zone_notes` trên giao diện NameNode. . . . 68

4.1 Trạng thái các container của stack L04 (`docker compose ps`) và danh sách 6 bảng nguồn trong PostgreSQL; container `jdbc-driver` kết thúc với mã 0 sau khi tải driver. . . . 78

4.2 Kết quả Sqoop nạp toàn phần `orders`: bounding query, 4 split, 1.009 MB trong 25.2306 s và 1,250 dòng mỗi file. 80
4.3 Nhật ký PostgreSQL ghi lại bounding query và 4 câu SELECT theo khoảng `order_seq` do các map task của Sqoop gửi. 81
4.4 JobHistory của job `job_1793356967101_0001` (`orders.jar`): 4 map task thành công, không có reduce task. 81
4.5 Số dòng mỗi file khi nạp `customers` chia theo `customer_zip_code_prefix`, thể hiện dữ liệu bị lệch. 82
4.6 Sqoop nạp `append` lần đầu, nạp `lastmodified` cơ sở và tạo saved job `orders_incr`. 84
4.7 Sau `simulate_changes` lần 1: saved job lấy 3 dòng mới, `lastmodified` kèm merge lấy 5 dòng, và export 5,000 record về PostgreSQL. 84
4.8 YARN ResourceManager UI: các MapReduce job do Sqoop nộp, tất cả ở trạng thái FINISHED/SUCCEEDED. 85
4.9 JobHistory Server liệt kê các job Sqoop đã hoàn tất. 85
4.10 Kết quả `spark-submit --step full`: số partition, thời gian đếm, số dòng mỗi partition, kết quả ghi Parquet và physical plan có `PartitionFilters`. 87
4.11 Nhật ký PostgreSQL ghi lại các câu `SELECT 1 FROM orders WHERE ...` theo khoảng do Spark sinh ra. 87
4.12 Spark UI, trang Jobs của ứng dụng `L04-jdbc-full`. 88
4.13 Spark UI, trang Stages: các stage đọc JDBC song song và stage ghi Parquet đều có 4 task. 89
4.14 Đầu ra Parquet phân vùng theo tháng trên HDFS và so sánh dung lượng với đầu ra văn bản của Sqoop. 89
4.15 Ba lần chạy Spark `--step incremental`: lần 1 đọc 5,003 dòng, lần 2 đọc 5 dòng thay đổi, lần 3 đọc 0 dòng. 92

5.1 Tab Environment của Spark UI: Java 17.0.20.1, Scala 2.13.17, `spark.driver.memory=2g`, event log được bật. 101
5.2 Đọc dữ liệu: schema 19 cột, 2,964,624 dòng, 4 partition; dựng ba transformation mất 0.096 s, `count()` mất 0.28 s. 103
5.3 Đếm NULL theo cột và số dòng vi phạm từng luật làm sạch; giữ lại 2,867,786 dòng (96.73 %). 104
5.4 Chế độ ANSI: `try_divide` trả NULL, phép chia thường ném lỗi `[DIVIDE_BY_ZERO]`. 104
5.5 Kết quả join hai lần với bảng khu vực, tổng hợp theo quận và theo giờ đón. . . 106
5.6 Hàm cửa sổ: top-3 khu vực đón mỗi quận và chuỗi theo ngày với `lag`, tổng lũy kế. 108
5.7 Kết quả Spark SQL trùng khớp DataFrame và cây kế hoạch vật lý của `by_borough`. 109
5.8 Phần kế hoạch thể hiện `HashAggregate` partial, `Exchange hashpartitioning` và `HashAggregate` final. 110
5.9 Tab Jobs: mỗi action sinh một hoặc nhiều job. 111

5.10 Tab SQL/DataFrame: 21 truy vấn đã hoàn thành cùng thời lượng và danh sách job. . . . 112
5.11 Đồ thị kế hoạch truy vấn 10 trong tab SQL: 2,867,786 dòng vào, 24 dòng ra khỏi `HashAggregate` partial. . . . 114
5.12 Job 21: một stage, 4/4 task, Input 347.3 MiB từ cache, Shuffle Write 2.3 KiB. . . 115
5.13 Job 22: stage 28 bị bỏ qua, stage 29 chạy 1 task với Shuffle Read 2.3 KiB. . . . 116
5.14 Tab Stages: 60 stage hoàn thành, 28 stage bị bỏ qua, kèm Input và Shuffle Read/Write. . . . 117
5.15 Tab Storage: `clean` được cache 4/4 partition, 100 %, 347.3 MiB trong bộ nhớ. . 118
5.16 Ghi Parquet phân vùng, đọc lại ngày 2024-01-15 (74,743 chuyến) và `PartitionFilters` trong kế hoạch. . . . 119
5.17 Đồ thị truy vấn 18 (ghi Parquet): `Exchange` do `repartition` shuffle toàn bộ 2,867,786 dòng. . . . 120

6.1 Đầu ra console của lần chạy thứ hai: thời gian, số task, median/max task và các nút kế hoạch của từng biến thể. . . . 131
6.2 Bảng kết quả Markdown do script in ra ở cuối lần chạy thứ hai. . . . 131
6.3 Tab SQL / DataFrame của Spark UI với 31 truy vấn đã hoàn tất trong phiên chạy. 134
6.4 Kế hoạch E1 khi AQE tắt: `Exchange` với 200 partition shuffle. . . . 135
6.5 Kế hoạch E1 khi AQE bật: `Exchange` 200 partition theo sau là `AQEShuffleRead` ở chế độ coalesced; truy vấn đọc 2,964,624 dòng, 47.6 MiB. . . . 136
6.6 Kế hoạch E2 với broadcast: `BroadcastExchange` và `BroadcastHashJoin`, phía trips không có `Exchange` trước join. . . . 137
6.7 Kế hoạch E2 với ngưỡng -1: `Exchange` và `Sort` ở cả hai phía trước `SortMergeJoin`. . . . 138
6.8 Phân vị thời gian task và số bản ghi shuffle mỗi task của ba stage join E3, lấy từ Spark REST API. . . . 139
6.9 Kế hoạch E3 baseline (AQE tắt): sort-merge join trên dữ liệu lệch. . . . 140
6.10 Summary Metrics của stage 42 (baseline): max 61.9 MiB / 6,264,438 bản ghi so với median 3.4 MiB / 262,619 bản ghi; task index 13 mất 2 s. . . . 141
6.11 Summary Metrics của stage 46 (salting 64): phân bố bản ghi đều hơn, max 917,267 bản ghi, nhưng median thời gian task tăng lên 590 ms. . . . 142
6.12 Kế hoạch E3 với AQE skew join: `AQEShuffleRead` báo 1 partition lệch được tách thành 4 split (16 thành 19 partition), sau đó là `SortMergeJoin(skew=true)`. . . . 143
6.13 Summary Metrics của stage 52 (AQE skew join): max 682 ms và 1,576,365 bản ghi mỗi task. . . . 144
6.14 Tab Storage của Spark UI: DataFrame `cleaned` được cache 8/8 partition, 100 %, 59.4 MiB trong bộ nhớ. . . . 145

Danh mục bảng

1.1 Phần cứng, phần mềm và cấu hình HDFS dùng trong thực nghiệm. 16
1.2 Ánh xạ địa chỉ IP sang hostname của các DataNode. 17
1.3 Số block của cùng một tệp khi dùng block 128 MB và 16 MB. 21
1.4 Các tệp block vật lý trên datanode1 và tệp checksum đi kèm. 22
1.5 Dung lượng logic và dung lượng thô theo `hdfs dfs -du -h` sau khi `setrep 2`. 22
1.6 Diễn biến sau khi dừng datanode3 theo `dfsadmin -report` và log NameNode. 23
1.7 Vị trí replica của tệp `small_blocks` (hệ số 2) trước và sau khi dừng datanode3. 26
1.8 Trạng thái các tệp trong `/data` khi chỉ còn 2 DataNode sống. 27
1.9 Danh mục tệp log và ảnh nộp kèm báo cáo. 32

2.1 Môi trường thực nghiệm. 36
2.2 Dữ liệu sử dụng trong thực nghiệm. 37
2.3 Counter của job đếm từ trên YARN. 39
2.4 So sánh counter giữa hai lần chạy không và có combiner. 41
2.5 Counter bổ sung của hai lần chạy khu vực (trích từ `nocombiner.log` và `combiner.log`). 41
2.6 Giá cước trung bình (`fare_amount`) của 10 khu vực đón khách nhiều chuyến nhất. 47
2.7 Danh mục tệp nộp kèm trong `submission/L02/`. 51

3.1 Môi trường thực nghiệm. 55
3.2 Dữ liệu đầu vào. 57
3.3 Nội dung bảng `TBLS` và `SDS` trong Metastore sau bước tạo bảng ngoài. 58
3.4 Phân vùng của `trips_orc` sau khi nạp bằng phân vùng động. 58
3.5 Kích thước lưu trữ cùng dữ liệu tháng 01/2024 theo định dạng. 62
3.6 Thời gian truy vấn theo định dạng (số liệu dùng để so sánh là lần chạy thứ hai). 62
3.7 Kết quả Q1 (giống nhau trên CSV, ORC và Parquet). 64
3.8 Kết quả Q2: số chuyến và doanh thu theo borough. 64
3.9 Kết quả kiểm chứng cắt tỉa phân vùng. 64
3.10 Các tệp bucket của `trips_bucketed`. 65
3.11 Các giao dịch trên bảng ACID `zone_notes` và thư mục delta tương ứng. 67
3.12 Các dòng có thời gian đón khách nằm ngoài tháng 01/2024. 71

3.13 Danh mục tệp nộp kèm trong `submission/L03/`. 73

4.1 Môi trường thực nghiệm (stack `l04_ingestion`). 77
4.2 Phân bố trạng thái của 5,000 đơn hàng ban đầu. 78
4.3 Các split của Sqoop khi nạp `orders` với `--split-by order_seq -m 4`. . . . 80
4.4 Số dòng mỗi file khi chia theo cột phân bố đều và cột bị lệch. 82
4.5 Phân bố `customer_zip_code_prefix` theo `width_bucket(..., 1003, 99990, 4)`. 82
4.6 Kết quả các bước nạp gia tăng, merge và export của Sqoop. 83
4.7 So sánh các cách đọc bảng `orders` bằng Spark JDBC. 86
4.8 Số dòng các bảng phụ được Spark nạp sang Parquet. 86
4.9 So sánh Sqoop và Spark JDBC khi nạp bảng `orders`. 90
4.10 Kết quả ba lần chạy Spark nạp gia tăng (thời điểm ngày 2026-10-30). 91
4.11 Danh mục tệp nộp kèm báo cáo L04. 96

5.1 Môi trường thực nghiệm và cấu hình Spark. 100
5.2 Dữ liệu đầu vào. 101
5.3 Số dòng vi phạm từng luật làm sạch (một dòng có thể vi phạm nhiều luật). . . . 103
5.4 Tổng hợp theo quận đón sau làm sạch (`by_borough`). 105
5.5 Top-3 khu vực đón tại Manhattan và Queens. 107
5.6 Số chuyến và doanh thu theo ngày với `lag` và tổng lũy kế (10 ngày đầu). 107
5.7 Một số truy vấn trong tab SQL (log `ui_sql_queries.txt`). 112
5.8 Các job và stage của truy vấn 10 (`by_borough.show()`). 113
5.9 So sánh ghi Parquet phân vùng theo ngày có và không có `repartition`. 118
5.10 Danh mục tệp nộp kèm. 124

6.1 Môi trường thực nghiệm. 129
6.2 Dữ liệu thực nghiệm. 130
6.3 Kết quả đầy đủ của lần chạy thứ hai (17 biến thể). 132
6.4 So sánh một số chỉ số giữa hai lần chạy. 134
6.5 Phân vị min / 25 % / median / 75 % / max tại stage join của E3 (lần chạy thứ hai). 139
6.6 So sánh với bảng kết quả tham chiếu của bài lab. 145
6.7 Danh mục tệp nộp kèm. 150

Chương 1

HDFS: tổ chức block, nhân bản và khả năng chịu lỗi DataNode

Tóm tắt

Bài thực hành khảo sát ba cơ chế của *Hadoop Distributed File System* (HDFS): chia tệp thành block, nhân bản block lên nhiều DataNode và phản ứng của hệ thống khi một DataNode ngừng hoạt động. Cluster Hadoop 3.4.3 gồm 1 NameNode và 3 DataNode chạy bằng Docker Compose; dữ liệu là tệp NYC Yellow Taxi tháng 01/2024 (49,961,641 byte). Cùng một tệp được lưu thành 1 block với block mặc định 128 MB và thành 3 block với block 16 MB, block cuối không bị đệm. Hạ hệ số nhân bản từ 3 xuống 2 làm dung lượng thô giảm từ 142.9 M xuống 95.3 M. Khi dừng datanode3, NameNode tuyên bố node chết sau **90 s**, đúng như công thức tính từ cấu hình. Tệp hệ số 2 được tái nhân bản sang datanode2, còn hai tệp hệ số 3 vẫn under-replicated nhưng `Status: HEALTHY` và md5 trùng khớp. Ngoài ra, fsimage không lưu vị trí block, và đây là lý do NameNode cần safe mode khi khởi động.

Từ khóa: HDFS, block, replication, DataNode, heartbeat, under-replication, safe mode, fsimage.

1.1 Giới thiệu

HDFS được thiết kế để lưu tệp rất lớn trên nhiều máy thông thường, trong đó hỏng phần cứng được coi là chuyện thường ngày [1], [2]. Hệ thống tách siêu dữ liệu (NameNode quản lý) khỏi dữ liệu (DataNode lưu dưới dạng block) và nhân bản mỗi block lên nhiều node. Các bài sau dùng MapReduce chạy trên HDFS, nên cần nắm các cơ chế này trước.

Mục tiêu của L01:

1. Khởi động cluster HDFS nhiều node bằng Docker Compose, đọc trạng thái qua CLI và giao diện web NameNode.

2. Dùng bộ lệnh `hdfs dfs` (mkdir, put, ls, du, cat, get, mv, rm, stat).
3. Dùng `hdfs fsck -files -blocks -locations` để chỉ ra cách một tệp được chia thành block và replica.
4. Thay đổi hệ số nhân bản bằng `setrep` và kích thước block lúc ghi bằng `-D dfs.blocksize`.
5. Dừng một DataNode, quan sát NameNode phát hiện sự cố, báo block under-replicated và tự tái nhân bản.
6. Dùng safe mode và kiểm tra nội dung fsimage.

Mục 1.2 tóm lược lý thuyết, Mục 1.3 mô tả môi trường và quy trình, Mục 1.4 trình bày kết quả, Mục 1.5 trả lời tám câu hỏi checkpoint và Mục 1.6 kết luận.

1.2 Cơ sở lý thuyết

1.2.1 Kiến trúc NameNode và DataNode

HDFS theo kiến trúc chủ tớ kế thừa từ Google File System [1], [2]. NameNode giữ toàn bộ *namespace* (cây thư mục, tên tệp, quyền, danh sách block ID của từng tệp) trong bộ nhớ và ghi bền vững vào *fsimage* (ảnh chụp namespace) cùng *edit log* (nhật ký thay đổi). DataNode lưu dữ liệu thật dưới dạng tệp Linux thông thường. Khi ghi, client hỏi NameNode danh sách DataNode đích rồi đẩy dữ liệu theo pipeline từ DataNode thứ nhất sang thứ hai, thứ ba. Dữ liệu tệp không bao giờ đi qua NameNode [3].

1.2.2 Block và replica

Mỗi tệp được cắt thành các block có kích thước tối đa `dfs.blocksize` (mặc định 128 MB). Khác với block của hệ thống tệp cục bộ, đây chỉ là giới hạn trên: block cuối chỉ lớn bằng phần dữ liệu còn lại [3], [5]. Mỗi block có `dfs.replication` replica, mặc định 3. Trên đĩa DataNode, một replica gồm tệp `blk_<id>` chứa dữ liệu và tệp `blk_<id>_<genstamp>.meta` chứa checksum. Hệ số nhân bản và kích thước block do client quyết định cho từng tệp; `hdfs-site.xml` chỉ cho giá trị mặc định [4].

1.2.3 Heartbeat, node stale và node dead

DataNode gửi *heartbeat* tới NameNode mỗi `dfs.heartbeat.interval` (3 s) và định kỳ gửi *block report* (danh sách block đang giữ). Node im lặng quá `dfs.namenode.stale.datanode.interval` bị coi là *stale*, client sẽ tránh node này. Node bị coi là *dead* sau

$$T_{dead} = 2 \times \text{recheck-interval} + 10 \times \text{heartbeat-interval}.$$

Với recheck-interval mặc định 5 phút, $T_{dead} = 2 \times 300 + 10 \times 3 = 630$ s, tức 10.5 phút [7].

1.2.4 Under-replication và tái nhân bản

Khi một node chết, NameNode gỡ các replica của node đó khỏi *block map* (bảng ánh xạ block sang DataNode). Block có số replica sống nhỏ hơn hệ số đích trở thành under-replicated và được xếp hàng sao chép từ một replica còn sống sang DataNode chưa có bản sao của block đó; HDFS không đặt hai replica của cùng block trên một DataNode [3]. Tệp vẫn `HEALTHY` khi mọi block còn ít nhất `dfs.namenode.replication.min` (= 1) replica sống. Mất dữ liệu chỉ ứng với trạng thái MISSING hoặc CORRUPT.

1.2.5 Safe mode và fsimage

Vị trí block không có trong fsimage hay edit log, nó chỉ nằm trong RAM của NameNode và được dựng lại từ block report sau mỗi lần khởi động [2], [3]. Trong lúc đó NameNode ở safe mode (chỉ đọc) cho tới khi tỉ lệ block đã được báo cáo đạt `dfs.namenode.safemode.threshold-pct` (99.9 %), cộng thêm khoảng chờ `dfs.namenode.safemode.extension`. Kleppmann [6] bàn rộng hơn về đánh đổi giữa nhân bản, độ bền và chi phí phục hồi trong hệ phân tán.

1.3 Môi trường và phương pháp thực nghiệm

1.3.1 Môi trường

Cluster chạy trên một máy, các thành phần Hadoop là container dùng chung image `de-labs/hadoop:3.4.3` (Bảng 1.1). Chúng tôi rút `recheck-interval` xuống 30 s (mặc định 5 phút) để thấy được sự cố DataNode trong khoảng 90 s thay vì phải chờ hơn 10 phút, và rút `safemode.extension` xuống 5 s.

Bảng 1.1. Phần cứng, phần mềm và cấu hình HDFS dùng trong thực nghiệm.

Thành phần	Giá trị
Máy chủ	WSL2, x86_64, 12 CPU, bộ nhớ Docker 31.9 GB
Phần mềm	Hadoop 3.4.3 (image `de-labs/hadoop:3.4.3`), Docker Compose
Dịch vụ	1 NameNode, 3 DataNode, ResourceManager, NodeManager, JobHistory
Topology	1 rack (`/default-rack`)
`dfs.replication`	3
`dfs.blocksize`	134,217,728 byte (128 MB, mặc định)
`dfs.heartbeat.interval`	3 s
`dfs.namenode.heartbeat.recheck-interval`	30,000 ms (mặc định 300,000 ms)

Thành phần	Giá trị
`dfs.namenode.stale.datanode.interval`	15,000 ms (mặc định 30,000 ms)
`dfs.namenode.redundancy.interval.seconds`	3 s
`dfs.namenode.safemode.extension`	5,000 ms (mặc định 30,000 ms)
`dfs.permissions.enabled`	false (chỉ dùng cho môi trường học tập)

`fsck` in vị trí replica theo IP, nên chúng tôi ghi lại ánh xạ IP sang hostname từ `hdfs dfsadmin -report` ngay sau khi khởi động (Bảng 1.2). Block Pool ID `BP-361839436-172.29.0.7-1793355266756` cho biết NameNode ở 172.29.0.7.

Bảng 1.2. Ánh xạ địa chỉ IP sang hostname của các DataNode.

Địa chỉ IP	Hostname
172.29.0.5	datanode1
172.29.0.2	datanode2
172.29.0.3	datanode3

1.3.2 Dữ liệu

Hai tệp NYC Taxi được đọc từ thư mục `/datasets` bind mount vào container: `taxi_zone_lookup.csv` (12,331 byte, 70 dòng chứa `Manhattan`) và `yellow_tripdata_2024-01.parquet` (49,961,641 byte, `hdfs dfs -du -h` hiển thị 47.6 M).

1.3.3 Quy trình

Mọi lệnh chạy trong container `namenode`, đóng vai trò edge node, trừ lệnh `docker compose` chạy trên host:

1. Khởi động cluster, chờ đủ 3 DataNode đăng ký, mở giao diện web NameNode ở cổng 9870.
2. Thao tác `hdfs dfs`: tạo thư mục, tải `taxi_zone_lookup.csv` lên `/data/taxi`, đọc, sao chép, đổi tên, tải về, xóa, đo dung lượng; in thuộc tính tệp bằng `hdfs dfs -stat`.
3. Tải tệp Parquet hai lần: block mặc định vào `/data/taxi/default_blocks` và `-D dfs.blocksize=16m` vào `/data/taxi/small_blocks`; so sánh bằng `fsck` và liệt kê tệp block vật lý trên datanode1.
4. Hạ hệ số của tệp `small_blocks` bằng `hdfs dfs -setrep -w 2`; ghi thêm một tệp với hệ số 1 trong một lệnh.
5. Chạy `fsck` tệp `small_blocks`, dừng `datanode3` bằng `docker compose stop`, thăm dò `hdfs dfsadmin -report` khoảng 8 s một lần; chạy lại `fsck`, kiểm tra md5 của tệp hệ số 3; khởi động lại datanode3.

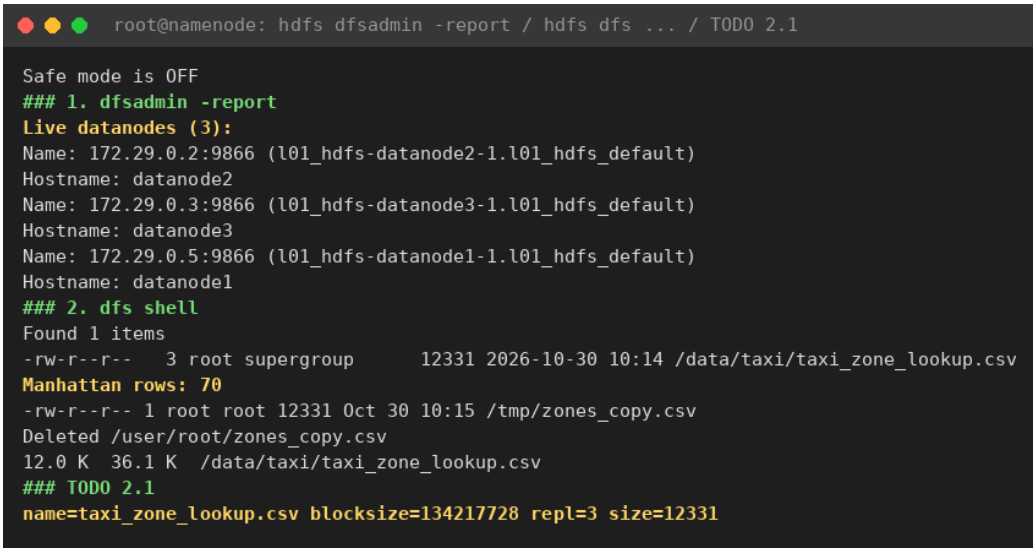
6. Bật, tắt safe mode thủ công, thử ghi và đọc; khởi động lại NameNode để xem safe mode tự động.
7. `saveNamespace`, chuyển fsimage sang XML bằng `hdfs oiv` và tìm thông tin vị trí block trong đó.

Đầu ra lưu trong các tệp `0*.txt` (Phụ lục A), ảnh chụp màn hình trong `img/`.

1.4 Kết quả thực nghiệm

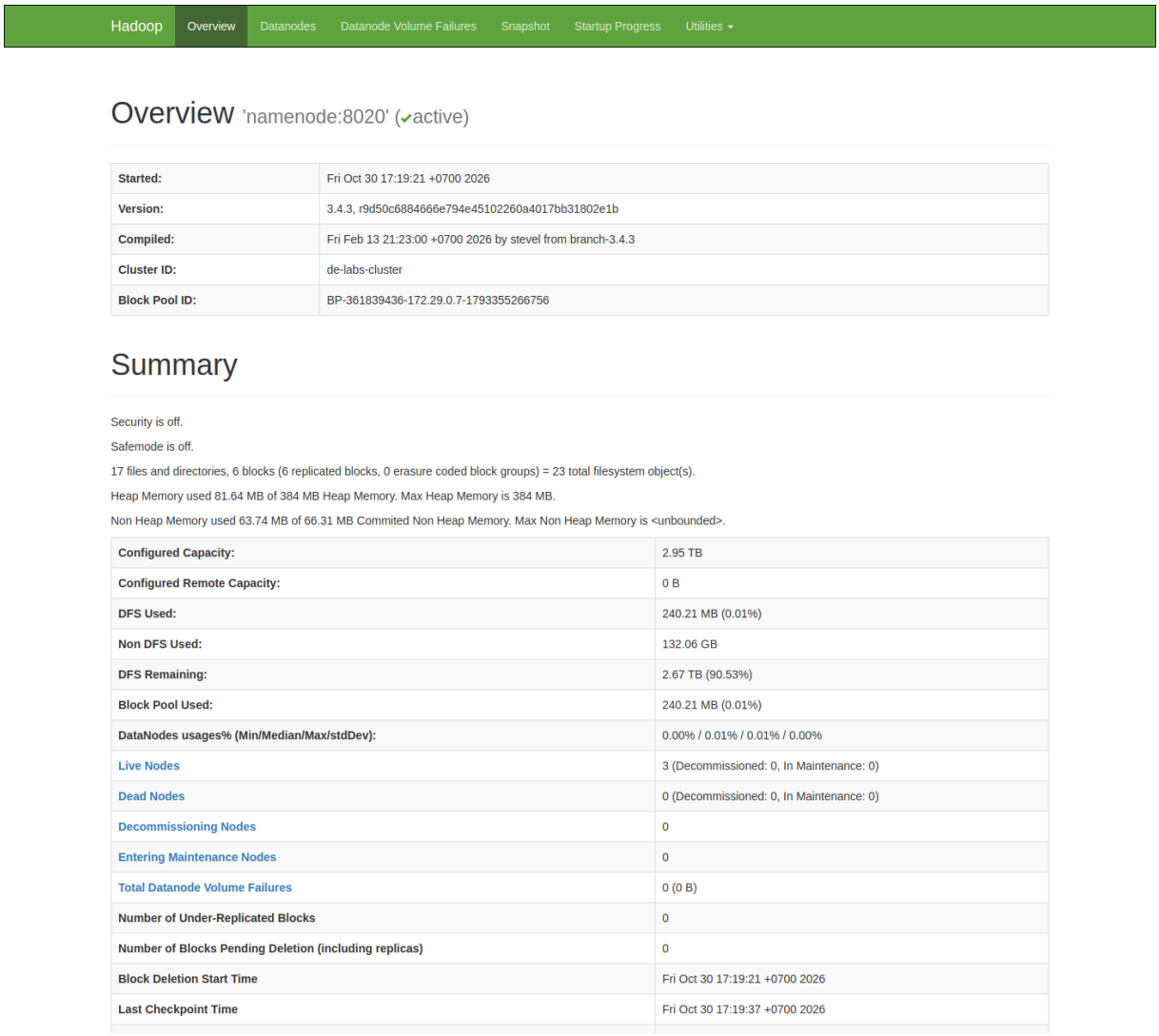
1.4.1 Khởi động cluster và bộ lệnh `hdfs dfs`

`hdfs dfsadmin -report` ghi nhận `Live datanodes (3)`, safe mode OFF. Các lệnh `hdfs dfs` đều chạy được (Hình 1.1). `taxi_zone_lookup.csv` có hệ số 3 (cột thứ hai của `ls`), `cat` kết hợp `grep -c` đếm được 70 dòng Manhattan, bản tải về `/tmp/zones_copy.csv` đúng 12,331 byte. `du -h` in 12.0 K dung lượng logic và 36.1 K dung lượng thô, gấp ba do nhân bản. `hdfs dfs -stat` trả về `blocksize=134217728 repl=3 size=12331`: tệp 12 KB vẫn được gán block size 128 MB.

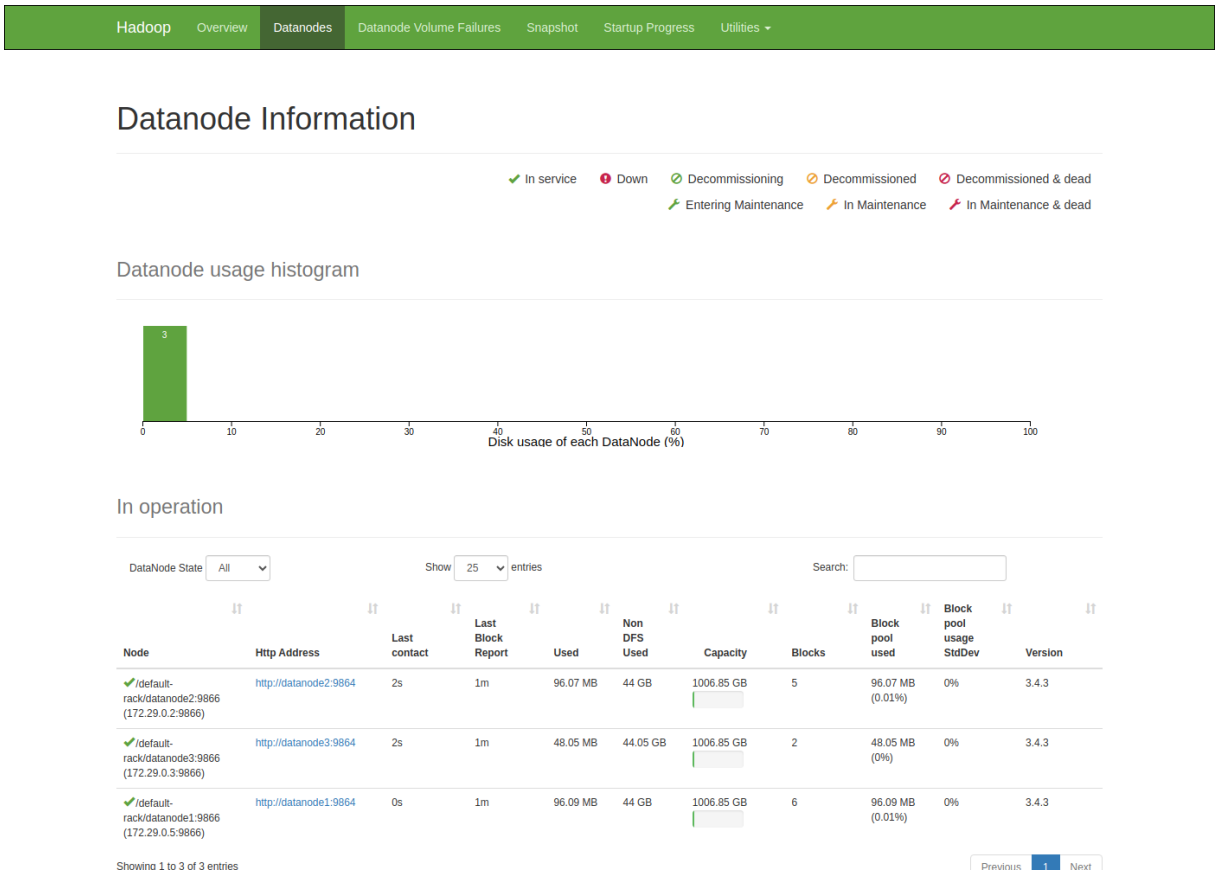


Hình 1.1. Đầu ra `dfsadmin -report` (3 DataNode sống, ánh xạ IP sang hostname), chuỗi lệnh `hdfs dfs` và kết quả `hdfs dfs -stat`.

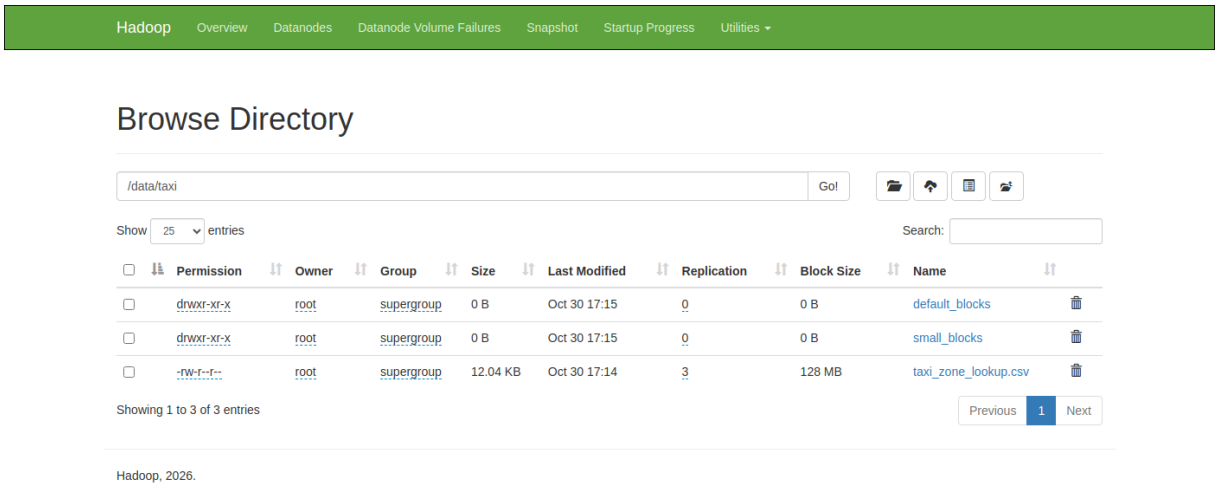
Trên giao diện web, trang Overview báo NameNode `active`, phiên bản 3.4.3, safe mode tắt, 3 node sống, 0 node chết, 0 block under-replicated (Hình 1.2). Trang Datanodes liệt kê cả ba node ở trạng thái In service (Hình 1.3). Trang Browse Directory cho thấy `/data/taxi` gồm `default_blocks`, `small_blocks` và `taxi_zone_lookup.csv` với Replication 3, Block Size 128 MB (Hình 1.4).



Hình 1.2. Trang Overview của NameNode khi cluster khỏe mạnh: 3 Live Nodes, 0 Dead Nodes, 0 block under-replicated.



Hình 1.3. Trang Datanodes của NameNode khi cả ba DataNode đều ở trạng thái In service.



Hình 1.4. Trang Browse Directory hiển thị thư mục `/data/taxi` với hệ số nhân bản và block size của từng mục.

1.4.2 Ảnh hưởng của kích thước block

Tệp Parquet được tải lên hai lần. `hdfs dfs -stat` cho bản thứ nhất `blocksize=134217728`, bản thứ hai `blocksize=16777216`, cả hai `repl=3`. Kết quả `fsck` ở Bảng 1.3 và Hình 1.5.

Bảng 1.3. Số block của cùng một tệp khi dùng block 128 MB và 16 MB.

Cách tải lên	Block size (byte)	Số block	Kích thước từng block (byte)
Mặc định	134,217,728	1	49,961,641
`-D dfs.blocksize=16m`	16,777,216	3	16,777,216 + 16,777,216 + 16,407,209

```
hdfs fsck -files -blocks -locations  (128 MB vs 16 MB blocks)

### 3. blocks
yellow_tripdata_2024-01.parquet blocksize=134217728 repl=3 size=49961641
yellow_tripdata_2024-01.parquet blocksize=16777216 repl=3 size=49961641
--- fsck default_blocks
/data/taxi/default_blocks <dir>
/data/taxi/default_blocks/yellow_tripdata_2024-01.parquet 49961641 bytes, replicated: replication=3, 1 block(s):  OK
0. BP-361839436-172.29.0.7-1793355266756:blk_1073741827_1003 len=49961641 Live_repl=3
  [DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK], DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
Status: HEALTHY

### fsck small_blocks rep3 (settled)
/data/taxi/small_blocks <dir>
/data/taxi/small_blocks/yellow_tripdata_2024-01.parquet 49961641 bytes, replicated: replication=3, 3 block(s):  OK
0. BP-361839436-172.29.0.7-1793355266756:blk_1073741828_1004 len=16777216 Live_repl=3  [DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK], DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK]]
1. BP-361839436-172.29.0.7-1793355266756:blk_1073741829_1005 len=16777216 Live_repl=3
  [DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK], DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
2. BP-361839436-172.29.0.7-1793355266756:blk_1073741830_1006 len=16407209 Live_repl=3
  [DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK], DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
Status: HEALTHY
 Under-replicated blocks:       0 (0.0 %)
```

Hình 1.5. Kết quả `fsck -files -blocks -locations` của tệp Parquet với block mặc định (1 block) và block 16 MB (3 block).

Ở lần chạy `fsck` đầu tiên trên `small_blocks` (10:15:21, ngay sau khi ghi), chúng tôi thấy hai block `blk_1073741829` và `blk_1073741830` chỉ có `Live_repl=2`, kèm `Under-replicated blocks: 2 (66.666664 %)` và `Missing replicas: 2 (22.222221 %)`, dù trạng thái vẫn `HEALTHY`. Lúc đầu chúng tôi nghĩ việc ghi bị lỗi, nhưng lần `fsck` tiếp theo (đầu tệp `02_setrep.txt`) đã cho cả ba block `Live_repl=3` và `Under-replicated blocks: 0`. Lý do là NameNode chỉ cập nhật vị trí replica khi DataNode báo về, nên sau khi ghi có một khoảng trễ ngắn.

Trên datanode1, mỗi block là một tệp `blk_<id>` kèm tệp `.meta` (Hình 1.6, Bảng 1.4). Tệp `blk_1073741830` chỉ có 16,407,209 byte, không bị đệm lên 16 MB. Tệp `.meta` 131,079 byte của block 16,777,216 byte khớp với 4 byte CRC cho mỗi 512 byte dữ liệu ($16{,}777{,}216/512 \times 4 = 131{,}072$) cộng 7 byte header, tức chi phí checksum khoảng 0.78 %.

Bảng 1.4. Các tệp block vật lý trên datanode1 và tệp checksum đi kèm.

Tệp block	Dữ liệu (byte)	`.meta` (byte)	Thuộc tệp
`blk_1073741825`	12,331	107	`taxi_zone_lookup.csv`
`blk_1073741827`	49,961,641	390,335	`default_blocks/...parquet`
`blk_1073741828`	16,777,216	131,079	`small_blocks/...parquet`, block 0
`blk_1073741829`	16,777,216	131,079	`small_blocks/...parquet`, block 1
`blk_1073741830`	16,407,209	128,191	`small_blocks/...parquet`, block 2

```
datanode1: find /hadoop/dfs/data -name 'blk_*'

-rw-r--r-- 1 root root 128191 Oct 30 10:15 blk_1073741830_1006.meta
-rw-r--r-- 1 root root 16407209 Oct 30 10:15 blk_1073741830
-rw-r--r-- 1 root root 16777216 Oct 30 10:15 blk_1073741829
-rw-r--r-- 1 root root 131079 Oct 30 10:15 blk_1073741829_1005.meta
-rw-r--r-- 1 root root 107 Oct 30 10:14 blk_1073741825_1001.meta
-rw-r--r-- 1 root root 49961641 Oct 30 10:15 blk_1073741827
-rw-r--r-- 1 root root 12331 Oct 30 10:14 blk_1073741825
-rw-r--r-- 1 root root 16777216 Oct 30 10:15 blk_1073741828
-rw-r--r-- 1 root root 390335 Oct 30 10:15 blk_1073741827_1003.meta
-rw-r--r-- 1 root root 131079 Oct 30 10:15 blk_1073741828_1004.meta
```

Hình 1.6. Tệp block vật lý `blk_<id>` và `blk_<id>_<genstamp>.meta` trong thư mục dữ liệu của datanode1.

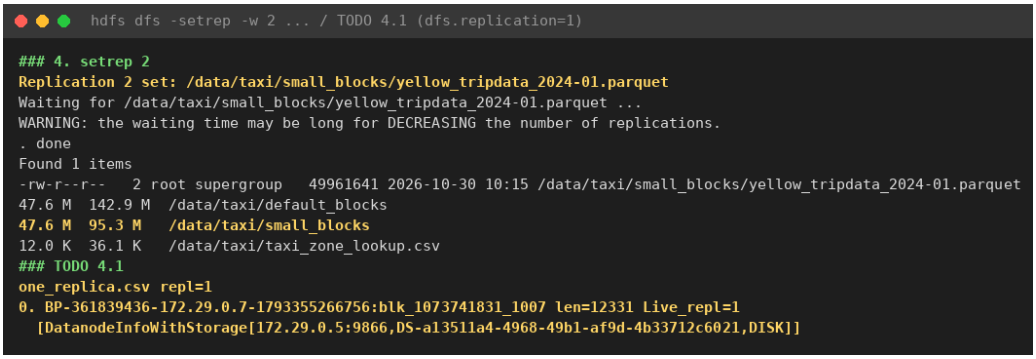
1.4.3 Thay đổi hệ số nhân bản

`hdfs dfs -setrep -w 2` trên tệp `small_blocks` in cảnh báo rằng việc giảm hệ số có thể phải chờ lâu, nhưng thực tế kết thúc gần như ngay (`. done`). Sau đó `ls` hiển thị hệ số 2 và `fsck` cho `Live_repl=2` ở cả ba block. Dung lượng thô của `small_blocks` giảm đúng một phần ba so với bản `default_blocks` còn hệ số 3 (Bảng 1.5).

Bảng 1.5. Dung lượng logic và dung lượng thô theo `hdfs dfs -du -h` sau khi `setrep 2`.

Đường dẫn	Hệ số	Dung lượng logic	Dung lượng thô
`/data/taxi/default_blocks`	3	47.6 M	142.9 M
`/data/taxi/small_blocks`	2	47.6 M	95.3 M
`/data/taxi/taxi_zone_lookup.csv`	3	12.0 K	36.1 K

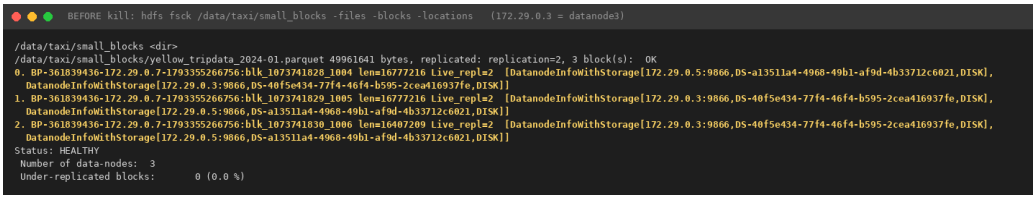
Để ghi tệp với một replica mà không cần `setrep`, chỉ cần ghi đè `-D dfs.replication=1` ngay trong lệnh `put`. Kết quả là `one_replica.csv repl=1`, block duy nhất `blk_1073741831` (12,331 byte, `Live_repl=1`) nằm trên 172.29.0.5, tức datanode1 (Hình 1.7).



Hình 1.7. Kết quả `setrep -w 2`, `du -h` sau khi giảm hệ số và tệp được ghi với hệ số nhân bản 1.

1.4.4 Sự cố DataNode và tái nhân bản

Ngay trước khi dừng node (10:15:58), cả ba block của `small_blocks` đều có replica trên datanode1 và datanode3, không block nào nằm trên datanode2 (Hình 1.8, cột "Trước sự cố" của Bảng 1.7). Vì vậy chúng tôi chọn dừng datanode3: node này giữ replica của mọi block thuộc tệp hệ số 2.



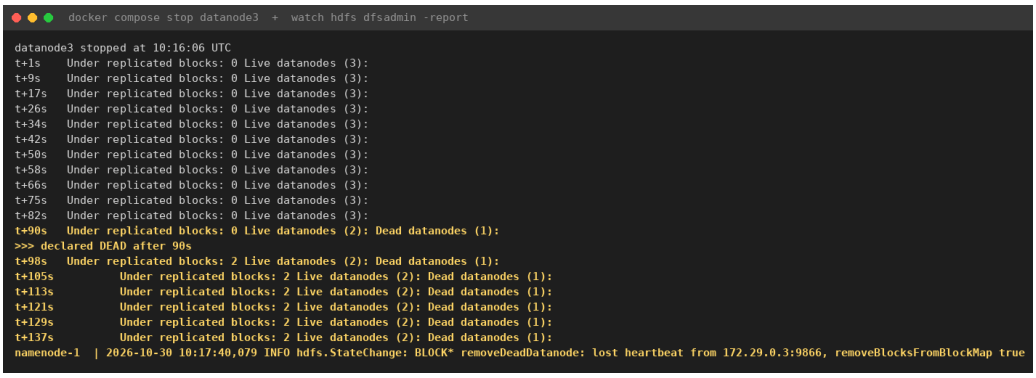
Hình 1.8. `fsck` của tệp block 16 MB trước khi dừng datanode3: mỗi block có 2 replica trên datanode1 và datanode3.

Datanode3 bị dừng lúc 10:16:06 UTC. Diễn biến theo kết quả thăm dò ở Bảng 1.6 và Hình 1.9.

Bảng 1.6. Diễn biến sau khi dừng datanode3 theo `dfsadmin -report` và log NameNode.

Thời điểm	Sự kiện
t+0 s (10:16:06)	`docker compose stop datanode3`
t+1 s đến t+82 s	`Live datanodes (3)`, `Under replicated blocks: 0` (11 lần thăm dò)
t+90 s	`Live datanodes (2)`, `Dead datanodes (1)`: node bị tuyên bố chết
t+94 s (10:17:40,079)	Log NameNode: `removeDeadDatanode: lost heartbeat from 172.29.0.3:9866`

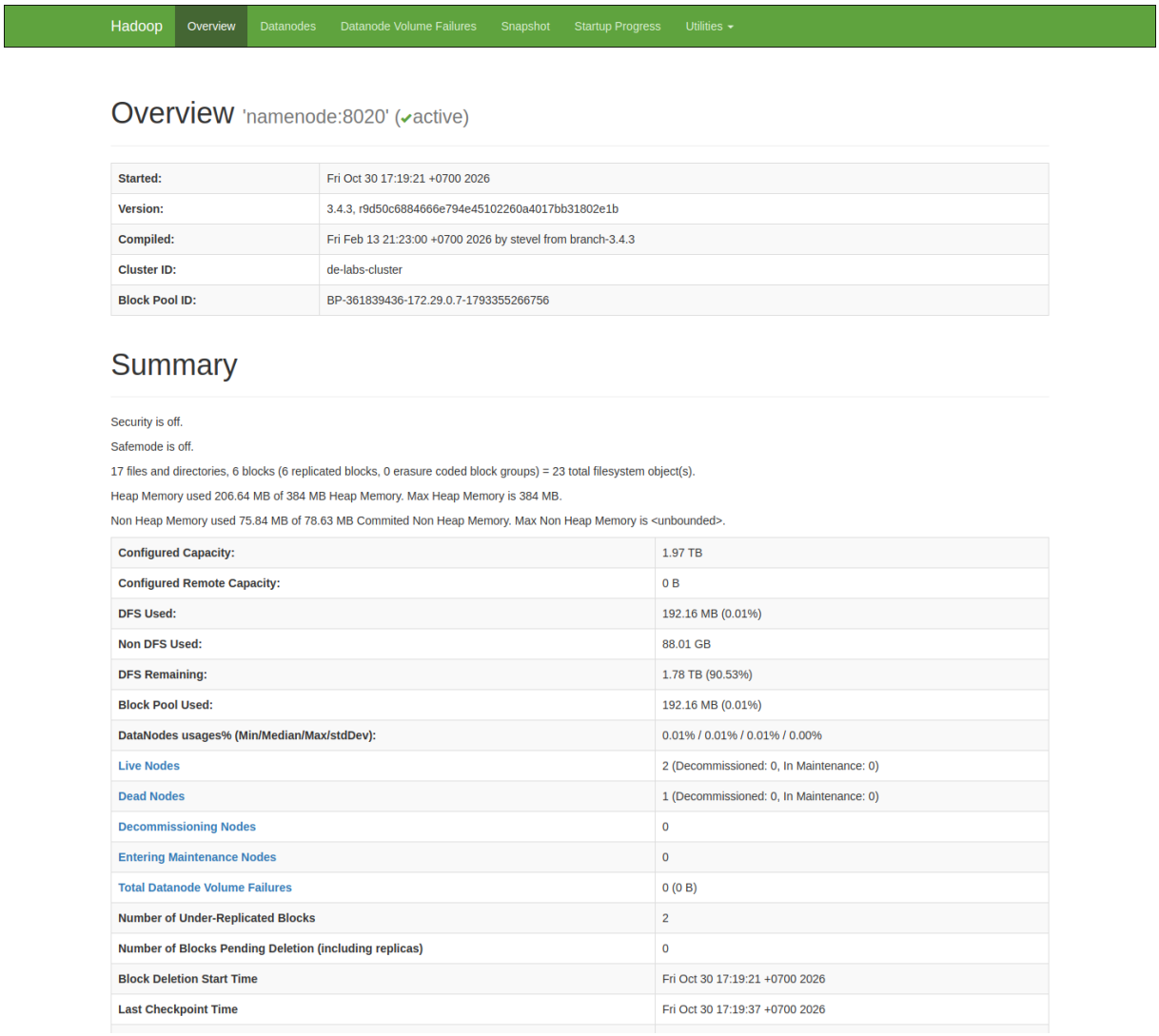
Thời điểm	Sự kiện
t+98 s đến t+137 s	`Under replicated blocks: 2` ổn định qua 6 lần thăm dò



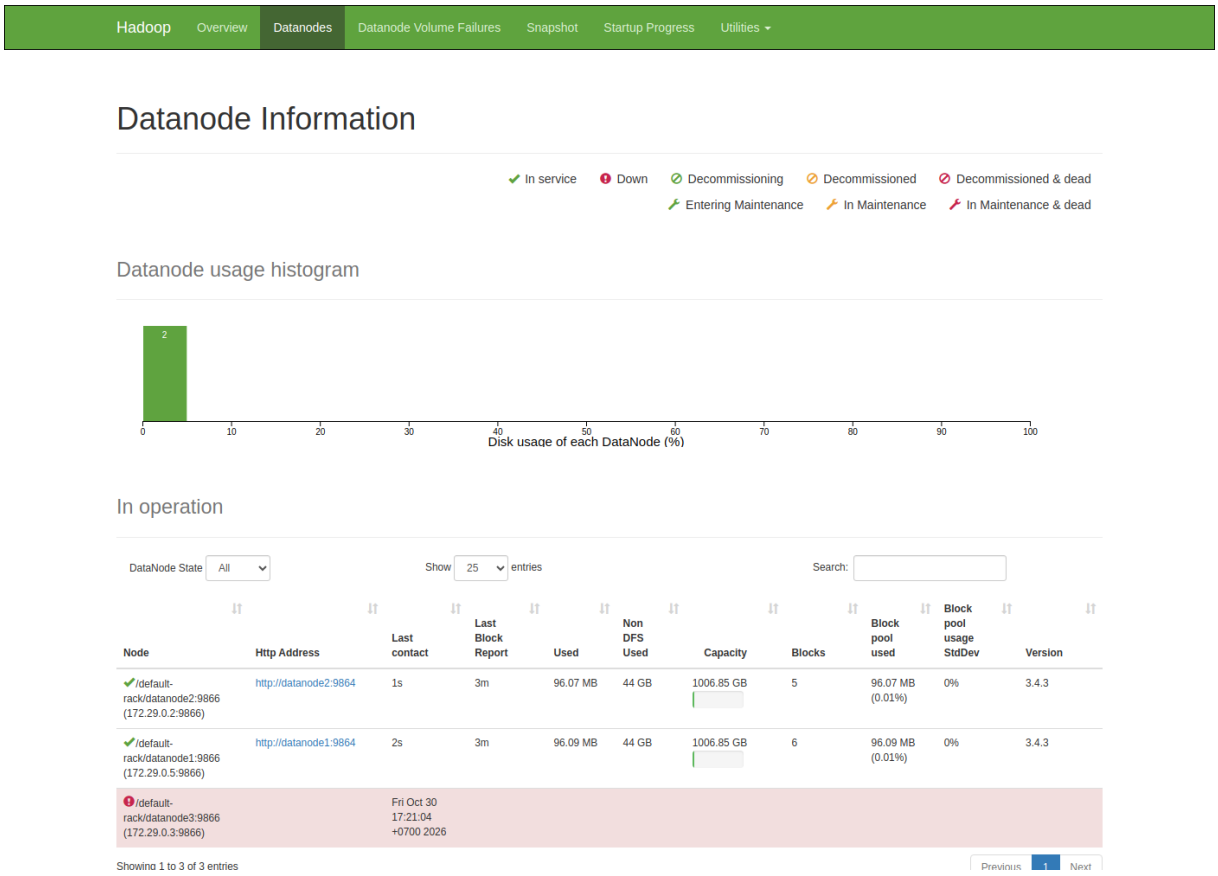
Hình 1.9. Dòng thời gian thăm dò `dfsadmin -report` sau khi dừng datanode3 và dòng log `removeDeadDatanode` của NameNode.

Con số 90 s khớp với công thức ở Mục 1.2: $2 \times 30 + 10 \times 3 = 90$ s. Replica bị gỡ khỏi block map muộn hơn 4 s vì bộ giám sát heartbeat chỉ chạy theo chu kỳ recheck-interval, và phải sau mốc này chỉ số under-replicated mới tăng. Có một điểm lúc đầu chúng tôi thấy lạ: ba block của tệp hệ số 2 cũng mất một replica, nhưng không lần thăm dò nào ghi nhận quá 2 block under-replicated. Cách giải thích hợp lý là ba block đó đã được tái nhân bản trong khoảng giữa t+94 s và t+98 s, nhanh hơn chu kỳ thăm dò. Giá trị 2 còn lại là hai block của các tệp hệ số 3 (Bảng 1.8).

Trên giao diện web, trang Overview hiện một Dead Node (Hình 1.10) và trang Datanodes đánh dấu datanode3 màu đỏ kèm thời điểm liên lạc cuối (Hình 1.11). Lần đo chính chúng tôi không kịp chụp giao diện, nên hai ảnh này được chụp ở một lần dừng datanode3 khác. Mốc "Last contact" 17:21:04 +0700 (10:21:04 UTC) muộn hơn dòng thời gian trong log, và hai ảnh không thuộc lần đo ở Bảng 1.6.



Hình 1.10. Trang Overview của NameNode khi datanode3 đã bị tuyên bố chết.

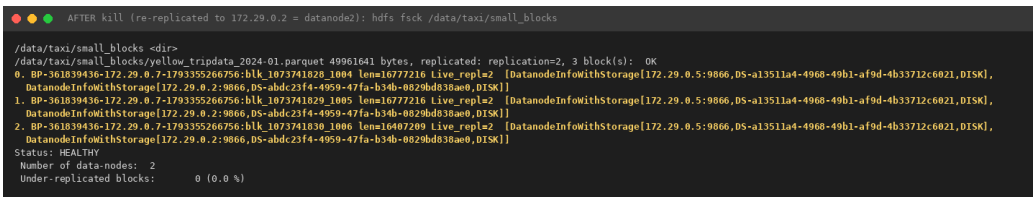


Hình 1.11. Trang Datanodes khi datanode3 chết: datanode1 và datanode2 In service, datanode3 ở trạng thái Down.

`fsck` lúc 10:18:38 cho thấy `small_blocks` đã trở lại `OK`: `Number of data-nodes: 2`, cả ba block `Live_repl=2`, `Under-replicated blocks: 0` (Hình 1.12). Replica trên datanode3 đã được chép sang datanode2, node còn sống duy nhất chưa giữ bản sao (Bảng 1.7).

Bảng 1.7. Vị trí replica của tệp `small_blocks` (hệ số 2) trước và sau khi dừng datanode3.

Block	Kích thước (byte)	Trước sự cố	Sau sự cố
`blk_1073741828`	16,777,216	datanode1, datanode3	datanode1, datanode2
`blk_1073741829`	16,777,216	datanode3, datanode1	datanode1, datanode2
`blk_1073741830`	16,407,209	datanode3, datanode1	datanode1, datanode2

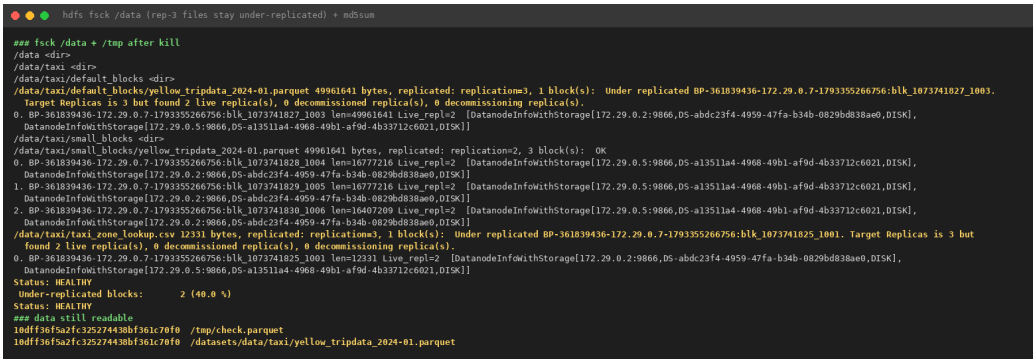


Hình 1.12. `fsck` của tệp block 16 MB sau khi dừng datanode3: replica đã chuyển sang datanode2, tệp trở lại trạng thái OK.

Với toàn bộ `/data`, `fsck` báo **`Under-replicated blocks: 2 (40.0 %)`** (2 trên 5 block) và `Status: HEALTHY` (Hình 1.13). Hai block này thuộc hai tệp hệ số 3, mỗi block còn 2 replica trên datanode2 và datanode1 (Bảng 1.8). Tệp Parquet hệ số 3 tải về có md5 `10dff36f5a2fc325274438bf361c70f0`, trùng với bản gốc. Tệp `/tmp/one_replica.csv` không bị MISSING vì replica duy nhất của nó nằm trên datanode1.

Bảng 1.8. Trạng thái các tệp trong `/data` khi chỉ còn 2 DataNode sống.

Tệp	Hệ số đích	Block	`Live_repl`	Trạng thái
`default_blocks/...parquet`	3	`blk_1073741827`	2	Under replicated
`small_blocks/...parquet`	2	3 block	2	OK
`taxi_zone_lookup.csv`	3	`blk_1073741825`	2	Under replicated



Hình 1.13. `fsck /data` khi datanode3 chết: hai tệp hệ số 3 under-replicated nhưng HEALTHY; md5 của tệp tải về trùng bản gốc.

Khởi động lại datanode3, `dfsadmin -report` trở về `Live datanodes (3)` và `fsck` báo `Under-replicated blocks: 0`, `Over-replicated blocks: 0`, `Status: HEALTHY` (Hình 1.14). Các block hệ số 3 đã có lại replica thứ ba. Tệp hệ số 2 lúc này có replica trên cả ba node, và NameNode đã xóa bớt bản thừa.



Hình 1.14. Trạng thái cluster sau khi datanode3 quay lại: không còn block under-replicated hay over-replicated.

1.4.5 Safe mode và fsimage

Khi bật safe mode thủ công, lệnh ghi bị từ chối với thông báo `put: Cannot create file/tmp/x.csv._COPYING_. Name node is in safe mode.`, còn `hdfs dfs -ls /data/taxi` vẫn trả về 3 mục (Hình 1.15). Sau `docker compose restart namenode`, `-safemode get` báo `Safe mode is ON` trong vài giây đầu rồi chuyển sang OFF khi đã nhận đủ block report.

`saveNamespace` tạo `fsimage_0000000000000000059` dung lượng 1,574 byte (kèm `.md5` 62 byte, `seen_txid` 3 byte, `VERSION` 189 byte). Bản XML do `hdfs oiv` sinh ra có tên thư mục và tệp (`tmp`, `hadoop-yarn`, `user`, `root`, `data`, `taxi`, `taxi_zone_lookup.csv`, …) cùng block ID, nhưng tìm thông tin DataNode (cổng 9866) thì được 0 dòng. fsimage không chứa vị trí block.

safe mode (manual + at restart) / fsimage via hdfs oiv

```
### 6. safe mode manual
Safe mode is OFF
Safe mode is ON
put: Cannot create file/tmp/x.csv._COPYING_. Name node is in safe mode.
Found 3 items
drwxr-xr-x   - root supergroup          0 2026-10-30 10:15 /data/taxi/default_blocks
drwxr-xr-x   - root supergroup          0 2026-10-30 10:15 /data/taxi/small_blocks
Safe mode is OFF
### automatic safe mode at restart
Safe mode is ON
Safe mode is OFF
### 7. fsimage
Safe mode is ON
Save namespace successful
Safe mode is OFF
-rw-r--r-- 1 root root    1574 Oct 30 10:19 fsimage_0000000000000000059
-rw-r--r-- 1 root root      62 Oct 30 10:19 fsimage_0000000000000000059.md5
-rw-r--r-- 1 root root       3 Oct 30 10:19 seen_txid
-rw-r--r-- 1 root root     189 Oct 30 10:19 VERSION
<name></name>
<name>tmp</name>
<name>hadoop-yarn</name>
<name>staging</name>
<name>mr-history</name>
<name>user</name>
<name>root</name>
<name>done</name>
<name>tmp</name>
<name>data</name>
<name>taxi</name>
<name>taxi_zone_lookup.csv</name>
block location info in fsimage? 0
```

Hình 1.15. Safe mode thủ công (ghi bị từ chối, đọc vẫn hoạt động), safe mode tự động khi khởi động lại và nội dung fsimage.

1.5 Thảo luận

1.5.1 Tệp 300 MB với cấu hình mặc định chiếm bao nhiêu block, replica và dung lượng?

Số block là $\lceil 300/128 \rceil = 3$: hai block 128 MB và một block cuối 44 MB. Với hệ số 3 có $3 \times 3 = 9$ replica và khoảng $3 \times 300 = 900$ MB dung lượng thô, cộng thêm tệp `.meta` khoảng 0.78 % (Bảng 1.4). Block cuối không chiếm trọn 128 MB, giống block 16,407,209 byte ở Bảng 1.3, và phép nhân ba dung lượng khớp với số đo ở Bảng 1.5. NameNode chỉ giữ 3 đối tượng block trong RAM, không phụ thuộc hệ số nhân bản.

1.5.2 Vì sao block cuối của tệp taxi nhỏ hơn 16 MB, và có tốn 16 MB đĩa không?

Vì tệp kết thúc ở đó: $49{,}961{,}641 - 2 \times 16{,}777{,}216 = 16{,}407{,}209$ byte. Tệp `blk_1073741830` trên datanode1 đúng bằng kích thước này (Hình 1.6), nên không có phần đĩa nào bị bỏ phí. Block của hệ thống tệp cục bộ (ví dụ 4 KB) thì khác, nó được cấp phát trọn và phần thừa bị bỏ trống. Tệp 12,331 byte được gán block size 128 MB cũng chỉ chiếm 12,331 byte trên mỗi DataNode.

1.5.3 Vị trí block được lưu ở đâu, và khi NameNode khởi động lại thì sao?

Ánh xạ block sang DataNode chỉ nằm trong RAM của NameNode. Thực nghiệm ở Hình 1.15 cho thấy fsimage có tên tệp và block ID nhưng không có thông tin DataNode nào. Khi khởi động lại, NameNode nạp namespace từ fsimage và edit log rồi dựng lại block map từ block report, nên phải ở safe mode cho tới khi nhận đủ báo cáo. Thiết kế này hợp lý vì DataNode mới biết chắc block nào đang nằm trên đĩa của nó. Trong lúc NameNode tắt, đĩa có thể hỏng hoặc node bị thay, nên danh sách vị trí lưu sẵn sẽ nhanh chóng sai [1], [2]. Hiện tượng thiếu replica tạm thời ngay sau khi ghi (Mục 1.4) cũng cho thấy NameNode chỉ biết vị trí replica qua báo cáo của DataNode.

1.5.4 Vì sao tệp hệ số 2 được phục hồi còn tệp hệ số 3 thì không?

Muốn tái nhân bản phải có một DataNode còn sống chưa giữ replica của block, vì hai replica trên cùng node không tăng khả năng chịu lỗi nên HDFS không cho phép. Sau khi datanode3 chết chỉ còn datanode1 và datanode2. Mỗi block của tệp hệ số 2 ban đầu nằm trên datanode1 và datanode3 (Bảng 1.7), nên datanode2 nhận bản sao mới và tệp trở lại `OK` (Hình 1.12). Block của hai tệp hệ số 3 thì đã có mặt trên cả hai node còn lại (Bảng 1.8), không còn chỗ cho bản thứ ba, nên phải chờ datanode3 quay lại (Hình 1.14). Số replica tối đa đạt được là $\min(\text{hệ số đích}, \text{số DataNode sống})$.

1.5.5 `fsck` báo gì khi có block under-replicated, và vì sao đó chưa phải lỗi?

`fsck /data` vẫn báo `Status: HEALTHY` khi có 2 block under-replicated (Hình 1.13). HEALTHY nghĩa là mọi block còn ít nhất `dfs.namenode.replication.min` (= 1) replica sống, nên dữ liệu vẫn đọc được, và md5 trùng khớp ở Mục 1.4 xác nhận điều này. Under-replication chỉ làm giảm độ dư thừa: hỏng thêm một node thì các block này còn một bản. Lỗi thật là MISSING hoặc CORRUPT, khi block không còn replica hợp lệ nào. Nếu replica duy nhất của `/tmp/one_replica.csv` nằm trên datanode3 thì tệp này đã bị báo MISSING; trong thực nghiệm nó may mắn nằm trên datanode1.

1.5.6 Nếu giữ `recheck-interval` mặc định 5 phút thì sao, và vì sao timeout dài lại tốt?

Node sẽ bị tuyên bố chết sau 2×5 phút $+10 \times 3$ s $= 10.5$ phút thay vì 90 s như Bảng 1.6. Trong môi trường thật, timeout dài tránh được *replication storm* (bão tái nhân bản). Một node khởi động lại để vá lỗi hoặc mất mạng một phút cũng có thể kéo theo việc sao chép hàng terabyte block, và khi node quay lại thì các bản sao đó thành thừa. Băng thông và I/O đĩa bị tiêu tốn đúng lúc cluster đang không ổn định. Hai ngưỡng phục vụ hai việc khác nhau. Ngưỡng stale ngắn (mặc định 30 s, trong lab 15 s) giúp client tránh node có dấu hiệu bất thường gần như ngay. Ngưỡng dead dài chỉ quyết định lúc bắt đầu tái nhân bản, việc tốn kém nhất. Ngoài ra còn độ trễ tối đa một chu kỳ recheck-interval trước khi replica bị gỡ khỏi block map, như 4 s đo được ở Bảng 1.6.

1.5.7 Vì sao NameNode khởi động trong safe mode và phải chặn ghi?

Lúc khởi động, NameNode biết namespace nhưng chưa biết block nằm ở đâu (Mục 1.5.3). Khi chưa đủ 99.9 % block được báo cáo, nó không phân biệt được block thật sự mất với block chỉ chưa kịp báo. Nếu cho ghi, xóa hay tái nhân bản lúc này, NameNode có thể lên lịch sao chép hàng loạt block chưa được báo cáo hoặc xóa nhầm replica tưởng là thừa. Safe mode giữ hệ thống ở chế độ chỉ đọc cho tới khi block map đáng tin, đúng như Hình 1.15: ghi bị từ chối, đọc vẫn chạy, trạng thái tự chuyển từ ON sang OFF sau khi khởi động lại.

1.5.8 Vì sao 1 triệu tệp 1 KB là vấn đề, còn 1 tệp 1 KB thì không?

Mỗi tệp, thư mục và block là một đối tượng trong heap NameNode, khoảng 150 byte mỗi đối tượng [5]. Một triệu tệp nhỏ tạo ít nhất 2 triệu đối tượng (một inode và một block mỗi tệp), xấp xỉ 300 MB heap cho khoảng 1 GB dữ liệu. Khi ghi còn có 1 triệu lời gọi RPC tới NameNode, và khi xử lý bằng MapReduce thì có 1 triệu map task vì mỗi tệp tạo ít nhất một split. Đây là *small files problem*. Trong thực nghiệm, tệp 12,331 byte vẫn chiếm riêng một block ID và một replica trên mỗi DataNode (Bảng 1.4). Cách xử lý thường dùng là gộp tệp nhỏ thành tệp lớn hoặc dùng

định dạng chứa như Parquet/ORC, SequenceFile, HAR.

1.5.9 Hạn chế của thực nghiệm

Cluster chạy trên một máy với một rack, nên chính sách đặt replica theo rack chưa được kiểm chứng. Các tham số `recheck-interval`, `stale.datanode.interval` và `safemode.extension` đã bị rút ngắn, nên thời gian đo không phản ánh môi trường sản xuất. Vòng thăm dò `sleep 5` cộng thời gian chạy `dfsadmin -report` cho chu kỳ khoảng 8 s, vì vậy thời điểm tái nhân bản xong chỉ xác định được trong khoảng t+94 s đến t+98 s. Dữ liệu chỉ khoảng 48 MB, chưa đủ để đánh giá chi phí mạng của tái nhân bản ở quy mô lớn.

1.6 Kết luận

Block size là giới hạn trên: tệp 49,961,641 byte thành 1 hoặc 3 block tùy tham số phía client, block cuối không bị đệm. Dung lượng thô tỉ lệ với hệ số nhân bản (Bảng 1.5). Thời gian phát hiện node chết đúng bằng $2 \times \text{recheck} + 10 \times \text{heartbeat}$, và NameNode tự tái nhân bản khi còn node hợp lệ. Block không phục hồi được vẫn HEALTHY và đọc được. Vị trí block chỉ nằm trong bộ nhớ NameNode, nên cần safe mode khi khởi động.

Từ kết quả này, chúng tôi rút ra rằng dữ liệu quan trọng nên giữ hệ số ít nhất 3 trên cluster có nhiều DataNode hơn hệ số đó thì mới tự phục hồi được khi một node hỏng. Hệ số 1 chỉ nên dùng cho dữ liệu tạm có thể tạo lại. Nên giữ timeout phát hiện node chết dài như mặc định và tránh lưu nhiều tệp nhỏ. Hướng làm tiếp có thể là thử *erasure coding* (ví dụ chính sách `XOR-2-1-1024k`) để so chi phí lưu trữ với nhân bản ba lần, so sánh decommission có kiểm soát với sự cố đột ngột, chạy balancer sau khi thêm DataNode thứ tư, hoặc cố ý làm hỏng một tệp block để xem cơ chế phát hiện lỗi checksum.

Tài liệu tham khảo

[1] S. Ghemawat, H. Gobioff, and S.-T. Leung, “The Google File System,” in *Proc. 19th ACM Symposium on Operating Systems Principles (SOSP)*, 2003, pp. 29–43.

[2] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, “The Hadoop Distributed File System,” in *Proc. IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.

[3] Apache Software Foundation, “HDFS Architecture,” Apache Hadoop 3.4 Documentation. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

[4] Apache Software Foundation, “HDFS Commands Guide,” Apache Hadoop 3.4 Documentation. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HDFSCommands.

[5] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA, USA: O'Reilly Media, 2015.

[6] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[7] Apache Software Foundation, "hdfs-default.xml," Apache Hadoop 3.4 Documentation. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/hdfs-default.xml

Phụ lục A: Danh mục tệp nộp kèm

Các tệp log và ảnh nộp kèm được liệt kê ở Bảng 1.9.

Bảng 1.9. Danh mục tệp log và ảnh nộp kèm báo cáo.

Tệp	Nội dung
`01_shell_blocks.txt`	`dfsadmin -report`, lệnh `hdfs dfs`, `hdfs dfs -stat`, `fsck` block 128 MB/16 MB, tệp block trên datanode1
`02_setrep.txt`	`fsck` tệp 16 MB khi đã ổn định, `setrep -w 2`, `du -h`, ghi tệp hệ số 1
`03_fsck_small_blocks_BEFORE_kill.txt`	`fsck` tệp 16 MB (hệ số 2) trước khi dừng datanode3
`04_kill_datanode_timeline.txt`	Dòng thời gian thăm dò sau khi dừng datanode3, log `removeDeadDatanode`
`05_fsck_small_blocks_AFTER_kill.txt`	`fsck` tệp 16 MB sau khi tái nhân bản
`06_after_kill_misc.txt`	`fsck /data` khi under-replicated, md5 của tệp tải về và bản gốc
`07_datanode_back.txt`	Trạng thái sau khi datanode3 khởi động lại
`08_safemode_fsimage.txt`	Safe mode thủ công và tự động, `saveNamespace`, phân tích fsimage
`img/01–img/10`	Ảnh chụp terminal của các bước 1 đến 7
`img/11–img/15`	Ảnh chụp giao diện web NameNode (khỏe mạnh, duyệt thư mục, datanode3 chết)

Phụ lục B: Trích đoạn mã nguồn

In tên, block size, hệ số nhân bản và kích thước của `taxi_zone_lookup.csv` (phần 2 của bài, ghi trong `01_shell_blocks.txt`):

```
hdfs dfs -stat "name=%n blocksize=%o repl=%r size=%b"
 ↪  /data/taxi/taxi_zone_lookup.csv
```

```
# name=taxi_zone_lookup.csv blocksize=134217728 repl=3 size=12331
```

Ghi tệp với hệ số 1 trong một lệnh, không dùng `setrep` (phần 4, ghi trong `02_setrep.txt`). `dfs.replication` là tham số phía client nên ghi đè được bằng `-D`:

```
hdfs dfs -D dfs.replication=1 -put
 ↪  /datasets/data/taxi/taxi_zone_lookup.csv /tmp/one_replica.csv
# one_replica.csv repl=1   (replica duy nhất trên 172.29.0.5 = datanode1)
```

Các lệnh chính của thí nghiệm block và sự cố DataNode:

```
hdfs dfs -D dfs.blocksize=16m -put
 ↪  /datasets/data/taxi/yellow_tripdata_2024-01.parquet \
    /data/taxi/small_blocks/
hdfs fsck /data/taxi/small_blocks -files -blocks -locations
hdfs dfs -setrep -w 2
 ↪  /data/taxi/small_blocks/yellow_tripdata_2024-01.parquet
docker compose stop datanode3          # chạy trên máy host
while true; do hdfs dfsadmin -report | grep -E "Live|Dead|Under"; sleep 5;
 ↪  done
hdfs dfs -get /data/taxi/default_blocks/yellow_tripdata_2024-01.parquet
 ↪  /tmp/check.parquet
md5sum /tmp/check.parquet
 ↪  /datasets/data/taxi/yellow_tripdata_2024-01.parquet
```

Chương 2

MapReduce với Hadoop Streaming, map-side join và combiner

Tóm tắt

Bài thực hành viết mapper và reducer bằng Python rồi chạy qua Hadoop Streaming trên cluster Hadoop dựng từ L01 (một NameNode, ba DataNode, ResourceManager, NodeManager, JobHistory Server). Có hai bài toán. Bài thứ nhất đếm từ trên bốn cuốn sách Project Gutenberg (khoảng 2.6 MB). Bài thứ hai đếm số chuyến taxi theo khu vực đón khách trên 1,000,000 dòng NYC Yellow Taxi tháng 01/2024 (92.8 MB), trong đó mã khu vực được đổi sang tên bằng map-side join với bảng tra cứu 265 dòng. Mọi chương trình đều được thử bằng đường ống Unix trước khi đưa lên YARN. Thí nghiệm chính chạy cùng một job hai lần, có và không có combiner. Combiner giảm `Reduce shuffle bytes` khoảng 754 lần, số bản ghi vào reducer từ 1,000,000 xuống 1,382, và kết quả hai lần chạy giống hệt nhau. Tuy vậy job có combiner lại chậm hơn khoảng 3 s, vì trên một máy shuffle gần như không tốn gì. Ở phần mở rộng, chúng tôi tính giá cước trung bình theo khu vực bằng một combiner dạng `(sum, count)`. Kết quả khớp với lần chạy không có combiner.

Từ khóa: MapReduce, Hadoop Streaming, YARN, shuffle, combiner, map-side join, counter, JobHistory.

2.1 Giới thiệu

MapReduce [1] tách việc xử lý dữ liệu lớn thành hai hàm do người lập trình viết, `map` và `reduce`. Phần còn lại do framework lo: chia dữ liệu, lập lịch, chuyển dữ liệu trung gian giữa các nút và chạy lại task lỗi. Ngày nay pipeline mới phần lớn viết bằng Spark, nhưng chuỗi map, shuffle/sort, reduce và ý tưởng gộp cục bộ vẫn còn nguyên, ví dụ bước `HashAggregate (partial)` trong kế hoạch thực thi Spark SQL [6]. Đọc counter của một job MapReduce là cách dễ nhất để thấy shuffle thực sự tốn bao nhiêu.

Bài thực hành có các mục tiêu sau:

1. Viết mapper và reducer bằng Python, kiểm thử cục bộ bằng đường ống `cat | mapper | sort | reducer`.
2. Chạy chương trình trên YARN bằng Hadoop Streaming và đọc job counter.
3. Đếm số chuyến theo khu vực đón khách, kèm map-side join qua distributed cache (tùy chọn `-files`).
4. Đo tác động của combiner lên `Map output materialized bytes` và `Reduce shuffle bytes`.
5. Xem lại các job trên YARN ResourceManager UI (cổng 8088) và JobHistory UI (cổng 19888).

2.2 Cơ sở lý thuyết

2.2.1 Mô hình MapReduce

Theo Dean và Ghemawat [1], hàm `map` biến mỗi bản ghi đầu vào thành không, một hoặc nhiều cặp khóa–giá trị trung gian. Framework nhóm các giá trị cùng khóa và gọi `reduce` một lần cho mỗi khóa. Đầu vào được chia thành các *input split*. Với tệp trên HDFS, mặc định mỗi block là một split, và mỗi split do một map task xử lý. Số reduce task thì người dùng tự đặt qua `mapreduce.job.reduces`, mặc định là 1 [4], [7].

2.2.2 Shuffle và sort

Đầu ra của mỗi map task được phân vùng theo khóa (mặc định dùng `HashPartitioner`), sắp xếp và ghi ra đĩa cục bộ. Sau đó các reducer kéo dữ liệu về qua HTTP rồi merge-sort. Giai đoạn này là shuffle. Nó bảo đảm mỗi reducer nhận khóa theo thứ tự, nên các bản ghi cùng khóa luôn nằm liền nhau [4]. Shuffle dùng cả đĩa, mạng và sắp xếp, nên thường là phần đắt nhất của job.

2.2.3 Hadoop Streaming

Hadoop Streaming [2] cho phép bất kỳ chương trình thực thi nào làm mapper hoặc reducer. Bản ghi đi vào qua stdin dưới dạng dòng văn bản, còn chương trình in `key<TAB>value` ra stdout. Reducer trong API Java nhận sẵn `(key, Iterable<values>)`, nhưng reducer Streaming chỉ thấy một luồng dòng đã sắp xếp và phải tự nhận ra chỗ đổi khóa. Muốn tăng counter tùy biến thì ghi dòng `reporter:counter:<group>,<counter>,<amount>` ra stderr.

2.2.4 Combiner

Combiner là một reducer thu nhỏ, chạy trên đầu ra của từng map task trước khi shuffle. Hadoop có thể gọi combiner 0, 1 hay nhiều lần. Vì vậy hàm gộp phải có tính kết hợp và giao hoán, và

đầu ra phải cùng định dạng với đầu vào [1], [4]. Phép cộng thỏa điều kiện này nên reducer đếm dùng lại được làm combiner. Phép lấy trung bình thì không.

2.2.5 Map-side join và distributed cache

Nếu một bên của phép nối đủ nhỏ, có thể gửi nó tới mọi task qua distributed cache, nạp vào bộ nhớ rồi tra cứu ngay trong mapper. Phép nối xong ở pha map và bên lớn không phải shuffle theo khóa nối. Khi cả hai bên đều lớn thì phải dùng *reduce-side join*, tức shuffle cả hai tập theo khóa [4], [5]. Spark gọi kỹ thuật thứ nhất là broadcast join [6].

2.2.6 YARN và JobHistory

YARN [3] tách quản lý tài nguyên (ResourceManager, NodeManager) khỏi logic của từng ứng dụng (ApplicationMaster). Mỗi map hoặc reduce task chạy trong một container. Khi job kết thúc, JobHistory Server lưu lại counter, cấu hình và log của các task.

2.3 Môi trường và phương pháp thực nghiệm

2.3.1 Môi trường

Bài này dùng lại cluster Docker của L01. Thư mục bài được gắn vào `/lab/L02` trong các container Hadoop, và mọi lệnh chạy trong container `namenode` vì container này có sẵn `python3`. Streaming cần trình thông dịch có mặt trên nút chạy task. Cấu hình xem ở Bảng 2.1.

Bảng 2.1. Môi trường thực nghiệm.

Thành phần	Cấu hình
HDFS	1 NameNode, 3 DataNode, replication factor = 3
YARN	1 ResourceManager (Capacity Scheduler), 1 NodeManager (1 Active Node trên UI)
Tài nguyên YARN	NodeManager cấp 1.5 GB: AM 512 MB + 2 task 384 MB chạy đồng thời
Phân bổ container	tối thiểu `<memory:128, vCores:1>`, tối đa `<memory:1536, vCores:4>`
Lịch sử job	JobHistory Server, cổng 19888; ResourceManager UI cổng 8088
Ngôn ngữ mapper/reducer	Python 3, chạy qua `hadoop-streaming-*.jar`

2.3.2 Dữ liệu

Bảng 2.2 liệt kê dữ liệu đã dùng. Bộ sách gồm bốn cuốn Project Gutenberg có mã 1342, 11, 84 và 2701. Dữ liệu taxi là 1,000,000 dòng đầu của tệp Parquet `yellow_tripdata_2024-01`. Streaming chỉ đọc văn bản nên chúng tôi dùng `parquet_to_csv.py` để chuyển sang CSV không có dòng tiêu đề. Bảng `taxi_zone_lookup.csv` ánh xạ `LocationID` sang `Borough/Zone`.

Bảng 2.2. Dữ liệu sử dụng trong thực nghiệm.

Bộ dữ liệu	Kích thước	Số bản ghi	Lưu trữ HDFS
Sách Gutenberg (4 tệp)	pg11 171K, pg1342 755K, pg2701 1.3M, pg84 439K (tổng 2.6M)	48,749 dòng	`/user/root/books`, 4 tệp
Taxi CSV 2024-01	92,847,108 byte (92.8 MB)	1,000,000 dòng	`/user/root/taxi_csv`, block 16 MB, 6 block
`taxi_zone_lookup.csv`	nhỏ	265 dòng	gửi qua `-files`

Tệp CSV được đưa lên với block 16 MB để job có nhiều split hơn. Theo tính toán, số block là $\lceil 92{,}847{,}108/16{,}777{,}216 \rceil = \lceil 5.53 \rceil = 6$, đúng với kết quả `hdfs fsck`.

2.3.3 Quy trình

1. Viết mapper và reducer đếm từ, thử bằng đường ống Unix có và không có `sort`.
2. Đưa sách lên HDFS, chạy job `wordcount` bằng Streaming với `-files mapper.py, reducer.py`.
3. Chuyển 1,000,000 dòng taxi sang CSV, đưa lên HDFS với `-D dfs.blocksize=16m`, kiểm tra số block bằng `hdfs fsck`.
4. Viết `zones/mapper.py`. Mapper nạp bảng tra cứu, đếm dòng hỏng và phát cặp `zone\t1`. Thử cục bộ trên 1,000 dòng.
5. Chạy job đếm theo khu vực hai lần, đều với `-D mapreduce.job.reduces=2` và chỉ khác nhau ở `-combiner`. Sau đó so counter và `diff` hai đầu ra.
6. Xem các job trên YARN UI, JobHistory UI và bằng CLI (`yarn application -list`, `mapred job -list all`).
7. Mở rộng: tính giá cước trung bình theo khu vực với combiner `(sum, count)`.

Lệnh chạy bản có combiner như dưới đây. Bản không combiner chỉ bỏ dòng `-combiner`.

```
hadoop jar $STREAM -D mapreduce.job.name=zones_combiner -D
↪  mapreduce.job.reduces=2 \
```

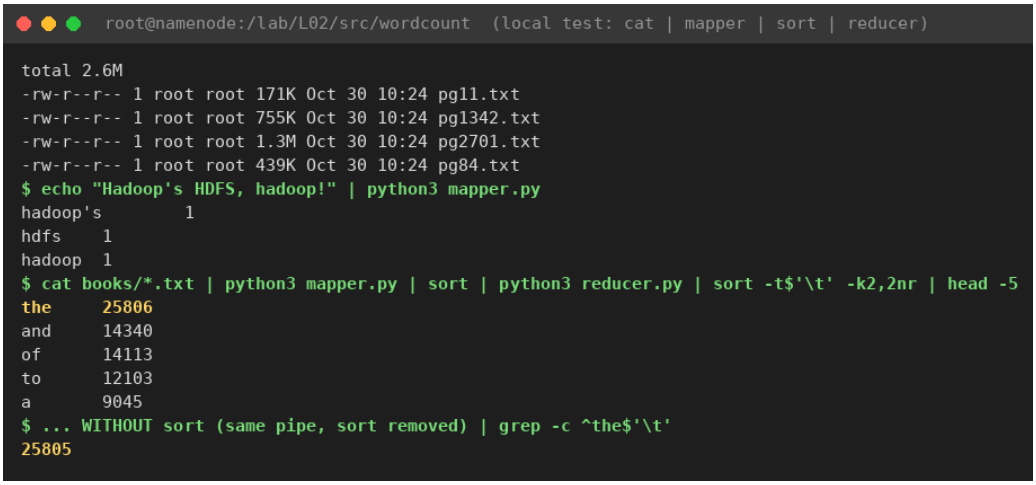
```
-files mapper.py,reducer.py,/datasets/data/taxi/taxi_zone_lookup.csv \
-mapper "python3 mapper.py" -combiner "python3 reducer.py" -reducer
 ↪  "python3 reducer.py" \
-input /user/root/taxi_csv -output /user/root/zones_out_combiner
```

Thời gian wall-clock của job taxi được tính bằng hiệu thời điểm giữa dòng log `Running job` và dòng `completed successfully` của job đó, không dùng lệnh `time`. Cách này bỏ qua phần khởi động client.

2.4 Kết quả thực nghiệm

2.4.1 Đếm từ, kiểm thử cục bộ

Mapper đổi văn bản sang chữ thường rồi lấy từ theo biểu thức `[a-z][a-z']*`. Với đầu vào `Hadoop's HDFS, hadoop!`, mapper in ra ba cặp `hadoop's 1`, `hdfs 1`, `hadoop 1`. Chạy cả đường ống trên bốn cuốn sách, năm từ nhiều nhất là `the 25806`, `and 14340`, `of 14113`, `to 12103` và `a 9045`. Khi bỏ `sort` khỏi đường ống, reducer in ra 25,805 dòng bắt đầu bằng `the` thay vì một dòng tổng (Hình 2.1). Con số này gần bằng tổng số lần xuất hiện, tức gần như lần nào cũng thành một nhóm riêng. Chỉ những chỗ `the` tình cờ đứng liền nhau mới được gộp.



Hình 2.1. Kiểm thử cục bộ đường ống `cat | mapper | sort | reducer` trên bốn cuốn sách: top 5 từ và số dòng `the` khi bỏ `sort`.

2.4.2 Đếm từ trên YARN

Job `wordcount` (`job_1793355278127_0001`) chạy thành công với `number of splits:4`. Counter xem ở Bảng 2.3 và Hình 2.2. Mỗi cuốn sách nhỏ hơn một block nên được một split. Vì vậy job có 4 map task và 1 reduce task theo mặc định. Số khóa vào reducer bằng số dòng đầu ra, và top 5 đọc từ HDFS giống bản chạy cục bộ.

Bảng 2.3. Counter của job đếm từ trên YARN.

Counter	Giá trị
Launched map tasks / reduce tasks	4 / 1
Map input records	48,749
Map output records	462,618
Map output bytes	3,408,811
Map output materialized bytes = Reduce shuffle bytes	4,334,071
Reduce input groups = Reduce output records	20,967
HDFS bytes written (`part-00000`)	226,341
CPU time spent	7,010 ms
Thời gian lệnh (`real`)	23.572 s



Hình 2.2. Counter cuối job `wordcount` trên YARN: 4 map task, `Reduce input groups=20967` và top 5 từ đọc từ HDFS.

2.4.3 Dữ liệu taxi và map-side join cục bộ

`hdfs fsck` báo `92847108 bytes, replicated: replication=3, 6 block(s): OK`. Lúc thử mapper khu vực ở máy cục bộ, mapper chưa đọc được bảng tra cứu vì `-files` chỉ có tác dụng trên cluster. Chúng tôi chép `taxi_zone_lookup.csv` vào thư mục hiện hành để giả lập distributed

cache. Trên 1,000 dòng đầu, ba khu vực nhiều chuyến nhất là `Manhattan/East Village 59`, `Manhattan/Lincoln Square East 55` và `Manhattan/Upper West Side South 45`. Tiếp theo chúng tôi đưa vào hai dòng hỏng, một dòng `garbage` và một dòng có `PULocationID` không phải số. Mapper không dừng lỗi mà ghi đúng hai dòng `reporter:counter:TaxiLab,BAD_RECORDS,1` ra stderr (Hình 2.3). Dữ liệu thật không có dòng hỏng nào. Vì thế nhóm counter `TaxiLab` không xuất hiện trong các job trên cluster, và `Map input records` bằng `Map output records`.

```
parquet_to_csv (1M rows) -> HDFS 16 MB blocks; zones mapper local test

wrote 1,000,000 rows -> /lab/L02/work/yellow_2024-01_1M.csv (92.8 MB)
$ head -2 yellow_2024-01_1M.csv
2,2024-01-01 00:57:55,2024-01-01 01:17:43,1,1.72,1,N,186,79,2,17.7,1,0.5,0,0,1,22.7,2.5,0
1,2024-01-01 00:03:00,2024-01-01 00:09:36,1,1.8,1,N,140,236,1,10,3.5,0.5,3.75,0,1,18.75,2.5,0
$ hdfs fsck /user/root/taxi_csv -files -blocks | grep "block(s)"
/user/root/taxi_csv/yellow_2024-01_1M.csv 92847108 bytes, replicated: replication=3, 6 block(s):  OK
$ head -1000 yellow_2024-01_1M.csv | python3 mapper.py | sort | python3 reducer.py | sort -t$'\t' -k2,2nr | head -3
Manhattan/East Village  59
Manhattan/Lincoln Square East   55
Manhattan/Upper West Side South 45
$ printf "garbage\nx,y,z,a,b,c,d,NOTNUM,1\n" | python3 mapper.py   # bad records -> stderr counter
reporter:counter:TaxiLab,BAD_RECORDS,1
reporter:counter:TaxiLab,BAD_RECORDS,1
```

Hình 2.3. Chuyển Parquet sang CSV, 6 block trên HDFS, kiểm thử cục bộ map-side join trên 1,000 dòng và counter `BAD_RECORDS`.

2.4.4 Có và không có combiner

Hai job `zones_nocombiner` (`job_1793355278127_0002`) và `zones_combiner` (`job_1793355278127_0003`) chạy liền nhau, cùng đầu vào và cùng 2 reducer. Bảng 2.4 gồm các counter đề bài yêu cầu. Bảng 2.5 gồm thêm vài counter lấy từ log đầy đủ.

Bảng 2.4. So sánh counter giữa hai lần chạy không và có combiner.

Counter	Không combiner	Có combiner
Launched map tasks	6	6
Map output records	1,000,000	1,000,000
Combine input records	0	1,000,000
Combine output records	0	1,382
Map output materialized bytes	31,039,071	41,169
Reduce shuffle bytes	31,039,071	41,169
Reduce input records	1,000,000	1,382
Wall-clock time	30.3 s	33.3 s

Bảng 2.5. Counter bổ sung của hai lần chạy khu vực (trích từ `nocombiner.log` và `combiner.log`).

Counter	Không combiner	Có combiner
Map output bytes	29,038,999	29,038,999

Counter	Không combiner	Có combiner
Reduce input groups = Reduce output records	247	247
Spilled Records	2,000,000	2,764
FILE: Number of bytes written	64,607,796	2,614,896
FILE: Number of bytes read	31,039,011	41,109
Total time spent by all map tasks	17,287 ms	17,398 ms
Total time spent by all reduce tasks	13,013 ms	11,353 ms
CPU time spent	15,050 ms	14,020 ms
Shuffled Maps / Merged Map outputs	12 / 12	12 / 12
Killed reduce tasks	1	không xuất hiện

```
zones job: WITHOUT vs WITH combiner (-D mapreduce.job.reduces=2)

== /tmp/nocombiner.log
2026-10-30 10:25:37,187 INFO mapreduce.Job: Running job: job_1793355278127_0002
2026-10-30 10:26:07,488 INFO mapreduce.Job: Job job_1793355278127_0002 completed successfully
                Launched map tasks=6
                Launched reduce tasks=2
                Map output records=1000000
                Map output materialized bytes=31039071
                Combine input records=0
                Combine output records=0
                Reduce shuffle bytes=31039071
                Reduce input records=1000000
WALL nocombiner 30.3 s (10:25:37.187 Running job -> 10:26:07.488 completed)
== /tmp/combiner.log
2026-10-30 10:26:10,419 INFO mapreduce.Job: Running job: job_1793355278127_0003
2026-10-30 10:26:43,754 INFO mapreduce.Job: Job job_1793355278127_0003 completed successfully
                Launched map tasks=6
                Launched reduce tasks=2
                Map output records=1000000
                Map output materialized bytes=41169
                Combine input records=1000000
                Combine output records=1382
                Reduce shuffle bytes=41169
                Reduce input records=1382
WALL combiner 33.3 s (10:26:10.419 Running job -> 10:26:43.754 completed)
$ bash top_zones.sh /user/root/zones_out_combiner
Queens/JFK Airport      59777
Manhattan/Midtown Center        48750
Manhattan/Upper East Side South 47260
Manhattan/Upper East Side North 45867
Manhattan/Lincoln Square East   37424
Manhattan/Penn Station/Madison Sq West  37342
Manhattan/Midtown East  36421
Manhattan/Times Sq/Theatre District     36291
Queens/LaGuardia Airport        33240
Manhattan/Upper West Side South 30849
$ diff <(nocombiner output) <(combiner output)
IDENTICAL results
```

Hình 2.4. Trích các counter so sánh từ hai log và top 10 khu vực đón khách; `diff` giữa hai đầu ra cho kết quả IDENTICAL.

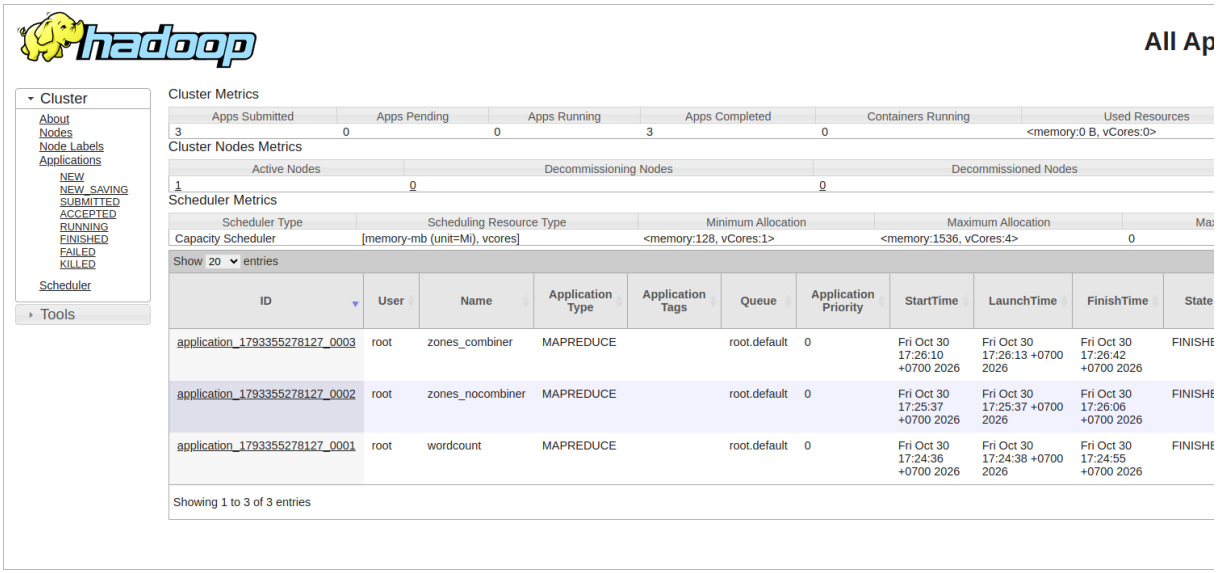
Mapper sinh ra cùng một lượng dữ liệu ở cả hai lần, vì `Map output bytes` không đổi. Combiner thu gọn dữ liệu đó trước khi ghi đĩa và shuffle. Lượng shuffle giảm $31{,}039{,}071/41{,}169 \approx 754$ lần, tức khoảng 99.9 %. Số bản ghi tràn đĩa và số byte ghi đĩa cục bộ cũng giảm theo cùng

cỡ. Cả hai lần chạy đều có 247 khu vực vào reducer, và `diff` hai đầu ra rỗng (Hình 2.4). Ba khu vực đứng đầu là `Queens/JFK Airport 59777`, `Manhattan/Midtown Center 48750` và `Manhattan/Upper East Side South 47260`.

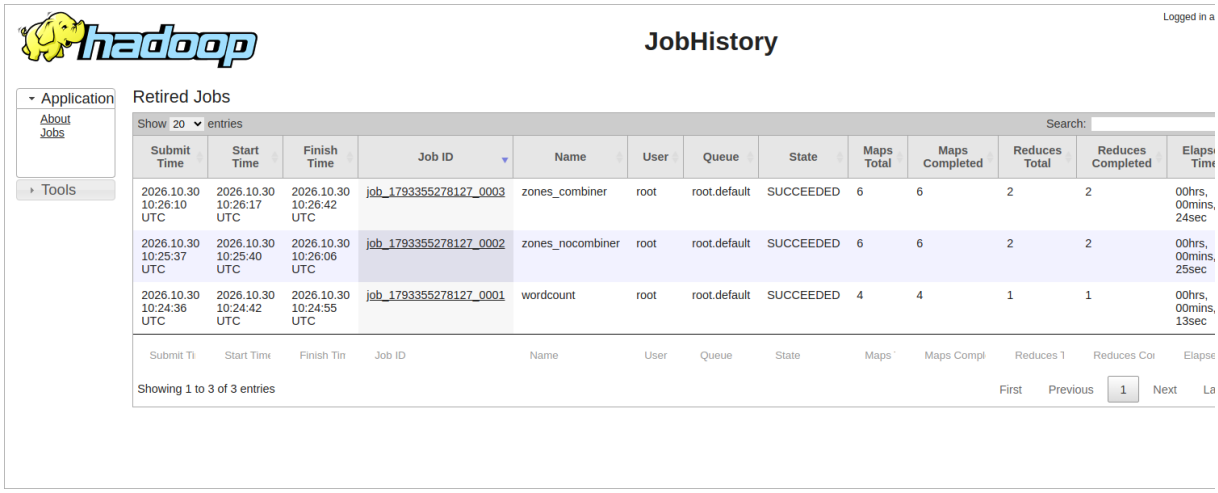
Riêng thời gian thì ngược với dự đoán: bản có combiner chậm hơn 3.0 s. Tổng thời gian map task gần như không đổi, chỉ có thời gian reduce giảm. Lần chạy không combiner có `Killed reduce tasks=1`. Đây là một attempt reduce bị hủy, còn cả hai reduce task vẫn chạy xong, nên kết quả không bị ảnh hưởng.

2.4.5 Giao diện YARN, JobHistory và CLI

ResourceManager UI liệt kê 3 ứng dụng đã xong (`wordcount`, `zones_nocombiner`, `zones_combiner`), 1 Active Node, Capacity Scheduler và giới hạn container như ở Bảng 2.1 (Hình 2.5). Danh sách job trên JobHistory ở Hình 2.6.

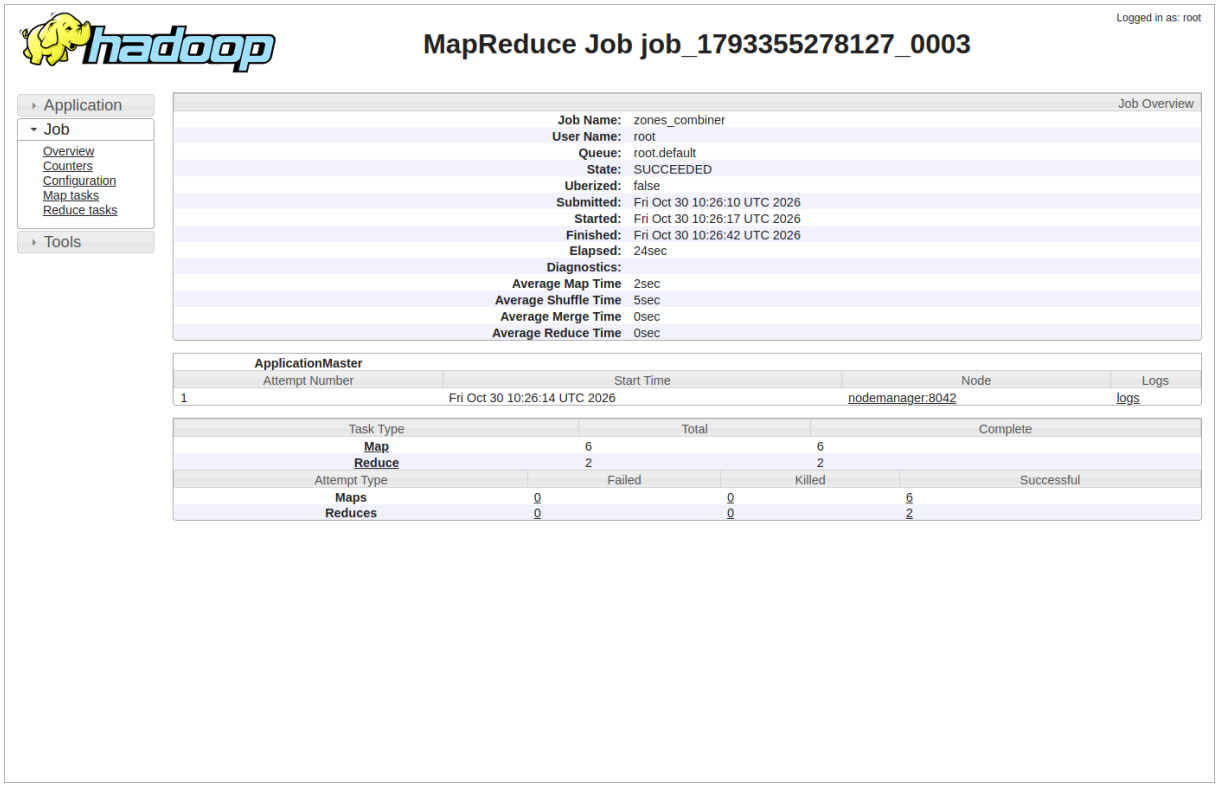


Hình 2.5. YARN ResourceManager UI (cổng 8088): ba ứng dụng MapReduce đã hoàn thành và các chỉ số cluster.



Hình 2.6. JobHistory UI (cổng 19888): danh sách các job đã hoàn thành.

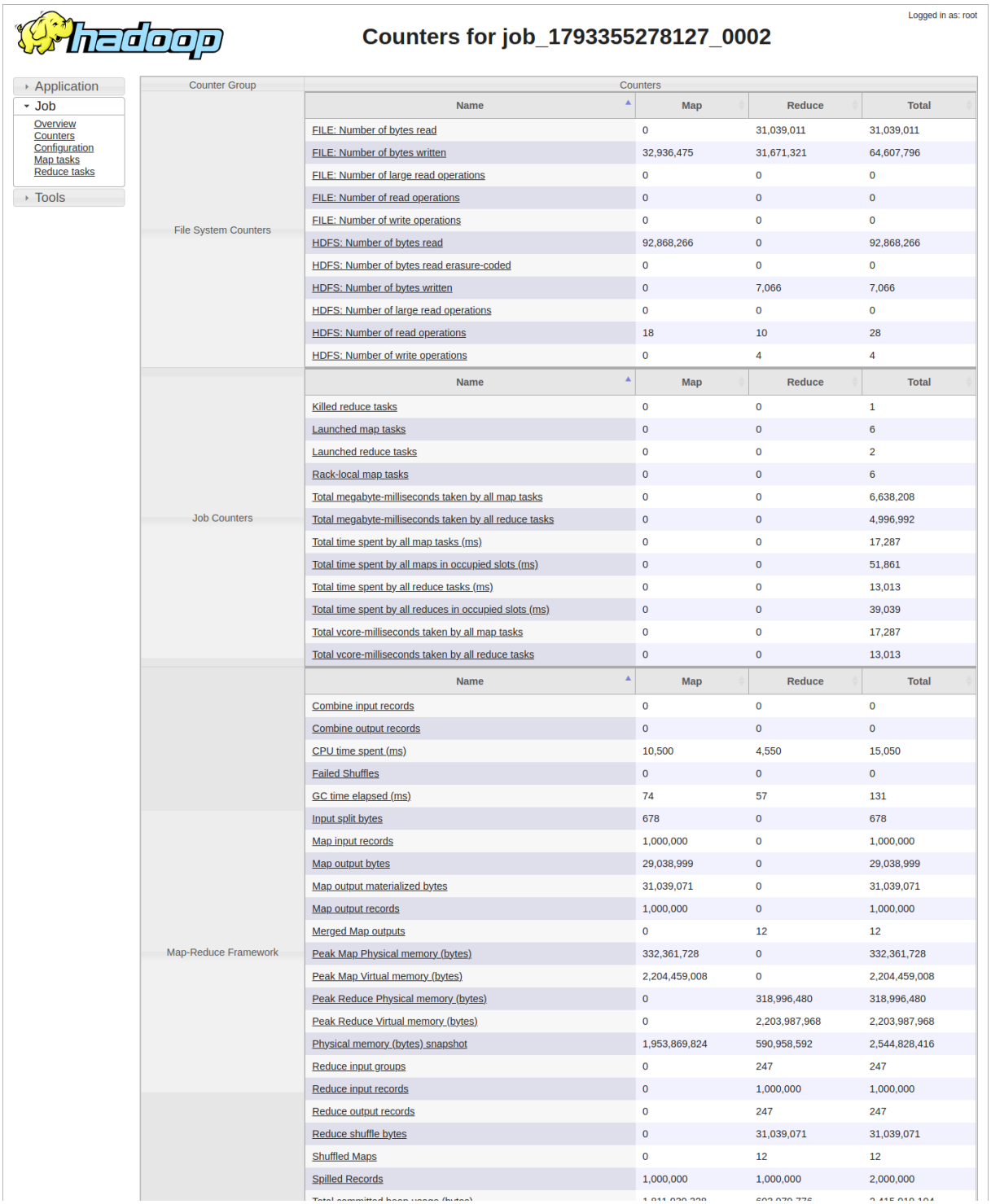
Trang tổng quan của `job_1793355278127_0003` (Hình 2.7) ghi các thông tin sau: SUCCEEDED, không chạy chế độ uber, Elapsed 24 s, map trung bình 2 s, shuffle trung bình 5 s. Cả 6 map và 2 reduce đều thành công, không có attempt nào lỗi hay bị hủy. Elapsed tính từ lúc job bắt đầu chạy nên ngắn hơn con số wall-clock ở Bảng 2.4. Con số wall-clock còn gồm thời gian chờ ApplicationMaster khởi động.



Hình 2.7. JobHistory UI: tổng quan job `zones_combiner` với 6 map task, 2 reduce task và thời gian từng pha.

Trang Counters của hai job (Hình 2.8, Hình 2.9) cho đúng các giá trị in trên console. Trên hai

trang này có thể so trực tiếp `Combine output records`, `Map output materialized bytes` và `Reduce shuffle bytes`.



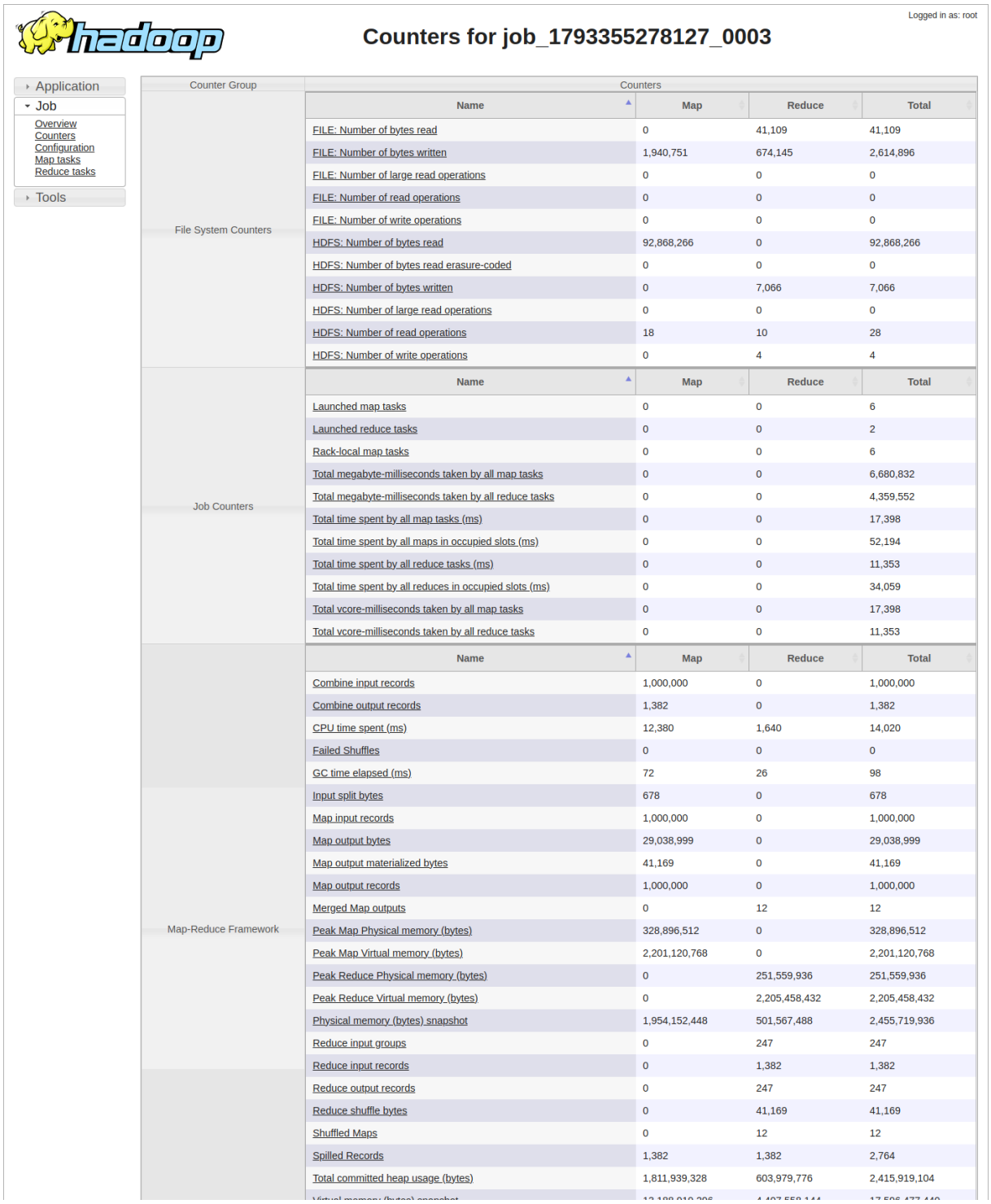
Logged in as: root

Counters for job_1793355278127_0002

- Application
- Job
  - Overview
  - Counters
  - Configuration
  - Map tasks
  - Reduce tasks
- Tools

Counter Group	Name	Map	Reduce	Total
File System Counters	FILE: Number of bytes read	0	31,039,011	31,039,011
	FILE: Number of bytes written	32,936,475	31,671,321	64,607,796
	FILE: Number of large read operations	0	0	0
	FILE: Number of read operations	0	0	0
	FILE: Number of write operations	0	0	0
	HDFS: Number of bytes read	92,868,266	0	92,868,266
	HDFS: Number of bytes read erasure-coded	0	0	0
	HDFS: Number of bytes written	0	7,066	7,066
	HDFS: Number of large read operations	0	0	0
	HDFS: Number of read operations	18	10	28
	HDFS: Number of write operations	0	4	4
Job Counters	Killed reduce tasks	0	0	1
	Launched map tasks	0	0	6
	Launched reduce tasks	0	0	2
	Rack-local map tasks	0	0	6
	Total megabyte-milliseconds taken by all map tasks	0	0	6,638,208
	Total megabyte-milliseconds taken by all reduce tasks	0	0	4,996,992
	Total time spent by all map tasks (ms)	0	0	17,287
	Total time spent by all maps in occupied slots (ms)	0	0	51,861
	Total time spent by all reduce tasks (ms)	0	0	13,013
	Total time spent by all reduces in occupied slots (ms)	0	0	39,039
	Total vcore-milliseconds taken by all map tasks	0	0	17,287
	Total vcore-milliseconds taken by all reduce tasks	0	0	13,013
Map-Reduce Framework	Combine input records	0	0	0
	Combine output records	0	0	0
	CPU time spent (ms)	10,500	4,550	15,050
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	74	57	131
	Input split bytes	678	0	678
	Map input records	1,000,000	0	1,000,000
	Map output bytes	29,038,999	0	29,038,999
	Map output materialized bytes	31,039,071	0	31,039,071
	Map output records	1,000,000	0	1,000,000
	Merged Map outputs	0	12	12
	Peak Map Physical memory (bytes)	332,361,728	0	332,361,728
	Peak Map Virtual memory (bytes)	2,204,459,008	0	2,204,459,008
	Peak Reduce Physical memory (bytes)	0	318,996,480	318,996,480
	Peak Reduce Virtual memory (bytes)	0	2,203,987,968	2,203,987,968
	Physical memory (bytes) snapshot	1,953,869,824	590,958,592	2,544,828,416
	Reduce input groups	0	247	247
	Reduce input records	0	1,000,000	1,000,000
	Reduce output records	0	247	247
	Reduce shuffle bytes	0	31,039,071	31,039,071
	Shuffled Maps	0	12	12
	Spilled Records	1,000,000	1,000,000	2,000,000

Hình 2.8. JobHistory UI: trang Counters của job không combiner.



Logged in as: root

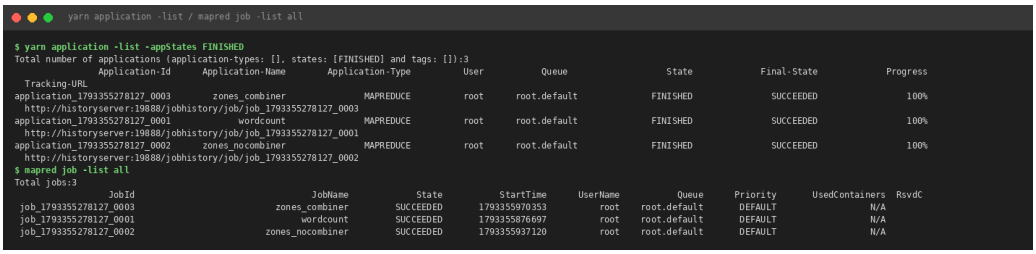
Counters for job_1793355278127_0003

- Application
- Job
  - Overview
  - Counters
  - Configuration
  - Map tasks
  - Reduce tasks
- Tools

Counter Group	Name	Map	Reduce	Total
File System Counters	FILE: Number of bytes read	0	41,109	41,109
	FILE: Number of bytes written	1,940,751	674,145	2,614,896
	FILE: Number of large read operations	0	0	0
	FILE: Number of read operations	0	0	0
	FILE: Number of write operations	0	0	0
	HDFS: Number of bytes read	92,868,266	0	92,868,266
	HDFS: Number of bytes read erasure-coded	0	0	0
	HDFS: Number of bytes written	0	7,066	7,066
	HDFS: Number of large read operations	0	0	0
	HDFS: Number of read operations	18	10	28
	HDFS: Number of write operations	0	4	4
Job Counters	Launched map tasks	0	0	6
	Launched reduce tasks	0	0	2
	Rack-local map tasks	0	0	6
	Total megabyte-milliseconds taken by all map tasks	0	0	6,680,832
	Total megabyte-milliseconds taken by all reduce tasks	0	0	4,359,552
	Total time spent by all map tasks (ms)	0	0	17,398
	Total time spent by all maps in occupied slots (ms)	0	0	52,194
	Total time spent by all reduce tasks (ms)	0	0	11,353
	Total time spent by all reduces in occupied slots (ms)	0	0	34,059
	Total vcore-milliseconds taken by all map tasks	0	0	17,398
	Total vcore-milliseconds taken by all reduce tasks	0	0	11,353
Map-Reduce Framework	Combine input records	1,000,000	0	1,000,000
	Combine output records	1,382	0	1,382
	CPU time spent (ms)	12,380	1,640	14,020
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	72	26	98
	Input split bytes	678	0	678
	Map input records	1,000,000	0	1,000,000
	Map output bytes	29,038,999	0	29,038,999
	Map output materialized bytes	41,169	0	41,169
	Map output records	1,000,000	0	1,000,000
	Merged Map outputs	0	12	12
	Peak Map Physical memory (bytes)	328,896,512	0	328,896,512
	Peak Map Virtual memory (bytes)	2,201,120,768	0	2,201,120,768
	Peak Reduce Physical memory (bytes)	0	251,559,936	251,559,936
	Peak Reduce Virtual memory (bytes)	0	2,205,458,432	2,205,458,432
	Physical memory (bytes) snapshot	1,954,152,448	501,567,488	2,455,719,936
	Reduce input groups	0	247	247
	Reduce input records	0	1,382	1,382
	Reduce output records	0	247	247
	Reduce shuffle bytes	0	41,169	41,169
	Shuffled Maps	0	12	12
	Spilled Records	1,382	1,382	2,764
	Total committed heap usage (bytes)	1,811,939,328	603,979,776	2,415,919,104

Hình 2.9. JobHistory UI: trang Counters của job có combiner.

Trên dòng lệnh, `yarn application -list -appStates FINISHED` cho 3 ứng dụng có Final-State SUCCEEDED, với Tracking-URL trỏ tới `historyserver:19888`. `mapred job -list all` báo `Total jobs:3` (Hình 2.10).



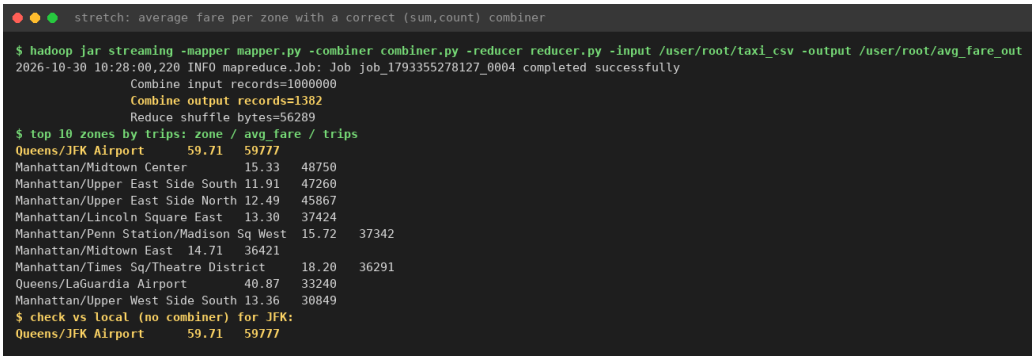
Hình 2.10. Kiểm tra các job đã hoàn thành bằng `yarn application -list` và `mapred job -list all`.

2.4.6 Mở rộng: giá cước trung bình theo khu vực

Mapper phát `zone\t<fare>,1`, trong đó `fare_amount` là cột thứ 10. Combiner cộng dồn cặp `(sum, count)` và xuất ra đúng định dạng nó nhận vào. Reducer cộng tiếp và chỉ chia `sum/count` ở bước cuối. Job `job_1793355278127_0004` chạy xong với `Combine input records=1000000`, `Combine output records=1382` và `Reduce shuffle bytes=56289`. Kết quả ở Bảng 2.6 và Hình 2.11.

Bảng 2.6. Giá cước trung bình (`fare_amount`) của 10 khu vực đón khách nhiều chuyến nhất.

Khu vực	Giá cước trung bình	Số chuyến
Queens/JFK Airport	59.71	59,777
Manhattan/Midtown Center	15.33	48,750
Manhattan/Upper East Side South	11.91	47,260
Manhattan/Upper East Side North	12.49	45,867
Manhattan/Lincoln Square East	13.30	37,424
Manhattan/Penn Station/Madison Sq West	15.72	37,342
Manhattan/Midtown East	14.71	36,421
Manhattan/Times Sq/Theatre District	18.20	36,291
Queens/LaGuardia Airport	40.87	33,240
Manhattan/Upper West Side South	13.36	30,849



Hình 2.11. Job tính giá cước trung bình theo khu vực với combiner `(sum, count)` và đối chiếu JFK với bản chạy không combiner.

Để kiểm tra, chúng tôi tính lại JFK bằng bản cục bộ không combiner và cũng được 59.71 trên 59,777 chuyến. Số chuyến từng khu vực khớp với job đếm ở phần trước. Số bản ghi shuffle vẫn là 1,382, nhưng số byte nhiều hơn job đếm vì mỗi giá trị mang thêm một tổng số thực. Về nội dung, hai sân bay có giá trung bình cao hơn hẳn các khu vực Manhattan, do chuyến đi từ sân bay thường dài hơn.

2.5 Thảo luận

2.5.1 Trong đường ống cục bộ, lệnh nào làm việc của shuffle, và thiếu nó thì sao?

Đó là lệnh `sort`. Nó đưa mọi dòng cùng khóa về cạnh nhau, giống bảo đảm mà shuffle cung cấp trên cluster (phân vùng theo khóa rồi merge-sort ở reducer). Không có `sort`, reducer gặp cùng một từ ở nhiều đoạn rời nhau và in nó ra nhiều lần với số đếm cục bộ. Đó là lý do `the` bị in 25,805 lần (Hình 2.1).

2.5.2 Vì sao reducer Streaming phải tự giữ `current_key`?

Streaming chỉ đưa cho reducer các dòng văn bản đã sắp xếp. Khái niệm nhóm `(key, Iterable<values>)` chỉ có trong API Java. Vì vậy reducer Python phải so khóa hiện tại với `current_key`, in tổng khi khóa đổi, và nhớ in khóa cuối cùng sau vòng lặp (Phụ lục B). Quên bước cuối thì mất từ cuối cùng. Reducer của bài không mắc lỗi này: `Reduce output records` bằng `Reduce input groups` ở Bảng 2.3.

2.5.3 Job taxi có bao nhiêu map task, bao nhiêu reduce task, và vì sao?

Job taxi có **6 map task**. Tệp CSV nằm trong 6 block 16 MB và mỗi block là một split (log ghi `number of splits:6`). Job đếm từ chỉ có 4 map task vì mỗi cuốn sách nhỏ hơn một block. Số reduce task là 2 vì chúng tôi đặt `-D mapreduce.job.reduces=2`. Con số này do người dùng chọn chứ không suy ra từ dữ liệu, và job đếm từ để mặc định là 1. Với 2 reducer, mỗi map task tạo hai phân vùng, nên `Shuffled Maps = 12` (6×2) ở Bảng 2.5. NodeManager chỉ chạy được hai task 384 MB cùng lúc, nên 6 map task chạy thành nhiều đợt. Log thể hiện điều này qua tiến độ map tăng dần 17 %, 33 %, 50 %, 67 %, 83 % rồi 100 %.

2.5.4 Combiner giảm `Reduce shuffle bytes` bao nhiêu lần, và vì sao đầu ra combiner xấp xỉ số khu vực nhân số map task?

Shuffle giảm khoảng **754 lần** (Bảng 2.4). Combiner chạy riêng trên đầu ra của từng map task, và mỗi task chỉ xuất một bản ghi cho mỗi khu vực mà nó gặp. Toàn bộ dữ liệu có 247 khu vực. Chia đều thì mỗi task ra khoảng $1{,}382/6 \approx 230$ bản ghi, tức mỗi split gặp gần hết các khu vực. Chỉ các tổng cục bộ này đi qua shuffle, nên `Reduce input records` cũng chỉ còn 1,382.

2.5.5 Vì sao combiner không làm job nhanh hơn ở đây, và khi nào nó có ích rõ?

Trên một máy, shuffle chỉ là chép dữ liệu qua đĩa cục bộ hoặc loopback, gần như không tốn gì. Trong khi đó combiner thêm một tiến trình Python phải đọc, phân tích và sắp xếp lại 1,000,000 bản ghi ở pha map. Theo Bảng 2.5, tổng thời gian map không giảm. Phần tiết kiệm ở reduce, khoảng 1.7 s, không bù được thời gian khởi động và độ dao động khi lập lịch container. Mỗi cấu hình mới chạy một lần, nên chênh lệch 3.0 s chưa đủ để kết luận combiner làm chậm. Bản tham chiếu của đề bài cũng ghi khoảng 28 s và 37 s, nên dao động cỡ này là bình thường. Trên cluster thật, shuffle đi qua mạng và mỗi reducer phải merge hàng GB từ hàng trăm mapper. Lúc đó việc bỏ được 99.9 % lượng shuffle và gần hết số bản ghi tràn đĩa sẽ lớn hơn nhiều so với chi phí CPU của combiner.

2.5.6 Có dùng lại reducer "trung bình" làm combiner được không?

Không được. Trung bình của các trung bình cho kết quả sai khi các phân vùng có số bản ghi khác nhau. Ví dụ, trung bình của $\{10\}$ và $\{20, 30\}$ là $(10 + 20 + 30)/3 = 20$, còn trung bình của hai trung bình cục bộ là $(10 + 25)/2 = 17.5$. Ngoài ra reducer trung bình xuất ra một số thực, khác định dạng đầu vào, nên không gộp tiếp được. Hadoop có thể gọi combiner 0, 1 hay nhiều lần, nên cách đúng là để combiner phát `(sum, count)` và chỉ reducer mới chia. Phần mở rộng làm theo cách này và kết quả JFK khớp với bản không combiner (Bảng 2.6).

2.5.7 Map-side join trong bài hoạt động thế nào, và khi nào nó không còn hợp?

Mỗi mapper nạp `taxi_zone_lookup.csv` (được phân phối qua `-files`) vào một `dict`, rồi đổi `PULocationID` thành `Borough/Zone` ngay khi đọc từng chuyến. Bảng lớn vì thế không phải shuffle theo khóa nối. Cách này không còn hợp khi bảng "nhỏ" không vừa thoải mái trong bộ nhớ task (ở đây mỗi task chỉ có 384 MB), hoặc khi gửi bảng tới mọi task tốn hơn cả shuffle. Khi đó phải chuyển sang reduce-side join. Spark gọi cùng ý tưởng này là broadcast join, sẽ gặp lại ở L06.

2.5.8 Counter `BAD_RECORDS` hiện ở đâu, và vì sao nên đếm dòng hỏng thay vì cho job dừng?

Counter này nằm trong job counters, tức phần in ra console cuối job và mục Counters trên JobHistory UI, nhóm `TaxiLab`. Counter chỉ hiện khi có ít nhất một dòng hỏng. Dữ liệu trong bài không có dòng hỏng nên nhóm này không xuất hiện. Phần thử cục bộ ở Hình 2.3 cho thấy mapper vẫn phát đúng tín hiệu. Một job xử lý hàng tỷ dòng không nên chết vì vài dòng lỗi, vì như vậy mọi công việc đã làm bị bỏ phí. Đếm số dòng hỏng thì vẫn thấy được vấn đề chất lượng dữ liệu, và có thể đặt ngưỡng để cảnh báo hoặc dừng job khi số dòng hỏng quá nhiều.

2.5.9 Hạn chế của thực nghiệm

Mọi thứ chạy trên một máy với một NodeManager. Các container dùng chung CPU và đĩa, còn shuffle không đi qua mạng thật, nên con số thời gian không áp dụng thẳng cho cluster nhiều nút được. Mỗi cấu hình chỉ chạy một lần. Dữ liệu chỉ là mẫu 1,000,000 dòng của một tháng và không có dòng hỏng, nên `BAD_RECORDS` mới được kiểm chứng ở chế độ cục bộ.

2.6 Kết luận

Đường ống `cat | mapper | sort | reducer` tái hiện đúng mô hình MapReduce, và bỏ `sort` cũng giống như bỏ shuffle. Số map task do số input split quyết định, còn số reduce task do người dùng đặt. Combiner giảm dữ liệu shuffle khoảng 754 lần mà không làm đổi kết quả. Tuy vậy trong môi trường một máy nó không làm job nhanh hơn, và chúng tôi cho rằng lý do nằm ở môi trường chứ không phải ở combiner. Với phép tổng hợp không có tính kết hợp như trung bình, cần một dạng trung gian như `(sum, count)` thì combiner mới dùng được.

Trong thực tế, nên dùng combiner hoặc tổng hợp cục bộ cho các phép tổng hợp có tính kết hợp khi lượng shuffle lớn và đi qua mạng. Map-side join phù hợp khi một bên nằm gọn trong bộ nhớ task. Bản ghi hỏng nên được đếm thay vì bỏ qua âm thầm. Nếu có thêm thời gian, chúng tôi muốn làm bài Top-N với một reducer dùng heap, xem `HashPartitioner` chia khóa thế nào qua `part-00000` và `part-00001`, cố ý làm hỏng một task để quan sát cơ chế thử lại, và so với `groupBy().agg()` của Spark ở L05/L06.

Tài liệu tham khảo

[1] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” in *Proc. 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.

[2] Apache Software Foundation, “Hadoop Streaming,” Apache Hadoop documentation. https://hadoop.apache.org/docs/stable/hadoop-streaming/HadoopStreaming.html

[3] V. K. Vavilapalli et al., “Apache Hadoop YARN: Yet Another Resource Negotiator,” in *Proc. 4th ACM Symposium on Cloud Computing (SoCC)*, 2013.

[4] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA, USA: O’Reilly Media, 2015.

[5] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O’Reilly Media, 2017.

[6] M. Armbrust et al., “Spark SQL: Relational Data Processing in Spark,” in *Proc. ACM SIGMOD International Conference on Management of Data*, 2015.

[7] Apache Software Foundation, “MapReduce Tutorial,” Apache Hadoop documentation.

https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html

Phụ lục A: Danh mục tệp nộp kèm

Bảng 2.7. Danh mục tệp nộp kèm trong `submission/L02/`.

Tệp	Nội dung
`wc_mapper.py`, `wc_reducer.py`	Mapper và reducer đếm từ
`zones_mapper.py`	Map-side join với `taxi_zone_lookup.csv`, counter `BAD_RECORDS`
`stretch_avg_fare/` (`mapper.py`, `combiner.py`, `reducer.py`)	Mở rộng: giá cước trung bình với combiner `(sum, count)`
`01_local_pipe.txt`	Kiểm thử cục bộ đếm từ, có và không có `sort`
`02_wordcount_yarn.txt`	Log và counter job `wordcount` trên YARN, top 5 từ HDFS
`03_taxi_csv_local_zones.txt`	Chuyển CSV, `hdfs fsck`, kiểm thử cục bộ mapper khu vực và `BAD_RECORDS`
`04_combiner_vs_nocombiner.txt`	Trích counter hai lần chạy, wall-clock, top 10 khu vực, kết quả `diff`
`nocombiner.log`, `combiner.log`	Log đầy đủ hai job khu vực
`05_yarn_cli.txt`	Kết quả `yarn application -list` và `mapred job -list all`
`06_stretch_avg_fare.txt`	Kết quả job giá cước trung bình
`img/`	Ảnh chụp terminal và giao diện web

Phụ lục B: Trích đoạn mã nguồn

Mapper đếm từ (`wc_mapper.py`) phát một cặp `word\t1` cho mỗi từ tìm được.

```
WORD = re.compile(r"[a-z][a-z']*")
for line in sys.stdin:
    for word in WORD.findall(line.lower()):
        print(f"{word}\t1")
```

Reducer đếm từ (`wc_reducer.py`) tự nhận ra chỗ đổi khóa và in thêm khóa cuối cùng sau vòng lặp.

```
current_word, current_count = None, 0
for line in sys.stdin:
    word, _, count = line.rstrip("\n").partition("\t")
```

```
    count = int(count)
    if word == current_word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")
        current_word, current_count = word, count
if current_word is not None:  # the last word's total
    print(f"{current_word}\t{current_count}")
```

Mapper khu vực (`zones_mapper.py`) nạp bảng tra cứu, báo dòng hỏng qua stderr và phát `zone\t1`.

```
zones = {}
if os.path.exists("taxi_zone_lookup.csv"):
    with open("taxi_zone_lookup.csv", newline="") as f:
        for row in csv.DictReader(f):
            zones[row["LocationID"]] = f"{row['Borough']}/{row['Zone']}"
for line in sys.stdin:
    fields = line.rstrip("\n").split(",")
    if len(fields) <= PU_COL or not fields[PU_COL].strip().isdigit():
        sys.stderr.write("reporter:counter:TaxiLab,BAD_RECORDS,1\n")
        continue
    pu = fields[PU_COL].strip()
    print(f"{zones.get(pu, pu)}\t1")
```

Combiner của phần mở rộng (`stretch_avg_fare/combiner.py`) cộng dồn `(sum, count)` và giữ định dạng `key\tsum,count`. Reducer có cùng logic, chỉ khác ở chỗ cuối cùng in `total / count` với hai chữ số thập phân kèm số chuyến.

```
for line in sys.stdin:
    key, _, value = line.rstrip("\n").partition("\t")
    s, _, c = value.partition(",")
    if key == current_key:
        total, count = total + float(s), count + int(c)
    else:
        if current_key is not None:
            print(f"{current_key}\t{total},{count}")
        current_key, total, count = key, float(s), int(c)
if current_key is not None:
    print(f"{current_key}\t{total},{count}")
```

Chương 3

Hive: truy vấn SQL trên HDFS, phân vùng, định dạng lưu trữ cột và bucketing

Tóm tắt

Bài thực hành dựng Apache Hive 4.2.1 trên HDFS, Metastore độc lập dùng PostgreSQL 18, Tez chạy ở chế độ cục bộ. Toàn bộ dữ liệu taxi vàng New York tháng 01/2024 (2,964,624 dòng, tệp CSV 274.2 MB) được đưa lên HDFS dưới dạng bảng ngoài, rồi chuyển sang hai bảng phân vùng ORC (nén ZLIB) và Parquet (nén Snappy) bằng phân vùng động. Chúng tôi đo kích thước lưu trữ và thời gian truy vấn, kiểm tra cắt tỉa phân vùng bằng `EXPLAIN`, và thử bucketing. ORC nhỏ hơn CSV 5.2 lần, Parquet nhỏ hơn 4.2 lần; truy vấn Q1 giảm từ 8.53 s trên CSV xuống 1.89 s (ORC) và 1.62 s (Parquet). Lọc theo cột phân vùng chỉ đọc 1 trên 5 phân vùng, còn lọc theo cột thời gian gốc phải đọc cả 5. Bảng bucketing có đúng 4 tệp, và toàn bộ 145,240 chuyến từ JFK nằm trong một tệp. Phân vùng động cũng làm lộ ra 18 dòng có thời gian nằm ngoài tháng, một lỗi chất lượng dữ liệu lẽ ra phải chặn từ trước khi ghi.

Từ khóa: Apache Hive, Metastore, ORC, Parquet, partition pruning, bucketing, ACID, Tez.

3.1 Giới thiệu

Hive là hệ thống đầu tiên đưa SQL lên Hadoop [1]. Ý tưởng chính của nó là tách siêu dữ liệu (lược đồ, vị trí, phân vùng) khỏi dữ liệu: siêu dữ liệu nằm trong một cơ sở dữ liệu quan hệ gọi là Metastore, còn dữ liệu là các tệp bình thường trên HDFS. Cách làm này về sau thành chuẩn chung. Ngày nay Spark SQL, Trino, Impala và cả bảng Iceberg vẫn dùng Metastore của Hive làm *catalog* (danh mục bảng), nên hiểu Hive tổ chức tệp và phân vùng ra sao vẫn còn giá trị.

Các mục tiêu của bài:

1. Mô tả kiến trúc Hive: HiveServer2, Metastore, Tez và HDFS.
2. Tạo bảng ngoài trên tệp có sẵn với `LazySimpleSerDe` và `OpenCSVSerde`.

3. Tạo bảng phân vùng ORC và Parquet, nạp dữ liệu bằng phân vùng động.
4. Đo ảnh hưởng của định dạng tệp (CSV, ORC, Parquet) lên kích thước và thời gian truy vấn.
5. Kiểm chứng cắt tỉa phân vùng bằng `EXPLAIN DEPENDENCY` và `EXPLAIN EXTENDED`.
6. Tạo bảng bucketing, lấy mẫu theo bucket; phần mở rộng thử bảng ACID và thống kê.

Lý thuyết ở Mục 3.2, cách làm ở Mục 3.3, kết quả ở Mục 3.4, các câu hỏi checkpoint ở Mục 3.5.

3.2 Cơ sở lý thuyết

3.2.1 Kiến trúc Hive

Máy khách (Beeline, JDBC) gửi câu lệnh tới HiveServer2 ở cổng 10000. HiveServer2 phân tích cú pháp, tối ưu kế hoạch bằng Calcite (CBO) rồi sinh một DAG Tez [6]. Lúc lập kế hoạch, nó hỏi Metastore (Thrift, cổng 9083) về lược đồ, định dạng và vị trí của từng bảng, từng phân vùng; Metastore giữ các thông tin này trong PostgreSQL ở các bảng `TBLS`, `SDS`, `PARTITIONS`. Các task Tez sau đó đọc và ghi tệp trên HDFS. Ở môi trường thật, task Tez chạy trong container YARN hoặc LLAP. Trong bài này Tez chạy cục bộ ngay trong HiveServer2, nhưng câu SQL, kế hoạch và tệp sinh ra không khác gì.

3.2.2 Bảng ngoài, bảng quản lý và schema-on-read

Hive theo kiểu *schema-on-read* (áp lược đồ lúc đọc): lược đồ chỉ dùng để diễn giải tệp khi truy vấn. Bảng ngoài (`EXTERNAL TABLE`) chỉ là siêu dữ liệu trỏ tới một thư mục, nên `DROP TABLE` không xóa tệp. Hive 4 bắt bảng quản lý phải là bảng ACID; vì vậy một câu `CREATE TABLE ... STORED AS ORC` bình thường bị Metastore chuyển thành bảng ngoài trong warehouse kèm `external.table.purge=TRUE`, và khi đó `DROP TABLE` vẫn xóa tệp [2].

3.2.3 Định dạng cột ORC và Parquet

ORC [3] và Parquet [4] đặt các giá trị cùng cột liền nhau trong từng *stripe* (ORC) hay *row group* (Parquet). Nhờ vậy có thể dùng dictionary encoding, *run-length encoding* (RLE), mã hóa delta, bit-packing, rồi nén thêm bằng ZLIB hoặc Snappy. Mỗi khối có thống kê min/max để bộ đọc bỏ qua khối không thỏa điều kiện (*predicate pushdown*), và bộ đọc chỉ lấy những cột cần (*column pruning*) [5].

3.2.4 Phân vùng và cắt tỉa phân vùng

Partition chia dữ liệu thành các thư mục con dạng `cot=gia_tri/`. Khi điều kiện lọc nằm trên cột phân vùng, trình lập kế hoạch chỉ liệt kê những thư mục khớp (*partition pruning*). Giá trị phân vùng có thể tĩnh (ghi trong câu lệnh) hoặc động (lấy từ cột cuối của `SELECT`). Mặc định Hive ở chế độ `strict`, bắt phải có ít nhất một giá trị tĩnh [2].

3.2.5 Bucketing

Bucketing chia dữ liệu theo hash(c) mod N vào đúng N tệp, N cố định trong DDL. Kỹ thuật này hợp với cột có nhiều giá trị phân biệt, dùng được cho `TABLESAMPLE`, bucket map join và sort-merge-bucket join [2], [7].

3.2.6 Bảng giao dịch ACID

Bảng ACID không sửa tệp tại chỗ. Mỗi giao dịch ghi một thư mục `delta_*` (dòng mới) hoặc `delete_delta_*` (đánh dấu dòng bị xóa). Bộ đọc hợp nhất các delta lúc đọc, và *compaction* gộp chúng lại về sau [5].

3.3 Môi trường và phương pháp thực nghiệm

3.3.1 Môi trường

Cả cụm chạy bằng Docker Compose trên một máy (Bảng 3.1).

Bảng 3.1. Môi trường thực nghiệm.

Thành phần	Phiên bản / cấu hình
Hive	4.2.1 (image chính thức `apache/hive:4.2.1`), HiveServer2 + Metastore độc lập
Bộ máy thực thi	Tez, chế độ cục bộ bên trong HiveServer2
Cơ sở dữ liệu Metastore	PostgreSQL 18 (database `metastore`)
HDFS	3.4.3, 1 NameNode + 1 DataNode (hệ số nhân bản 1)
Cổng dịch vụ	HiveServer2 10000 (JDBC), 10002 (Web UI); Metastore 9083; NameNode UI 9870
Máy khách	Beeline qua `run_sql.sh` (in thời gian từng câu lệnh)

Năm container đều `Up`, NameNode và PostgreSQL ở trạng thái `healthy` (Hình 3.1). Giao diện web của HiveServer2 liệt kê các truy vấn đã chạy xong, tất cả trên `tez` (Hình 3.2).

```
L03 — docker compose ps + CSV lên HDFS

$ docker compose ps
NAME                  STATUS                   PORTS
l03_hive-datanode-1       Up 14 seconds
l03_hive-hiveserver2-1    Up 6 seconds             0.0.0.0:10000->10000/tcp, [::]:10000->10000/tcp, 0.0.0.0:10002->10002/tcp, [::]:10002->10002/tcp
l03_hive-metastore-1      Up 6 seconds             0.0.0.0:9083->9083/tcp, [::]:9083->9083/tcp
l03_hive-namenode-1       Up 14 seconds (healthy)  0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp
l03_hive-postgres-1       Up 14 seconds (healthy)  5432/tcp

$ parquet_to_csv + hdfs dfs -put
wrote 2,964,624 rows -> /datasets/data/taxi/csv/yellow_2024-01.csv (274.2 MB)
Found 1 items
-rw-r--r--   1 root supergroup    261.5 M 2026-10-30 10:30 /data/taxi_csv/yellow_2024-01.csv
Found 1 items
-rw-r--r--   1 root supergroup     12.0 K 2026-10-30 10:30 /data/zones/taxi_zone_lookup.csv
```

Hình 3.1. Khởi động cụm Hive (`docker compose ps`) và tải tệp CSV cùng bảng tra cứu zone lên HDFS.

Home | Local logs | Metrics Dump | Hive Configuration | Stack Trace | Llap Daemons | Configure logging

HiveServer2

Active Sessions

User Name	IP Address	Operation Count	Active Time (s)	Idle Time (s)

Total number of sessions: 0

Open Queries

User Name	Query	Execution Engine	State	Opened Timestamp	Opened (s)	Latency (s)	Drilldown Link

Total number of queries: 0

Last Max 25 Closed Queries

User Name	Query	Execution Engine	State	Opened (s)	Closed Timestamp	Latency (s)	Drilldown Link
hive	SELECT count(*) AS stats_count FROM trips_orc	tez	FINISHED	2	Fri Oct 30 10:36:42 GMT 2026	2	Drilldown
hive	EXPLAIN SELECT count(*) FROM trips_orc	tez	FINISHED	0	Fri Oct 30 10:36:40 GMT 2026	0	Drilldown
hive	USE taxi	tez	FINISHED	0	Fri Oct 30 10:36:40 GMT 2026	0	Drilldown
hive	DESCRIBE FORMATTED zone_notes	tez	FINISHED	0	Fri Oct 30 10:36:22 GMT 2026	0	Drilldown
hive	SELECT * FROM zone_notes ORDER BY locationid	tez	FINISHED	1	Fri Oct 30 10:36:22 GMT 2026	1	Drilldown
hive	DELETE FROM zone_notes WHERE locationid = 161	tez	FINISHED	3	Fri Oct 30 10:36:20 GMT 2026	3	Drilldown
hive	UPDATE zone_notes SET note = 'JFK Airport' WHERE locationid = 132	tez	FINISHED	4	Fri Oct 30 10:36:17 GMT 2026	4	Drilldown
hive	INSERT INTO zone_notes VALUES (132, 'JFK'), (138, 'LaGuardia'), (161, 'Midtown')	tez	FINISHED	3	Fri Oct 30 10:36:13 GMT 2026	3	Drilldown
hive	CREATE TABLE zone_notes (locationid INT, note STRING) STORED AS ORC TBLPROPERTIES ('transactional'='true')	tez	FINISHED	0	Fri Oct 30 10:36:09 GMT 2026	0	Drilldown
hive	DROP TABLE IF EXISTS zone_notes	tez	FINISHED	0	Fri Oct 30 10:36:09 GMT 2026	0	Drilldown
hive	SELECT count(*) AS stats_count FROM trips_orc	tez	FINISHED	1	Fri Oct 30 10:36:09 GMT 2026	1	Drilldown

Hình 3.2. Giao diện web HiveServer2 (cổng 10002): danh sách truy vấn đã hoàn tất, tất cả chạy trên Tez.

3.3.2 Dữ liệu

Tệp `yellow_tripdata_2024-01.parquet` được chuyển sang CSV không có dòng tiêu đề (Bảng 3.2). Đề bài gợi ý dùng mẫu 1 triệu dòng, nhưng chúng tôi dùng nguyên tháng, nên

các con số trong bài lớn hơn số tham khảo.

Bảng 3.2. Dữ liệu đầu vào.

Tệp trên HDFS	Kích thước	Số dòng
`/data/taxi_csv/yellow_2024-01.csv`	274,217,811 B (274.2 MB, 261.5 MiB)	2,964,624
`/data/zones/taxi_zone_lookup.csv`	12.0 K	265 zone + 1 dòng tiêu đề

3.3.3 Quy trình

Mỗi bước chạy bằng `run_sql.sh`, log gốc lưu trong các tệp `*.txt`:

1. Khởi động cụm, chuyển Parquet sang CSV, tải lên `/data/taxi_csv` và `/data/zones`.
2. `01_external_tables.sql`: tạo bảng ngoài `trips_csv` (`ROW FORMAT DELIMITED`) và `zones` (`OpenCSVSerde`), rồi truy vấn thẳng bảng `TBLS`, `SDS` của Metastore trong PostgreSQL.
3. `02_partitioned_tables.sql`: bật `hive.exec.dynamic.partition.mode=nonstrict`, tính `pickup_month` bằng `date_format(..., 'yyyy-MM')`, tạo và nạp `trips_orc` (ZLIB) và `trips_parquet` (Snappy). Đo kích thước bằng `hdfs dfs -du`.
4. `03_compare_formats.sql`, chạy hai lần với `hive.compute.query.using.stats=false`, `hive.query.results.cache.enabled=false` và `hive.fetch.task.conversion=none` để Hive phải quét thật. Chúng tôi lấy số của lần thứ hai, khi JVM và Tez đã khởi động xong.
5. `04_partition_pruning.sql`: thêm một `EXPLAIN DEPENDENCY` lọc theo `tpep_pickup_datetime` để đối chiếu.
6. `05_bucketing.sql`: bảng `CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS`.
7. Phần mở rộng: `ANALYZE TABLE` cho thống kê cột và bảng ACID `zone_notes`.

3.4 Kết quả thực nghiệm

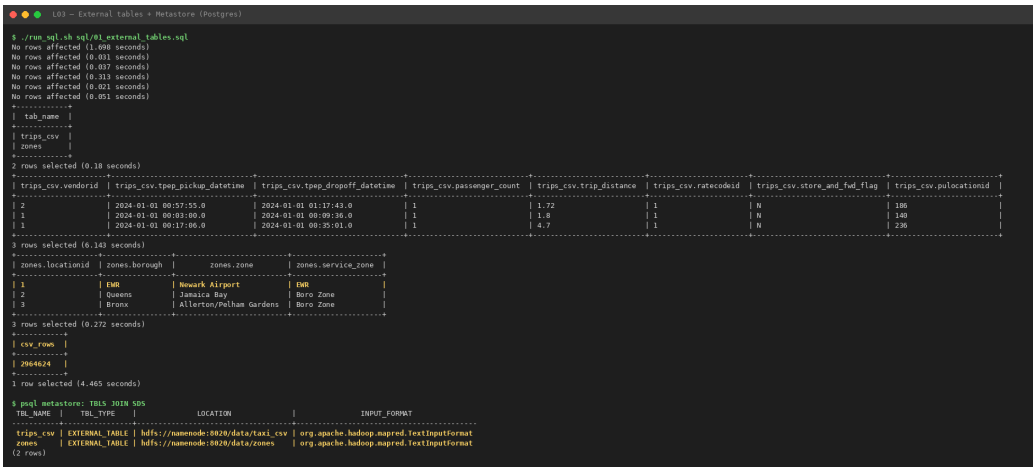
3.4.1 Bảng ngoài và nội dung Metastore

Cả hai bảng ngoài đọc được ngay. `SELECT * FROM zones LIMIT 3` trả về `1 | EWR | Newark Airport | EWR`, `2 | Queens | Jamaica Bay | Boro Zone` và `3 | Bronx | Allerton/Pelham Gardens | Boro Zone`, tức `OpenCSVSerde` đã bỏ dòng tiêu đề và dấu ngoặc kép. `count(*)` trên `trips_csv` ra `csv_rows = 2,964,624` sau 4.465 s, khớp số dòng của tệp.

Truy vấn `TBLS JOIN SDS` trong PostgreSQL (Bảng 3.3, Hình 3.3) chỉ thấy tên bảng, loại bảng, vị trí và lớp định dạng đầu vào. Không có dòng dữ liệu nào trong Metastore.

Bảng 3.3. Nội dung bảng `TBLS` và `SDS` trong Metastore sau bước tạo bảng ngoài.

TBL_NAME	TBL_TYPE	LOCATION	INPUT_FORMAT
trips_csv	EXTERNAL_TABLE	`hdfs://namenode:8020/data/taxi_csv`	`org.apache.hadoop.mapred.TextInputFormat`
zones	EXTERNAL_TABLE	`hdfs://namenode:8020/data/zones`	`org.apache.hadoop.mapred.TextInputFormat`



Hình 3.3. Kết quả truy vấn bảng ngoài và nội dung Metastore (PostgreSQL).

3.4.2 Bảng phân vùng ORC và Parquet

`INSERT OVERWRITE` ghi 2,964,624 dòng vào `trips_orc` trong 42.761 s và vào `trips_parquet` trong 45.134 s. Chúng tôi chờ 4 phân vùng như kết quả mẫu của đề, nhưng `SHOW PARTITIONS` trả về 5 (Bảng 3.4, Hình 3.4): dữ liệu cả tháng có thêm phân vùng `2024-02`.

Bảng 3.4. Phân vùng của `trips_orc` sau khi nạp bằng phân vùng động.

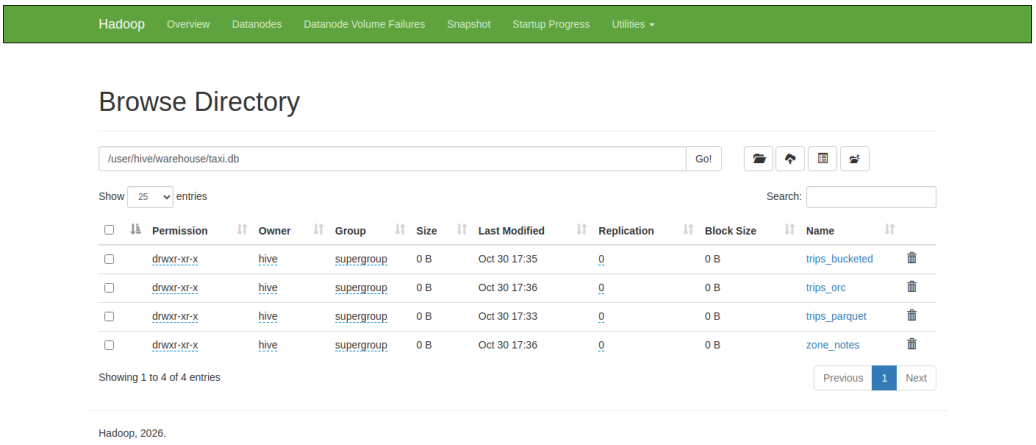
pickup_month	Số chuyến	Tệp ORC	Kích thước tệp
2002-12	2	`000007_0`	1,637 B
2009-01	3	`000004_0`	1,922 B
2023-12	10	`000001_0`	2,234 B
2024-01	2,964,606	`000005_0`	52,621,888 B
2024-02	3	`000005_0`	1,993 B

```
L03 — Dynamic partitioning ORC + Parquet

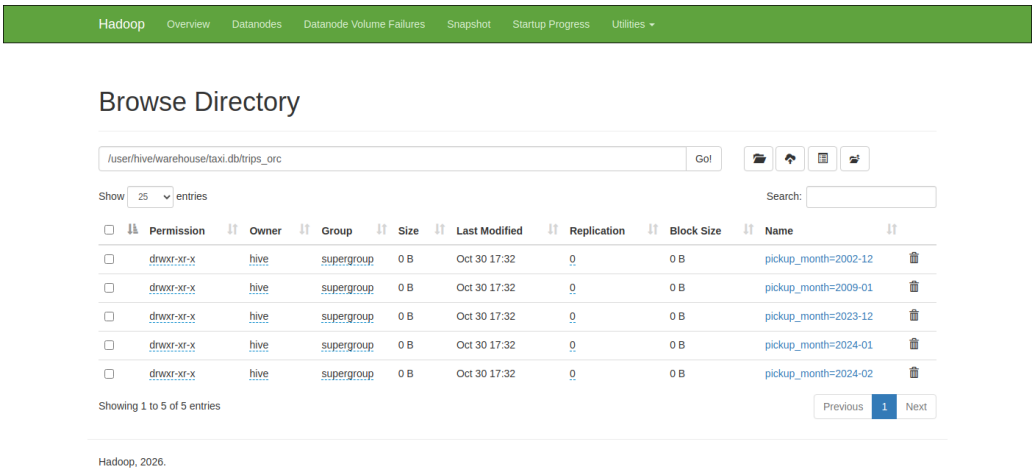
$ ./run_sql.sh sql/02_partitioned_tables.sql
No rows affected (0.06 seconds)
No rows affected (0.006 seconds)
No rows affected (0.004 seconds)
No rows affected (0.017 seconds)
No rows affected (0.07 seconds)
2,964,624 rows affected (42.761 seconds)
No rows affected (0.017 seconds)
No rows affected (0.052 seconds)
2,964,624 rows affected (45.134 seconds)
+-----------------------+
|       partition       |
+-----------------------+
| pickup_month=2002-12  |
| pickup_month=2009-01  |
| pickup_month=2023-12  |
| pickup_month=2024-01  |
| pickup_month=2024-02  |
+-----------------------+
5 rows selected (0.077 seconds)
+---------------+----------+
| pickup_month  |  trips   |
+---------------+----------+
| 2002-12       | 2        |
| 2009-01       | 3        |
| 2023-12       | 10       |
| 2024-01       | 2964606  |
| 2024-02       | 3        |
+---------------+----------+
5 rows selected (2.24 seconds)
```

Hình 3.4. `SHOW PARTITIONS` và số chuyến theo `pickup_month`.

Trên giao diện NameNode, thư mục `taxi.db` có bốn bảng `trips_bucketed`, `trips_orc`, `trips_parquet`, `zone_notes` (Hình 3.5), và `trips_orc` có năm thư mục con `pickup_month=...` (Hình 3.6). Bốn phân vùng nhỏ chỉ có một tệp khoảng 2 KB, một phiên bản thu nhỏ của vấn đề tệp nhỏ ở Mục 3.5.6.

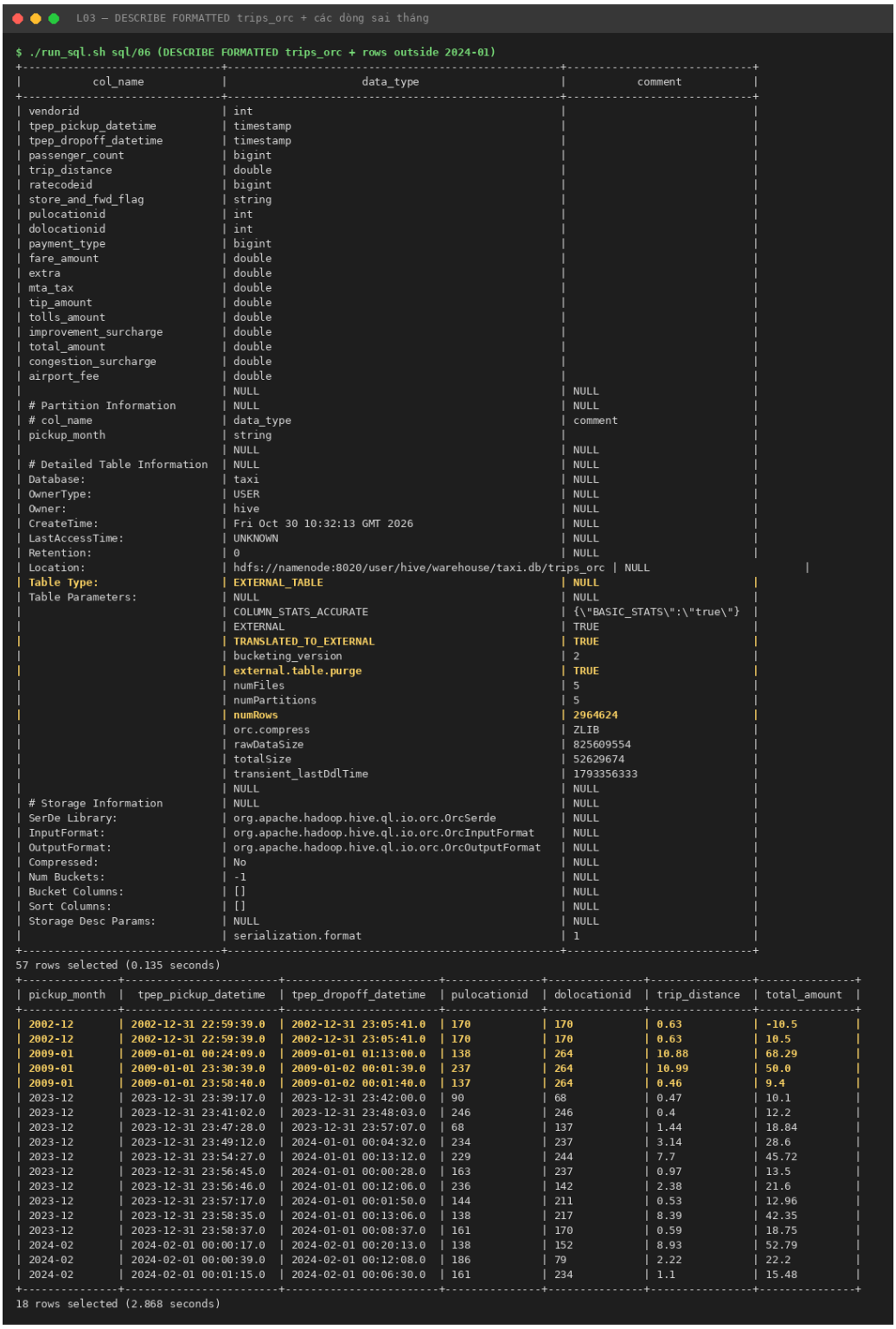


Hình 3.5. Thư mục warehouse `/user/hive/warehouse/taxi.db` trên giao diện NameNode.



Hình 3.6. Năm thư mục phân vùng của bảng `trips_orc` trên giao diện NameNode.

`DESCRIBE FORMATTED trips_orc` (Hình 3.7) ghi `Table Type: EXTERNAL_TABLE` cùng `TRANSLATED_TO_EXTERNAL=TRUE` và `external.table.purge=TRUE`, đúng cách Hive 4 xử lý bảng "managed" không phải ACID. Thống kê trong Metastore: `numFiles=5`, `numPartitions=5`, `numRows=2964624`, `rawDataSize=825609554`, `totalSize=52629674`. Hình này cũng liệt kê 18 dòng có `pickup_month` khác `2024-01`, bàn ở Mục 3.5.8.



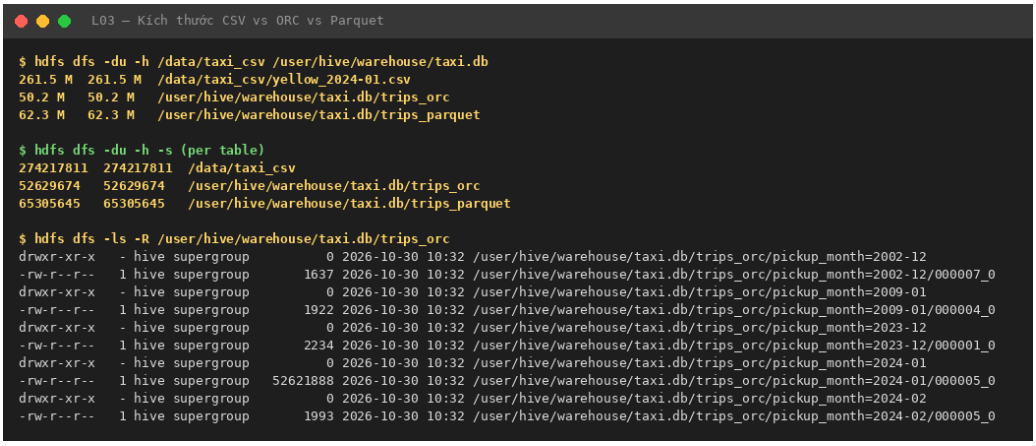
Hình 3.7. `DESCRIBE FORMATTED trips_orc` và 18 dòng nằm ngoài tháng 01/2024.

3.4.3 Kích thước lưu trữ theo định dạng

Kích thước đo bằng `hdfs dfs -du -s` nằm ở Bảng 3.5 và Hình 3.8. Tỉ lệ nén tính bằng $r = S_{\text{CSV}} / S_{\text{định dạng}}$, ví dụ $274{,}217{,}811 / 52{,}629{,}674 \approx 5.2$.

Bảng 3.5. Kích thước lưu trữ cùng dữ liệu tháng 01/2024 theo định dạng.

Định dạng	Kích thước (byte)	Kích thước (MiB)	Nhỏ hơn CSV
CSV (text)	274,217,811	261.5	1×
ORC + ZLIB	52,629,674	50.2	**5.2×**
Parquet + Snappy	65,305,645	62.3	4.2×
ORC bucketed (chỉ 5 cột)	25,271,072	24.1	10.9×



Hình 3.8. Kích thước CSV, ORC và Parquet trên HDFS.

ORC + ZLIB nhỏ hơn Parquet + Snappy khoảng 19 %. Điều này hợp lý vì ZLIB thiên về tỉ lệ nén, Snappy thiên về tốc độ. Bảng bucketed chỉ giữ 5 trên 19 cột nên con số 10.9× không đem so với ba dòng trên được.

3.4.4 So sánh thời gian truy vấn CSV, ORC và Parquet

Q1 tìm năm zone đón khách nhiều nhất trong tháng 01/2024, chỉ dùng `pulocationid`, `fare_amount` và điều kiện thời gian. Trên CSV, điều kiện là `tpep_pickup_datetime`; trên ORC và Parquet là `pickup_month = '2024-01'`. Q2 nối `trips_orc` với `zones`. Thời gian cả hai lần chạy ở Bảng 3.6, ảnh lần chạy thứ hai ở Hình 3.9.

Bảng 3.6. Thời gian truy vấn theo định dạng (số liệu dùng để so sánh là lần chạy thứ hai).

Truy vấn	Lần chạy	CSV	ORC	Parquet
Q1 top 5 zone đón khách	1	9.222 s	1.987 s	1.767 s
Q1 top 5 zone đón khách	2	8.528 s	1.885 s	1.618 s
Q2 nối `zones` theo borough	1		2.939 s	
Q2 nối `zones` theo borough	2		2.655 s	

```
L03 — Q1/Q2 trên CSV vs ORC vs Parquet (lần chạy 2)

$ ./run_sql.sh sql/03_compare_formats.sql   # RUN 2 (CSV, ORC, Parquet, join)
+---------------+---------+-----------+
| pulocationid  |  trips  | avg_fare  |
+---------------+---------+-----------+
| 132           | 145240  | 59.4      |
| 161           | 143469  | 15.21     |
| 237           | 142707  | 12.18     |
| 236           | 136464  | 12.71     |
| 162           | 106717  | 14.79     |
+---------------+---------+-----------+
5 rows selected (8.528 seconds)
+---------------+---------+-----------+
| pulocationid  |  trips  | avg_fare  |
+---------------+---------+-----------+
| 132           | 145240  | 59.4      |
| 161           | 143469  | 15.21     |
| 237           | 142707  | 12.18     |
| 236           | 136464  | 12.71     |
| 162           | 106717  | 14.79     |
+---------------+---------+-----------+
5 rows selected (1.885 seconds)
+---------------+---------+-----------+
| pulocationid  |  trips  | avg_fare  |
+---------------+---------+-----------+
| 132           | 145240  | 59.4      |
| 161           | 143469  | 15.21     |
| 237           | 142707  | 12.18     |
| 236           | 136464  | 12.71     |
| 162           | 106717  | 14.79     |
+---------------+---------+-----------+
5 rows selected (1.618 seconds)
+----------------+----------+-----------+
|   z.borough    |  trips   |  revenue  |
+----------------+----------+-----------+
| Manhattan      | 2646933  | 59247250  |
| Queens         | 273125   | 18639629  |
| Brooklyn       | 25258    | 828808    |
| Unknown        | 10360    | 304152    |
| Bronx          | 6905     | 246496    |
| N/A            | 1658     | 154901    |
| EWR            | 295      | 30738     |
| Staten Island  | 72       | 3966      |
+----------------+----------+-----------+
8 rows selected (2.655 seconds)
```

Hình 3.9. Lần chạy thứ hai của `03_compare_formats.sql`.

Ở lần thứ hai, ORC nhanh hơn CSV khoảng $8.528/1.885 \approx 4.5$ lần, Parquet khoảng $8.528/1.618 \approx 5.3$ lần. Ba nguồn cho cùng một kết quả (Bảng 3.7), nên việc đổi định dạng không làm sai dữ liệu. Zone `132` (sân bay JFK) đứng đầu, và giá cước trung bình ở đây cao gấp mấy lần các zone trong nội đô.

Bảng 3.7. Kết quả Q1 (giống nhau trên CSV, ORC và Parquet).

pulocationid	Số chuyến	Giá cước trung bình
132	145,240	59.40
161	143,469	15.21
237	142,707	12.18
236	136,464	12.71
162	106,717	14.79

Theo Q2 (Bảng 3.8), Manhattan chiếm 89.3 % số chuyến. Cộng số chuyến các borough lại ra đúng 2,964,606, bằng số dòng của phân vùng `2024-01`.

Bảng 3.8. Kết quả Q2: số chuyến và doanh thu theo borough.

Borough	Số chuyến	Doanh thu (`total_amount`)
Manhattan	2,646,933	59,247,250
Queens	273,125	18,639,629
Brooklyn	25,258	828,808
Unknown	10,360	304,152
Bronx	6,905	246,496
N/A	1,658	154,901
EWR	295	30,738
Staten Island	72	3,966

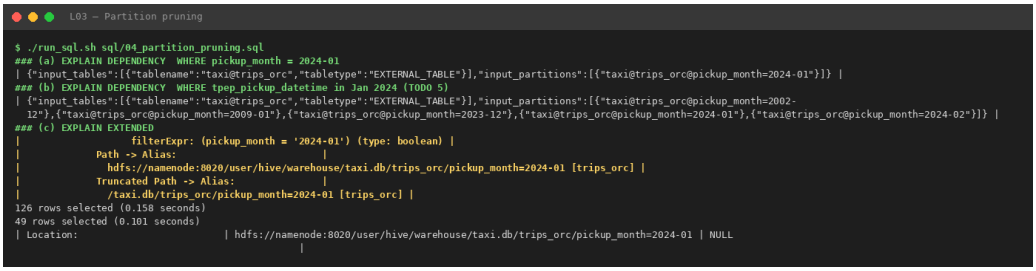
3.4.5 Cắt tỉa phân vùng

Ba câu `EXPLAIN` được đặt cạnh nhau ở Bảng 3.9 và Hình 3.10.

Bảng 3.9. Kết quả kiểm chứng cắt tỉa phân vùng.

Thí nghiệm	Điều kiện lọc	Kết quả
(a) `EXPLAIN DEPENDENCY`	`pickup_month = '2024-01'`	`input_partitions` có **1** phân vùng (`pickup_month=2024-01`)
(b) `EXPLAIN DEPENDENCY`	`tpep_pickup_datetime` trong tháng 1	`input_partitions` có cả 5 phân vùng

Thí nghiệm	Điều kiện lọc	Kết quả
(c) `EXPLAIN EXTENDED`	`pickup_month = '2024-01'`	`filterExpr: (pickup_month = '2024-01')`; `Path -> Alias` chỉ có `.../trips_orc/pickup_month=2024-01`



Hình 3.10. Kết quả `EXPLAIN DEPENDENCY` và `EXPLAIN EXTENDED`.

Trong kế hoạch (c), `TableScan` ước lượng `Num rows: 2964606`, `Data size: 849321344`, `Basic stats: COMPLETE`, đúng bằng số dòng của riêng phân vùng `2024-01`. Cả `Map 1` và `Reducer 2` đều chạy `Execution mode: vectorized`. Ở (b), Hive không biết `tpep_pickup_datetime` và `pickup_month` có liên quan với nhau, vì trong lược đồ đó là hai cột độc lập. Nó phải liệt kê mọi phân vùng rồi lọc từng dòng.

3.4.6 Bucketing

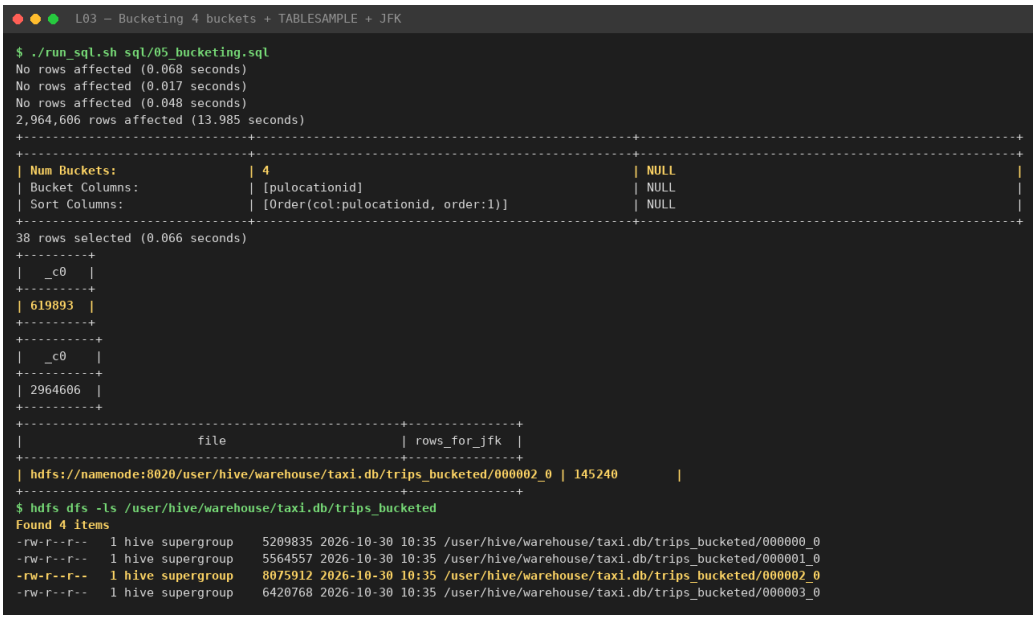
Bảng `trips_bucketed` (5 cột) nạp 2,964,606 dòng từ phân vùng `2024-01` trong 13.985 s. `DESCRIBE FORMATTED` ghi `Num Buckets: 4`, `Bucket Columns: [pulocationid]` và `Sort Columns: [Order(col:pulocationid, order:1)]`. Thư mục bảng có đúng bốn tệp (Bảng 3.10, Hình 3.12).

Bảng 3.10. Các tệp bucket của `trips_bucketed`.

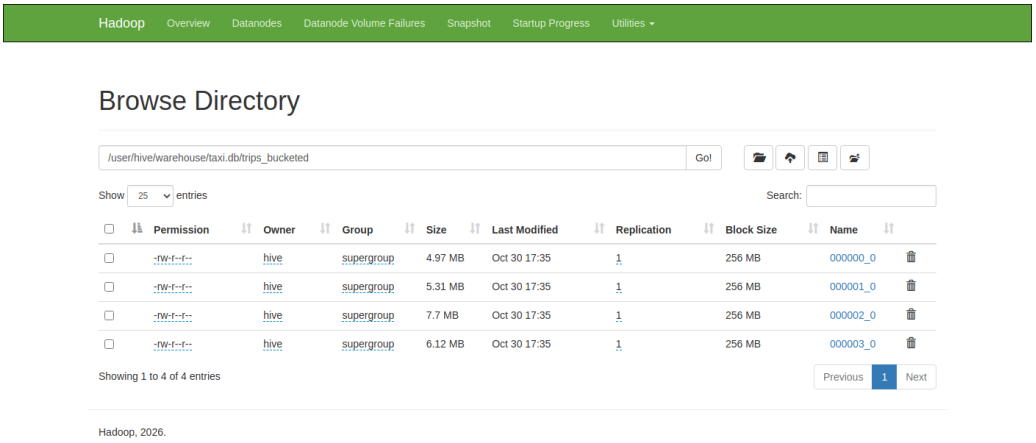
Tệp	Kích thước	Ghi chú
`000000_0`	5,209,835 B	
`000001_0`	5,564,557 B	
`000002_0`	8,075,912 B	chứa toàn bộ 145,240 dòng JFK (zone 132)
`000003_0`	6,420,768 B	
Tổng	25,271,072 B	`numFiles=4`, `numRows=2964606`

`TABLESAMPLE(BUCKET 1 OUT OF 4 ON pulocationid)` trả về 619,893 dòng sau 1.606 s, tức khoảng 20.9 % chứ không phải 25 %. Lý do là hàm băm chia theo giá trị khóa chứ không theo

số dòng; bucket nào nhận phải các zone đông khách sẽ to hơn. Truy vấn `INPUT__FILE__NAME` cho zone 132 chỉ trả về một tệp `000002_0` (Hình 3.11), nên tệp này cũng là tệp lớn nhất.



Hình 3.11. Kết quả bucketing: mô tả bảng, lấy mẫu một bucket và vị trí các dòng JFK.



Hình 3.12. Bốn tệp bucket của `trips_bucketed` trên giao diện NameNode.

3.4.7 Phần mở rộng: thống kê và bảng ACID

Trước khi có thống kê, `SELECT count(*)` trên `trips_orc` mất 2.132 s. Câu `ANALYZE TABLE trips_orc PARTITION (pickup_month) COMPUTE STATISTICS FOR COLUMNS` mất 18.354 s. Chúng tôi chờ `count(*)` sau đó sẽ trả lời ngay từ Metastore khi bật `hive.compute.query.`

`using.stats=true`, nhưng không phải vậy: kết quả vẫn là 2,964,624 và vẫn chạy DAG Tez, mất 1.569 s lần đầu và 2.592 s khi đo lại. `EXPLAIN` lần đo lại vẫn có `TableScan [TS_0] (rows= 2964624 width=290)` với `Tbl:COMPLETE, Col:COMPLETE`. Thống kê đã đủ mà không được dùng; nguyên nhân ở Mục 3.5.9.

Bảng `zone_notes (locationid INT, note STRING) STORED AS ORC TBLPROPERTIES ('transactional'='true')` mới là `MANAGED_TABLE` thật. Các thao tác và thư mục sinh ra trên HDFS nằm ở Bảng 3.11, Hình 3.13 và Hình 3.14. Cuối cùng còn 2 dòng: `132 \| JFK Airport` và `138 \| LaGuardia`.

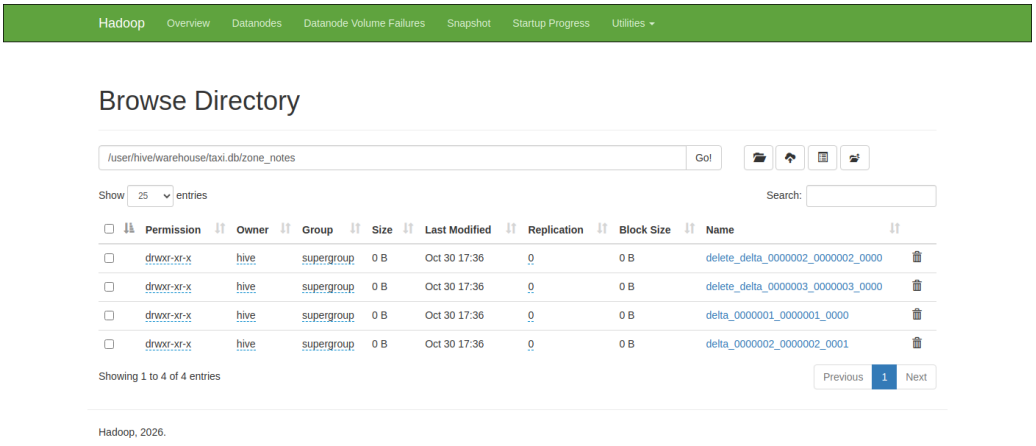
Bảng 3.11. Các giao dịch trên bảng ACID `zone_notes` và thư mục delta tương ứng.

Giao dịch	Câu lệnh	Thời gian	Thư mục sinh ra
1	`INSERT` (132 JFK), (138 LaGuardia), (161 Midtown)	3.342 s (3 dòng)	`delta_0000001_ 0000001_0000`
2	`UPDATE ... SET note='JFK Airport' WHERE locationid=132`	4.34 s (1 dòng)	`delete_delta_ 0000002_0000002_ 0000` + `delta_ 0000002_0000002_ 0001`
3	`DELETE ... WHERE locationid=161`	3.121 s (1 dòng)	`delete_delta_ 0000003_0000003_ 0000`

```
L03 — Stretch: statistics + ACID (delta / delete_delta)

$ stretch: statistics + ACID
+-------------+
| scan_count  |
+-------------+
| 2964624     |
+-------------+
1 row selected (2.132 seconds)
+--------------+
| stats_count  |
+--------------+
| 2964624      |
+--------------+
1 row selected (1.569 seconds)
3 rows affected (3.342 seconds)
1 row affected (4.34 seconds)
1 row affected (3.121 seconds)
+-----------------------+------------------+
| zone_notes.locationid | zone_notes.note  |
+-----------------------+------------------+
| 132                   | JFK Airport      |
| 138                   | LaGuardia        |
+-----------------------+------------------+
2 rows selected (1.578 seconds)
| Table Type:                | MANAGED_TABLE                                  | NULL                |
|                            | transactional                                  | true                |
|                            | transactional_properties                       | default             |
$ hdfs dfs -ls -R .../zone_notes
  zone_notes/delete_delta_0000002_0000002_0000
  zone_notes/delete_delta_0000002_0000002_0000/bucket_00000_0
  zone_notes/delete_delta_0000003_0000003_0000
  zone_notes/delete_delta_0000003_0000003_0000/bucket_00000_0
  zone_notes/delta_0000001_0000001_0000
  zone_notes/delta_0000001_0000001_0000/bucket_00000_0
  zone_notes/delta_0000002_0000002_0001
  zone_notes/delta_0000002_0000002_0001/bucket_00000_0
```

Hình 3.13. Phần mở rộng: thống kê cột và các thao tác ACID.



Hình 3.14. Các thư mục `delta_*` và `delete_delta_*` của bảng `zone_notes` trên giao diện NameNode.

Không tệp nào bị sửa tại chỗ. `UPDATE` thực chất là một lần xóa (`delete_delta`) cộng một lần ghi mới (`delta`); mỗi thư mục có một tệp `bucket_00000_0` khoảng 700–800 B và tệp `_orc_acid_version`. Bộ đọc hợp nhất các delta khi truy vấn, sau này compaction sẽ gộp chúng thành thư mục `base_*`.

3.5 Thảo luận

3.5.1 Metastore giữ gì, HDFS giữ gì, và `DROP TABLE` xóa những gì?

Metastore (các bảng `TBLS`, `SDS`, `PARTITIONS`… trong PostgreSQL) giữ toàn bộ siêu dữ liệu: database, tên bảng, tên và kiểu cột, SerDe, lớp định dạng vào/ra, loại bảng, `LOCATION` của bảng và từng phân vùng, giá trị phân vùng, thống kê (`numRows`, `totalSize`…) và trạng thái giao dịch ACID. HDFS chỉ chứa tệp dữ liệu. Ở Bảng 3.3, hai bảng ngoài chỉ là hai dòng siêu dữ liệu trỏ tới `/data/taxi_csv` và `/data/zones`.

`DROP TABLE trips_csv` (bảng ngoài) chỉ xóa siêu dữ liệu, tệp trong `/data/taxi_csv` vẫn còn. `DROP TABLE trips_orc` thì khác: bảng này là bảng ngoài được chuyển đổi với `external.table.purge=TRUE` (Hình 3.7), nên lệnh xóa cả siêu dữ liệu lẫn thư mục `/user/hive/warehouse/taxi.db/trips_orc`, giống một bảng quản lý.

3.5.2 Vì sao `zones` cần `OpenCSVSerde` còn `trips_csv` chỉ cần `ROW FORMAT DELIMITED`?

`ROW FORMAT DELIMITED` (`LazySimpleSerDe`) chỉ cắt trường theo dấu phân cách. Nó không bỏ dấu ngoặc kép và không hiểu dấu phẩy nằm trong ngoặc kép. Tệp trips sinh ra từ Parquet, không tiêu đề, không ngoặc kép, nên vậy là đủ. Tệp zones có dòng tiêu đề và giá trị như `"Allerton/Pelham Gardens"`, nên cần `OpenCSVSerde` cùng `skip.header.line.count=1`. Đổi lại, `OpenCSVSerde` coi mọi cột là `STRING` (Q2 phải viết `CAST(z.locationid AS INT)`), phân tích chậm hơn và không xử lý được giá trị NULL đặc biệt.

3.5.3 Vì sao ORC và Parquet nhỏ hơn CSV 4–5 lần?

Số đo được ở Bảng 3.5 có hai nguyên nhân. Thứ nhất là lưu theo cột: giá trị cùng cột, cùng kiểu nằm cạnh nhau nên dictionary, RLE, delta và bit-packing nén rất tốt, nhất là với các cột như `vendorid`, `payment_type`, `ratecodeid` chỉ có vài giá trị. Thứ hai là kiểu nhị phân cộng nén khối: một timestamp chiếm 8 byte thay vì chuỗi 19 ký tự, số không còn là chuỗi chữ số kèm dấu phân cách, rồi cả khối lại được nén bằng ZLIB hoặc Snappy. ORC và Parquet chênh nhau chủ yếu do codec. Một chi tiết thú vị: `rawDataSize` so với `totalSize` của `trips_orc` (Mục 3.4.2) cho thấy dữ liệu khi giải nén trong bộ nhớ lớn gấp khoảng 15.7 lần trên đĩa.

3.5.4 Vì sao Q1 chạy nhanh hơn trên ORC và Parquet?

Thời gian Q1 giảm 4.5 đến 5.3 lần (Bảng 3.6), và chúng tôi thấy có bốn cơ chế cùng góp phần. Trước hết là column pruning: Q1 chỉ cần 2–3 trong 19 cột, ORC và Parquet chỉ đọc đúng các *column chunk* đó, còn CSV phải đọc và phân tích cả 274 MB từng dòng một. Thứ hai là partition pruning: điều kiện lọc trên `pickup_month` khiến Hive chỉ mở thư mục `pickup_month=2024-01` (Bảng 3.9), trong khi bảng CSV không phân vùng nên phải quét hết rồi lọc theo timestamp. Thứ

ba, min/max và chỉ mục dòng của từng stripe/row group cho phép bỏ qua những khối không thể thỏa điều kiện. Cuối cùng, bộ đọc ORC/Parquet xử lý theo lô 1,024 dòng (`Execution mode: vectorized` ở Mục 3.4.5) và không mất công đổi văn bản thành số. Thí nghiệm của chúng tôi không tách riêng được phần đóng góp của từng cơ chế.

3.5.5 Phân vùng tĩnh khác phân vùng động ra sao, và vì sao mặc định là `strict`?

Phân vùng tĩnh ghi giá trị ngay trong câu lệnh, ví dụ `PARTITION (pickup_month='2024-01')`, và mọi dòng đều vào phân vùng đó. Phân vùng động lấy giá trị từ cột cuối của `SELECT`. Chế độ `strict` là mặc định để một câu truy vấn viết sai không lỡ tay tạo ra hàng nghìn phân vùng, hàng nghìn tệp và đối tượng trong Metastore. Bài này gặp đúng chuyện đó ở quy mô nhỏ: chỉ 18 dòng bất thường đã đẻ ra bốn phân vùng "lạ" (Bảng 3.4).

3.5.6 Phân vùng theo `pulocationid` cộng ngày, hoặc theo `tpep_pickup_datetime`, thì sao?

Theo `pulocationid` cộng ngày sẽ có khoảng $265 \times 31 \approx 8{,}000$ phân vùng mỗi tháng, mỗi phân vùng một tệp từ vài KB đến vài trăm KB. Đó là vấn đề tệp nhỏ (*small files problem*) [7], [8]. NameNode phải giữ hàng chục nghìn đối tượng trong heap. Metastore phải lưu hàng nghìn dòng `PARTITIONS`/`SDS`, và việc lập kế hoạch chậm đi vì phải liệt kê và tải siêu dữ liệu của từng phân vùng. Mỗi tệp nhỏ lại thành một split/task riêng, nên chi phí khởi tạo lấn át thời gian xử lý thật. Bốn phân vùng lẻ của `trips_orc`, mỗi cái một tệp 1,637–2,234 B, là dấu hiệu sớm của chuyện này. Phân vùng theo timestamp gốc còn tệ hơn: gần như mỗi giây một phân vùng, tức hàng triệu thư mục chứa tệp vài trăm byte. Theo chúng tôi, nên phân vùng theo cột ít giá trị mà truy vấn hay lọc, sao cho mỗi phân vùng có từ vài trăm MB đến khoảng 1 GB.

3.5.7 Khi nào dùng partitioning, khi nào dùng bucketing, và vì sao số tệp bucket cố định?

Partitioning tạo thư mục theo giá trị cột, hợp với cột ít giá trị mà truy vấn hay lọc (ngày, tháng, quốc gia), để bỏ qua nguyên thư mục khi đọc. Bucketing chia theo $\text{hash}(c) \bmod N$ vào N tệp, hợp với cột nhiều giá trị như mã định danh hay `pulocationid`. Nó cho phép lấy mẫu bằng `TABLESAMPLE`, bucket map join, sort-merge-bucket join, và tránh shuffle khi hai bảng được bucket giống nhau trên khóa join. Số tệp cố định vì N đã chốt trong DDL (`INTO 4 BUCKETS`): mỗi dòng luôn rơi vào bucket hash mod 4, dữ liệu nhiều hay ít cũng vậy. Kết quả ở Bảng 3.10 khớp với điều này. Hai kỹ thuật dùng chung được, ví dụ phân vùng theo tháng rồi trong mỗi phân vùng chia bucket theo zone.

3.5.8 Những dòng nào rơi vào phân vùng lạ, và pipeline thật nên xử lý ra sao?

18 dòng ngoài tháng 01/2024 (Hình 3.7) được tóm lại ở Bảng 3.12.

Bảng 3.12. Các dòng có thời gian đón khách nằm ngoài tháng 01/2024.

Phân vùng	Số dòng	Mô tả	Đánh giá
2002-12	2	2002-12-31 22:59:39, zone 170, một cặp `total_amount` +10.5/−10.5	Lỗi đồng hồ; một chuyến bị hoàn tiền
2009-01	3	Ngày 2009-01-01, cả ba trả khách tại zone 264	Lỗi đồng hồ thiết bị
2023-12	10	2023-12-31 23:39–23:58	Chuyến thật bắt đầu trước giao thừa
2024-02	3	2024-02-01 00:00:17–00:01:15	Chuyến thật bắt đầu ngay sau nửa đêm

Hai nhóm này cần xử lý khác nhau. Dòng năm 2002 và 2009 là lỗi đồng hồ, còn dòng 2023-12 và 2024-02 là chuyến thật ở biên tháng. Trong một pipeline thật, chúng tôi sẽ so thời gian sự kiện với tháng của tệp nguồn, đẩy dòng ngoài khoảng hợp lệ vào một bảng quarantine kèm lý do và cảnh báo khi số dòng này tăng. Dòng biên tháng hợp lệ thì ghi vào đúng phân vùng của nó, hoặc giữ theo tháng của tệp nguồn, tùy quy ước của nhóm. Điều quan trọng là không để phân vùng động lặng lẽ tạo phân vùng rác. Iceberg giải quyết một phần chuyện này bằng phân vùng ẩn và cho phép đổi cách phân vùng về sau.

3.5.9 Vì sao `count(*)` không nhanh hơn sau khi đã có thống kê?

Kết quả này làm chúng tôi bất ngờ lúc chạy. Thống kê đã `COMPLETE` nhưng `count(*)` vẫn quét bằng Tez, và thời gian xấp xỉ lần quét trước khi có thống kê (Mục 3.4.7). Lời giải thích hợp lý nhất là `trips_orc` thực chất là bảng EXTERNAL (bảng được chuyển đổi), và Hive không tin thống kê của bảng ngoài để trả lời thay cho việc quét, vì tệp có thể bị sửa từ bên ngoài Hive. Với bảng quản lý ACID, câu trả lời sẽ lấy thẳng từ Metastore và kế hoạch chỉ còn `Fetch`. Chúng tôi chưa kiểm chứng điều này trên `zone_notes`, nên đây vẫn là giả thuyết.

3.5.10 Hạn chế của thực nghiệm

Mọi thứ chạy trên một máy, Tez ở chế độ cục bộ, HDFS chỉ một DataNode với hệ số nhân bản 1, nên kết quả không phản ánh tính song song hay chi phí mạng của cụm thật. Mỗi truy vấn chỉ đo

hai lần, chưa đủ để nói về phương sai; riêng lần `count(*)` đo lại sau thống kê đã chậm hơn lần trước gần 1 s. Dữ liệu chỉ một tháng (khoảng 274 MB), nên chi phí khởi động JVM/Tez chiếm phần đáng kể trong các thời gian 1.6–2.7 s.

3.6 Kết luận

Metastore chỉ là một cơ sở dữ liệu quan hệ chứa lược đồ và vị trí, dữ liệu thật nằm trên HDFS. Chuyển CSV sang định dạng cột giảm kích thước 4–5 lần (Bảng 3.5) và giảm thời gian Q1 tương ứng (Bảng 3.6), nhờ cắt tỉa cột, cắt tỉa phân vùng, predicate pushdown và thực thi vector hóa. Hive chỉ cắt tỉa phân vùng khi điều kiện lọc nằm đúng trên cột phân vùng; lọc trên cột thời gian gốc thì vẫn đọc cả 5 phân vùng. Bucketing cho số tệp cố định và gom các dòng cùng khóa vào một tệp. Bảng ACID ghi thay đổi thành thư mục delta chứ không sửa tệp.

Từ bài này, chúng tôi rút ra vài điều cho thực tế. Nên lưu dữ liệu phân tích ở ORC hoặc Parquet, phân vùng theo ngày hoặc tháng và viết truy vấn lọc thẳng trên cột phân vùng. Tránh phân vùng theo cột nhiều giá trị, và kiểm tra thời gian sự kiện trước khi ghi bằng phân vùng động. Nếu có thêm thời gian, chúng tôi muốn nạp tháng thứ hai bằng phân vùng tĩnh, so sánh khi bật và tắt vector hóa, thử schema-on-read bằng cách đổi `LOCATION`, và chuyển sang Iceberg để so với phân vùng kiểu thư mục.

Tài liệu tham khảo

[1] A. Thusoo, J. S. Sarma, N. Jain, Z. Shao, P. Chakka, S. Anthony, H. Liu, P. Wyckoff, R. Murthy, “Hive: A Warehousing Solution Over a Map-Reduce Framework,” *Proc. VLDB Endowment*, vol. 2, no. 2, pp. 1626–1629, 2009.

[2] Apache Software Foundation, “Apache Hive Language Manual: DDL,” Apache Hive Wiki. https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

[3] Apache Software Foundation, “ORC Specification v1,” Apache ORC. https://orc.apache.org/specificatio

[4] Apache Software Foundation, “Apache Parquet Documentation,” Apache Parquet. https://parquet.apache.org/docs/

[5] Y. Huai, A. Chauhan, A. Gates, G. Hagleitner, E. N. Hanson, O. O’Malley, J. Pandey, Y. Yuan, R. Lee, X. Zhang, “Major Technical Advancements in Apache Hive,” in *Proc. ACM SIGMOD*, 2014, pp. 1235–1246.

[6] B. Saha, H. Shah, S. Seth, G. Vijayaraghavan, A. Murthy, C. Curino, “Apache Tez: A Unifying Framework for Modeling and Building Data Processing Applications,” in *Proc. ACM SIGMOD*, 2015, pp. 1357–1369.

[7] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O’Reilly Media, 2015.

[8] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

Phụ lục A: Danh mục tệp nộp kèm

Danh sách tệp nộp kèm ở Bảng 3.13.

Bảng 3.13. Danh mục tệp nộp kèm trong `submission/L03/`.

Tệp	Nội dung
`01_external_tables.sql`	Bảng ngoài `trips_csv`, `zones`
`02_partitioned_tables.sql`	Bảng phân vùng `trips_orc`, `trips_parquet`
`03_compare_formats.sql`	Q1 trên CSV/ORC/Parquet và Q2 nối `zones`
`04_partition_pruning.sql`	`EXPLAIN DEPENDENCY`/`EXPLAIN EXTENDED`
`05_bucketing.sql`	Bảng `trips_bucketed` 4 bucket
`01_stack_and_upload.txt`	Trạng thái container, chuyển đổi và tải CSV lên HDFS
`02_external_tables.txt`	Log chạy `01_external_tables.sql`
`03_metastore_tbls.txt`	Truy vấn `TBLS JOIN SDS` trong PostgreSQL
`04_partitioned_tables.txt`	Log nạp ORC/Parquet và `SHOW PARTITIONS`
`05_sizes_hdfs.txt`	`hdfs dfs -du` và `hdfs dfs -ls -R` của `trips_orc`
`06_describe_and_bad_rows.txt`	`DESCRIBE FORMATTED trips_orc` và 18 dòng ngoài tháng
`07_compare_formats_run1.txt`, `08_compare_formats_run2.txt`	Hai lần chạy so sánh định dạng
`09_partition_pruning.txt`	Log `EXPLAIN` cắt tỉa phân vùng
`10_bucketing.txt`	Log bucketing và danh sách tệp bucket
`11_stretch_stats_acid.txt`, `12_stretch_stats_answer.txt`	Phần mở rộng thống kê và ACID
`clean.sh`	Lọc log, chỉ giữ bảng kết quả, số dòng, thời gian và lỗi
`img/`	Ảnh chụp terminal và giao diện web

Phụ lục B: Trích đoạn mã nguồn

Trong `01_external_tables.sql`, bảng `zones` dùng `OpenCSVSerde` và bỏ dòng tiêu đề:

```
CREATE EXTERNAL TABLE zones (
  locationid STRING, borough STRING, zone STRING, service_zone STRING
```

```
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
STORED AS TEXTFILE
LOCATION '/data/zones'
TBLPROPERTIES ('skip.header.line.count' = '1');
```

Trong `02_partitioned_tables.sql`, chúng tôi bật phân vùng động, tính `pickup_month` ở cột cuối của `SELECT` và khai báo bảng Parquet nén Snappy (câu nạp `trips_parquet` giống hệt câu nạp `trips_orc`):

```
SET hive.exec.dynamic.partition.mode = nonstrict;

INSERT OVERWRITE TABLE trips_orc PARTITION (pickup_month)
SELECT t.*,
       date_format(t.tpep_pickup_datetime, 'yyyy-MM') AS pickup_month
FROM trips_csv t;

CREATE TABLE trips_parquet ( ... cùng 19 cột như trips_orc ... )
PARTITIONED BY (pickup_month STRING)
STORED AS PARQUET
TBLPROPERTIES ('parquet.compression' = 'SNAPPY');
```

Trong `04_partition_pruning.sql`, câu `EXPLAIN` lọc theo cột thời gian gốc để so với cách lọc theo cột phân vùng:

```
EXPLAIN DEPENDENCY
SELECT count(*) FROM trips_orc
WHERE tpep_pickup_datetime >= '2024-01-01' AND tpep_pickup_datetime <
 ↪  '2024-02-01';
```

Trong `05_bucketing.sql`, bảng bucketing chia theo `pulocationid` vào 4 bucket:

```
CREATE TABLE trips_bucketed (
  tpep_pickup_datetime TIMESTAMP, pulocationid INT, dolocationid INT,
  fare_amount DOUBLE, total_amount DOUBLE
)
CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS
STORED AS ORC;
```

Chương 4

Nạp dữ liệu từ cơ sở dữ liệu OLTP vào HDFS: Sqoop và Spark JDBC, nạp toàn phần và nạp gia tăng

Tóm tắt

Bài thực hành này nạp dữ liệu theo lô từ PostgreSQL (bộ dữ liệu Olist, 6 bảng, 5,000 đơn hàng) vào HDFS bằng hai công cụ: Apache Sqoop 1.4.7 chạy MapReduce trên YARN và Spark 4.1.3 qua nguồn dữ liệu JDBC. Chúng tôi bật `log_statement=all` trên PostgreSQL để đọc trực tiếp câu SQL mà mỗi công cụ gửi xuống. Khi chia theo `order_seq`, Sqoop tạo 4 khoảng đều nhau (1,250 dòng mỗi map); chia theo `customer_zip_code_prefix` thì lệch nặng (2,450 so với 427 dòng). Spark JDBC đọc song song 4 partition, thời gian `count()` giảm từ 1.17 s xuống 0.22 s, và đầu ra Parquet phân vùng theo tháng chiếm 680.4 KB so với 1.0 MB văn bản của Sqoop. Ở chế độ gia tăng, `append` chỉ bắt được 3 dòng INSERT; `lastmodified` và watermark trên `updated_at` bắt được 5 dòng (3 INSERT, 2 UPDATE), lần chạy không có thay đổi đọc 0 dòng. Spark JDBC thay được Sqoop, nhưng mọi cách nạp dựa trên truy vấn đều bỏ sót DELETE và các phiên bản trung gian, nên vẫn cần CDC.

Từ khóa: Sqoop, Spark JDBC, PostgreSQL, HDFS, Parquet, split-by, partitionColumn, incremental load, watermark, CDC.

4.1 Giới thiệu

Dữ liệu nghiệp vụ phần lớn nằm trong các cơ sở dữ liệu giao dịch (OLTP). Muốn phân tích trên nền tảng dữ liệu lớn thì phải định kỳ chép dữ liệu đó sang HDFS. Có hai yêu cầu đi kèm: đọc song song đủ nhanh mà không làm sập cơ sở dữ liệu nguồn, và ở các lần sau chỉ đọc phần đã thay đổi.

Công cụ truyền thống trong hệ Hadoop là Apache Sqoop [1]. Dự án này đã vào Apache Attic từ tháng 6 năm 2021, nên hiện nay người ta thường dùng Spark JDBC [2] hoặc CDC thay thế. Bài này nhắm tới bốn việc:

1. Chạy Sqoop nạp toàn phần với `--split-by` và `-m`, đọc câu SQL Sqoop sinh ra để hiểu cách chia split, và so sánh cột phân bố đều (`order_seq`) với cột lệch (`customer_zip_code_prefix`).
2. Chạy Sqoop nạp gia tăng ở chế độ `append` và `lastmodified`, kèm saved job, `--merge-key` và `sqoop export`.
3. Viết phần còn thiếu của `spark_jdbc_ingest.py`: đọc JDBC song song, ghi Parquet phân vùng theo tháng, nạp gia tăng bằng watermark và dựng snapshot khử trùng lặp.
4. So sánh hai công cụ và chỉ ra giới hạn của nạp gia tăng dựa trên truy vấn so với CDC.

4.2 Cơ sở lý thuyết

4.2.1 Nạp toàn phần và nạp gia tăng

Nạp toàn phần đọc lại cả bảng mỗi lần, đơn giản và luôn nhất quán nhưng tốn kém khi bảng lớn. Nạp gia tăng chỉ đọc dòng mới hoặc dòng đã đổi, dựa vào một cột đơn điệu: khóa tăng dần (bắt INSERT) hoặc cột thời điểm cập nhật (bắt cả INSERT và UPDATE). Giá trị lớn nhất đã xử lý được lưu lại làm watermark cho lần chạy sau [5].

4.2.2 Cách Sqoop chia split

Mỗi lệnh `import` được biên dịch thành một MapReduce job chỉ có pha map [1], [6]. Sqoop chạy một *bounding query* `SELECT MIN(col), MAX(col)` trên cột `--split-by`, rồi chia đoạn $[\min, \max]$ thành `-m` khoảng có độ rộng giá trị bằng nhau. Mỗi map mở một kết nối JDBC và chạy `SELECT ... WHERE col >= lo AND col < hi` (khoảng cuối dùng <=). Số dòng mỗi map chỉ đều khi giá trị của cột chia phân bố đều.

4.2.3 Đọc JDBC song song trong Spark

Nguồn JDBC của Spark SQL [2], [3] nhận `partitionColumn`, `lowerBound`, `upperBound` và `numPartitions`. Spark tính bước $s = (\text{upper} - \text{lower})/n$ và sinh n mệnh đề WHERE; partition đầu là `col < lower + s OR col IS NULL`, partition cuối là `col >= lower + (n-1)s`, cả hai không có cận ngoài. Như vậy hai cận chỉ quyết định bước chia chứ không lọc dữ liệu. Cột chia phải có kiểu số, ngày hoặc timestamp; mỗi partition là một task của RDD [4] và một kết nối riêng.

4.2.4 Watermark, change log và snapshot

Pipeline gia tăng thường ghi các dòng thay đổi vào một change log chỉ nối thêm, rồi dựng snapshot bằng cách giữ phiên bản mới nhất của mỗi khóa, ví dụ với `row_number()`. Cận trên của cửa sổ phải chốt trước khi đọc, và watermark chỉ lưu sau khi ghi xong. Kết quả là ngữ nghĩa *at-least-once* cộng với bước khử trùng lặp idempotent [5].

4.2.5 Change Data Capture

CDC dựa trên nhật ký giao dịch không truy vấn bảng mà đọc write-ahead log (WAL) qua *logical decoding* [7]. Các công cụ như Debezium [8] phát ra mọi INSERT, UPDATE và DELETE theo thứ tự commit, kèm ảnh trước và sau của dòng.

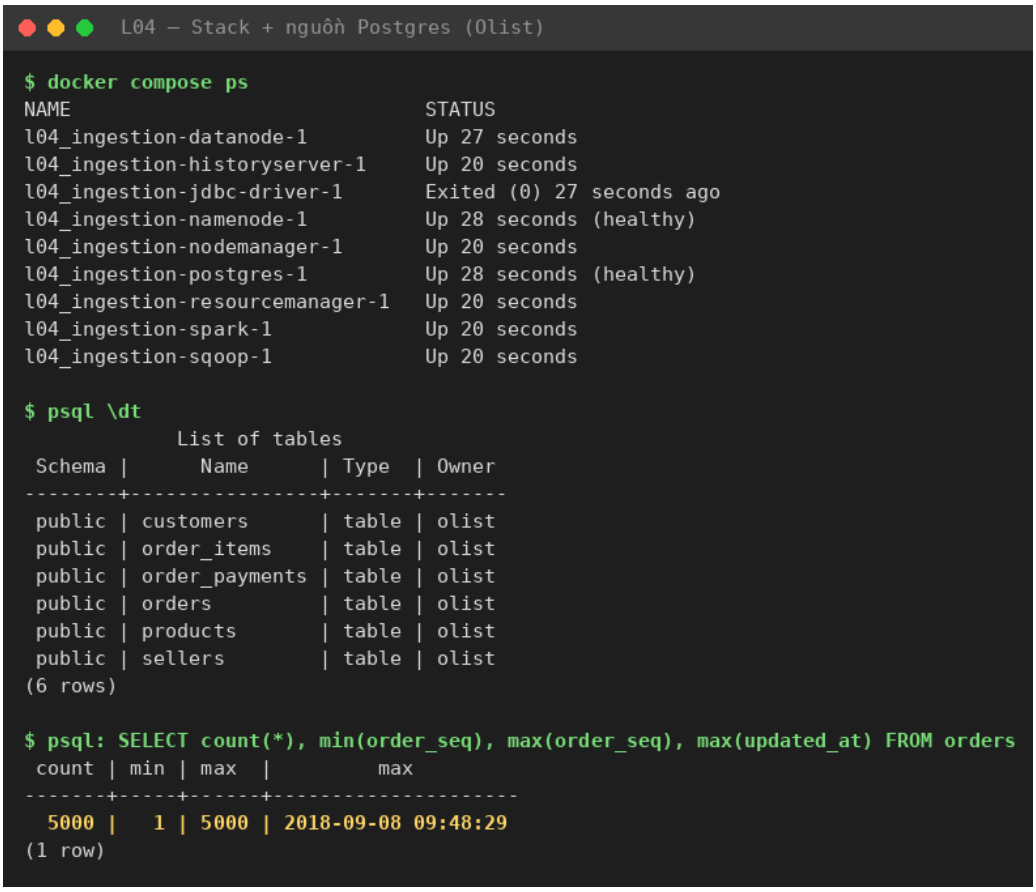
4.3 Môi trường và phương pháp thực nghiệm

4.3.1 Môi trường

Toàn bộ hệ thống chạy bằng Docker Compose trên một máy (Bảng 4.1). Lúc khởi động, container `jdbc-driver` chỉ tải driver rồi thoát với mã 0; ban đầu chúng tôi tưởng nó lỗi, nhưng đó là hành vi đúng (Hình 4.1).

Bảng 4.1. Môi trường thực nghiệm (stack `l04_ingestion`).

Thành phần	Phiên bản / cấu hình	Vai trò
PostgreSQL	18.6, `log_statement=all`	Cơ sở dữ liệu nguồn Olist, ghi lại mọi câu SQL
HDFS	3.4.3, 1 NameNode + 1 DataNode	Đích lưu trữ
YARN	ResourceManager, NodeManager, JobHistory Server	Chạy MapReduce job của Sqoop
Sqoop	1.4.7 (image khóa học `de-labs/sqoop:1.4.7`)	Công cụ nạp truyền thống
Spark	4.1.3, local mode, Spark UI cổng 4040	Công cụ nạp hiện đại
JDBC driver	PostgreSQL JDBC (`/jars/postgresql.jar`)	Kết nối cho Sqoop và Spark



Hình 4.1. Trạng thái các container của stack L04 (`docker compose ps`) và danh sách 6 bảng nguồn trong PostgreSQL; container `jdbc-driver` kết thúc với mã 0 sau khi tải driver.

4.3.2 Dữ liệu

Nguồn là cơ sở dữ liệu Olist tổng hợp gồm 6 bảng: `customers`, `orders`, `order_items`, `order_payments`, `products`, `sellers`. Bảng `orders` có 5,000 dòng, `order_seq` chạy từ 1 đến 5000, `max(updated_at)` là `2018-09-08 09:48:29`. `order_seq` là khóa thay thế kiểu số, tăng theo thời gian mua; các khóa gốc như `order_id`, `customer_id` là chuỗi hash 32 ký tự. Mỗi bảng có cột `updated_at` do trigger tự cập nhật khi UPDATE. Phân bố trạng thái đơn hàng lấy bằng `sqoop eval` nằm ở Bảng 4.2.

Bảng 4.2. Phân bố trạng thái của 5,000 đơn hàng ban đầu.

`order_status`	Số đơn
delivered	4,793
shipped	102
unavailable	41
canceled	30
processing	19
invoiced	11

order_status	Số đơn
approved	3
created	1

Script `sql/simulate_changes.sql` mô phỏng một ngày nghiệp vụ: chèn 3 khách hàng và 3 đơn mới (trạng thái `created`), đổi 2 đơn `shipped` đầu tiên thành `delivered`. Chúng tôi chạy script hai lần: lần 1 trước phần Sqoop gia tăng (tạo `order_seq` 5001–5003, cập nhật đơn 139 và 278), lần 2 giữa hai lần chạy Spark gia tăng (tạo 5004–5006, cập nhật đơn 337 và 355). Vì thế lúc chạy Spark, `orders` và `customers` đã có 5,003 dòng chứ không phải 5,000.

4.3.3 Quy trình

1. Khởi động stack, kiểm tra 6 bảng và thống kê `orders`.
2. Sqoop phần 1 (`sqoop_part1.sh`): `list-tables`, `eval`; nạp toàn phần `orders` với `--split-by order_seq -m 4`; nạp `customers` với `--split-by customer_zip_code_prefix -m 4`; nạp append lần đầu với `-m 2`, `--last-value 0`; nạp `lastmodified` cơ sở với `-m 1`, `--last-value '1970-01-01 00:00:00'`; tạo saved job `orders_incr`. Đếm số dòng từng file `part-m-*`.
3. Chạy `simulate_changes.sql` lần 1.
4. Sqoop phần 2: `sqoop job --exec orders_incr`; `lastmodified` với `--merge-key order_id`; `sqoop export` từ `orders_full` sang bảng mới `orders_copy` với `-m 2`.
5. Spark `--step full`: đọc naive, đọc song song theo `order_seq` và theo `order_purchase_timestamp`, ghi Parquet phân vùng `purchase_month`, kiểm tra partition pruning, nạp 5 bảng còn lại.
6. Spark `--step incremental` ba lần: lần 1; chạy `simulate_changes.sql` lần 2; lần 2; lần 3 khi không có thay đổi.
7. Trong suốt quá trình, thu nhật ký PostgreSQL để đối chiếu câu SQL; chụp YARN UI, JobHistory và Spark UI.

Các tham số thay đổi giữa các thí nghiệm là cột chia (`order_seq`, `customer_zip_code_prefix`, `order_purchase_timestamp`), mức song song (`-m 1/2/4`, `numPartitions` 1 hoặc 4) và chế độ gia tăng.

4.4 Kết quả thực nghiệm

4.4.1 Sqoop nạp toàn phần bảng `orders`

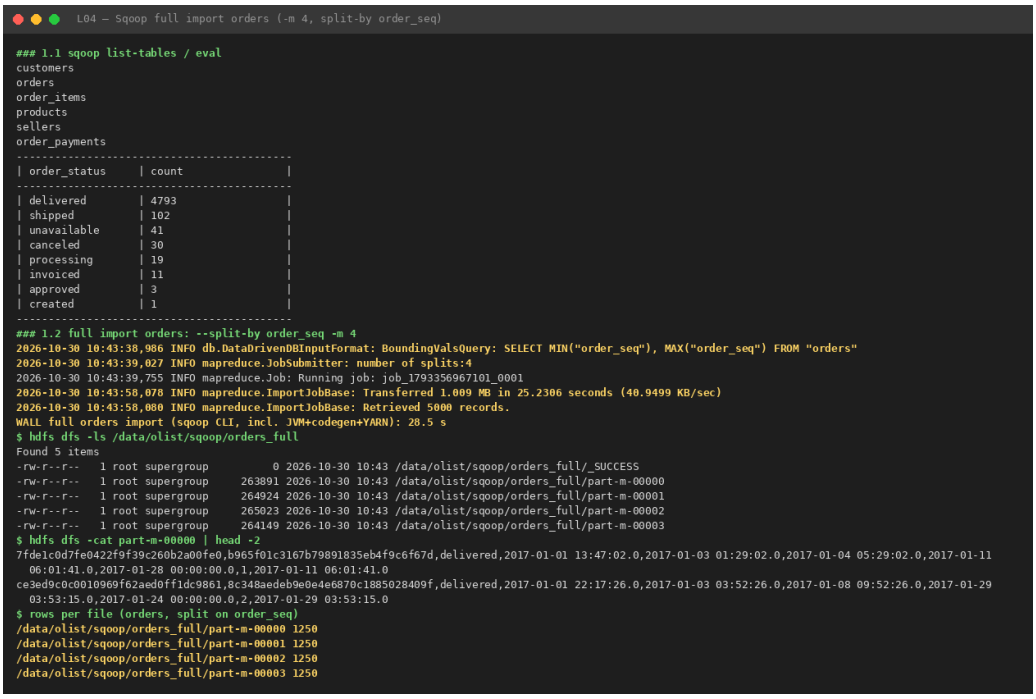
Sqoop gửi `SELECT MIN("order_seq"), MAX("order_seq") FROM "orders"`, nhận (1, 5000) và báo `number of splits:4`. Nhật ký PostgreSQL có đúng 4 câu SELECT của 4 map (Bảng 4.3, Hình 4.3). Job `job_1793356967101_0001` báo `Transferred 1.009 MB in 25.2306 seconds`

`(40.9499 KB/sec)` và `Retrieved 5000 records`. Cả lệnh CLI mất 28.5 s, vì còn phải khởi động JVM, sinh và biên dịch lớp record rồi nộp job lên YARN.

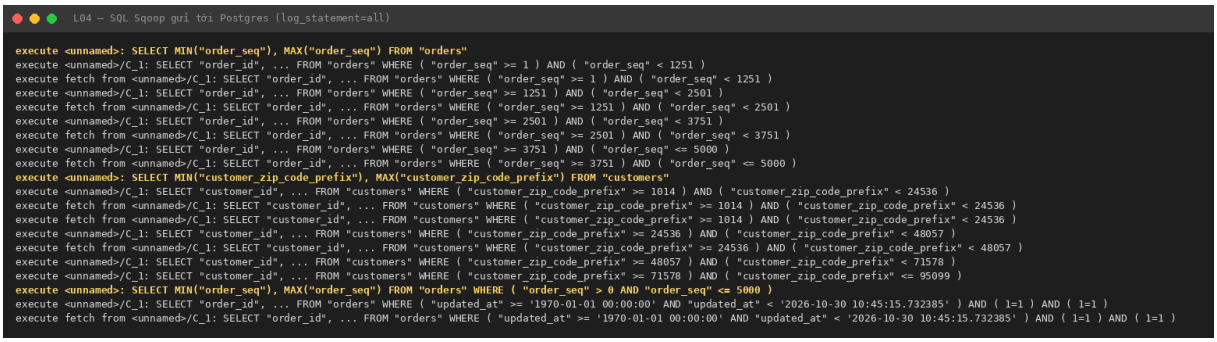
Bảng 4.3. Các split của Sqoop khi nạp `orders` với `--split-by order_seq -m 4`.

Map	Điều kiện WHERE	Số dòng	Kích thước file
0	`order_seq >= 1 AND order_seq < 1251`	1,250	263,891 B
1	`order_seq >= 1251 AND order_seq < 2501`	1,250	264,924 B
2	`order_seq >= 2501 AND order_seq < 3751`	1,250	265,023 B
3	`order_seq >= 3751 AND order_seq <= 5000`	1,250	264,149 B

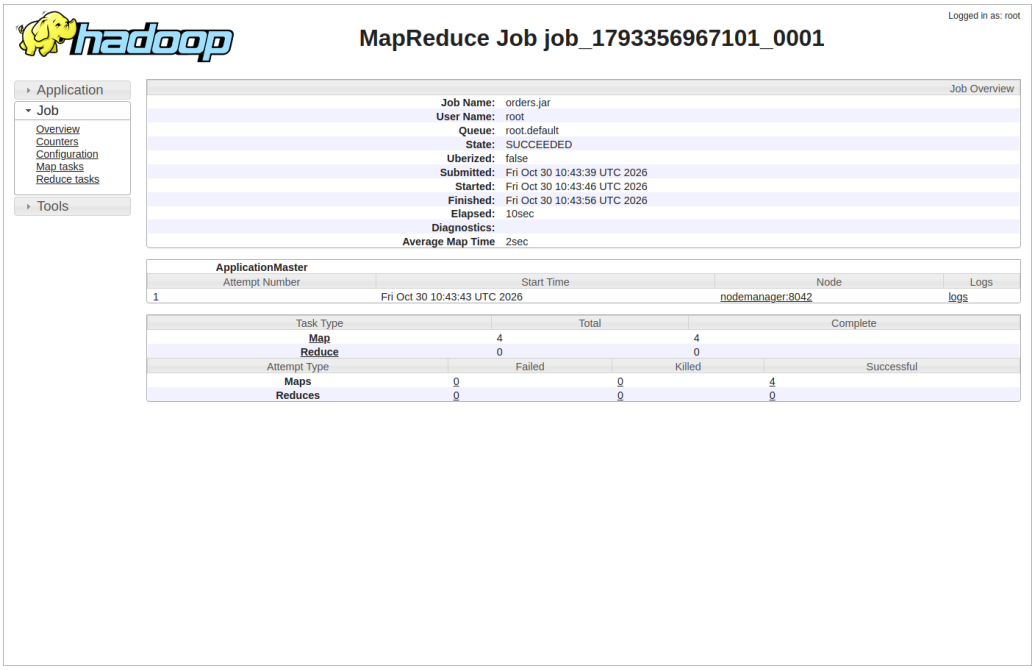
Bốn file `part-m-00000` đến `part-m-00003` gần như bằng nhau cả về số dòng lẫn kích thước (Hình 4.2). Dữ liệu là văn bản CSV, timestamp có dạng `2017-01-03 01:29:02.0`, NULL được ghi là `\N`. JobHistory xác nhận job có 4 map, 0 reduce, tất cả thành công (Hình 4.4).



Hình 4.2. Kết quả Sqoop nạp toàn phần `orders`: bounding query, 4 split, 1.009 MB trong 25.2306 s và 1,250 dòng mỗi file.



Hình 4.3. Nhật ký PostgreSQL ghi lại bounding query và 4 câu SELECT theo khoảng `order_seq` do các map task của Sqoop gửi.



Hình 4.4. JobHistory của job `job_1793356967101_0001` (`orders.jar`): 4 map task thành công, không có reduce task.

4.4.2 Sqoop với cột chia bị lệch

Với `customers`, chúng tôi chia theo `customer_zip_code_prefix -m 4`. Bounding query trả về (1014, 95099), và nhật ký PostgreSQL cho thấy 4 khoảng rộng xấp xỉ 23,521 với các điểm cắt 24536, 48057 và 71578. Số dòng mỗi file nằm ở Bảng 4.4, đặt cạnh trường hợp `orders` để dễ so.

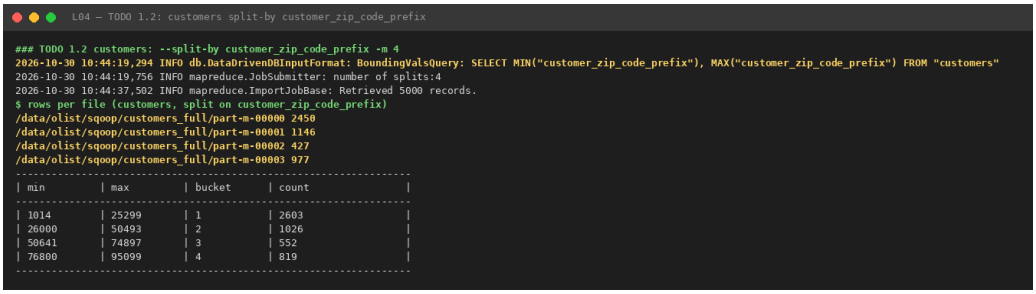
Bảng 4.4. Số dòng mỗi file khi chia theo cột phân bố đều và cột bị lệch.

Bảng / cột split	part-m-00000	part-m-00001	part-m-00002	part-m-00003
`orders` / `order_seq` (1–5000)	1,250	1,250	1,250	1,250
`customers` / `customer_zip_code_prefix` (1014–95099)	**2,450**	1,146	**427**	977

Split đầu nhận gần nửa bảng, gấp khoảng 5.7 lần split thứ ba (Hình 4.5). Để kiểm tra độc lập, chúng tôi đếm zip code bằng `width_bucket` trên 4 thùng (Bảng 4.5) và thấy cùng xu hướng: thùng chứa vùng São Paulo (01xxx–19xxx) chiếm phần lớn khách hàng.

Bảng 4.5. Phân bố `customer_zip_code_prefix` theo `width_bucket(..., 1003, 99990, 4)`.

Thùng	min zip	max zip	Số khách hàng
1	1014	25299	2,603
2	26000	50493	1,026
3	50641	74897	552
4	76800	95099	819



Hình 4.5. Số dòng mỗi file khi nạp `customers` chia theo `customer_zip_code_prefix`, thể hiện dữ liệu bị lệch.

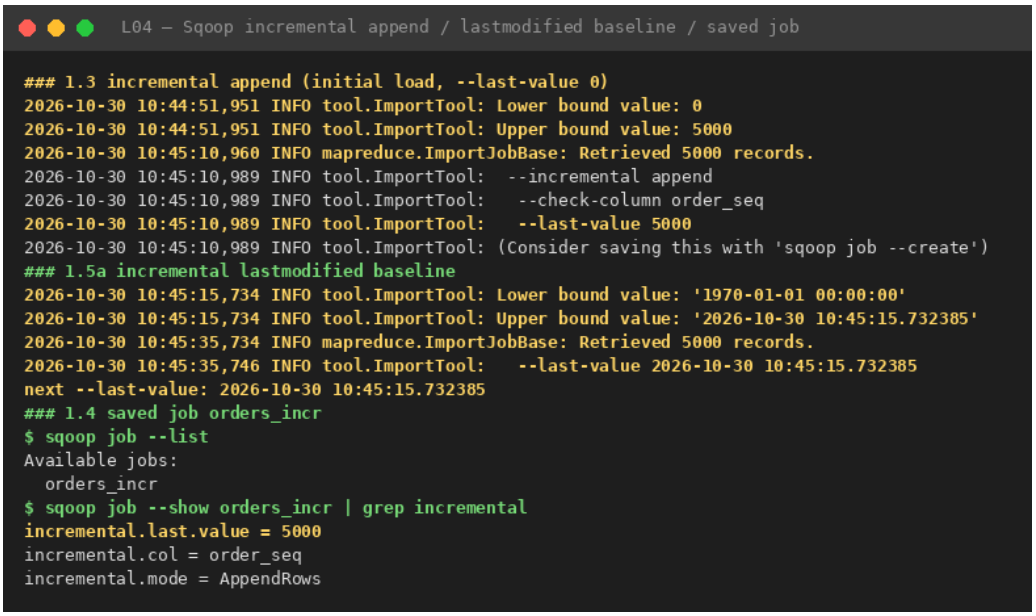
4.4.3 Sqoop nạp gia tăng, merge và export

Chuỗi thao tác gia tăng được tóm ở Bảng 4.6. Lần `append` đầu với `-m 2` có điều kiện `order_seq > 0 AND order_seq <= 5000` và hai split `[1, 2501)`, `[2501, 5000]`. Cuối lệnh, Sqoop gợi ý `--last-value 5000` và đề nghị lưu thành saved job. Saved job `orders_incr` lưu `incremental.col = order_seq` và `incremental.mode = AppendRows` trong metastore của Sqoop.

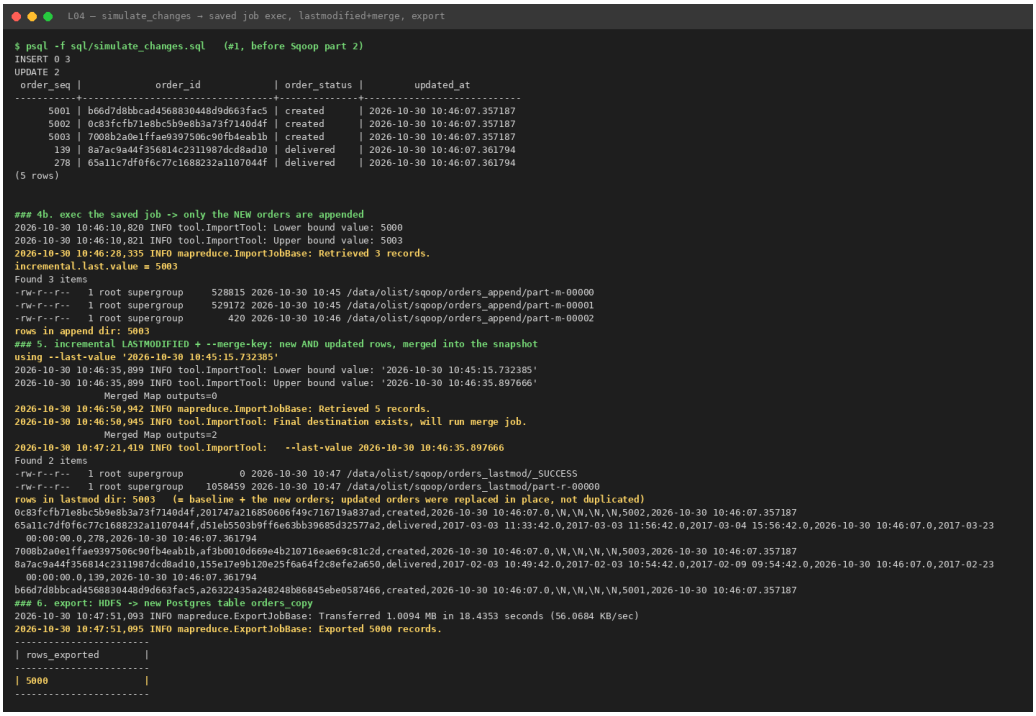
Bảng 4.6. Kết quả các bước nạp gia tăng, merge và export của Sqoop.

Bước	Cửa sổ	Kết quả
`append`, lần đầu (`-m 2`)	lower 0, upper 5000	5,000 dòng, gợi ý `--last-value 5000`
`lastmodified`, cơ sở (`-m 1`)	`'1970-01-01 00:00:00'` đến `'2026-10-30 10:45:15.732385'`	5,000 dòng
Saved job `orders_incr`		`incremental.last.value = 5000`
`simulate_changes` lần 1		tạo `order_seq` 5001–5003; đơn 139 và 278 chuyển `delivered`
`sqoop job --exec orders_incr`	lower 5000, upper 5003	**3** dòng; `last.value` thành 5003; thư mục có 5,003 dòng
`lastmodified` + `--merge-key order_id`	`'...10:45:15.732385'` đến `'...10:46:35.897666'`	**5** dòng; merge job tạo `part-r-00000` 5,003 dòng
`sqoop export` sang `orders_copy` (`-m 2`)		1.0094 MB trong 18.4353 s, 5,000 record

Saved job chạy `... WHERE ( "order_seq" > 5000 AND "order_seq" <= 5003 )` và chỉ thêm một file nhỏ `part-m-00002` (420 B) cạnh hai file 528,815 B và 529,172 B của lần đầu. Đơn 139 và 278 vừa cập nhật không được nạp lại, vì `append` chỉ nhìn khóa mới. `lastmodified` thì lấy đủ 5 dòng. Do thư mục đích đã tồn tại, Sqoop chạy thêm một merge job (`Merged Map outputs=2`) để thay bản cũ của cùng `order_id` bằng bản mới. Kết quả là một file `part-r-00000` (1,058,459 B), không trùng lặp, trong đó đơn 139 và 278 mang trạng thái `delivered` và `updated_at` mới. Một chi tiết chúng tôi không để ý cho tới khi đọc log: `lastmodified` dùng cửa sổ nửa mở `updated_at >= lower AND updated_at < upper`, ngược với Spark ở phần sau. Các bước này ở Hình 4.6 và Hình 4.7.

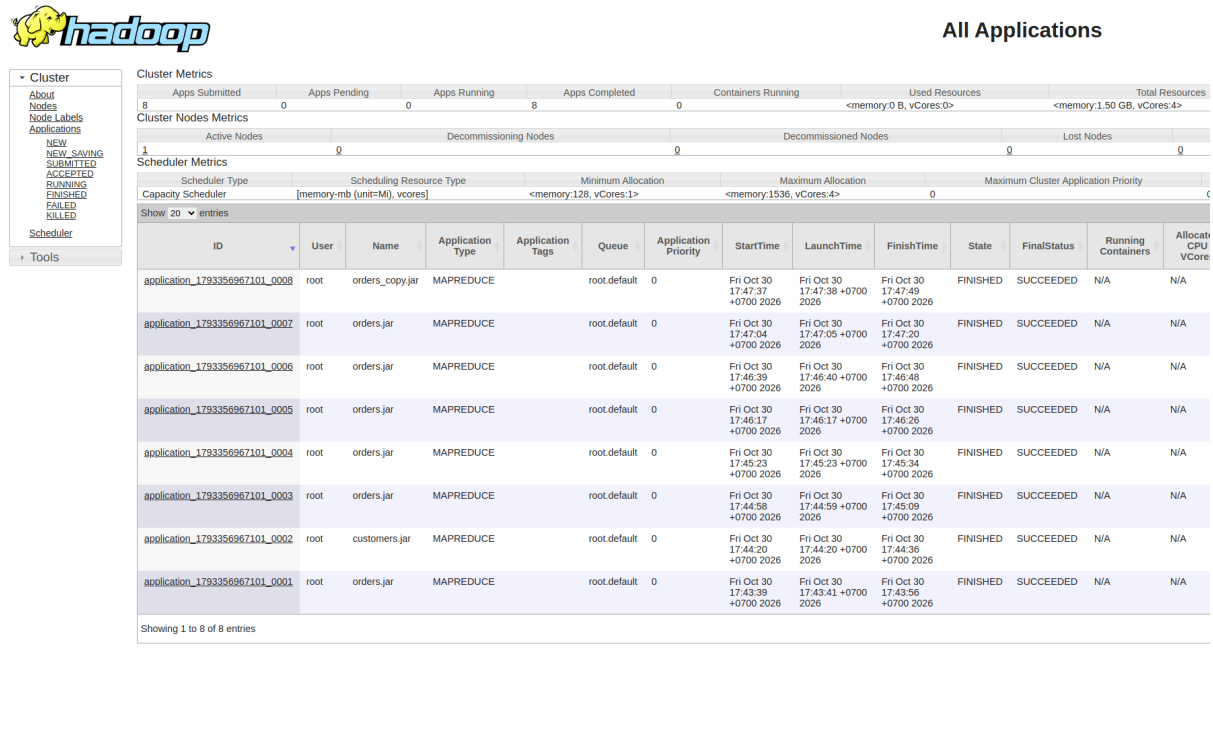


Hình 4.6. Sqoop nạp append lần đầu, nạp `lastmodified` cơ sở và tạo saved job `orders_incr`.

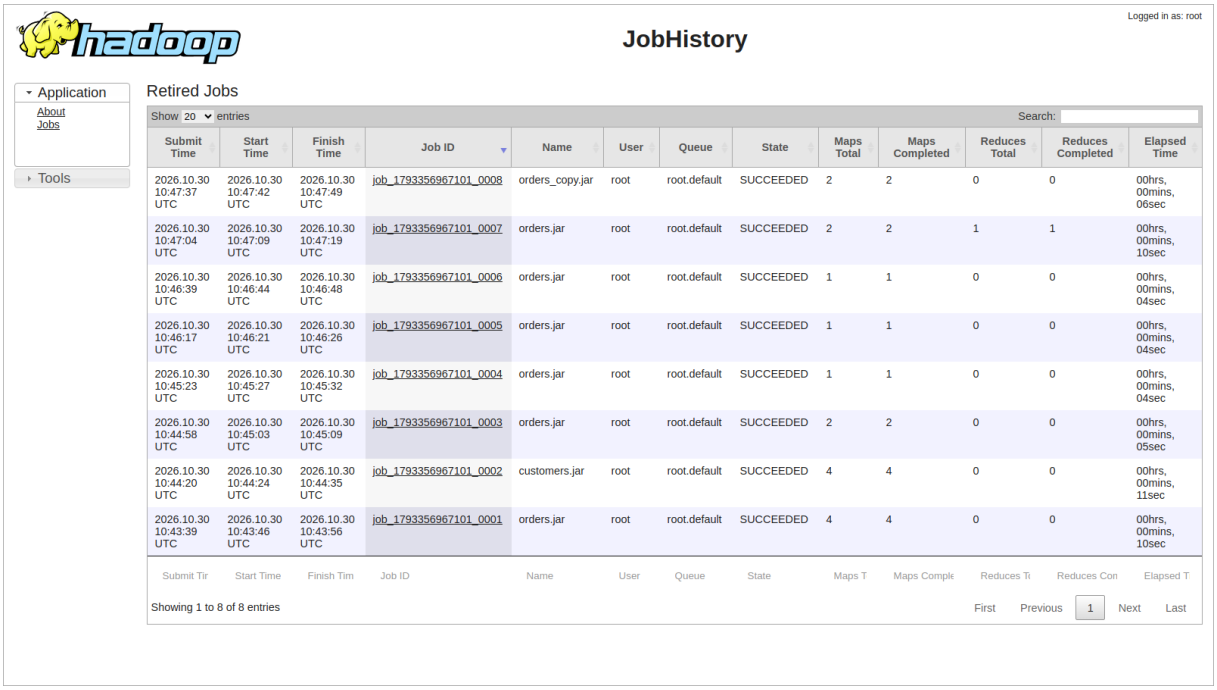


Hình 4.7. Sau `simulate_changes` lần 1: saved job lấy 3 dòng mới, `lastmodified` kèm merge lấy 5 dòng, và export 5,000 record về PostgreSQL.

Mỗi lệnh Sqoop là một ứng dụng MapReduce trên YARN. ResourceManager liệt kê 8 ứng dụng (`orders.jar`, `customers.jar`, `orders_copy.jar`) đều kết thúc thành công (Hình 4.8); JobHistory tương ứng ở Hình 4.9.



Hình 4.8. YARN ResourceManager UI: các MapReduce job do Sqoop nộp, tất cả ở trạng thái FINISHED/SUCCEEDED.



Hình 4.9. JobHistory Server liệt kê các job Sqoop đã hoàn tất.

4.4.4 Spark JDBC: đọc song song và ghi Parquet

Script lấy cận bằng `SELECT min(order_seq) AS lo, max(order_seq) AS hi FROM orders`, nhận `1 .. 5003`, rồi đọc theo ba cách (Bảng 4.7).

Bảng 4.7. So sánh các cách đọc bảng `orders` bằng Spark JDBC.

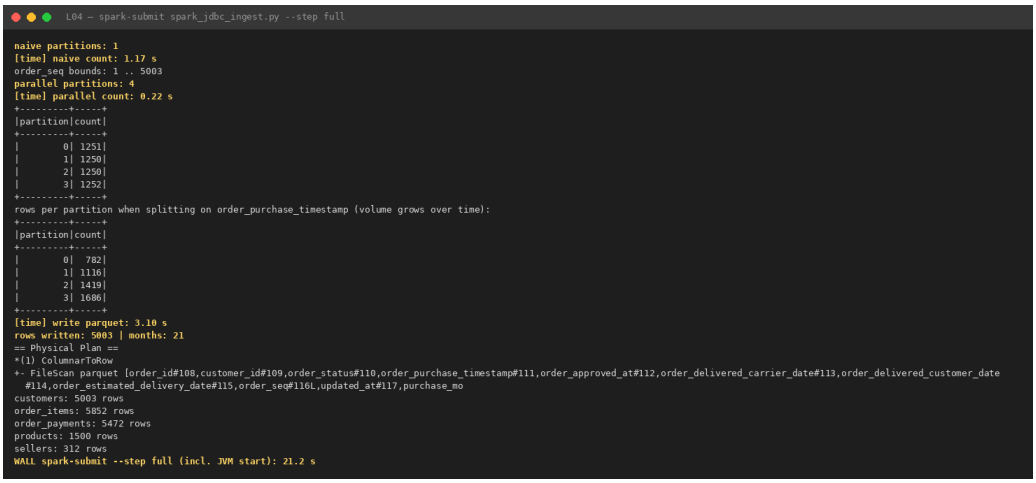
Cách đọc	Số partition	Thời gian `count()`	Số dòng mỗi partition
Naive (không có `partitionColumn`)	1	1.17 s	5,003 dòng trong 1 task
`partitionColumn=order_seq`, bounds 1..5003	4	0.22 s	1,251 / 1,250 / 1,250 / 1,252
`partitionColumn=order_purchase_timestamp`, 2017-01-01 đến 2018-09-01	4	không đo	782 / 1,116 / 1,419 / 1,686

Nhật ký PostgreSQL (Hình 4.11) có 4 câu do Spark sinh: `WHERE "order_seq" < 1252 or "order_seq" is null`, `>= 1252 AND < 2502`, `>= 2502 AND < 3752` và `WHERE "order_seq" >= 3752`. Bước chia là $(5003 - 1)/4 \approx 1250$, nên partition đầu có 1,251 dòng (1 đến 1251) và partition cuối có 1,252 dòng (3752 đến 5003). Vì lệnh là `count()`, Spark chỉ gửi `SELECT 1`, tức column pruning đã được đẩy xuống SQL. Với cột timestamp, partition đầu là `< '2017-06-02 00:00:00' or ... is null` và partition cuối là `>= '2018-04-02 00:00:00'`; số dòng tăng dần theo thời gian vì lượng đơn hàng của Olist tăng dần.

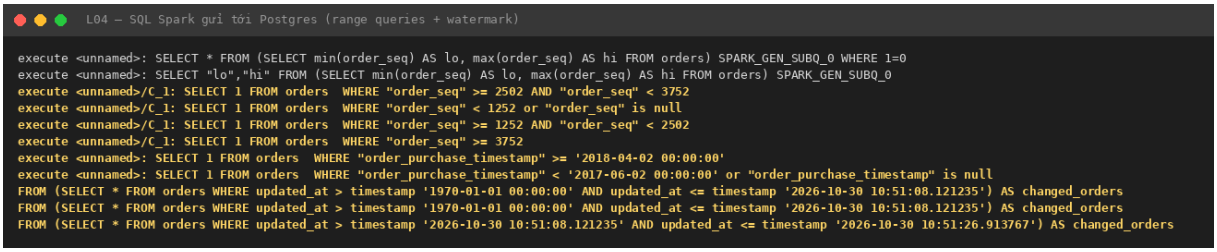
Ghi Parquet với `partitionBy("purchase_month")` mất 3.10 s, tạo 21 thư mục tháng (20 tháng lịch sử và tháng của các đơn mới) chứa đủ 5,003 dòng. Truy vấn lọc `purchase_month = '2018-08'` có physical plan chứa `PartitionFilters: [isnotnull(purchase_month#118), (purchase_month#118 = 2018-08)]`, nghĩa là partition pruning có tác dụng. Năm bảng còn lại được nạp mỗi bảng một partition (Bảng 4.8). Toàn bộ `spark-submit --step full`, tính cả khởi động JVM, mất 21.2 s (Hình 4.10).

Bảng 4.8. Số dòng các bảng phụ được Spark nạp sang Parquet.

Bảng	Số dòng
customers	5,003
order_items	5,852
order_payments	5,472
products	1,500
sellers	312

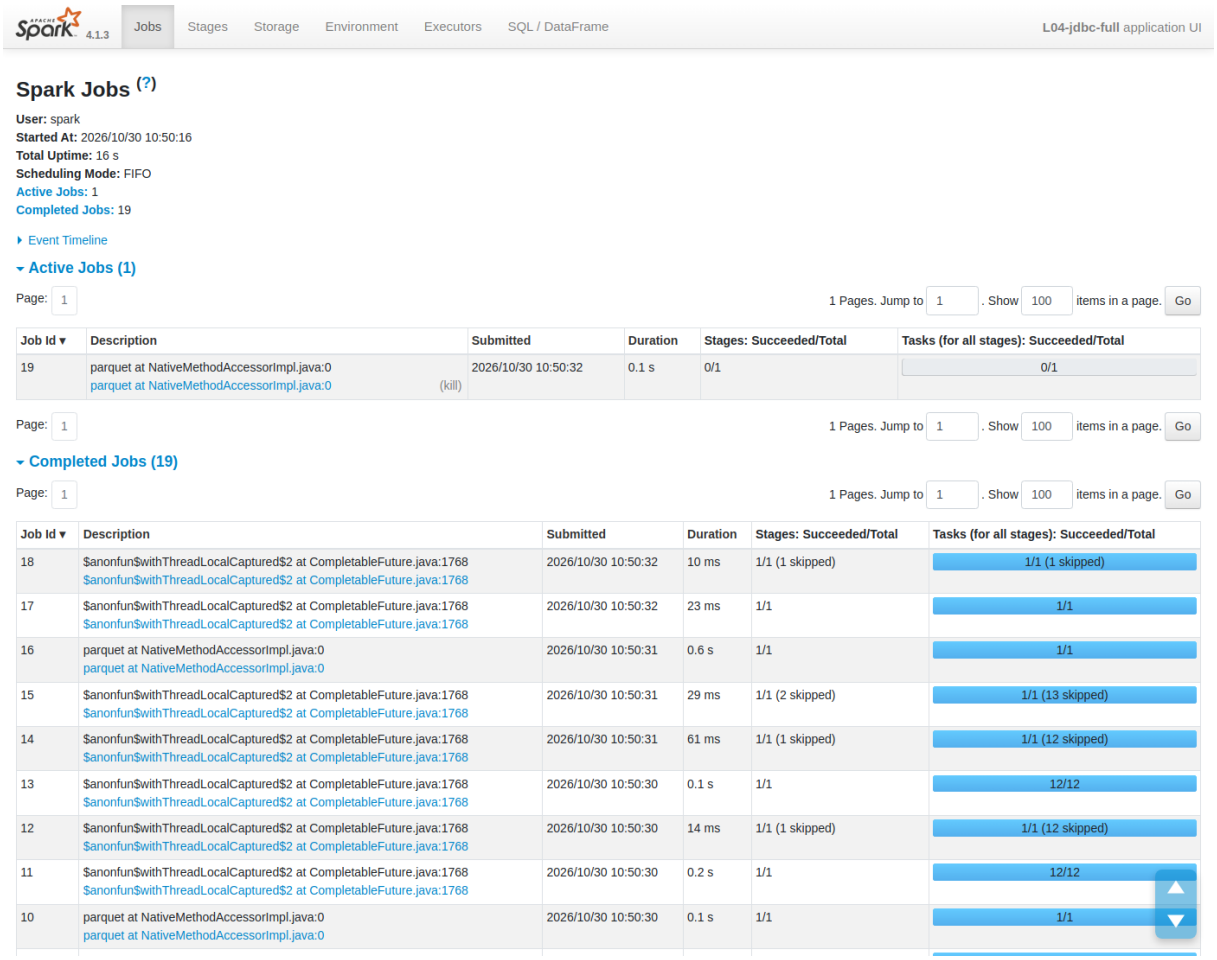


Hình 4.10. Kết quả `spark-submit --step full`: số partition, thời gian đếm, số dòng mỗi partition, kết quả ghi Parquet và physical plan có `PartitionFilters`.

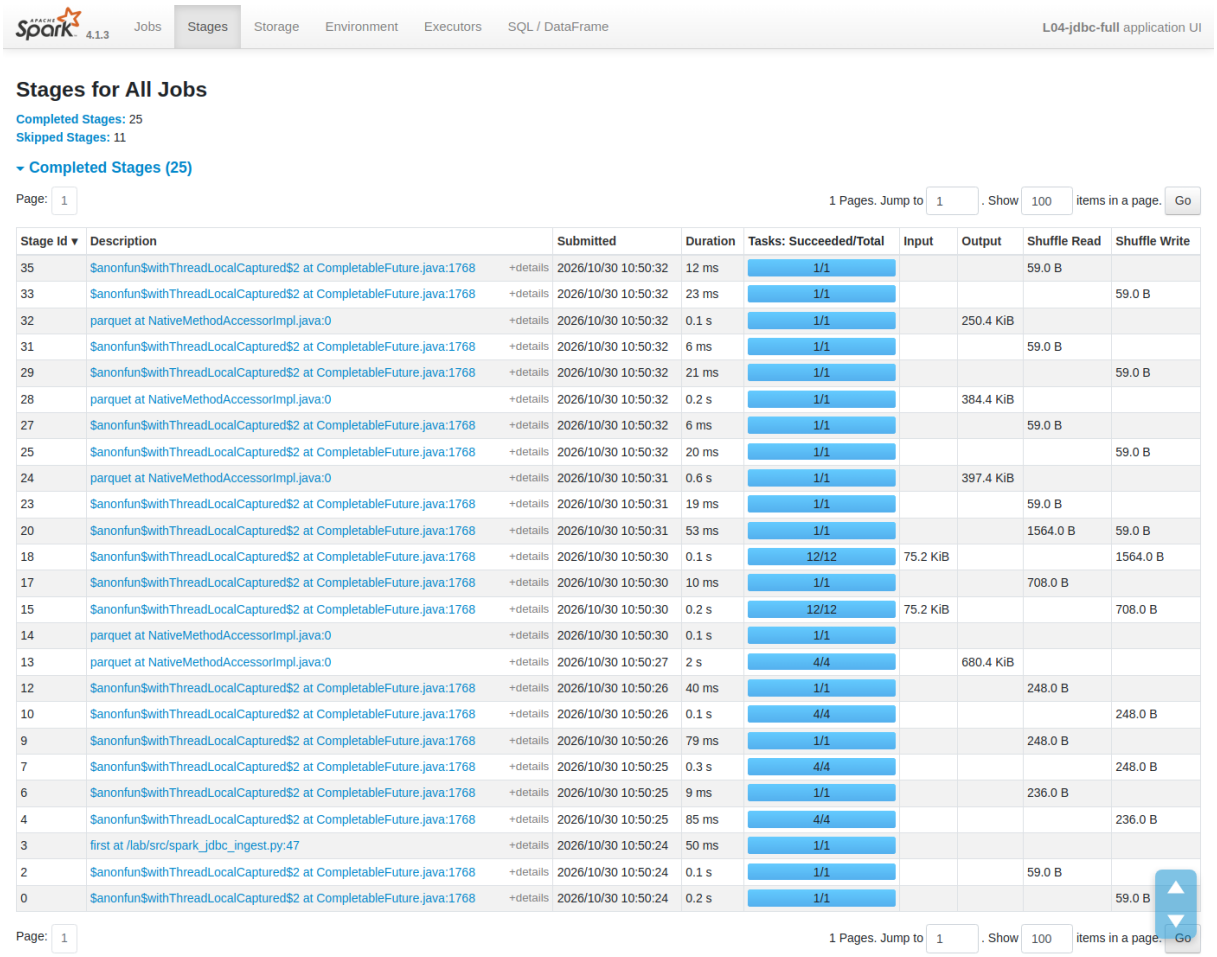


Hình 4.11. Nhật ký PostgreSQL ghi lại các câu `SELECT 1 FROM orders WHERE ...` theo khoảng do Spark sinh ra.

Trên Spark UI của ứng dụng `L04-jdbc-full`, các stage đọc song song có 4/4 task và stage ghi Parquet có 4/4 task với đầu ra 680.4 KiB (Hình 4.12, Hình 4.13).



Hình 4.12. Spark UI, trang Jobs của ứng dụng `L04-jdbc-full`.



Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
35	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	12 ms	1/1			59.0 B	
33	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	23 ms	1/1				59.0 B
32	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:32	0.1 s	1/1		250.4 KiB		
31	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	6 ms	1/1			59.0 B	
29	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	21 ms	1/1				59.0 B
28	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:32	0.2 s	1/1		384.4 KiB		
27	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	6 ms	1/1			59.0 B	
25	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	20 ms	1/1				59.0 B
24	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:31	0.6 s	1/1		397.4 KiB		
23	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:31	19 ms	1/1			59.0 B	
20	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:31	53 ms	1/1			1564.0 B	59.0 B
18	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:30	0.1 s	12/12	75.2 KiB			1564.0 B
17	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:30	10 ms	1/1			708.0 B	
15	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:30	0.2 s	12/12	75.2 KiB			708.0 B
14	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:30	0.1 s	1/1				
13	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:27	2 s	4/4		680.4 KiB		
12	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:26	40 ms	1/1			248.0 B	
10	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:26	0.1 s	4/4				248.0 B
9	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:26	79 ms	1/1			248.0 B	
7	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:25	0.3 s	4/4				248.0 B
6	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:25	9 ms	1/1			236.0 B	
4	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:25	85 ms	4/4				236.0 B
3	first at /lab/src/spark_jdbc_ingest.py:47 +details	2026/10/30 10:50:24	50 ms	1/1				
2	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:24	0.1 s	1/1			59.0 B	
0	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:24	0.2 s	1/1				59.0 B

Hình 4.13. Spark UI, trang Stages: các stage đọc JDBC song song và stage ghi Parquet đều có 4 task.

Kiểm tra bằng `hdfs dfs -ls` và `hdfs dfs -du` (Hình 4.14): `/data/olist/spark/orders` có 22 mục (`_SUCCESS` và 21 thư mục `purchase_month=`), dung lượng 680.4 K, còn `/data/olist/sqoop/orders_full` là 1.0 M.

```
L04 — Parquet partitionBy(purchase_month) trên HDFS

$ hdfs dfs -ls /data/olist/spark/orders | head
Found 22 items
-rw-r--r--   1 spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/_SUCCESS
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-01
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-02
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-03
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-04
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-05
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-06
...
$ hdfs dfs -du -s -h /data/olist/spark/orders /data/olist/sqoop/orders_full
680.4 K  680.4 K  /data/olist/spark/orders
1.0 M    1.0 M    /data/olist/sqoop/orders_full
```

Hình 4.14. Đầu ra Parquet phân vùng theo tháng trên HDFS và so sánh dung lượng với đầu ra văn bản của Sqoop.

4.4.5 So sánh Sqoop và Spark JDBC

Bảng 4.9 là bảng so sánh của Phần 3 trong đề bài, bổ sung thời gian đo được trong bài này.

Bảng 4.9. So sánh Sqoop và Spark JDBC khi nạp bảng `orders`.

Tiêu chí	Sqoop 1.4.7 (MapReduce)	Spark JDBC
Tham số song song	`-m 4` + `--split-by`	`numPartitions=4` + `partitionColumn`
Cách tính khoảng	Sqoop tự chạy `SELECT MIN, MAX` (BoundingValsQuery)	người dùng truyền `lowerBound`/`upperBound` (chỉ là stride)
Nạp gia tăng	Có sẵn: `--incremental append/lastmodified`, saved job lưu last value	Tự viết: truy vấn watermark + file trạng thái
Xử lý UPDATE	`--merge-key` (thêm một MR job)	`row_number()` khử trùng lặp, hoặc MERGE vào table format
Định dạng đầu ra	Văn bản CSV, 1.0 MB	Parquet Snappy phân vùng theo tháng, 680.4 KB
Biến đổi khi nạp	Không (chỉ SQL qua `--query`)	Đầy đủ DataFrame API (`withColumn`, `partitionBy`, …)
Thời gian nạp `orders`	25.2 s MR job (28.5 s cả lệnh)	3.10 s ghi Parquet (21.2 s cả `spark-submit` 6 bảng, gồm khởi động JVM)
Trạng thái dự án	Apache Attic (ngừng từ 2021)	Đang phát triển

Sqoop chậm chủ yếu vì mỗi lệnh phải sinh mã, biên dịch lớp record và dựng một MapReduce job riêng với container YARN. Spark dùng một SparkSession cho cả 6 bảng và ghi thẳng ra định dạng cột có nén, nên đầu ra nhỏ hơn khoảng một phần ba.

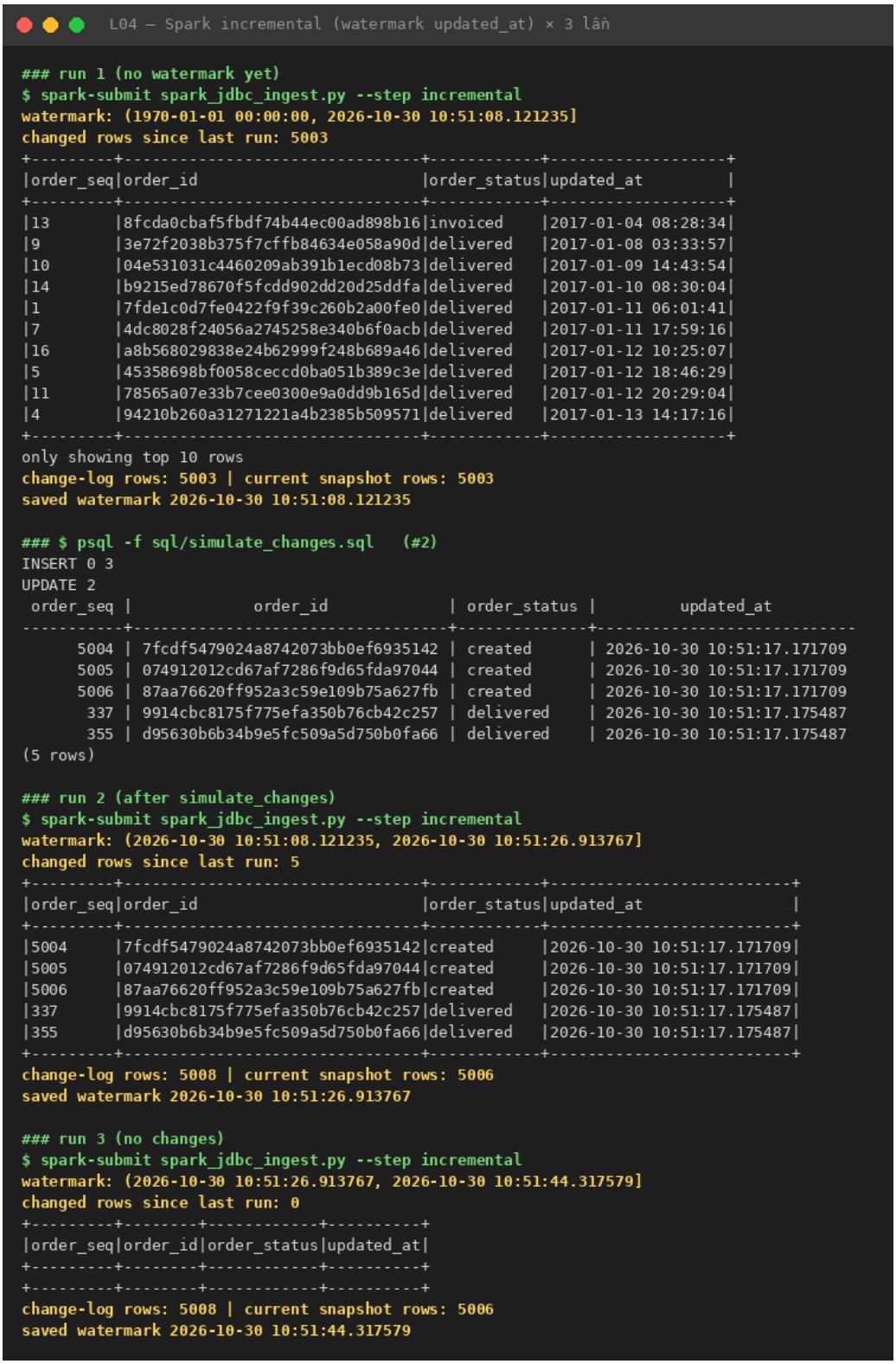
4.4.6 Spark nạp gia tăng bằng watermark

Bước `incremental` được chạy ba lần (Bảng 4.10, Hình 4.15).

Bảng 4.10. Kết quả ba lần chạy Spark nạp gia tăng (thời điểm ngày 2026-10-30).

Lần chạy	Cửa sổ watermark	Dòng đọc được	Change log	Snapshot
1 (chưa có watermark)	`(1970-01-01 00:00:00, 2026-10-30 10:51:08.121235]`	5,003	5,003	5,003
`simulate_changes` lần 2	tạo 5004–5006, cập nhật đơn 337 và 355			
2	`(10:51:08.121235, 10:51:26.913767]`	**5**	5,008	5,006
3 (không có thay đổi)	`(10:51:26.913767, 10:51:44.317579]`	0	5,008	5,006

Change log giữ mọi phiên bản: $5003 + 5 = 5008$ dòng. Snapshot dùng `row_number()` theo `order_id`, sắp `updated_at DESC, _ingested_at DESC`, nên chỉ còn phiên bản mới nhất của mỗi đơn: $5003 + 3 = 5006$, hai đơn được cập nhật không bị nhân đôi. Lần thứ ba đọc 0 dòng, tức watermark đã được lưu đúng và chạy lại không làm sai kết quả. Cận trên của lần trước trùng khít với cận dưới của lần sau.



Hình 4.15. Ba lần chạy Spark `--step incremental`: lần 1 đọc 5,003 dòng, lần 2 đọc 5 dòng thay đổi, lần 3 đọc 0 dòng.

4.5 Thảo luận

4.5.1 Sqoop chạy câu SQL nào trước khi import, và từ đó ra 4 split bằng cách nào?

Trước khi import, Sqoop chạy `SELECT MIN("order_seq"), MAX("order_seq") FROM "orders"` (Hình 4.3) và nhận (1, 5000). Bộ chia số nguyên cắt đoạn này thành 4 khoảng bằng nhau về giá trị, mỗi khoảng rộng khoảng $(5000 - 1)/4 \approx 1250$. Mỗi map chạy `SELECT ... WHERE order_seq >= lo AND order_seq < hi`, riêng khoảng cuối dùng <= để lấy cả giá trị MAX. Vì `order_seq` liên tục từ 1 đến 5000 nên mỗi map nhận đúng 1,250 dòng (Bảng 4.3). Điểm cần nhớ là Sqoop chia theo giá trị, không theo số dòng, và câu hỏi tiếp theo cho thấy hệ quả của việc đó.

4.5.2 Vì sao các file `customers` không đều, và nếu dùng `--split-by customer_id` thì sao?

Zip code của khách hàng Olist dồn về São Paulo (01xxx–19xxx), nên khoảng [1014, 24536) nhận 2,450 dòng trong khi [48057, 71578) chỉ có 427 dòng (Bảng 4.4, Bảng 4.5). Đây là data skew: task nặng nhất quyết định thời gian của cả job. Còn `customer_id` là chuỗi hash kiểu text. Mặc định Sqoop từ chối chia theo cột text và báo lỗi, trừ khi bật `-Dorg.apache.sqoop.splitter.allow_text_splitter=true`. Kể cả khi bật, việc chia theo thứ tự chuỗi phụ thuộc collation nên split vẫn có thể lệch hoặc rỗng. Cột chia tốt nhất là cột số, phân bố đều và có chỉ mục, như `order_seq`.

4.5.3 Với `lowerBound=1, upperBound=5003, numPartitions=4`, Spark có đọc dòng `order_seq` = 9,999 không?

Có đọc. `lowerBound` và `upperBound` chỉ dùng để tính bước chia, không phải bộ lọc. Nhật ký ở Hình 4.11 cho thấy partition cuối là `order_seq >= 3752` và không có cận trên, nên dòng 9,999 rơi vào partition cuối. Tương tự, giá trị nhỏ hơn `lowerBound` và NULL rơi vào partition đầu. Cận sai không làm mất dữ liệu, nhưng nếu nhiều dòng nằm ngoài cận thì partition hai đầu sẽ phình to và gây skew. Vì vậy script lấy MIN/MAX thật trước khi đọc, và nhờ đó 4 partition gần đều nhau (Bảng 4.7).

4.5.4 Vì sao phải chốt cận trên trước khi đọc, và chỉ lưu watermark sau khi ghi xong?

Script lấy `wm_new = now()` của cơ sở dữ liệu trước khi đọc, rồi truy vấn `updated_at > wm_old AND updated_at <= wm_new`. Nếu lấy `now()` sau khi đọc, dòng nào được cập nhật trong lúc đang đọc sẽ có `updated_at` nhỏ hơn watermark mới mà lại không nằm trong dữ liệu vừa đọc, và lần sau sẽ bỏ qua nó luôn. Cận trên cố định còn bảo đảm các partition JDBC song song

cùng nhìn một cửa sổ, và các cửa sổ liên tiếp liền nhau mà không chồng lên nhau. Bảng 4.10 cho thấy điều đó: lần 2 bắt đầu đúng tại 10:51:08.121235, lần 3 tại 10:51:26.913767.

Về thứ tự lưu, nếu job chết giữa chừng thì watermark cũ vẫn còn, lần chạy lại sẽ đọc lại cả cửa sổ. Dữ liệu có thể bị trùng nhưng không mất (at-least-once), và phần trùng được loại bằng `row_number()` theo `order_id` khi dựng snapshot. Ngược lại, nếu lưu watermark trước rồi mới ghi và bước ghi thất bại, cửa sổ đó sẽ không bao giờ được đọc lại.

4.5.5 Chế độ append hay `lastmodified` bắt được UPDATE, và chế độ nào bắt được DELETE?

`append` chỉ dựa vào `order_seq > last_value` nên chỉ bắt được INSERT: saved job lấy 3 dòng mới (Bảng 4.6). `lastmodified` dựa vào `updated_at` nên bắt được cả INSERT và UPDATE: 5 dòng gồm 3 đơn mới cùng đơn 139 và 278; khi đó cần `--merge-key` để gộp bản cũ với bản mới. Không chế độ nào bắt được DELETE, vì dòng đã xóa không còn trong bảng để truy vấn.

4.5.6 Watermark trên `updated_at` không giải quyết được gì, và CDC khác ở đâu?

Vấn đề đầu tiên là DELETE: dòng bị xóa không bao giờ xuất hiện trong truy vấn, nên bản sao trên HDFS giữ lại dữ liệu đã không còn ở nguồn, trừ khi ứng dụng dùng soft delete hoặc so sánh cả snapshot. Thứ hai là các lần UPDATE trung gian: nếu một đơn đổi từ `shipped` sang `delivered` rồi `canceled` trong cùng một chu kỳ, chúng tôi chỉ thấy trạng thái cuối. Ngoài hai điểm đó, bản thân `updated_at` có thể không đáng tin. Ứng dụng có thể quên cập nhật cột, đồng hồ máy chủ có thể lệch, hoặc một transaction dài commit sau khi watermark đã vượt qua timestamp của nó, và dòng đó bị sót. Cách này cũng tạo tải quét lên nguồn (cần chỉ mục trên `updated_at`), còn độ trễ thì bằng chu kỳ lập lịch.

CDC (Debezium, sẽ làm ở L08) không truy vấn bảng mà đọc WAL của PostgreSQL qua logical replication [7], [8]. Mọi INSERT, UPDATE và DELETE thành sự kiện theo thứ tự commit, kèm ảnh trước và sau, không phụ thuộc cột `updated_at`, gần như không tạo tải truy vấn và trễ chỉ vài giây.

4.5.7 Điều gì xảy ra với cơ sở dữ liệu nguồn khi đặt `-m 50` hoặc `numPartitions=50`?

Khi đó có 50 kết nối đồng thời, mỗi kết nối chạy một range scan. Con số này dễ chạm `max_connections` (PostgreSQL mặc định 100) và ứng dụng production sẽ không lấy được kết nối. Các truy vấn còn tranh CPU, I/O và buffer cache với tải OLTP; nếu cơ sở dữ liệu đã là nút thắt thì tốc độ gần như không tăng. Với bảng 5,000 dòng, mỗi task chỉ còn khoảng 100 dòng và chi phí khởi tạo lớn hơn lợi ích. Thực tế nên giữ khoảng 4–8 kết nối và thống nhất với DBA, đọc từ read replica hoặc ngoài giờ cao điểm, và đặt `fetchsize` hợp lý.

4.5.8 Hạn chế của thực nghiệm

Mọi thứ chạy trên một máy: HDFS một DataNode, YARN một NodeManager, Spark local mode, các task cùng dùng chung một máy và một cơ sở dữ liệu. Thời gian đo vì vậy phản ánh chi phí khởi động nhiều hơn thông lượng, và với khoảng 1 MB dữ liệu thì so sánh 25.2 s với 3.10 s chỉ nên hiểu theo nghĩa định tính. Thời gian `count()` chỉ đo một lần, và lần đọc song song chạy sau lần naive nên có thể được lợi từ JVM đã nóng; con số 0.22 s chưa đủ để kết luận về hiệu năng. Cuối cùng, do `simulate_changes.sql` chạy hai lần nên số dòng (5,003 trong Spark, 5,006 trong snapshot) khác với giá trị tham chiếu trong README.

4.6 Kết luận

Qua nhật ký SQL thật, bài này xác nhận cả Sqoop và Spark JDBC đều chia bảng theo khoảng giá trị của một cột. Sqoop tự tính cận bằng bounding query, còn Spark dùng cận do người dùng truyền vào và để hở partition hai đầu. Vì thế độ đều của các task phụ thuộc vào phân bố của cột chia: `order_seq` cho 4 phần gần bằng nhau, còn zip code (2,450 so với 427 dòng) và timestamp (782 so với 1,686 dòng) đều lệch. Ở phần nạp gia tăng, `append` chỉ bắt INSERT, còn `lastmodified` hay watermark trên `updated_at` bắt thêm UPDATE. Pipeline Spark chốt cận trên trước khi đọc, lưu watermark sau khi ghi và khử trùng lặp bằng `row_number()`; khi chạy lại nó cho kết quả đúng: 5 dòng rồi 0 dòng.

Nếu làm thật, chúng tôi sẽ dùng Spark JDBC thay Sqoop, chọn cột chia kiểu số có chỉ mục và lấy MIN/MAX thật làm cận, giới hạn số kết nối, đọc từ read replica và ghi ra Parquet phân vùng. Các bước tiếp theo có thể thử là dùng `predicates` để chia theo `order_status`, kiểm tra predicate pushdown, ghi vào table format có MERGE (Delta Lake, Iceberg), và chuyển sang CDC bằng Debezium (L08) để bắt DELETE cùng các phiên bản trung gian.

Tài liệu tham khảo

[1] The Apache Software Foundation, “Sqoop User Guide (v1.4.7),” 2017. https://sqoop.apache.org/docs/1.4

[2] The Apache Software Foundation, “JDBC To Other Databases,” Spark SQL Guide. https://spark.apache.org/docs/latest/sql-data-sources-jdbc.html

[3] M. Armbrust et al., “Spark SQL: Relational Data Processing in Spark,” in *Proc. ACM SIGMOD*, 2015, pp. 1383–1394.

[4] M. Zaharia et al., “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” in *Proc. USENIX NSDI*, 2012.

[5] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O’Reilly Media, 2017.

[6] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O’Reilly Media, 2015.

[7] The PostgreSQL Global Development Group, “Logical Decoding,” PostgreSQL Documentation. https://www.postgresql.org/docs/current/logicaldecoding.html

[8] Debezium Community, “Debezium Connector for PostgreSQL,” Debezium Documentation. https://debezium.io/documentation/reference/stable/connectors/postgresql.html

Phụ lục A: Danh mục tệp nộp kèm

Bảng 4.11. Danh mục tệp nộp kèm báo cáo L04.

Tệp	Nội dung
`sqoop_part1.sh`	Lệnh Sqoop phần 1: explore, nạp toàn phần, nạp `customers` theo zip code, `append`, `lastmodified` cơ sở, saved job
`spark_jdbc_ingest.py`	Đọc JDBC song song, ghi Parquet phân vùng, nạp gia tăng theo watermark, snapshot khử trùng lặp
`01_stack_and_source.txt`	`docker compose ps`, danh sách bảng, thống kê `orders`
`02_sqoop_part1.txt`	Log Sqoop phần 1, số dòng mỗi file, phân bố zip code, saved job
`03_postgres_log_sqoop_queries.txt`	Nhật ký PostgreSQL: bounding query và các câu SELECT của Sqoop
`04_simulate_changes_1.txt`	Kết quả `simulate_changes.sql` lần 1
`05_sqoop_part2_after_changes.txt`	Saved job, `lastmodified` + merge, export
`06_spark_full.txt`	Kết quả `spark-submit --step full`
`07_postgres_log_spark_queries.txt`	Nhật ký PostgreSQL: các truy vấn Spark (kèm một số truy vấn Sqoop append/saved job ở đầu)
`08_spark_output_hdfs.txt`	Liệt kê và dung lượng đầu ra Spark và Sqoop trên HDFS
`09_spark_incremental.txt`	Ba lần chạy `--step incremental` và `simulate_changes.sql` lần 2
`img/`	Ảnh chụp terminal, YARN UI, JobHistory và Spark UI

Phụ lục B: Trích đoạn mã nguồn

Các tệp được liệt kê ở Bảng 4.11. Đoạn đầu trong `sqoop_part1.sh` nạp `customers` chia theo zip code:

```
sqoop import "${CONN[@]}" --table customers --split-by
 ↪  customer_zip_code_prefix -m 4 "${NULLS[@]}" \
  --target-dir /data/olist/sqoop/customers_full --delete-target-dir
```

Trong `spark_jdbc_ingest.py`, bước `--step full` đọc song song theo `order_seq`, đọc theo timestamp rồi ghi Parquet phân vùng theo tháng:

```
orders = jdbc(spark, dbtable="orders", partitionColumn="order_seq",
              lowerBound=str(b.lo), upperBound=str(b.hi),
 ↪  numPartitions="4", fetchsize="1000")
by_ts = jdbc(spark, dbtable="orders",
 ↪  partitionColumn="order_purchase_timestamp",
             lowerBound="2017-01-01 00:00:00", upperBound="2018-09-01
 ↪  00:00:00",
             numPartitions="4")
out = orders.withColumn("purchase_month",
                        F.date_format("order_purchase_timestamp", "yyyy-MM"))
out.write.mode("overwrite").partitionBy("purchase_month").parquet(f"{BASE
 ↪  }/orders")
```

Bước `--step incremental` chốt cận trên bằng `now()` trước, sau đó mới đọc cửa sổ và nối vào change log:

```
wm_new = jdbc(spark, query="SELECT to_char(now(), 'YYYY-MM-DD
 ↪  HH24:MI:SS.US') AS ts").first().ts
changed_sql = (f"(SELECT * FROM orders WHERE updated_at > timestamp
 ↪  '{wm_old}'"
               f" AND updated_at <= timestamp '{wm_new}') AS
                ↪  changed_orders")
changes = jdbc(spark, dbtable=changed_sql) \
    .withColumn("_ingested_at", F.lit(wm_new).cast("timestamp"))
changes.write.mode("append").parquet(f"{BASE}/orders_changes")
```

Cuối bước này, script dựng snapshot từ change log, và watermark chỉ được ghi ở dòng cuối, sau khi snapshot đã ghi xong:

```
log = spark.read.parquet(f"{BASE}/orders_changes")
w = Window.partitionBy("order_id") \
    .orderBy(F.col("updated_at").desc(), F.col("_ingested_at").desc())
current = log.withColumn("_rn", F.row_number().over(w)).filter("_rn =
 ↪  1").drop("_rn")
current.write.mode("overwrite").parquet(f"{BASE}/orders_current")
spark.createDataFrame([(wm_new,)], "watermark string").coalesce(1) \
    .write.mode("overwrite").json(f"{BASE}/_state/orders_watermark")
```

Chương 5

PySpark cơ bản: DataFrame, Spark SQL, kế hoạch thực thi và Spark UI

Tóm tắt

Bài thực hành này dùng API DataFrame của PySpark (Spark 4.1.3, chế độ `local[4]` trong Docker) để xử lý dữ liệu taxi vàng New York tháng 01/2024 gồm 2,964,624 chuyến. Chúng tôi đọc Parquet và CSV, kiểm tra cơ chế lazy evaluation, làm sạch dữ liệu theo năm luật, join với bảng khu vực, tổng hợp theo quận, dùng hàm cửa sổ, viết lại bằng Spark SQL, đọc kế hoạch vật lý cùng Spark UI và cuối cùng ghi Parquet phân vùng theo ngày. Dựng ba transformation mất 0.096 s và không sinh job, còn `count()` mất 0.28 s. Sau làm sạch còn 2,867,786 dòng (96.73 %). Kết quả SQL trùng với DataFrame. Nhờ tổng hợp partial trước shuffle, chỉ 24 dòng đi qua shuffle thay vì 2.87 triệu. Khi ghi không `repartition` thu được 56 tệp, có `repartition` thu được đúng 31 tệp, đổi lại thời gian ghi tăng từ 6.8 s lên 8.2 s. Ranh giới stage nằm ở các toán tử `Exchange`, và tổng hợp hai pha của Spark làm cùng việc với combiner trong MapReduce.

Từ khóa: Apache Spark, PySpark, DataFrame, Spark SQL, lazy evaluation, Catalyst, AQE, broadcast join, Spark UI, partitioned Parquet.

5.1 Giới thiệu

Các bài trước đã đi qua HDFS, MapReduce và Hive. Bài này chuyển sang Apache Spark, hệ thống xử lý phân tán giữ dữ liệu trung gian trong bộ nhớ [1]. Với MapReduce, người lập trình tự viết pha map và reduce. Với Spark, ta chỉ mô tả phép tính qua DataFrame hoặc SQL, còn bộ tối ưu Catalyst [2] quyết định cách chạy. Vì vậy muốn dùng Spark cho đúng thì phải biết nó làm gì với đoạn mã: lúc nào job thật sự chạy, stage bị cắt ở đâu, dữ liệu có bị shuffle hay không.

Mục tiêu cụ thể:

1. Tạo `SparkSession`, đọc Parquet và CSV, kiểm tra schema và số partition đầu vào.

2. Kiểm chứng lazy evaluation: transformation chỉ dựng kế hoạch, action mới chạy job.
3. Làm sạch dữ liệu thực (NULL, giá cước âm, quãng đường hoặc thời lượng bất thường, ngày ngoài phạm vi) và xử lý chế độ ANSI mặc định của Spark 4.
4. Thực hiện join, `groupBy/agg` và hàm cửa sổ (`dense_rank`, `lag`, tổng lũy kế).
5. Viết lại cùng logic bằng Spark SQL và so sánh kết quả.
6. Đọc kế hoạch vật lý bằng `explain("formatted")` và đối chiếu với các stage trong Spark UI.
7. Ghi Parquet phân vùng và kiểm tra partition pruning khi đọc lại.

Mục 5.2 tóm lược lý thuyết, Mục 5.3 mô tả môi trường và quy trình, Mục 5.4 trình bày kết quả, Mục 5.5 trả lời tám câu hỏi checkpoint, Mục 5.6 kết luận.

5.2 Cơ sở lý thuyết

5.2.1 Driver, job, stage, task

Một ứng dụng Spark gồm một driver điều phối và các executor thực thi. Mỗi action (`count`, `show`, `collect`, `write`) sinh một hoặc nhiều job. DAG scheduler chia job thành các stage tại những chỗ cần trao đổi dữ liệu giữa các partition (shuffle), mỗi stage có một task cho mỗi partition [1], [3]. Ở `local[4]`, driver và bốn luồng task chạy chung một JVM. Đây là mô hình cluster thu nhỏ nhưng vẫn giữ đủ các khái niệm trên.

5.2.2 Transformation, action và lazy evaluation

`filter`, `withColumn`, `select`, `join`, `groupBy` là transformation. Chúng chỉ trả về một DataFrame mới mang cây kế hoạch logic, chưa đọc dữ liệu. Khi gọi action, Spark mới tối ưu cả kế hoạch rồi thực thi. Nhờ vậy Catalyst có thể gộp bộ lọc và đẩy chúng xuống nguồn dữ liệu trước khi đọc [2].

5.2.3 Catalyst, AQE và tổng hợp hai pha

Catalyst chuyển kế hoạch logic thành kế hoạch vật lý, áp dụng các quy tắc như predicate pushdown (đẩy điều kiện lọc xuống reader) và column pruning (chỉ đọc cột cần) [2]. Adaptive Query Execution (AQE) tối ưu lại kế hoạch ngay khi chạy, dựa trên thống kê thực của từng query stage, ví dụ gộp các partition shuffle nhỏ (`AQEShuffleRead coalesced`) [5]. Phép tổng hợp được tách hai pha: `HashAggregate` partial trước shuffle và final sau shuffle, tương đương combiner trong MapReduce [7].

5.2.4 Chiến lược join

Nếu một phía của join nhỏ hơn `spark.sql.autoBroadcastJoinThreshold` (mặc định 10 MB), Spark gửi bản sao phía đó tới mọi task (broadcast hash join) và không phải shuffle phía lớn. Khi

cả hai phía đều lớn, Spark shuffle cả hai theo khóa rồi dùng sort-merge join [3], [5].

5.2.5 Chế độ ANSI trong Spark 4

Từ Spark 4.0, `spark.sql.ansi.enabled` mặc định là `true`: chia cho 0 hay ép kiểu sai sẽ báo lỗi thay vì trả NULL như Spark 3. Muốn nhận NULL thì phải gọi tường minh `try_divide`, `try_cast` [6].

5.2.6 Parquet phân vùng và partition pruning

`DataFrameWriter.partitionBy(col)` ghi dữ liệu thành các thư mục con `col=value`. Khi đọc lại với bộ lọc trên cột phân vùng, Spark chỉ liệt kê và mở các thư mục khớp điều kiện (`PartitionFilters`) [4].

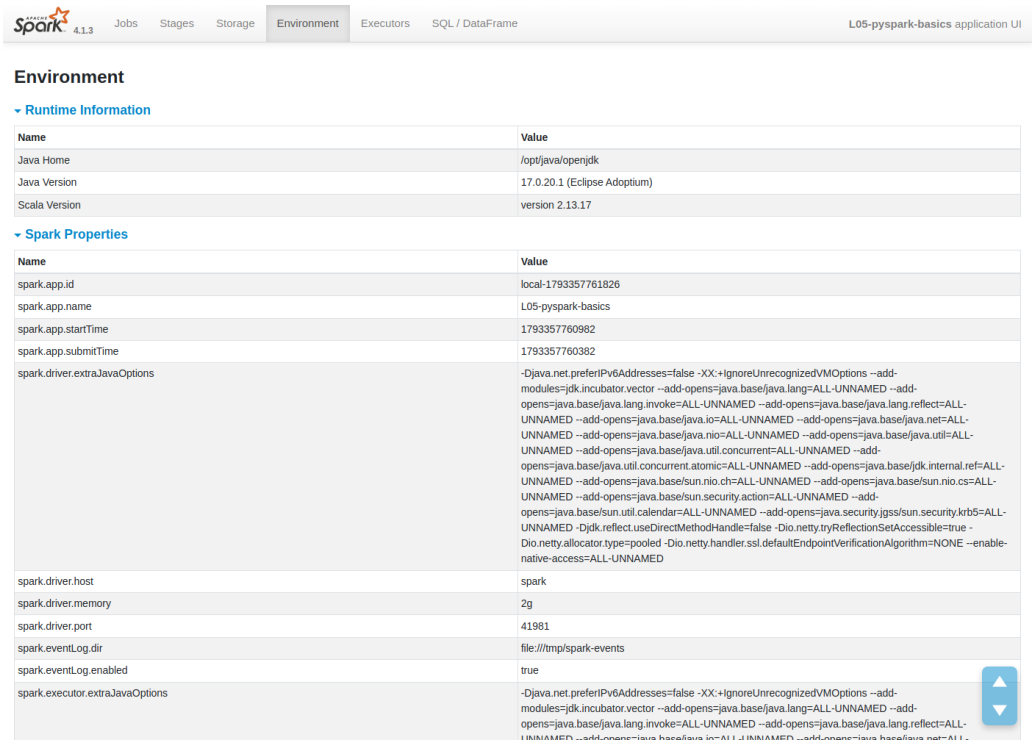
5.3 Môi trường và phương pháp thực nghiệm

5.3.1 Môi trường

Cấu hình ở Bảng 5.1 lấy từ `conf/spark-defaults.conf`, kiểm tra lại trong tab Environment của Spark UI (Hình 5.1).

Bảng 5.1. Môi trường thực nghiệm và cấu hình Spark.

Thành phần	Giá trị
Phiên bản Spark	4.1.3 (image Docker `spark:4.1.3-python3`)
Java / Scala / Python	Java 17 (17.0.20.1, Eclipse Adoptium) / Scala 2.13.17 / Python 3.10
Chế độ chạy	`local[4]`: driver và 4 luồng task trong một JVM
Bộ nhớ driver	2 GB (`spark.driver.memory=2g`)
`spark.sql.shuffle.partitions`	8 (thay cho mặc định 200)
`spark.sql.session.timeZone`	UTC
`spark.sql.ansi.enabled`	`true` (mặc định của Spark 4)
Event log	bật, ghi vào `file:///tmp/spark-events`
Spark UI	cổng 4040, giữ sống bằng tham số `--hold 300`



Hình 5.1. Tab Environment của Spark UI: Java 17.0.20.1, Scala 2.13.17, `spark.driver.memory=2g`, event log được bật.

5.3.2 Dữ liệu

Hai tệp công khai của NYC Taxi and Limousine Commission [8] được mô tả ở Bảng 5.2.

Bảng 5.2. Dữ liệu đầu vào.

Tệp	Định dạng	Kích thước	Số dòng	Số cột
`yellow_tripdata_2024-01.parquet`	Parquet	khoảng 48 MB	2,964,624	19
`taxi_zone_lookup.csv`	CSV có header	12.0 KiB	265	4

Bảng chuyến đi có hai cột thời gian `tpep_pickup_datetime`, `tpep_dropoff_datetime` kiểu `timestamp_ntz`, các cột tiền (`fare_amount`, `tip_amount`, `total_amount`…) kiểu `double` và mã khu vực `PULocationID`, `DOLocationID`. Bảng khu vực gồm `LocationID`, `Borough`, `Zone`, `service_zone`.

5.3.3 Quy trình

Toàn bộ thực nghiệm nằm trong script `l05_basics.py`, chạy bằng `spark-submit` trong container. Các bước:

1. Đọc Parquet và CSV, in schema, số dòng và số partition đầu vào.
2. Dựng ba transformation (`filter`, `withColumn`, `select`) rồi gọi `count()`, đo thời gian từng giai đoạn.
3. Đếm NULL theo cột, đếm số dòng vi phạm từng luật làm sạch trong một lần quét, lọc, điền giá trị mặc định, tính `tip_pct` bằng `F.try_divide` và `cache()` DataFrame `clean`.
4. Join hai lần với bảng khu vực, theo điểm đón và theo điểm trả.
5. Tổng hợp theo quận đón và theo giờ đón.
6. Xếp hạng top-3 khu vực đón mỗi quận bằng `dense_rank`; tính số chuyến theo ngày, chênh lệch với ngày trước (`lag`) và doanh thu lũy kế.
7. Tạo temp view `trips`, viết lại phép tổng hợp theo quận bằng SQL và so sánh với DataFrame.
8. In `by_borough.explain("formatted")`.
9. Ghi `enriched` thành Parquet phân vùng theo `pickup_date`, đếm tệp, đọc lại ngày 2024-01-15.

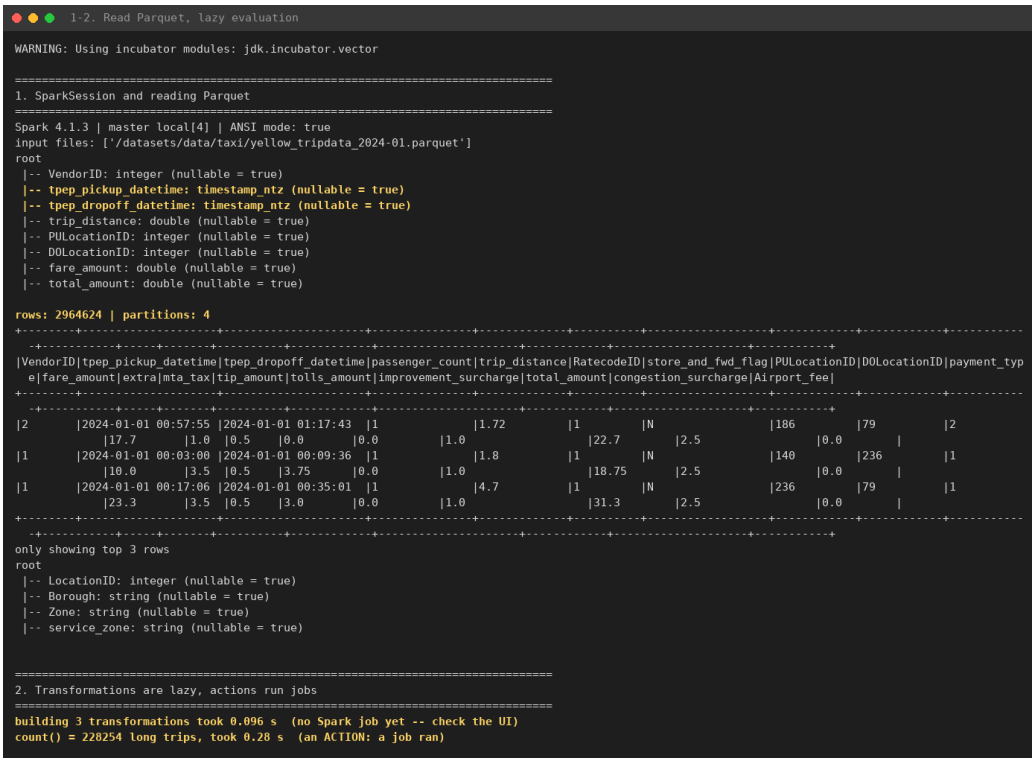
Tham số thay đổi giữa hai lần chạy là có hay không `repartition("pickup_date")` trước khi ghi. Để khỏi sửa mã giữa hai lần, chúng tôi thêm cờ `--no-repartition` vào script. Lần đầu chạy với cờ này. Lần thứ hai (lần chạy chính) dùng `repartition` kèm `--hold 300` để Spark UI còn sống trong lúc chụp màn hình. Hành vi ANSI khi chia cho 0 được kiểm tra riêng bằng một đoạn thử nhỏ.

5.4 Kết quả thực nghiệm

Số liệu dưới đây lấy từ log `03_run_with_repartition.txt` nếu không ghi chú khác. Lần chạy không `repartition` (`01_run_no_repartition.txt`) cho kết quả giống hệt, trừ phần ghi Parquet và thời gian đo.

5.4.1 Đọc dữ liệu và đánh giá trễ

Spark đọc được 19 cột, 2,964,624 dòng, chia thành 4 partition đầu vào (Hình 5.2). Ba transformation ở bước 2 được dựng trong 0.096 s và không sinh job nào. `count()` ngay sau đó trả về 228,254 chuyến dài hơn 10 dặm sau 0.28 s. Ở lần chạy không `repartition`, hai con số là 0.076 s và 0.26 s, cùng bậc độ lớn.



Hình 5.2. Đọc dữ liệu: schema 19 cột, 2,964,624 dòng, 4 partition; dựng ba transformation mất 0.096 s, `count()` mất 0.28 s.

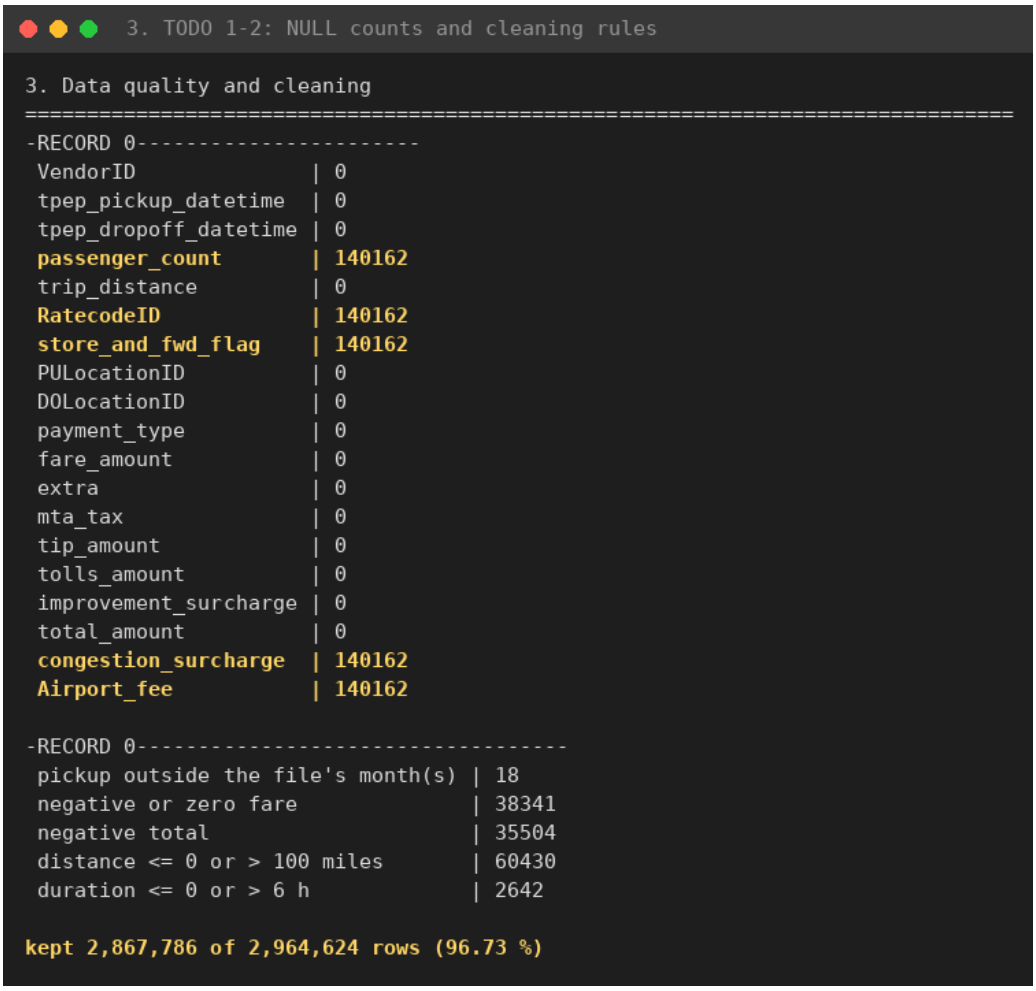
5.4.2 Chất lượng dữ liệu và làm sạch

Số NULL của mọi cột được đếm trong một phép `select` duy nhất. 14 cột không có NULL, còn đúng năm cột `passenger_count`, `RatecodeID`, `store_and_fwd_flag`, `congestion_surcharge`, `Airport_fee` cùng có 140,162 NULL (Hình 5.3). Số dòng vi phạm từng luật được đếm bằng tổng hợp có điều kiện trong một lần quét (Bảng 5.3).

Bảng 5.3. Số dòng vi phạm từng luật làm sạch (một dòng có thể vi phạm nhiều luật).

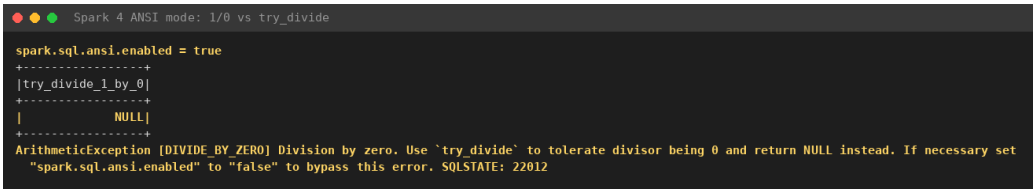
Luật	Số dòng vi phạm
Thời điểm đón nằm ngoài tháng của tệp (2024-01)	18
Giá cước `fare_amount` ≤ 0	38,341
Tổng tiền `total_amount` < 0	35,504
Quãng đường ≤ 0 hoặc > 100 dặm	60,430
Thời lượng ≤ 0 hoặc > 6 giờ	2,642
Giữ lại sau làm sạch	**2,867,786 / 2,964,624 (96.73 %)**

Tổng số dòng bị loại là 96,838, ít hơn tổng cột bên phải vì các luật chồng lên nhau: chuyến có giá cước âm thường cũng có tổng tiền âm. Dòng có NULL không bị loại mà được điền mặc định `passenger_count=1`, `RatecodeID=1`, `congestion_surcharge=0.0`, `Airport_fee=0.0`.



Hình 5.3. Đếm NULL theo cột và số dòng vi phạm từng luật làm sạch; giữ lại 2,867,786 dòng (96.73 %).

Vì Spark 4 bật ANSI mode, phép chia `tip_amount / fare_amount` thông thường sẽ làm hỏng cả job nếu gặp mẫu số bằng 0, nên tỷ lệ tip được tính bằng `F.try_divide`. Đoạn thử riêng (Hình 5.4) xác nhận `spark.sql.ansi.enabled = true`: `try_divide(1, 0)` trả NULL, còn `1/0` ném `ArithmeticException [DIVIDE_BY_ZERO]` với SQLSTATE 22012.



Hình 5.4. Chế độ ANSI: `try_divide` trả NULL, phép chia thường ném lỗi `[DIVIDE_BY_ZERO]`.

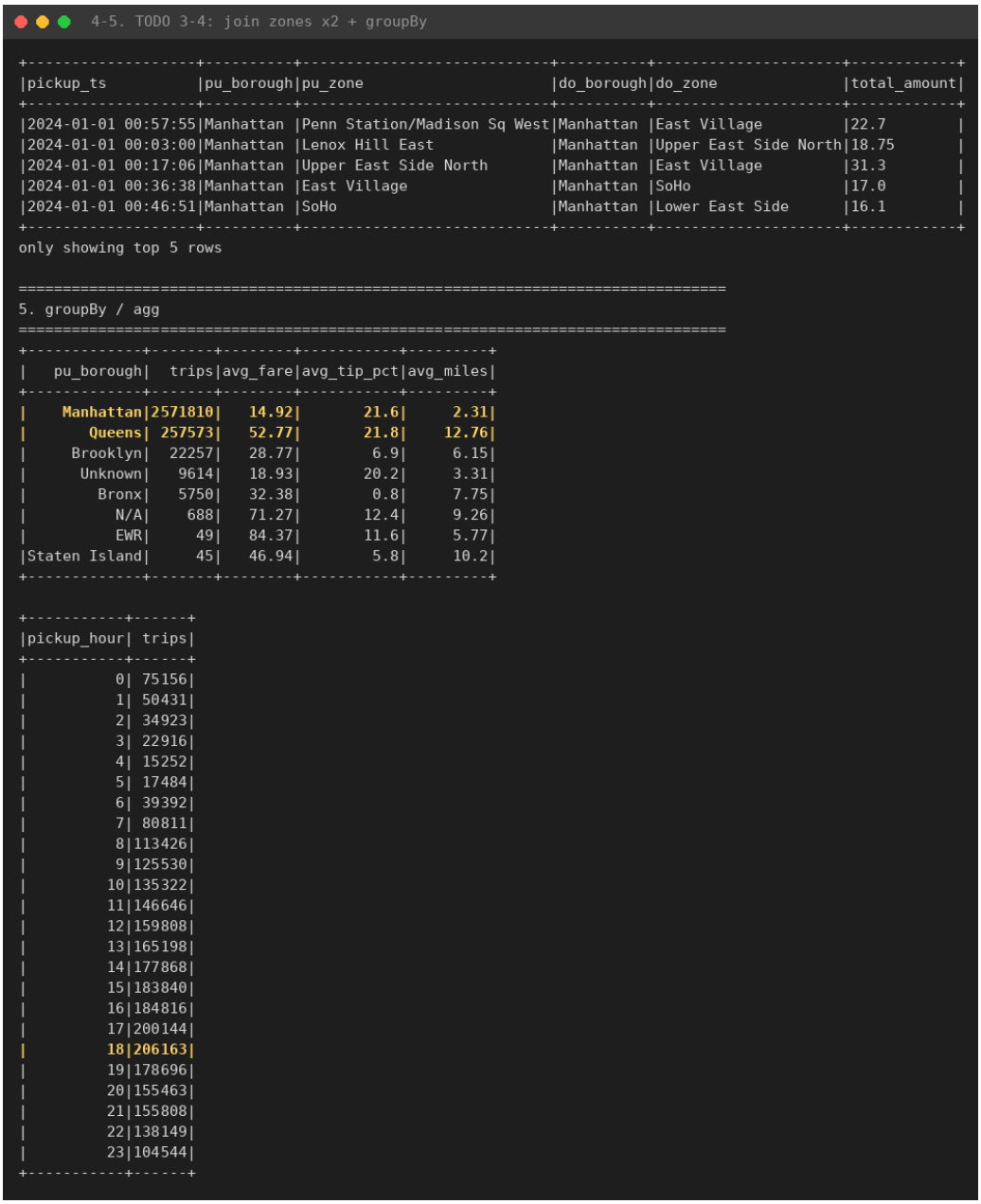
5.4.3 Join với bảng khu vực và tổng hợp theo quận

Trước mỗi lần join, bảng khu vực được đổi tên cột (`PULocationID`/`pu_borough`/`pu_zone` và `DOLocationID`/`do_borough`/`do_zone`). Nếu không đổi tên, hai lần join cùng một bảng sẽ sinh hai cột `Borough` trùng tên và Spark báo `AMBIGUOUS_REFERENCE`. Kết quả join và tổng hợp ở Hình 5.5, số liệu theo quận đón ở Bảng 5.4.

Bảng 5.4. Tổng hợp theo quận đón sau làm sạch (`by_borough`).

Quận đón	Số chuyến	Giá cước TB	Tip TB (%)	Quãng đường TB (dặm)
Manhattan	2,571,810	14.92	21.6	2.31
Queens	257,573	52.77	21.8	12.76
Brooklyn	22,257	28.77	6.9	6.15
Unknown	9,614	18.93	20.2	3.31
Bronx	5,750	32.38	0.8	7.75
N/A	688	71.27	12.4	9.26
EWR	49	84.37	11.6	5.77
Staten Island	45	46.94	5.8	10.2

Manhattan chiếm khoảng 89.7 % số chuyến, toàn chuyến ngắn và rẻ. Giá cước trung bình ở Queens cao gấp khoảng 3.5 lần Manhattan, quãng đường cũng dài hơn nhiều, dễ hiểu vì JFK và LaGuardia đều nằm ở Queens. Tip ở Bronx chỉ 0.8 %; chúng tôi đoán phần lớn tiền boa ở đây trả bằng tiền mặt nên không được ghi nhận, nhưng dữ liệu không đủ để khẳng định. Theo giờ đón, khung 18:00 đông nhất với 206,163 chuyến, khung 04:00 vắng nhất với 15,252 chuyến.



Hình 5.5. Kết quả join hai lần với bảng khu vực, tổng hợp theo quận và theo giờ đón.

5.4.4 Hàm cửa sổ

Top-3 khu vực đón mỗi quận dùng `dense_rank()` trên cửa sổ `partitionBy("pu_borough")`. Bảng 5.5 ghi kết quả cho hai quận lớn nhất. Riêng Staten Island trả về hơn 3 dòng, do `dense_rank` để nhiều khu vực đồng hạng: ba khu vực cùng 4 chuyến ở hạng 2 và bốn khu vực cùng 2 chuyến ở hạng 3.

Bảng 5.5. Top-3 khu vực đón tại Manhattan và Queens.

Quận	Hạng 1	Hạng 2	Hạng 3
Manhattan	Midtown Center (140,065)	Upper East Side South (140,059)	Upper East Side North (133,891)
Queens	JFK Airport (138,296)	LaGuardia Airport (87,657)	East Elmhurst (11,861)

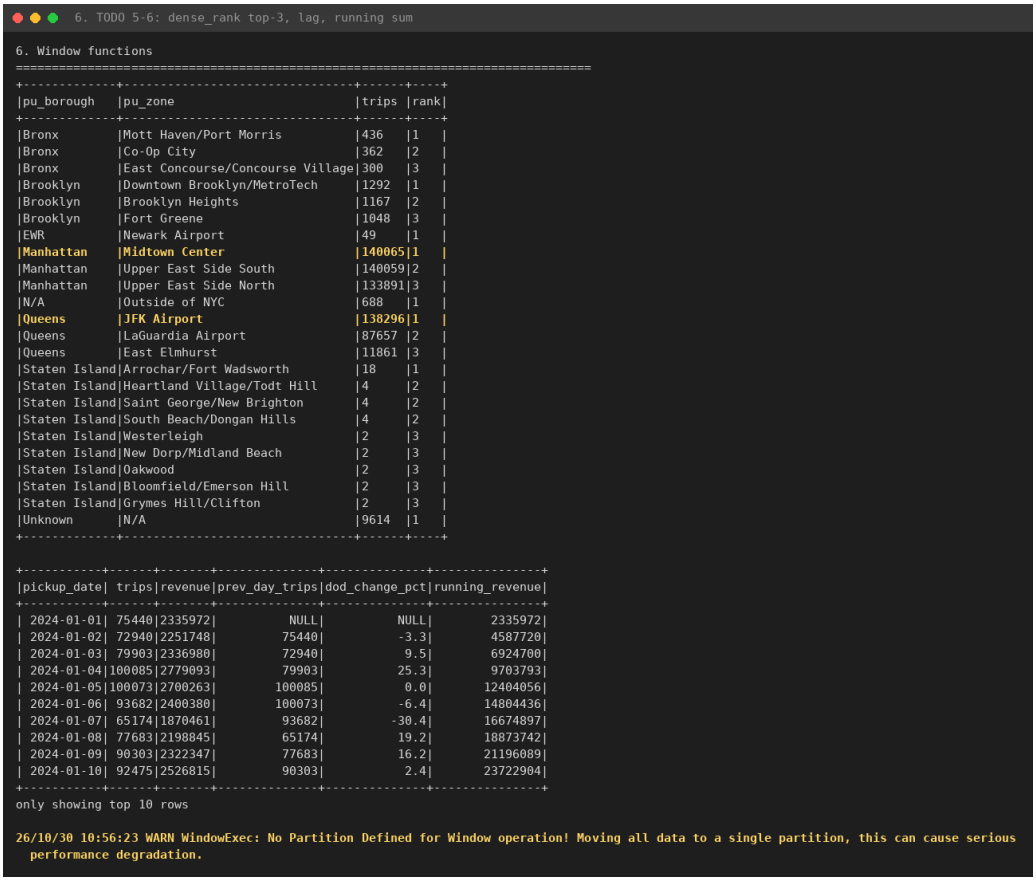
Chuỗi theo ngày dùng `lag` và tổng lũy kế trên `Window.orderBy("pickup_date")`. Bảng 5.6 và Hình 5.6 cho mười ngày đầu. Ngày 2024-01-07 (Chủ nhật) giảm 30.4 % so với hôm trước, còn 2024-01-04 tăng 25.3 %.

Bảng 5.6. Số chuyến và doanh thu theo ngày với `lag` và tổng lũy kế (10 ngày đầu).

Ngày	Số chuyến	Doanh thu	Ngày trước	Thay đổi (%)	Doanh thu lũy kế
2024-01-01	75,440	2,335,972	NULL	NULL	2,335,972
2024-01-02	72,940	2,251,748	75,440	-3.3	4,587,720
2024-01-03	79,903	2,336,980	72,940	9.5	6,924,700
2024-01-04	100,085	2,779,093	79,903	25.3	9,703,793
2024-01-05	100,073	2,700,263	100,085	0.0	12,404,056
2024-01-06	93,682	2,400,380	100,073	-6.4	14,804,436
2024-01-07	65,174	1,870,461	93,682	-30.4	16,674,897
2024-01-08	77,683	2,198,845	65,174	19.2	18,873,742
2024-01-09	90,303	2,322,347	77,683	16.2	21,196,089
2024-01-10	92,475	2,526,815	90,303	2.4	23,722,904

Lúc chạy bước này, log in ra cả chục dòng cảnh báo *“No Partition Defined for Window operation! Moving all data to a single partition”*. Lúc đầu chúng tôi tưởng đó là lỗi, nhưng ở đây cảnh báo vô hại: cửa sổ chỉ áp lên DataFrame `daily` đã tổng hợp còn 31 dòng. Nếu áp

cùng cửa sổ không `partitionBy` lên 2.87 triệu chuyến gốc thì toàn bộ dữ liệu dồn vào một task, mất song song và có thể tràn bộ nhớ.



Hình 5.6. Hàm cửa sổ: top-3 khu vực đón mỗi quận và chuỗi theo ngày với `lag`, tổng lũy kế.

5.4.5 Spark SQL và kế hoạch vật lý

Truy vấn SQL trên temp view `trips` cho đúng bảng ở Bảng 5.4, script in ra `SQL result == DataFrame result: True` (Hình 5.7). Hai API đi qua cùng Catalyst nên sinh cùng kế hoạch vật lý.



Hình 5.7. Kết quả Spark SQL trùng khớp DataFrame và cây kế hoạch vật lý của `by_borough`.

Kế hoạch `explain("formatted")` của `by_borough` (Hình 5.7, Hình 5.8) đọc từ dưới lên như sau. `Scan parquet` (3) mang `PushedFilters: [IsNotNull(tpep_pickup_datetime), IsNotNull(fare_amount), IsNotNull(total_amount), IsNotNull(trip_distance), GreaterThan(fare_amount,0.0), GreaterThanOrEqual(total_amount,0.0), GreaterThan(trip_distance,0.0), LessThanOrEqual(trip_distance,100.0)]`, nghĩa là các luật làm sạch đơn giản đã được đẩy xuống reader Parquet. Luật theo tháng và theo thời lượng cần `date_format`, `unix_timestamp` nên vẫn nằm ở nút `Filter` (5). `InMemoryTableScan` (1) đọc từ cache với `rowCount=2.87E+6`. Hai nhánh `Scan csv` đi qua `BroadcastExchange` (`Statistics(sizeInBytes=1026.1 KiB, rowCount=265)`) vào `BroadcastHashJoin LeftOuter BuildRight` (13) và (20). `HashAggregate` (22) với `partial_count(1)`, `partial_avg(...)` phát ra các cặp sum/count, qua `Exchange hashpartitioning(pu_borough, 8)` (23) có `ShuffleQueryStage` thống kê `sizeInBytes=1968.0 B, rowCount=24`, qua `AQEShuffleRead coalesced` (25) rồi tới `HashAggregate`

(26) final. Cuối cùng là `Exchange rangepartitioning(trips DESC NULLS LAST, 8)` (27) với `rowCount=8`, `AQEShuffleRead` (29) và `Sort` (30) cho `orderBy`.

```
8. partial HashAggregate -> Exchange -> final HashAggregate

(22) HashAggregate [codegen id : 3]
Keys [1]: [pu_borough#1112]
Functions [4]: [partial_count(1), partial_avg(fare_amount#10), partial_avg(tip_pct#337), partial_avg(trip_distance#4)]
Aggregate Attributes [7]: [count#2026L, sum#2027, count#2028L, sum#2029, count#2030L, sum#2031, count#2032L]
Results [8]: [pu_borough#1112, count#2033L, sum#2034, count#2035L, sum#2036, count#2037L, sum#2038, count#2039L]

(23) Exchange
Arguments: hashpartitioning(pu_borough#1112, 8), ENSURE_REQUIREMENTS, [plan_id=2521]

(24) ShuffleQueryStage
Output [8]: [pu_borough#1112, count#2033L, sum#2034, count#2035L, sum#2036, count#2037L, sum#2038, count#2039L]
Arguments: 3

(25) AQEShuffleRead
Arguments: coalesced

(26) HashAggregate [codegen id : 4]
Keys [1]: [pu_borough#1112]
Functions [4]: [count(1), avg(fare_amount#10), avg(tip_pct#337), avg(trip_distance#4)]
Aggregate Attributes [4]: [count(1)#1657L, avg(fare_amount#10)#1658, avg(tip_pct#337)#1659, avg(trip_distance#4)#1660]
Results [5]: [pu_borough#1112, count(1)#1657L AS trips#1625L, round(avg(fare_amount#10)#1658, 2) AS avg_fare#1626, round(avg(tip_pct#337)#1659, 1) AS
  avg_tip_pct#1627, round(avg(trip_distance#4)#1660, 2) AS avg_miles#1628]

(27) Exchange
Arguments: rangepartitioning(trips#1625L DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=2558]

(28) ShuffleQueryStage
Output [5]: [pu_borough#1112, trips#1625L, avg_fare#1626, avg_tip_pct#1627, avg_miles#1628]
Arguments: 4

(29) AQEShuffleRead
Arguments: coalesced

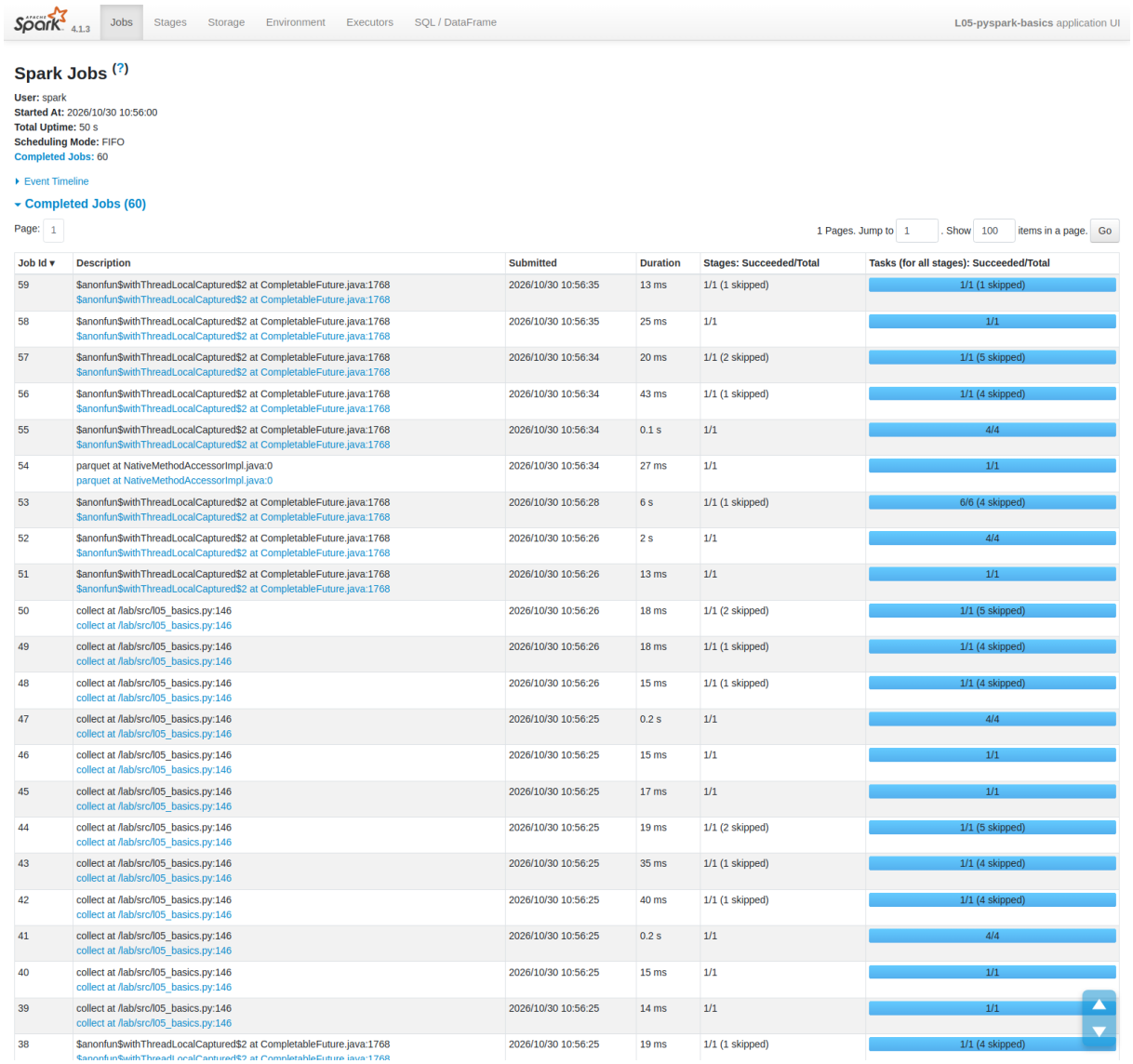
(30) Sort [codegen id : 5]
Arguments: [trips#1625L DESC NULLS LAST], true, 0

(31) ResultQueryStage
Output [5]: [pu_borough#1112, trips#1625L, avg_fare#1626, avg_tip_pct#1627, avg_miles#1628]
Arguments: 5
```

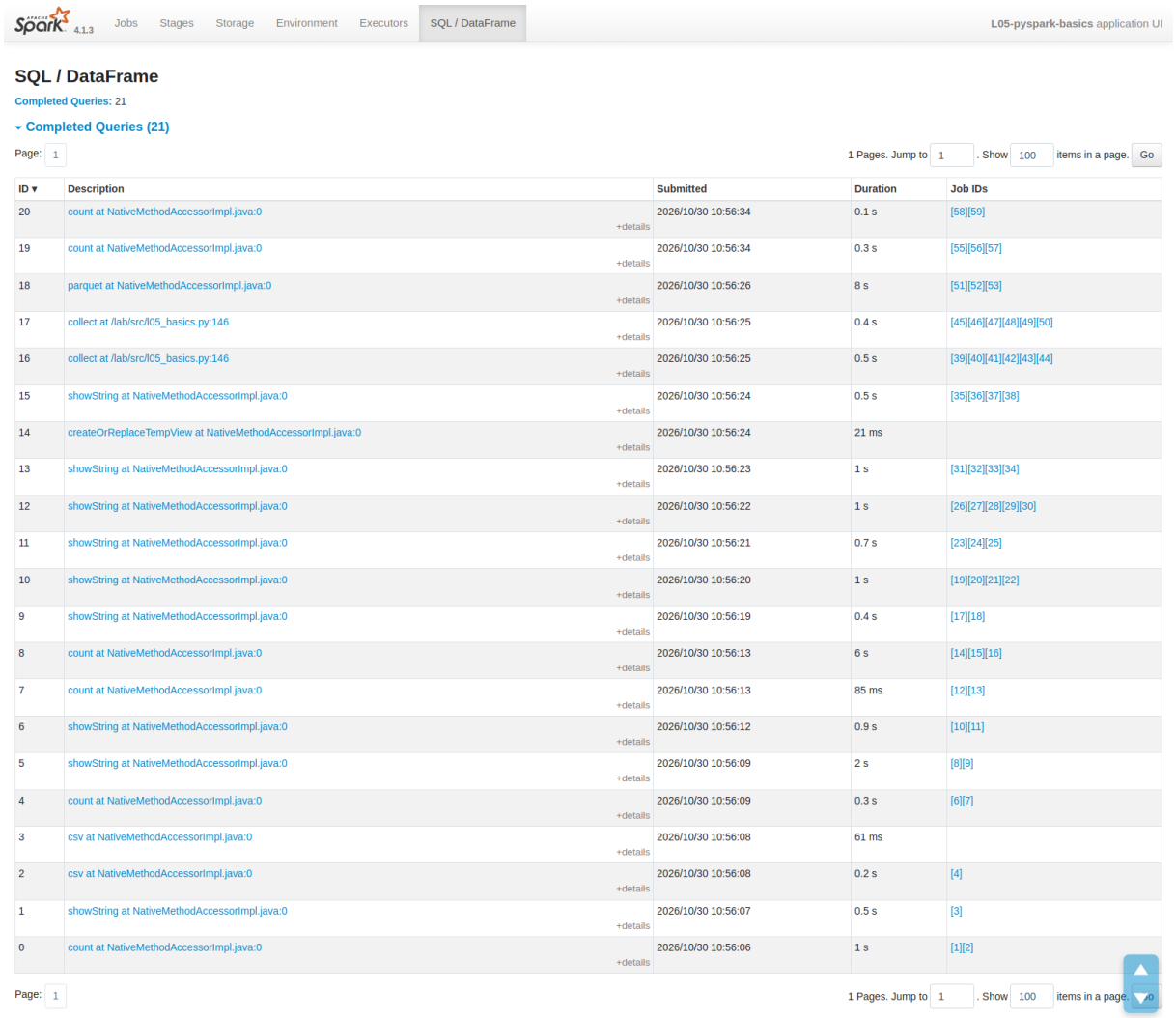
Hình 5.8. Phần kế hoạch thể hiện `HashAggregate` partial, `Exchange hashpartitioning` và `HashAggregate` final.

5.4.6 Spark UI

Trong tab Jobs (Hình 5.9), mỗi action (`count`, `showString`, `collect`, `parquet`) sinh một hoặc nhiều job. Tab SQL/DataFrame (Hình 5.10) liệt kê 21 truy vấn; Bảng 5.7 chọn ra các truy vấn đáng xem theo log `ui_sql_queries.txt`.



Hình 5.9. Tab Jobs: mỗi action sinh một hoặc nhiều job.



Hình 5.10. Tab SQL/DataFrame: 21 truy vấn đã hoàn thành cùng thời lượng và danh sách job.

Bảng 5.7. Một số truy vấn trong tab SQL (log `ui_sql_queries.txt`).

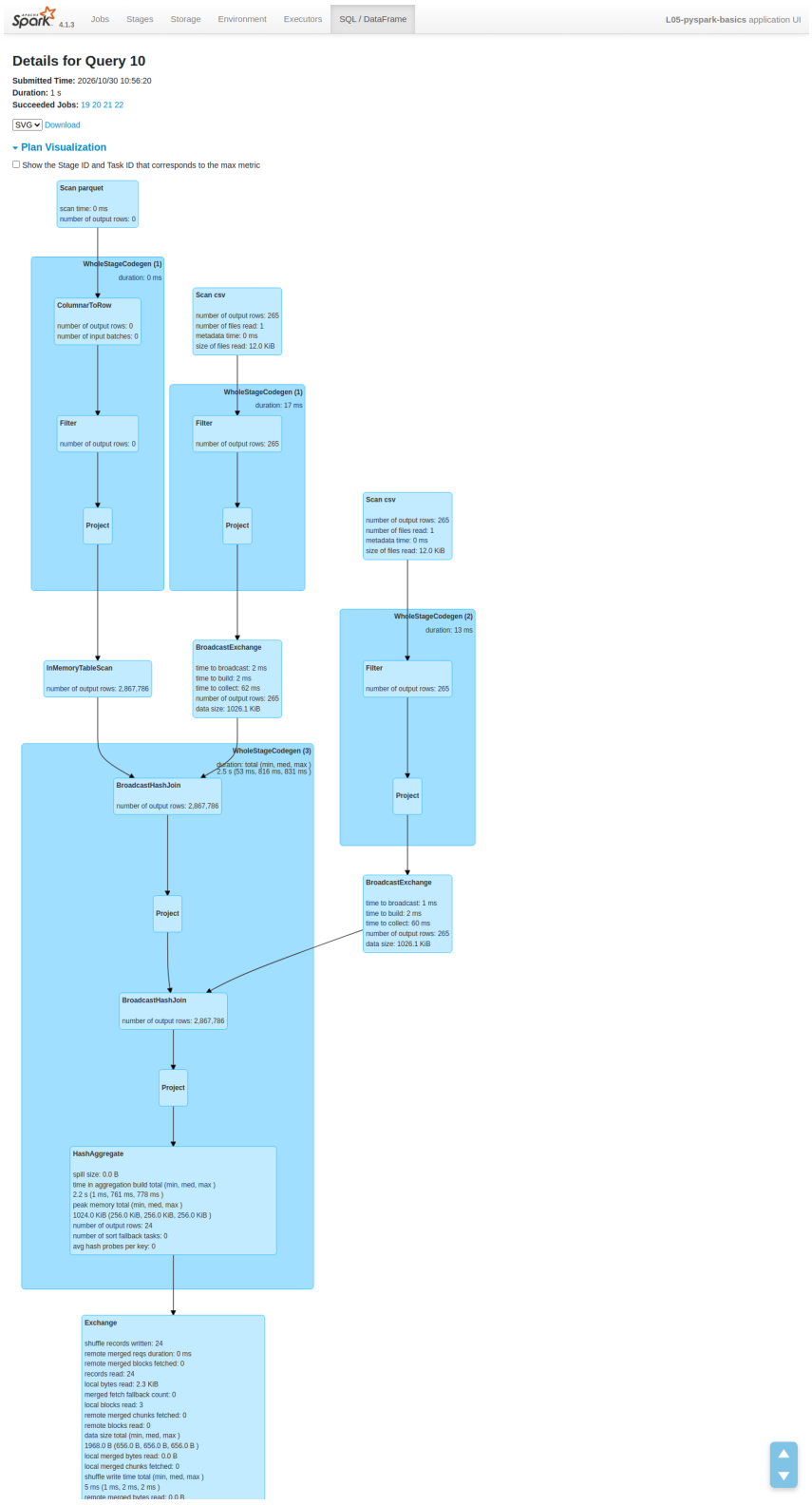
Query	Action	Thời lượng (ms)	Ý nghĩa
0	`count`	1,477	đếm số dòng đầu vào
4	`count`	268	`count()` ở bước 2 (228,254 chuyến dài)
8	`count`	6,225	`clean.count()` lần đầu, materialize cache
10	`showString`	1,473	`by_borough.show()`
16, 17	`collect`	511, 418	so sánh SQL và DataFrame
18	`parquet`	8,127	ghi Parquet phân vùng có `repartition`
19	`count`	336	đọc lại ngày 2024-01-15

Truy vấn 10 (`by_borough.show()`) sinh bốn job 19–22, chi tiết ở Bảng 5.8 (lấy từ `ui_by_borough_jobs.txt` và ảnh chụp job).

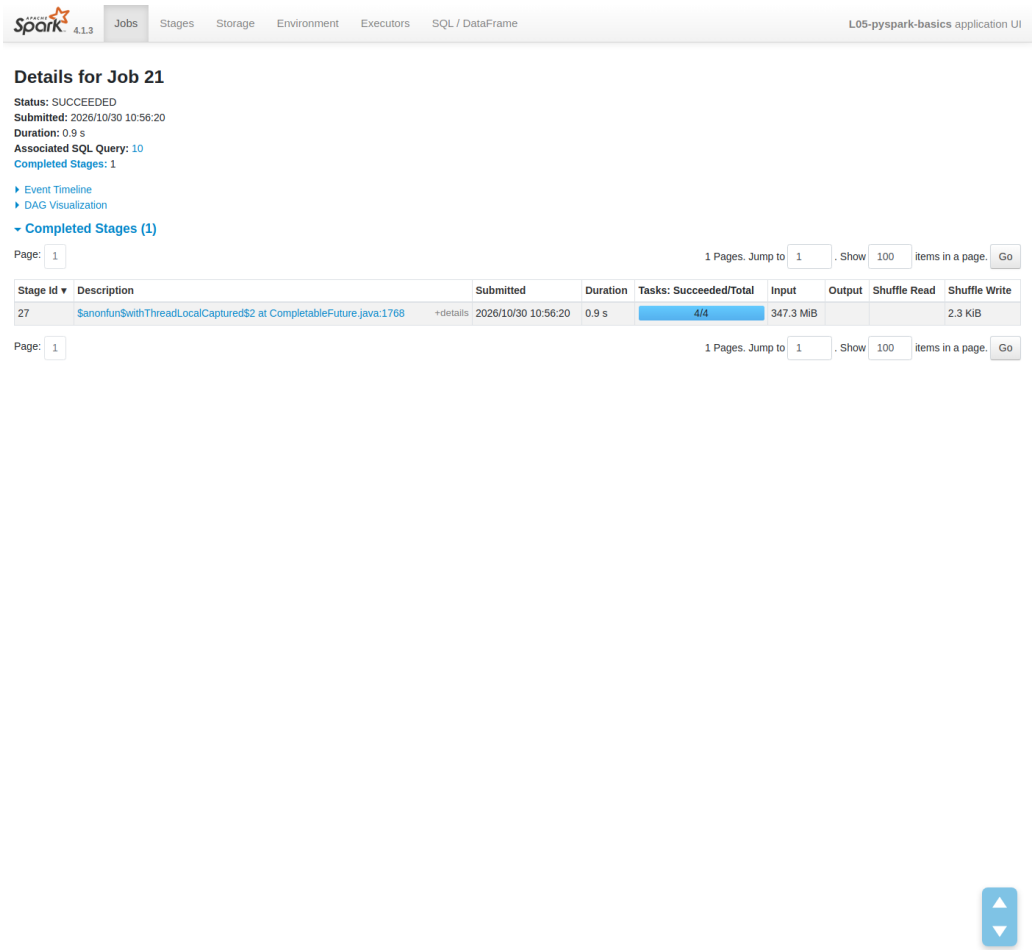
Bảng 5.8. Các job và stage của truy vấn 10 (`by_borough.show()`).

Job	Stage	Số task	Công việc
19	25	1	dựng broadcast bảng khu vực cho điểm đón (`BroadcastExchange`)
20	26	1	dựng broadcast bảng khu vực cho điểm trả
21	27	4	`InMemoryTableScan`, 2 × `BroadcastHashJoin`, `HashAggregate` partial, ghi shuffle
22	28 (skipped), 29	1	`AQEShuffleRead coalesced`, `HashAggregate` final, `TakeOrderedAndProject`

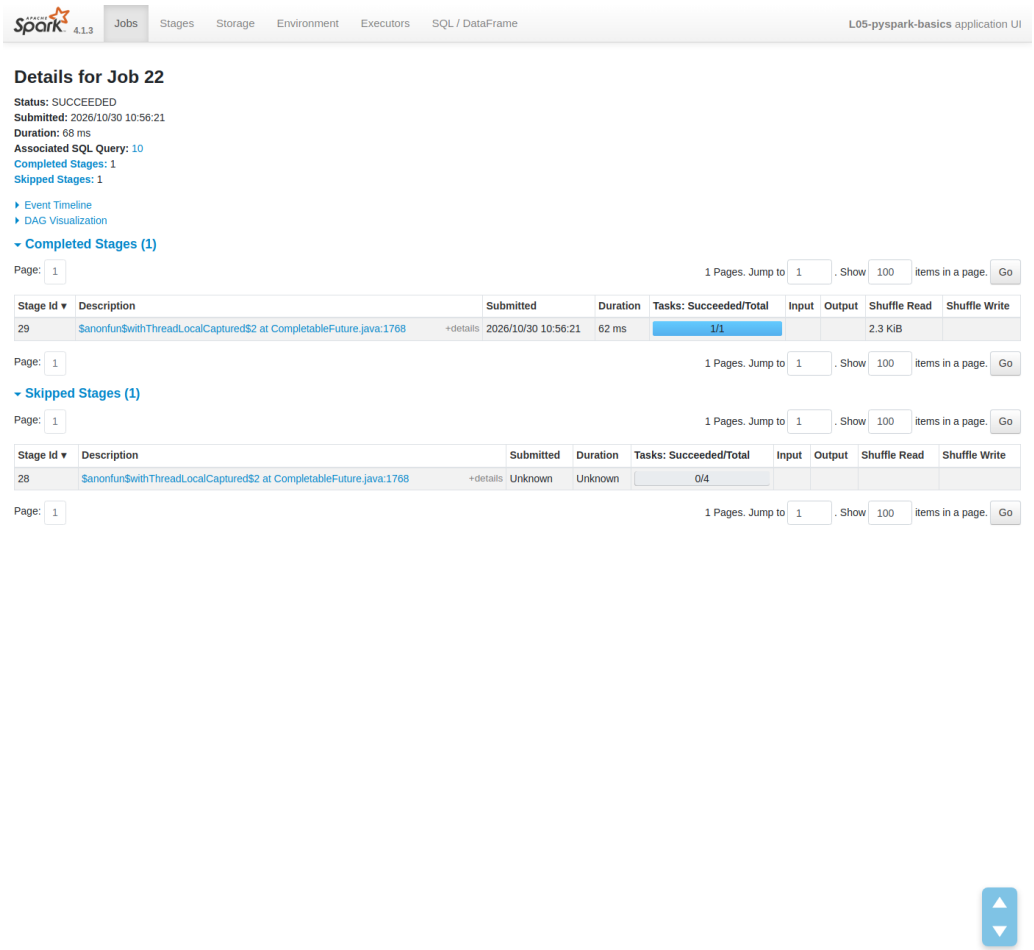
Trên đồ thị của truy vấn 10 (Hình 5.11), `InMemoryTableScan` xuất 2,867,786 dòng, mỗi nhánh `BroadcastExchange` 265 dòng (1026.1 KiB), hai `BroadcastHashJoin` giữ nguyên số dòng, còn `HashAggregate` partial chỉ xuất 24 dòng và `Exchange` ghi 24 bản ghi shuffle (1968.0 B). Job 21 (Hình 5.12) chạy 4/4 task trong 0.9 s, đọc 347.3 MiB từ cache và chỉ ghi 2.3 KiB shuffle. Ở job 22 (Hình 5.13), stage 28 bị skipped (0/4 task, vì output shuffle của stage 27 đã có sẵn), stage 29 chạy 1 task trong 62 ms với Shuffle Read 2.3 KiB.



Hình 5.11. Đồ thị kế hoạch truy vấn 10 trong tab SQL: 2,867,786 dòng vào, 24 dòng ra khỏi `HashAggregate` partial.

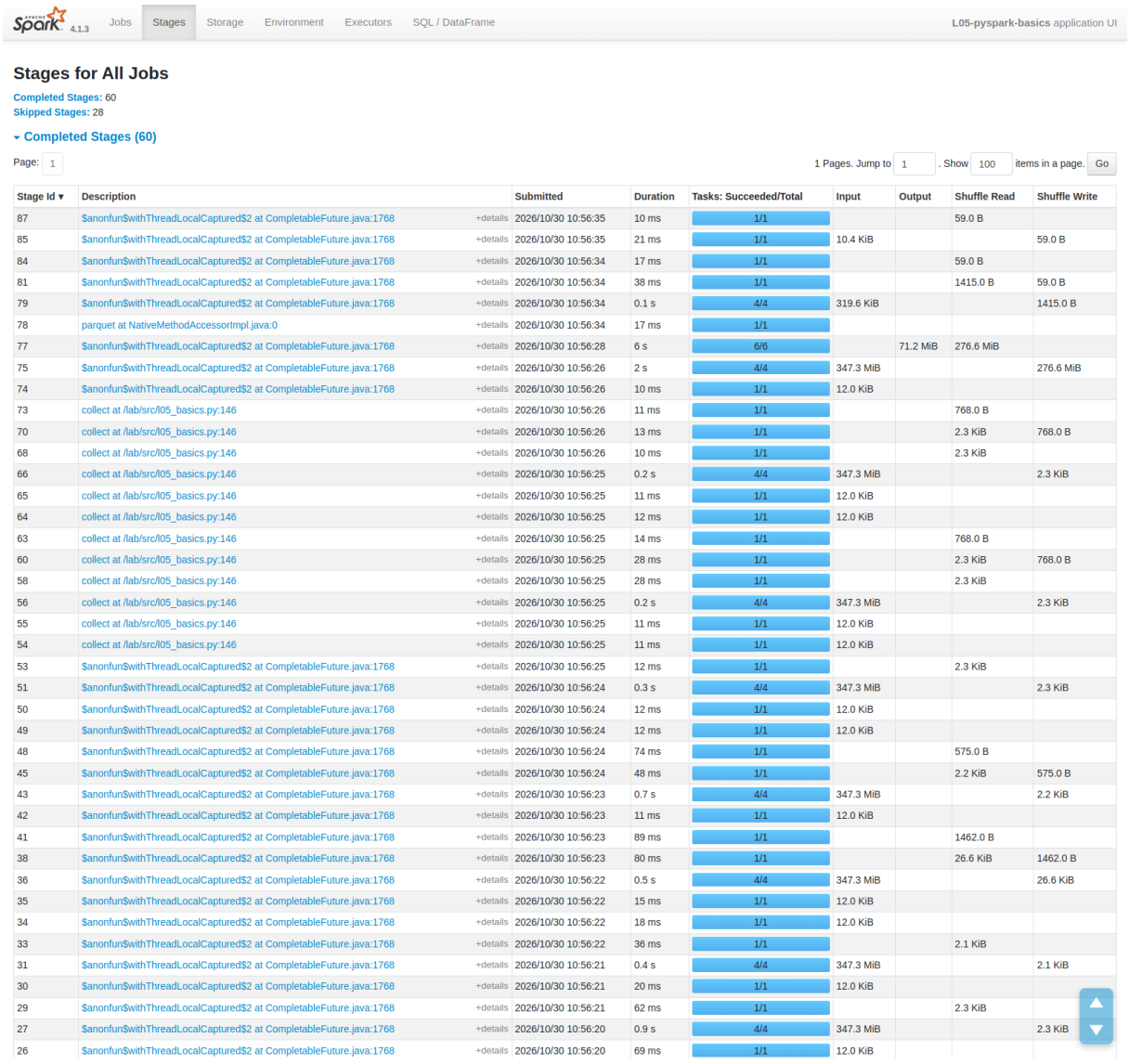


Hình 5.12. Job 21: một stage, 4/4 task, Input 347.3 MiB từ cache, Shuffle Write 2.3 KiB.



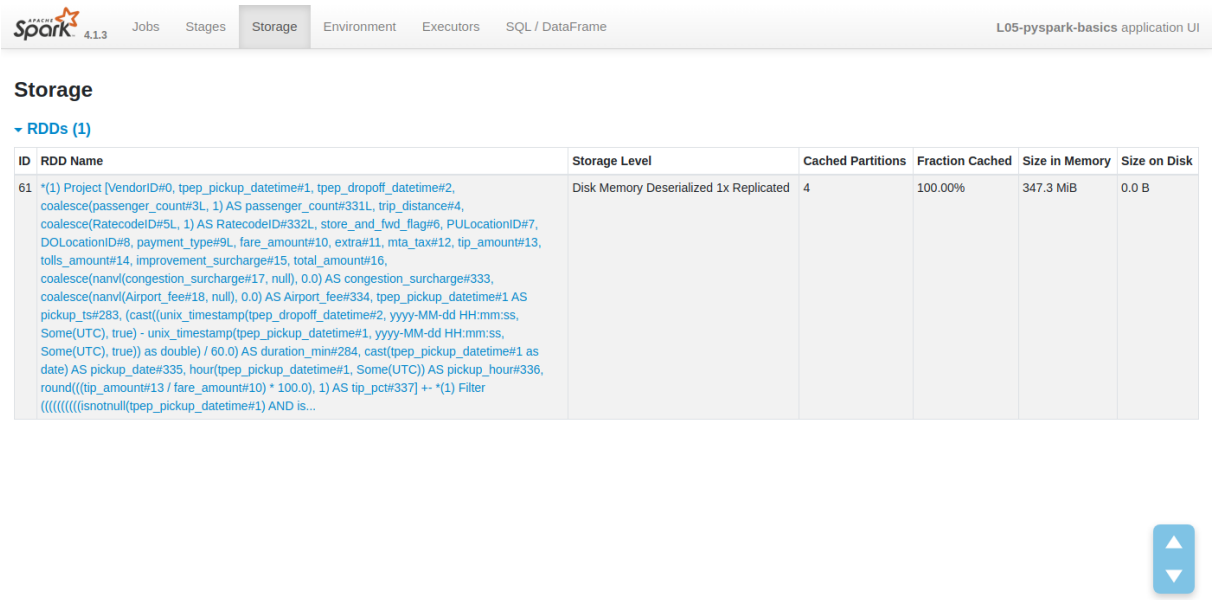
Hình 5.13. Job 22: stage 28 bị bỏ qua, stage 29 chạy 1 task với Shuffle Read 2.3 KiB.

Tab Stages (Hình 5.14) có 60 stage hoàn thành và 28 stage bị bỏ qua. Các stage 4 task đọc 347.3 MiB từ cache, các stage 1 task đọc 12.0 KiB từ tệp CSV khu vực. Tab Storage (Hình 5.15) cho thấy `clean` được cache ở mức *Disk Memory Deserialized 1x Replicated*, 4/4 partition (100 %), 347.3 MiB trong bộ nhớ, 0.0 B trên đĩa.



Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
87	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:35	10 ms	1/1			59.0 B	
85	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:35	21 ms	1/1	10.4 KiB			59.0 B
84	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:34	17 ms	1/1			59.0 B	
81	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:34	38 ms	1/1			1415.0 B	59.0 B
79	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:34	0.1 s	4/4	319.6 KiB			1415.0 B
78	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:56:34	17 ms	1/1				
77	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:28	6 s	6/6		71.2 MiB	276.6 MiB	
75	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:26	2 s	4/4	347.3 MiB			276.6 MiB
74	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:26	10 ms	1/1	12.0 KiB			
73	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:26	11 ms	1/1			768.0 B	
70	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:26	13 ms	1/1			2.3 KiB	768.0 B
68	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:26	10 ms	1/1			2.3 KiB	
66	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	0.2 s	4/4	347.3 MiB			2.3 KiB
65	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	11 ms	1/1	12.0 KiB			
64	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	12 ms	1/1	12.0 KiB			
63	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	14 ms	1/1			768.0 B	
60	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	28 ms	1/1			2.3 KiB	768.0 B
58	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	28 ms	1/1			2.3 KiB	
56	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	0.2 s	4/4	347.3 MiB			2.3 KiB
55	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	11 ms	1/1	12.0 KiB			
54	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	11 ms	1/1	12.0 KiB			
53	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:25	12 ms	1/1			2.3 KiB	
51	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:24	0.3 s	4/4	347.3 MiB			2.3 KiB
50	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:24	12 ms	1/1	12.0 KiB			
49	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:24	12 ms	1/1	12.0 KiB			
48	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:24	74 ms	1/1			575.0 B	
45	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:24	48 ms	1/1			2.2 KiB	575.0 B
43	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:23	0.7 s	4/4	347.3 MiB			2.2 KiB
42	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:23	11 ms	1/1	12.0 KiB			
41	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:23	89 ms	1/1			1462.0 B	
38	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:23	80 ms	1/1			26.6 KiB	1462.0 B
36	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:22	0.5 s	4/4	347.3 MiB			26.6 KiB
35	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:22	15 ms	1/1	12.0 KiB			
34	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:22	18 ms	1/1	12.0 KiB			
33	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:22	36 ms	1/1			2.1 KiB	
31	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:21	0.4 s	4/4	347.3 MiB			2.1 KiB
30	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:21	20 ms	1/1	12.0 KiB			
29	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:21	62 ms	1/1			2.3 KiB	
27	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:20	0.9 s	4/4	347.3 MiB			2.3 KiB
26	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:20	69 ms	1/1	12.0 KiB			

Hình 5.14. Tab Stages: 60 stage hoàn thành, 28 stage bị bỏ qua, kèm Input và Shuffle Read/Write.



Hình 5.15. Tab Storage: `clean` được cache 4/4 partition, 100 %, 347.3 MiB trong bộ nhớ.

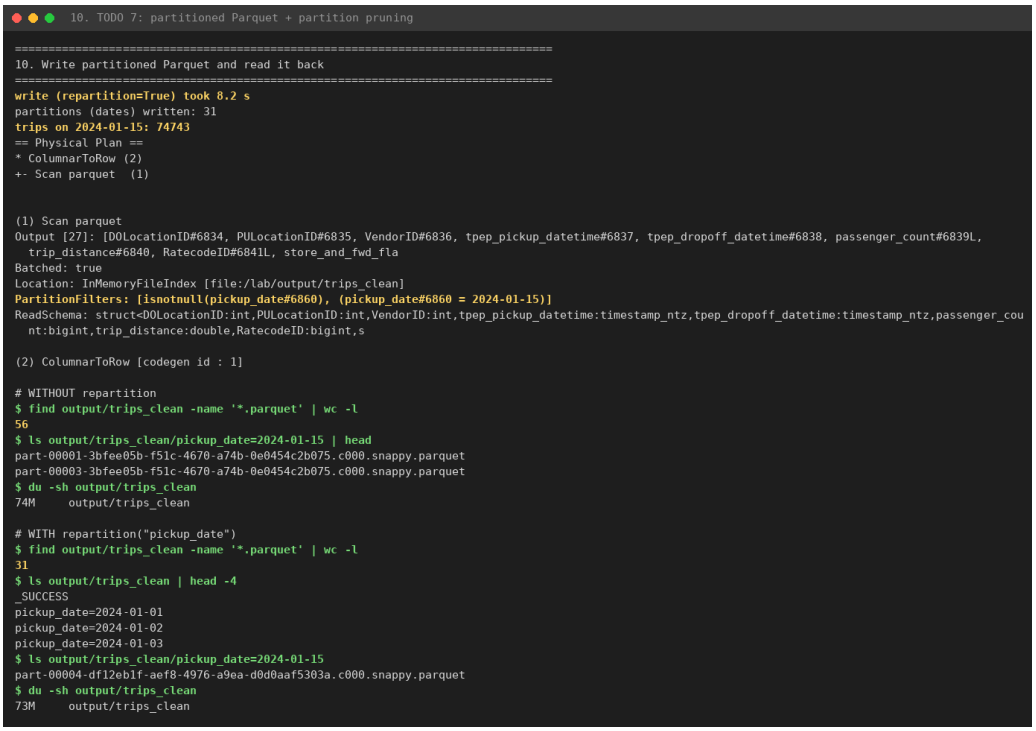
5.4.7 Ghi Parquet phân vùng và partition pruning

Bảng 5.9 so sánh hai cách ghi, dựa trên `02_files_no_repartition.txt`, `04_files_with_repartition.txt` và dòng thời gian ghi trong hai log chạy.

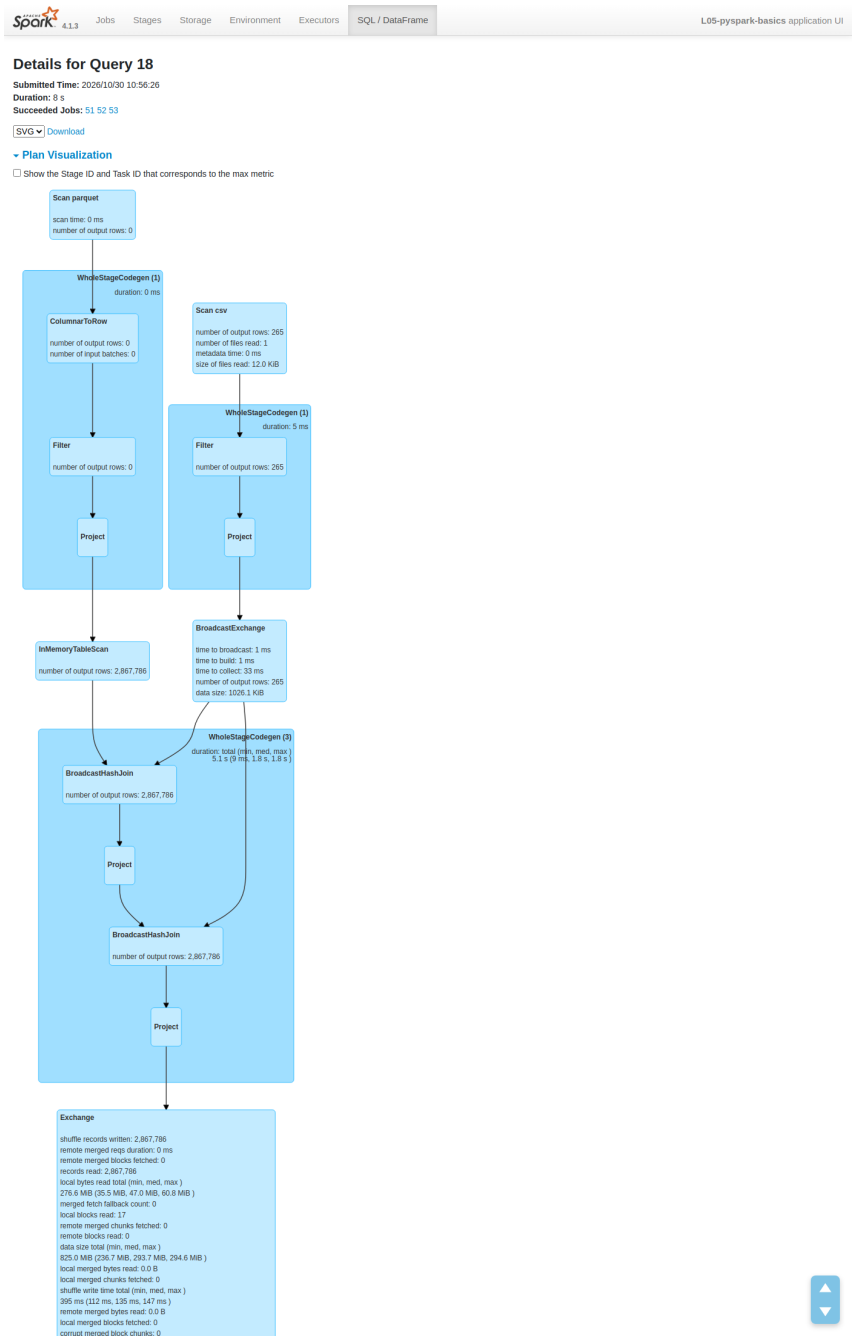
Bảng 5.9. So sánh ghi Parquet phân vùng theo ngày có và không có `repartition`.

Chỉ số	Không `repartition`	`repartition("pickup_date")`
Số tệp `.parquet`	56	31 (1 tệp/ngày)
Số tệp trong `pickup_date=2024-01-15`	2 (`part-00001`, `part-00003`)	1 (`part-00004`)
Dung lượng thư mục output	74 MB	73 MB
Thời gian ghi	6.8 s	8.2 s
Tổng thời gian chạy script	35.70 s	không so sánh được (có `--hold 300`)

Cả hai lần đều ghi ra 31 thư mục ngày. Đọc lại với `filter(pickup_date == '2024-01-15')` trả về 74,743 chuyến, và nút `Scan parquet` có `PartitionFilters: [isnotnull(pickup_date), (pickup_date = 2024-01-15)]` (Hình 5.16), tức chỉ một thư mục được mở. Ở đồ thị truy vấn 18 (Hình 5.17), `repartition` thêm một `Exchange` mà cả 2,867,786 dòng phải đi qua. Trong tab Stages, stage 75 ghi 276.6 MiB shuffle, stage 77 (6 task sau khi AQE gộp partition) đọc lại lượng đó và xuất 71.2 MiB Parquet (Hình 5.14).



Hình 5.16. Ghi Parquet phân vùng, đọc lại ngày 2024-01-15 (74,743 chuyến) và `PartitionFilters` trong kế hoạch.



Hình 5.17. Đồ thị truy vấn 18 (ghi Parquet): `Exchange` do `repartition` shuffle toàn bộ 2,867,786 dòng.

5.5 Thảo luận

5.5.1 Vì sao tệp 48 MB lại có 4 partition đầu vào?

Spark cắt tệp thành các split có kích thước tối đa maxSplitBytes $=$ min(maxPartitionBytes, max(openCostIn
với `spark.sql.files.maxPartitionBytes` mặc định 128 MB, `openCostInBytes` mặc định 4 MB, còn `bytesPerCore` là tổng dung lượng (cộng chi phí mở tệp) chia cho `defaultParallelism`. Ở `local[4]`, `defaultParallelism` bằng 4 nên split bị hạ xuống khoảng 48 MB / 4 thay vì 128 MB, ra 4 split và 4 partition (Hình 5.2). Có điều Parquet chỉ cắt

được ở ranh giới row group: task nào nhận điểm bắt đầu của row group thì đọc cả row group đó. Chúng tôi không tự kiểm tra số row group, nhưng theo đáp án tham khảo của bài lab tệp này có 3 row group, nên một partition có thể rỗng. Quan sát ở Bảng 5.9 khớp với điều này: các tệp của ngày 2024-01-15 mang số `part-00001` và `part-00003`, không có task 2.

5.5.2 Dòng nào là transformation, dòng nào là action, và sao dựng plan nhanh vậy?

`filter`, `withColumn`, `select` là transformation. Chúng chỉ thêm nút vào kế hoạch logic, không chạm dữ liệu, nên mất 0.096 s và tab Jobs không có job mới. `count()` là action: Catalyst phân tích và tối ưu kế hoạch, DAG scheduler tạo stage và task, rồi mới đọc tệp. Thời gian 0.28 s khớp với truy vấn 4 (268 ms) trong Bảng 5.7. `show`, `collect`, `take`, `write` cũng là action. Dựng plan nhanh vì đó chỉ là thao tác trên cây biểu thức trong bộ nhớ driver, không có I/O và không cần liên lạc với executor.

5.5.3 `by_borough.show()` sinh bao nhiêu job, stage, và stage bị cắt ở đâu?

Theo Bảng 5.8, truy vấn 10 sinh **4 job (19–22) với 5 stage (25–29)**, trong đó stage 28 bị bỏ qua nên chỉ 4 stage thực sự chạy. Job 19 và 20 mỗi job 1 task, dựng hai bảng broadcast. Job 21 chạy 4 task, mỗi task một partition của cache, làm quét cache, hai broadcast join, tổng hợp partial và ghi shuffle. Job 22 đọc shuffle đã được AQE gộp từ 8 xuống 1 partition, tổng hợp final và lấy kết quả có thứ tự cho `show()`.

Ranh giới stage nằm ở các `Exchange`: giữa stage 27 và stage 29 là `Exchange hashpartitioning(pu_borough, 8)`. Broadcast phải chạy thành job riêng vì `BroadcastExchange` cần thu dữ liệu về driver rồi phát lại trước khi phía lớn join được. Với AQE, mỗi query stage được nộp như một job riêng để Spark tối ưu lại khi đã có thống kê thật, vì thế stage 28 (bản sao của stage 27 trong DAG của job 22) hiện là skipped (Hình 5.13). Với `show()`, Spark dùng `TakeOrderedAndProject` thay cho sort toàn cục. Nếu tính trọn `by_borough` (ví dụ `collect`), kế hoạch có thêm `Exchange rangepartitioning(trips DESC, 8)` và `Sort`, tức thêm một stage, như trong `explain` (Hình 5.7). Trước khi gọi `show()`, các phép join và `groupBy` ở bước 4–5 chưa chạy gì, chúng chỉ dựng DAG.

5.5.4 Vì sao năm cột có cùng 140,162 NULL, và nên xử lý thế nào?

Năm cột trùng số NULL (Hình 5.3) vì đó là cùng 140,162 chuyến: các bản ghi này thiếu cả nhóm trường metadata của chuyến (thường từ một nhà cung cấp, hoặc chuyến đón vẫy tay ngoài đường), chứ không phải năm lỗi độc lập. Nếu độc lập thì gần như không thể trùng số tuyệt đối ở cả năm cột. Có ba cách xử lý: (i) giữ dòng và điền mặc định như script đã làm (`passenger_count=1`, `RatecodeID=1`, phụ phí bằng 0); (ii) giữ NULL và chỉ bỏ các dòng này khỏi những chỉ số cần đến các cột đó; (iii) tách sang vùng quarantine để kiểm tra sau. Cách nào thì cũng nên

thống nhất với chủ dữ liệu và ghi vào tài liệu pipeline. Chúng tôi chọn (i) cho đơn giản, nhưng điền `passenger_count=1` có thể làm lệch chỉ số số khách trung bình nếu sau này cần tới.

5.5.5 `HashAggregate` partial trước `Exchange` làm gì, ứng với khái niệm nào ở L02?

`HashAggregate` (22) trước `Exchange` là pha partial: mỗi task gộp trước các dòng của mình theo `pu_borough`. Hàm `avg` bị tách thành cặp `(sum, count)` (thấy trong `Aggregate Attributes` gồm `count#...`, `sum#...`), vì trung bình của các trung bình không bằng trung bình chung, còn tổng và số đếm thì cộng dồn được. Mỗi task chỉ phát một dòng cho mỗi quận nó gặp, nên 4 task cộng lại chỉ có **24 dòng** qua shuffle (1968.0 B, Hình 5.11) thay vì 2,867,786 dòng. Sau `Exchange hashpartitioning(pu_borough, 8)`, cũng là ranh giới stage, `AQEShuffleRead coalesced` gộp các partition nhỏ, rồi `HashAggregate` (26) final dùng `count(1), avg(...)` để hợp nhất các kết quả partial.

Cơ chế partial/final này là combiner của MapReduce ở L02 [7]: combiner chạy phía map để giảm dữ liệu trung gian trước khi gửi sang reducer. Việc tách `avg` thành `(sum, count)` giống cách làm ở bài mở rộng L02 (combiner tính giá cước trung bình), nơi combiner không được phát ra trung bình mà phải phát cặp tổng và số đếm. Lúc làm L02 chúng tôi phải tự nghĩ ra cặp này; ở Spark, Catalyst tự làm.

5.5.6 Vì sao bảng khu vực được broadcast, và khi nào Spark chuyển sang sort-merge join?

Bảng khu vực chỉ có 265 dòng (tệp 12 KB, khoảng 1 MB trong bộ nhớ theo `Statistics(sizeInBytes= 1026.1 KiB)`), nhỏ hơn nhiều so với ngưỡng `spark.sql.autoBroadcastJoinThreshold` mặc định 10 MB. Spark gửi một bản sao tới mọi task nên 2,867,786 chuyến không phải shuffle; trên Hình 5.11, `BroadcastHashJoin` nằm chung một khối WholeStageCodegen với phép quét cache. Khi cả hai phía đều vượt ngưỡng, khi ngưỡng đặt bằng -1, hoặc khi thống kê ước lượng vượt ngưỡng, Spark shuffle cả hai phía theo khóa join và dùng SortMergeJoin. Bài L06 sẽ đo trực tiếp chênh lệch giữa hai chiến lược.

5.5.7 `cache()` thay đổi gì trong plan và UI, và khi nào không nên cache?

Sau `cache()`, kế hoạch đọc từ `InMemoryRelation`/`InMemoryTableScan` thay vì quét và lọc lại Parquet. Nhánh `Scan parquet` vẫn còn trên đồ thị nhưng ra 0 dòng (Hình 5.11). Tab Storage hiện RDD được cache (Hình 5.15). Theo Bảng 5.7, `clean.count()` lần đầu (truy vấn 8) mất 6,225 ms vì phải materialize cache, còn các truy vấn dùng cache sau đó (9–17) chỉ mất 396 ms đến 1,473 ms.

Không nên cache khi dữ liệu chỉ dùng một lần, hoặc khi nó không vừa bộ nhớ và gây spill, eviction, áp lực GC. Ở đây cache chiếm 347 MiB so với khoảng 48 MB Parquet nén, gấp khoảng

7 lần, nên với dữ liệu cả năm thì driver 2 GB khó mà chứa nổi. Cache cũng có thể thừa khi nguồn đọc vốn đã rẻ: Parquet có column pruning đôi khi nhanh hơn một cache dạng hàng rộng. Cuối cùng, quên `unpersist()` thì bộ nhớ bị giữ vô ích.

5.5.8 Vì sao cần `repartition("pickup_date")` trước `partitionBy("pickup_date")`?

`partitionBy` ghi một tệp cho mỗi cặp (task, giá trị phân vùng) mà task đó chứa. Không `repartition` thì mỗi task có thể chứa dòng của nhiều ngày, số tệp tối đa là $N \times 31$ với N task, ở đây là $4 \times 31 = 124$. Chúng tôi đo được 56 tệp (Bảng 5.9), ít hơn giới hạn khá nhiều. Lý do là tệp nguồn gần như đã sắp theo thời gian, mỗi task chỉ chứa một dải ngày liền nhau và chỉ vài ngày bị tách giữa hai task (như 2024-01-15 có `part-00001` và `part-00003`). Với dữ liệu không có thứ tự và hàng trăm task trên cluster thật, con số sẽ lên tới hàng nghìn tệp nhỏ, gây áp lực cho NameNode và làm chậm việc đọc.

`repartition("pickup_date")` shuffle trước để mọi dòng của một ngày về cùng một task, nên ra **đúng 31 tệp, mỗi ngày một tệp**. Cái giá là một lần shuffle toàn bộ dữ liệu (276.6 MiB, Hình 5.17), thời gian ghi tăng từ 6.8 s lên 8.2 s. Với dữ liệu lệch (skew), một ngày quá lớn sẽ dồn vào một task và sinh một tệp khổng lồ; khi đó có thể dùng `repartition(n, "pickup_date", salt)` hoặc `maxRecordsPerFile`.

5.5.9 Hạn chế của thực nghiệm

Mọi thứ chạy ở `local[4]` trên một máy, không có độ trễ mạng thật, shuffle chỉ là ghi và đọc đĩa cục bộ. Dữ liệu chỉ một tháng (khoảng 48 MB), nên các khác biệt dưới một giây bị nhiễu khởi động JVM ảnh hưởng nhiều. Mỗi cấu hình chỉ chạy một lần, nên chênh lệch 6.8 s và 8.2 s mới là một mẫu, chưa đủ để kết luận chắc về chi phí của `repartition`. Lần chạy chính kết thúc với mã 137 vì tiến trình bị dừng trong lúc `--hold 300`, khi mọi kết quả đã in xong, nên số liệu không bị ảnh hưởng; cũng vì `--hold` mà tổng thời gian của lần chạy này (WALL 213.90 s) không dùng để so sánh được.

5.6 Kết luận

Script chạy đủ các bước và cho đúng các số liệu tham khảo: 2,964,624 dòng trong 4 partition, 228,254 chuyến dài, 2,867,786 dòng giữ lại sau làm sạch, Manhattan 2,571,810 chuyến, giờ cao điểm 18:00, 31 phân vùng ngày và 74,743 chuyến ngày 2024-01-15. Qua bài này chúng tôi thấy được trên log và UI những điều trước đó chỉ đọc trong sách: transformation chỉ dựng kế hoạch, action mới chạy job; stage bị cắt ở `Exchange`; tổng hợp hai pha đưa lượng dữ liệu shuffle từ 2,867,786 xuống 24 dòng giống combiner của MapReduce; bảng nhỏ được broadcast tự động; DataFrame và SQL cho cùng kết quả vì dùng chung Catalyst.

Về thực hành, với Spark 4 nên dùng `try_divide`/`try_cast` ngay từ đầu, đặt `spark.sql.shuffle.partitions` theo kích thước dữ liệu, chỉ `cache()` thứ được dùng lại nhiều lần và `repartition` theo cột phân vùng trước khi ghi. Nếu làm tiếp, chúng tôi muốn thử dữ liệu nhiều tháng với thêm cấp phân vùng `pickup_month`, so sánh UDF Python với hàm dựng sẵn, và dùng History Server để xem lại ứng dụng đã kết thúc thay vì phải giữ UI bằng `--hold`.

Tài liệu tham khảo

[1] M. Zaharia *et al.*, "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," in *Proc. 9th USENIX NSDI*, 2012.

[2] M. Armbrust *et al.*, "Spark SQL: Relational Data Processing in Spark," in *Proc. ACM SIGMOD*, 2015.

[3] B. Chambers and M. Zaharia, *Spark: The Definitive Guide*. Sebastopol, CA: O'Reilly Media, 2018.

[4] Apache Software Foundation, "Spark SQL, DataFrames and Datasets Guide," Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-programming-guide.html

[5] Apache Software Foundation, "Performance Tuning," Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-performance-tuning.html

[6] Apache Software Foundation, "ANSI Compliance," Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-ref-ansi-compliance.html

[7] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," in *Proc. 6th USENIX OSDI*, 2004.

[8] NYC Taxi and Limousine Commission, "TLC Trip Record Data." https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Phụ lục A: Danh mục tệp nộp kèm

Các tệp nộp kèm được liệt kê ở Bảng 5.10.

Bảng 5.10. Danh mục tệp nộp kèm.

Tệp	Nội dung
`l05_basics.py`	Script hoàn chỉnh, có thêm cờ `--no-repartition`
`01_run_no_repartition.txt`	Log lần chạy ghi không `repartition` (WALL 35.70 s)

Tệp	Nội dung
`02_files_no_repartition.txt`	Đếm tệp output khi không `repartition` (56 tệp, 74 MB)
`03_run_with_repartition.txt`	Log lần chạy chính có `repartition` và `--hold 300` để chụp UI
`04_files_with_repartition.txt`	Đếm tệp output khi có `repartition` (31 tệp, 73 MB)
`05_ansi_divide_by_zero.txt`	Thử nghiệm ANSI mode: `try_divide` và `[DIVIDE_BY_ZERO]`
`ui_sql_queries.txt`	Danh sách 21 truy vấn SQL kèm thời lượng (ms)
`ui_by_borough_jobs.txt`	Các job 19–22 của truy vấn 10 cùng stage và số task
`img/`	Ảnh chụp terminal (`01`–`08`) và Spark UI (`20`–`28`)

Phụ lục B: Trích đoạn mã nguồn

Đếm NULL của mọi cột trong một lần quét và tính tỷ lệ tip an toàn với ANSI mode (phần làm sạch trong `l05_basics.py`):

```
null_counts = trips.select([F.sum(F.col(c).isNull().cast("int")).alias(c)
                            for c in trips.columns])

# ANSI mode: try_divide trả NULL khi fare_amount = 0 thay vì làm hỏng job
.withColumn("tip_pct",
            F.round(F.try_divide(F.col("tip_amount"), F.col("fare_amount"))
 ↪  * 100, 1))
```

Ba luật làm sạch bổ sung vào từ điển luật; mỗi luật là một biểu thức boolean để đếm vi phạm trong một lần quét:

```
"negative total": F.col("total_amount") < 0,
"distance <= 0 or > 100 miles": (F.col("trip_distance") <= 0)
                                | (F.col("trip_distance") > 100),
"duration <= 0 or > 6 h": (F.col("duration_min") <= 0) |
 ↪  (F.col("duration_min") > 360),
```

Join hai lần với bảng khu vực, đổi tên cột trước mỗi lần để tránh `AMBIGUOUS_REFERENCE`:

```
pu = zones.select(F.col("LocationID").alias("PULocationID"),
```

```
                  F.col("Borough").alias("pu_borough"),
 ↪  F.col("Zone").alias("pu_zone"))
do = zones.select(F.col("LocationID").alias("DOLocationID"),
                  F.col("Borough").alias("do_borough"),
 ↪  F.col("Zone").alias("do_zone"))
enriched = clean.join(pu, "PULocationID", "left").join(do, "DOLocationID",
 ↪  "left")
```

Xếp hạng khu vực theo quận và chuỗi theo ngày với `lag`, tổng lũy kế:

```
w_rank = Window.partitionBy("pu_borough").orderBy(F.desc("trips"))
top3 = (zone_counts.withColumn("rank", F.dense_rank().over(w_rank))
        .filter("rank <= 3").orderBy("pu_borough", "rank"))

w_day = Window.orderBy("pickup_date")
daily = (daily.withColumn("prev_day_trips", F.lag("trips").over(w_day))
         .withColumn("dod_change_pct",
                     F.round((F.col("trips") / F.col("prev_day_trips") - 1)
 ↪  * 100, 1))
         .withColumn("running_revenue",

 ↪  F.sum("revenue").over(w_day.rowsBetween(Window.unboundedPreceding,
 ↪  0)))
         .orderBy("pickup_date"))
```

Ghi Parquet phân vùng; cờ `--no-repartition` cho phép chạy cả hai cấu hình mà không sửa mã:

```
out = enriched.drop("pickup_ts")
if not args.no_repartition:
    out = out.repartition("pickup_date")
out.write.mode("overwrite").partitionBy("pickup_date").parquet(OUTPUT)
```

Chương 6

Tối ưu hiệu năng Spark: shuffle, chiến lược join, data skew, AQE, phân vùng và cache

Tóm tắt

Bài này đo ảnh hưởng của một số tham số tinh chỉnh Apache Spark 4.1.3 trên một máy, chế độ `local[4]`. Dữ liệu gồm tập chuyến taxi New York tháng 01/2024 (2,964,624 dòng) và một bảng fact tổng hợp 10 triệu dòng, trong đó 60 % số dòng có cùng một khóa. Mỗi biến thể được ép chạy hết qua *noop sink*; thời gian đo bằng đồng hồ tường, kế hoạch vật lý cuối và thống kê task lấy từ Spark REST API. Với dữ liệu nhỏ, 200 partition shuffle chậm hơn khoảng 6.7 lần so với 8 partition, còn *Adaptive Query Execution* (AQE) tự gộp partition và về gần mức tinh chỉnh tay. Trong phép join bị lệch, task chậm nhất lâu hơn trung vị khoảng 14 lần; AQE skew join rút thời gian từ 4.10 s xuống 2.22 s, trong khi salting chỉ cắt được task chậm nhất mà thời gian tổng gần như giữ nguyên. Cache một DataFrame đắt được dùng lại ba lần giảm thời gian ba truy vấn từ 6.40 s xuống 0.99 s. Cuối bài là một số khuyến nghị cho job production trên cluster 20 node.

Từ khóa: Apache Spark, shuffle partitions, Adaptive Query Execution, broadcast hash join, sort-merge join, data skew, salting, repartition, coalesce, cache.

6.1 Giới thiệu

Tốc độ một job Spark phụ thuộc nhiều vào cách dữ liệu được chia cho các task: có bao nhiêu partition sau shuffle, planner chọn kiểu join nào, khóa có bị lệch không, ghi ra bao nhiêu file, và kết quả trung gian có được dùng lại không. Những lựa chọn này hay được đặt theo thói quen, hoặc khi chậm thì người ta thêm máy. Bài lab L06 yêu cầu đo thay vì đoán.

Năm thí nghiệm được thực hiện:

1. E1: ảnh hưởng của `spark.sql.shuffle.partitions` và cơ chế gộp partition của AQE.
2. E2: *broadcast hash join* so với *sort-merge join*, vai trò của `autoBroadcastJoinThreshold` và gợi ý `broadcast()`.
3. E3: data skew trong join và hai cách xử lý là salting và AQE skew join.
4. E4: `repartition` so với `coalesce` trước khi ghi (thời gian, số file, mức song song).
5. E5: khi nào `cache`/`persist` có lợi.

Mục 6.2 nhắc lại phần lý thuyết cần dùng, Mục 6.3 mô tả môi trường và cách đo, Mục 6.4 trình bày kết quả, Mục 6.5 trả lời tám câu hỏi checkpoint, và Mục 6.6 đưa ra khuyến nghị.

6.2 Cơ sở lý thuyết

6.2.1 Shuffle và số partition shuffle

Spark chia một truy vấn thành các stage, ranh giới là các *wide dependency* (phụ thuộc rộng) như `groupBy` hay `join`, nơi các bản ghi cùng khóa phải được đưa về cùng một partition [1]. Khi shuffle, phía map ghi dữ liệu đã băm theo khóa ra đĩa cục bộ, phía reduce đọc lại thành `spark.sql.shuffle.partitions` partition (mặc định 200). Mỗi partition là một task, nên con số này quyết định cả mức song song lẫn chi phí lập lịch. Quá ít partition thì task to, dễ spill hoặc thiếu bộ nhớ; quá nhiều thì có hàng trăm task tí hon mà phần chi phí khởi tạo còn lớn hơn phần việc thật [3], [4].

6.2.2 Adaptive Query Execution

AQE tối ưu lại kế hoạch vật lý trong lúc chạy, dựa trên thống kê thật của các stage shuffle đã xong (*map output statistics*) thay vì ước lượng từ kích thước file [3]. Bài này dùng ba tính năng của AQE: gộp các partition nhỏ liền kề tới khoảng `advisoryPartitionSizeInBytes` (nút `AQEShuffleRead coalesced`), đổi sang broadcast khi một phía thật sự nhỏ, và tách partition bị lệch trong sort-merge join (`SortMergeJoin(skew=true)`). AQE được bật mặc định từ Spark 3.2.

6.2.3 Chiến lược join

Spark SQL [2] chọn kiểu join theo kích thước ước lượng của hai phía. Broadcast hash join (BHJ) gom bảng nhỏ về driver rồi gửi một bản sao tới mọi executor, nên bảng lớn không phải shuffle. Sort-merge join (SMJ) shuffle cả hai phía theo khóa, sort từng partition rồi trộn. `spark.sql.autoBroadcastJoinThreshold` (mặc định 10 MB) quyết định khi nào broadcast được chọn tự động; đặt -1 sẽ tắt việc chọn tự động, nhưng gợi ý `broadcast()` viết trong mã vẫn có hiệu lực.

6.2.4 Data skew và salting

Skew xảy ra khi vài khóa chiếm phần lớn số bản ghi. Shuffle băm theo khóa nên mọi bản ghi của một *hot key* (khóa nóng) dồn vào một partition, sinh ra một *straggler* (task tụt hậu) trong khi các core khác ngồi chờ. Dấu hiệu dễ nhận nhất là thời gian task lớn nhất vượt xa trung vị trong cùng stage. Salting gắn thêm một khóa phụ ngẫu nhiên cho hot key và nhân bản dòng tương ứng ở bảng dimension, để hot key được rải ra nhiều partition [4], [5].

6.2.5 repartition, coalesce và cache

`repartition(n)` luôn shuffle để chia đều dữ liệu. `coalesce(n)` chỉ gộp các partition có sẵn qua *narrow dependency* (phụ thuộc hẹp), không shuffle, nhưng có thể kéo giảm song song của cả stage đứng trước nó [4]. `cache`/`persist` giữ kết quả trung gian trong bộ nhớ hoặc trên đĩa để các action sau không phải tính lại cả lineage [1]. Nó chỉ có lợi khi chi phí tính lại lớn hơn chi phí materialize cộng với bộ nhớ bị chiếm.

6.3 Môi trường và phương pháp thực nghiệm

6.3.1 Môi trường

Mọi thứ chạy trong một container Docker, Spark ở chế độ local với 4 core, nên không có chi phí mạng giữa các executor (Bảng 6.1).

Bảng 6.1. Môi trường thực nghiệm.

Thành phần	Giá trị
Máy chủ	Laptop 12 luồng CPU, 32 GB RAM
Ảo hóa	Docker Desktop trên WSL2, cấp 31 GB
Spark	4.1.3 (image `spark:4.1.3-python3`), PySpark
Chế độ chạy	`local[4]` (4 core, `defaultParallelism` = 4), driver 2 GB
Thu thập số liệu	`time.time()` quanh mỗi action; Spark REST API (`/sql`, `/stages/.../taskSummary`)
Ngày thực hiện	2026-10-30

6.3.2 Dữ liệu

Bảng fact tổng hợp được sinh bằng `spark.range`: 60 % số dòng có khóa 0 (`HOT_SHARE = 0.6`), phần còn lại rải đều trên 200,000 khóa. Đếm lại thì được 5,998,366 dòng hot và 4,001,634 dòng thường. Các tập dữ liệu được liệt kê ở Bảng 6.2.

Bảng 6.2. Dữ liệu thực nghiệm.

Tập dữ liệu	Nguồn	Kích thước	Dùng trong
Trips	`yellow_tripdata_2024-01.parquet`	2,964,624 dòng, 47.6 MiB, 3 row group	E1, E2, E4, E5
Zones	`taxi_zone_lookup.csv`	265 dòng	E2
Fact tổng hợp	`spark.range(10,000,000)`, 60 % key = 0	10,000,000 dòng	E3
Dim tổng hợp	`spark.range(200,000)`, cột `segment` = `seg_{key % 50}`	200,000 dòng	E3

6.3.3 Quy trình

Script `l06_experiments.py` (trích đoạn ở Phụ lục B) chạy từng biến thể theo các bước sau:

1. Tạo SparkSession và chạy một `count()` để khởi động JVM, liệt kê file và làm nóng OS cache; lần này không tính.
2. Với mỗi biến thể, đặt lại cấu hình (`reset_conf`), đổi tham số cần khảo sát, rồi ép thực thi bằng `df.write.format("noop")` (tính đủ nhưng không ghi file). E4 ghi Parquet thật để đếm số file; E5 dùng `collect()` cho ba truy vấn.
3. Sau mỗi action, hàm `last_sql_summary()` gọi REST API lấy các nút của kế hoạch cuối (sau AQE), tổng số task, và median/max thời gian task tại stage lệch nhất (stage có hiệu max trừ median lớn nhất).
4. Tham số thay đổi: E1 `shuffle.partitions` $\in \{200, 8, 1\}$ khi tắt AQE và 200 khi bật AQE; E2 ngưỡng broadcast 10 MB, -1, và -1 kèm `broadcast()`; E3 SMJ 16 partition tắt AQE, sau đó salting với 64 giá trị salt, sau đó AQE skew join với `advisoryPartitionSizeInBytes=4m`, `skewedPartitionThresholdInBytes=16m`, `skewedPartitionFactor=3` và tắt coalesce; E4 bốn cách phân vùng trước khi ghi; E5 ba phép tổng hợp có và không có `persist(MEMORY_AND_DISK)`.
5. Chúng tôi chạy cả script hai lần, kết quả nối vào `results.csv` (34 dòng = 2 lần × 17 biến thể). Báo cáo dùng lần chạy thứ hai; lần đầu để xem số liệu có ổn định không. Ở lần có cờ `--hold`, Spark UI được giữ lại để chụp kế hoạch và Summary Metrics.

6.4 Kết quả thực nghiệm

6.4.1 Tổng quan hai lần chạy

Hình 6.1 là đầu ra console của lần chạy thứ hai, Hình 6.2 là bảng Markdown script in ra ở cuối. Toàn bộ 17 biến thể được gom lại ở Bảng 6.3; hai cột median/max là thời gian task ở stage lệch nhất của truy vấn. Tab SQL của phiên có giữ UI liệt kê 31 truy vấn đã xong (Hình 6.3), và chúng tôi dùng danh sách này để tìm kế hoạch của từng biến thể.

```
spark-submit l06_experiments.py (run 2)

Spark 4.1.3 | cores 4 | input ['/datasets/data/taxi/yellow_tripdata_2024-01.parquet']

===== E1: e1_shuffle_partitions
[E1] AQE off, shuffle.partitions=200           2.20 s | tasks=204 skewiest stage: median=6.0 ms max=60.0 ms | Exchange
[E1] AQE off, shuffle.partitions=8             0.33 s | tasks=12 skewiest stage: median=177.0 ms max=178.0 ms | Exchange
[E1] AQE off, shuffle.partitions=1             0.44 s | tasks=5 skewiest stage: median=232.0 ms max=233.0 ms | Exchange
[E1] AQE on,  shuffle.partitions=200           0.53 s | tasks=5 skewiest stage: median=271.0 ms max=274.0 ms | AQEShuffleRead Exchange

===== E2: e2_broadcast_vs_smj
[E2] AQE off, broadcast (threshold 10MB)       1.20 s | tasks=13 skewiest stage: median=626.0 ms max=635.0 ms | BroadcastExchange BroadcastHashJoin
  Exchange
[E2] AQE off, sort-merge (threshold -1)        1.51 s | tasks=21 skewiest stage: median=386.0 ms max=591.0 ms | Exchange SortMergeJoin
[E2] AQE on,  threshold -1 + broadcast() hint  0.40 s | tasks=6 skewiest stage: median=157.0 ms max=168.0 ms | AQEShuffleRead BroadcastExchange
  BroadcastHashJoin Exchange

===== E3: e3_skew
+----------+-------+
|is_hot_key|  count|
+----------+-------+
|      true|5998366|
|     false|4001634|
+----------+-------+

[E3] skewed SMJ, AQE off                       4.10 s | tasks=40 skewiest stage: median=165.0 ms max=2366.0 ms | Exchange SortMergeJoin
[E3] salted SMJ (salt=64), AQE off             4.01 s | tasks=56 skewiest stage: median=590.0 ms max=1197.0 ms | Exchange SortMergeJoin
[E3] skewed SMJ, AQE skewJoin on               2.22 s | tasks=27 skewiest stage: median=186.0 ms max=682.0 ms | AQEShuffleRead Exchange
  SortMergeJoin(skew=true)
same result with and without salting: True (500061629.0)

===== E4: e4_repartition_vs_coalesce
[E4] repartition(64)                          7.25 s | files=64 | tasks=68 | Exchange
[E4] repartition(4)                           4.40 s | files=4 | tasks=8 | Exchange
[E4] coalesce(4)                              3.12 s | files=3 | tasks=4 | Coalesce
[E4] coalesce(1)                              6.95 s | files=1 | tasks=1 | Coalesce

===== E5: e5_cache
[E5] no cache                                 6.40 s
[E5] with cache (MEMORY_AND_DISK)             0.99 s
```

Hình 6.1. Đầu ra console của lần chạy thứ hai: thời gian, số task, median/max task và các nút kế hoạch của từng biến thể.

```
results table (run 2)

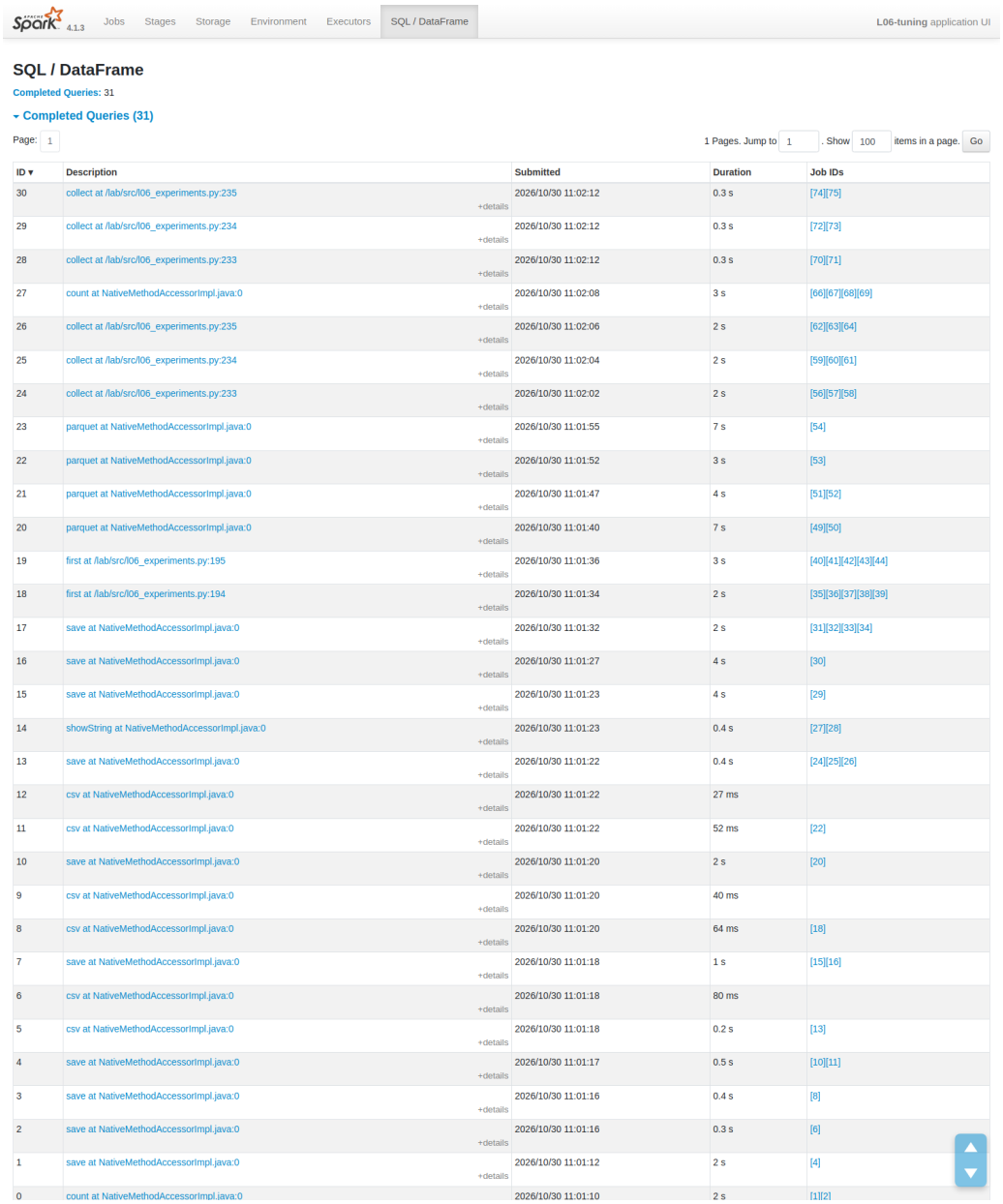
 experiment | variant | seconds | total tasks | task median ms* | task max ms* | plan (final, after AQE)
---|---|---|---|---|---|---
 E1 | AQE off, shuffle.partitions=200 | 2.2 | 204 | 6.0 | 60.0 | Exchange
 E1 | AQE off, shuffle.partitions=8 | 0.33 | 12 | 177.0 | 178.0 | Exchange
 E1 | AQE off, shuffle.partitions=1 | 0.44 | 5 | 232.0 | 233.0 | Exchange
 E1 | AQE on,  shuffle.partitions=200 | 0.53 | 5 | 271.0 | 274.0 | AQEShuffleRead Exchange
 E2 | AQE off, broadcast (threshold 10MB) | 1.2 | 13 | 626.0 | 635.0 | BroadcastExchange BroadcastHashJoin Exchange
 E2 | AQE off, sort-merge (threshold -1) | 1.51 | 21 | 386.0 | 591.0 | Exchange SortMergeJoin
 E2 | AQE on,  threshold -1 + broadcast() hint | 0.4 | 6 | 157.0 | 168.0 | AQEShuffleRead BroadcastExchange BroadcastHashJoin Exchange
 E3 | skewed SMJ, AQE off | 4.1 | 40 | 165.0 | 2366.0 | Exchange SortMergeJoin
 E3 | salted SMJ (salt=64), AQE off | 4.01 | 56 | 590.0 | 1197.0 | Exchange SortMergeJoin
 E3 | skewed SMJ, AQE skewJoin on | 2.22 | 27 | 186.0 | 682.0 | AQEShuffleRead Exchange SortMergeJoin(skew=true)
 E4 | repartition(64) | 7.25 | 68 | 152.0 | 980.0 | Exchange
 E4 | repartition(4) | 4.4 | 8 | 2185.0 | 2210.0 | Exchange
 E4 | coalesce(4) | 3.12 | 4 | 2907.0 | 2970.0 | Coalesce
 E4 | coalesce(1) | 6.95 | 1 | 6800.0 | 6800.0 | Coalesce
 E5 | no cache | 6.4 |  |  |  |
 E5 | cache materialisation (count) | 3.42 |  |  |  |
 E5 | with cache (MEMORY_AND_DISK) | 0.99 |  |  |  |
* median / max task run time of the most skewed stage of that query
```

Hình 6.2. Bảng kết quả Markdown do script in ra ở cuối lần chạy thứ hai.

Bảng 6.3. Kết quả đầy đủ của lần chạy thứ hai (17 biến thể).

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E1	AQE off, shuffle.partitions=200	2.20	204	6	60	Exchange	
E1	AQE off, shuffle.partitions=8	0.33	12	177	178	Exchange	
E1	AQE off, shuffle.partitions=1	0.44	5	232	233	Exchange	
E1	AQE on, shuffle.partitions=200	0.53	5	271	274	AQEShuffleRead Exchange	AQE gộp partition nhỏ
E2	AQE off, broadcast (threshold 10MB)	1.20	13	626	635	BroadcastExchange BroadcastHashJoin Exchange	
E2	AQE off, sort-merge (threshold -1)	1.51	21	386	591	Exchange SortMergeJoin	
E2	AQE on, threshold -1 + broadcast() hint	0.40	6	157	168	AQEShuffleRead BroadcastExchange BroadcastHashJoin Exchange	Ràng buộc thắng threshold
E3	skewed SMJ, AQE off	4.10	40	165	**2366**	Exchange SortMergeJoin	
E3	salted SMJ (salt=64), AQE off	4.01	56	590	1197	Exchange SortMergeJoin	cùng kết quả (500061629)
E3	skewed SMJ, AQE skewJoin on	2.22	27	186	682	AQEShuffleRead Exchange SortMergeJoin(skew)	partition skew, 4 split (skew=true)
E4	repartition(64)	4.25	68	152	980	Exchange	64 file

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E4	repartition(4)	4.40	8	2185	2210	Exchange	4 file
E4	coalesce(4)	3.12	4	2907	2970	Coalesce	3 file
E4	coalesce(1)	6.95	1	6800	6800	Coalesce	1 file
E5	no cache (3 queries)	6.40					3 aggregation
E5	cache materialisation (count)	3.42					2,869,714 dòng, 59.4 MiB
E5	with cache (3 queries)	0.99					3 aggregation



Hình 6.3. Tab SQL / DataFrame của Spark UI với 31 truy vấn đã hoàn tất trong phiên chạy.

Lần chạy đầu (tổng 71.71 s) cho cùng xu hướng ở mọi thí nghiệm (Bảng 6.4). Phần lớn chênh lệch nằm trong khoảng ±20 % mà đề bài cho phép. Ngoại lệ là các biến thể rất ngắn: E2 broadcast khi tắt AQE lần đầu chỉ 0.77 s, lần hai lên 1.20 s, và E1 một partition từ 0.30 s lên 0.44 s. Với truy vấn chạy dưới một giây, vài trăm mili giây dao động đã là tỉ lệ lớn, nên chúng tôi không rút kết luận từ chênh lệch nhỏ giữa các biến thể loại này.

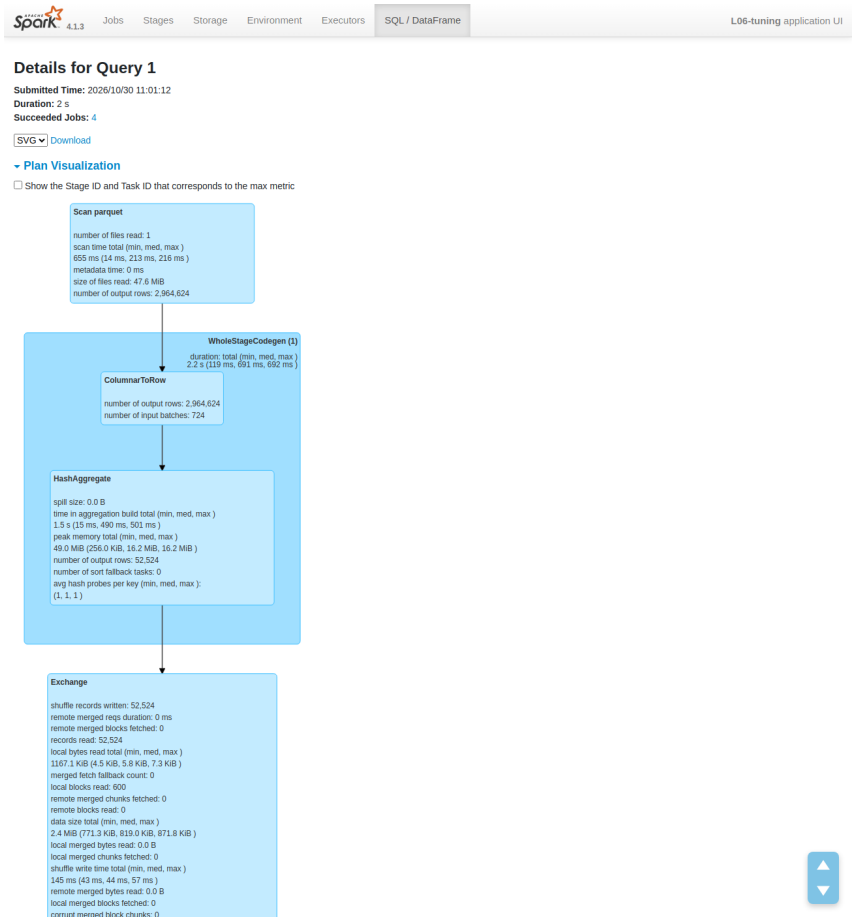
Bảng 6.4. So sánh một số chỉ số giữa hai lần chạy.

Chỉ số	Lần 1	Lần 2
E1, 200 partition, AQE off	2.15 s	2.20 s
E2, broadcast, AQE off	0.77 s	1.20 s

Chỉ số	Lần 1	Lần 2
E3 baseline, task max	2,686 ms	2,366 ms
E3 salting	4.28 s	4.01 s
E3 AQE skew join	2.28 s	2.22 s
E5 không cache	6.07 s	6.40 s
E5 với cache	0.88 s	0.99 s

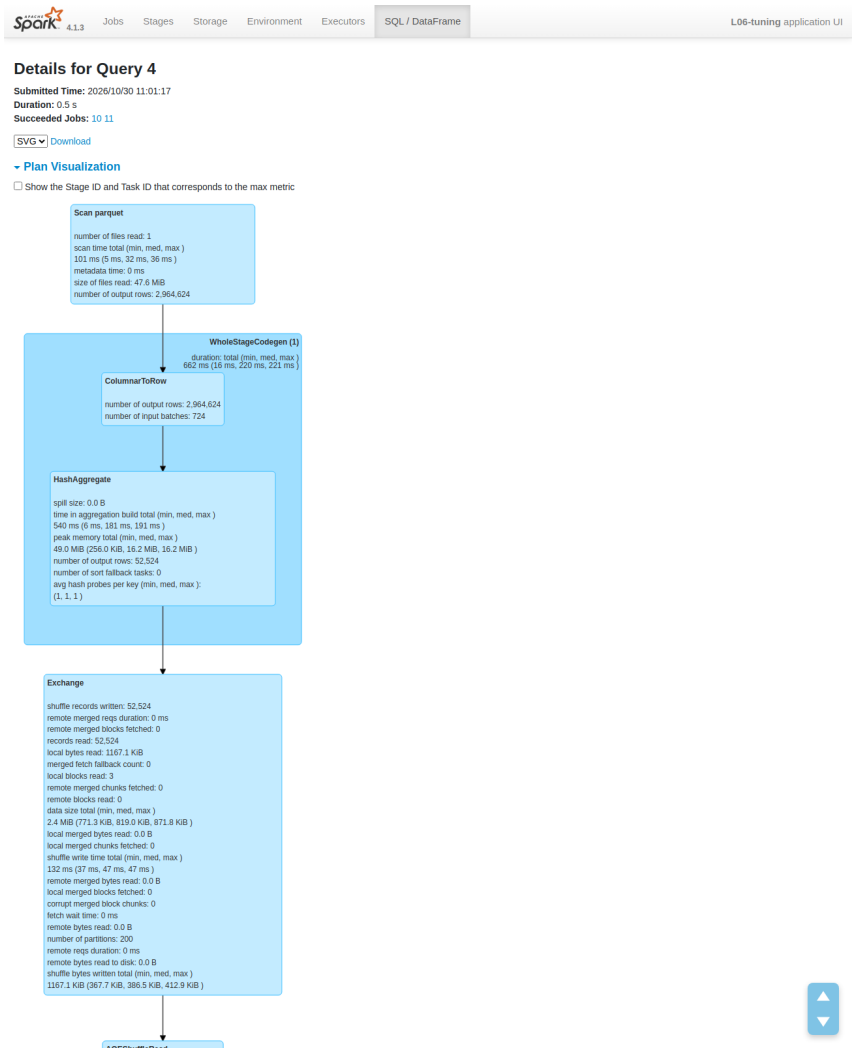
6.4.2 E1: số partition shuffle và AQE coalescing

Truy vấn `groupBy(PULocationID, DOLocationID).agg(count, avg)` đọc 47.6 MiB nhưng chỉ ghi khoảng 1167.1 KiB dữ liệu shuffle. Với 200 partition và AQE tắt, truy vấn sinh 204 task (4 map, 200 reduce), mỗi task reduce chạy khoảng 6 ms, và đây là biến thể chậm nhất. Kế hoạch ở Hình 6.4 chỉ có một `Exchange` 200 partition.



Hình 6.4. Kế hoạch E1 khi AQE tắt: `Exchange` với 200 partition shuffle.

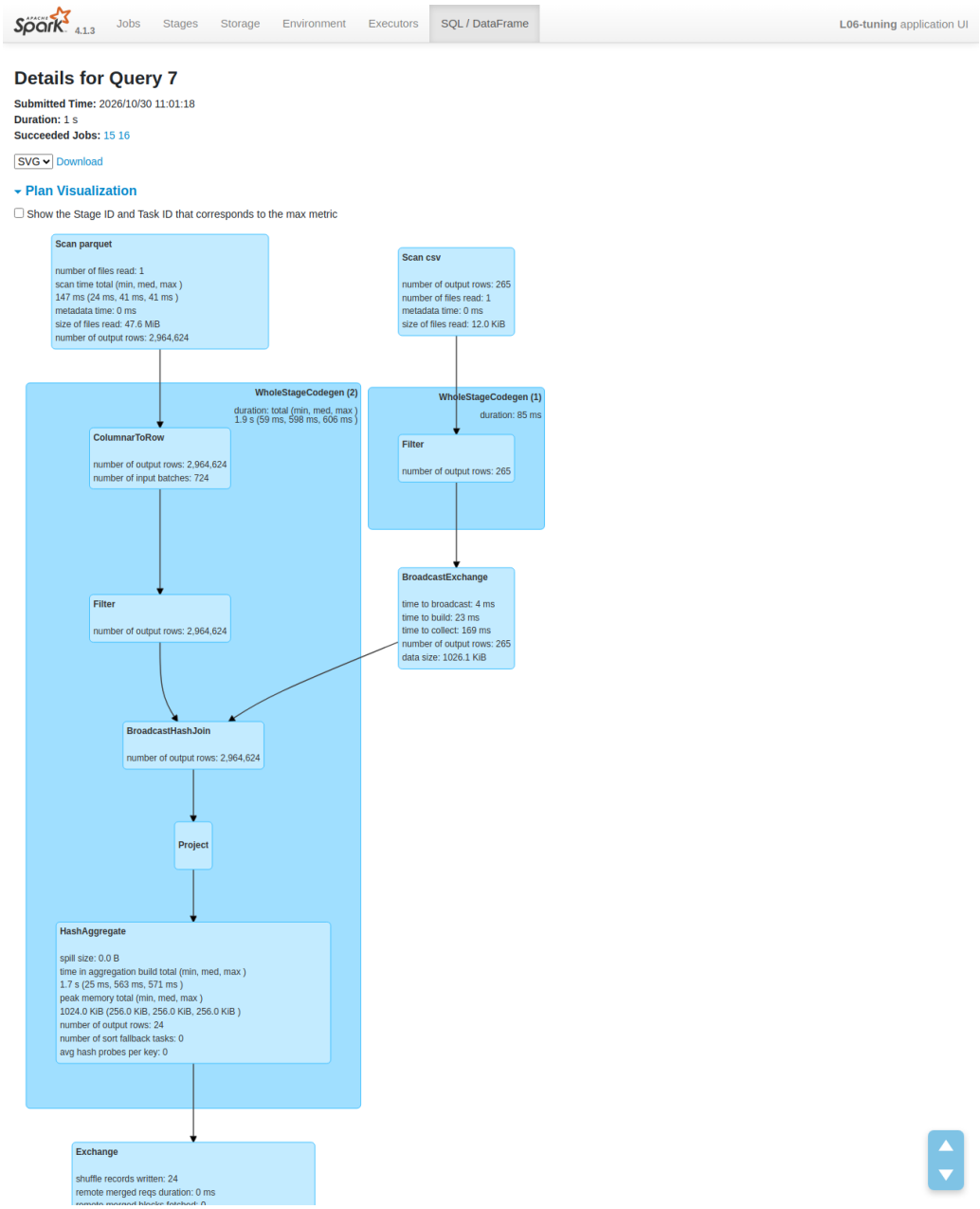
Bật AQE mà vẫn để 200 partition thì sau `Exchange` xuất hiện `AQEShuffleRead` (Hình 6.5), các partition nhỏ được gộp lại, tổng chỉ còn 5 task, thời gian xấp xỉ bản tinh chỉnh tay 8 partition. Bản 1 partition cũng nhanh, nhưng chỉ vì kết quả quá nhỏ (khoảng 20 nghìn nhóm); Mục 6.5.1 giải thích vì sao không nên dùng nó.



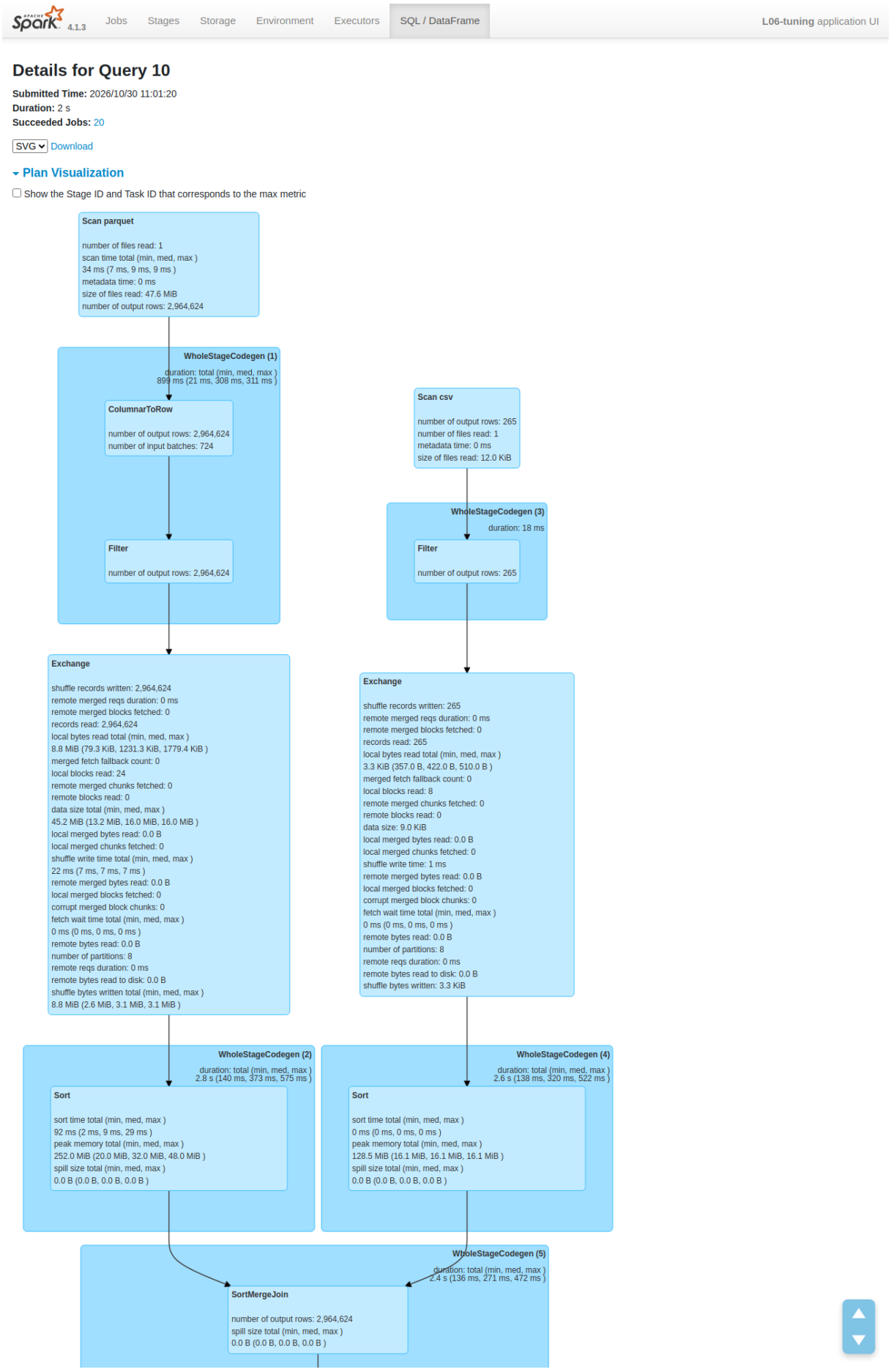
Hình 6.5. Kế hoạch E1 khi AQE bật: `Exchange` 200 partition theo sau là `AQEShuffleRead` ở chế độ coalesced; truy vấn đọc 2,964,624 dòng, 47.6 MiB.

6.4.3 E2: broadcast hash join và sort-merge join

Phép join trips ⋈ zones (zones có 265 dòng) được chạy theo ba cách. Khi ngưỡng là -1, Spark phải dùng `SortMergeJoin`: khoảng 3 triệu chuyến bị shuffle cùng bảng zone rồi sort (Hình 6.7). Khi broadcast, chỉ 265 dòng được gửi đi và phía trips không có `Exchange` trước join (Hình 6.6).



Hình 6.6. Kế hoạch E2 với broadcast: `BroadcastExchange` và `BroadcastHashJoin`, phía trips không có `Exchange` trước join.



Hình 6.7. Kế hoạch E2 với ngưỡng -1: `Exchange` và `Sort` ở cả hai phía trước `SortMergeJoin`.

Gợi ý `F.broadcast()` vẫn có tác dụng khi ngưỡng là -1 và là biến thể nhanh nhất của E2 (0.40 s, 6 task). Chúng tôi hơi bất ngờ khi thấy broadcast tắt AQE (1.20 s) chỉ nhanh hơn SMJ (1.51 s) một chút. Lý do có lẽ là cấu hình mặc định của script chỉ dùng 8 partition nên SMJ ở đây khá nhẹ, cộng thêm dao động giữa các lần chạy đã nói ở trên. Khác biệt thật nằm ở kế hoạch: broadcast bỏ được `Exchange` và `Sort` ở phía bảng lớn, và khoản tiết kiệm này sẽ lớn dần theo kích thước dữ liệu.

6.4.4 E3: data skew, salting và AQE skew join

Bảng fact có 5,998,366 dòng mang khóa 0. Bằng REST API, chúng tôi lấy phân vị thời gian task và số bản ghi shuffle đọc mỗi task tại stage join của ba biến thể (Bảng 6.5, Hình 6.8).

Bảng 6.5. Phân vị min / 25 % / median / 75 % / max tại stage join của E3 (lần chạy thứ hai).

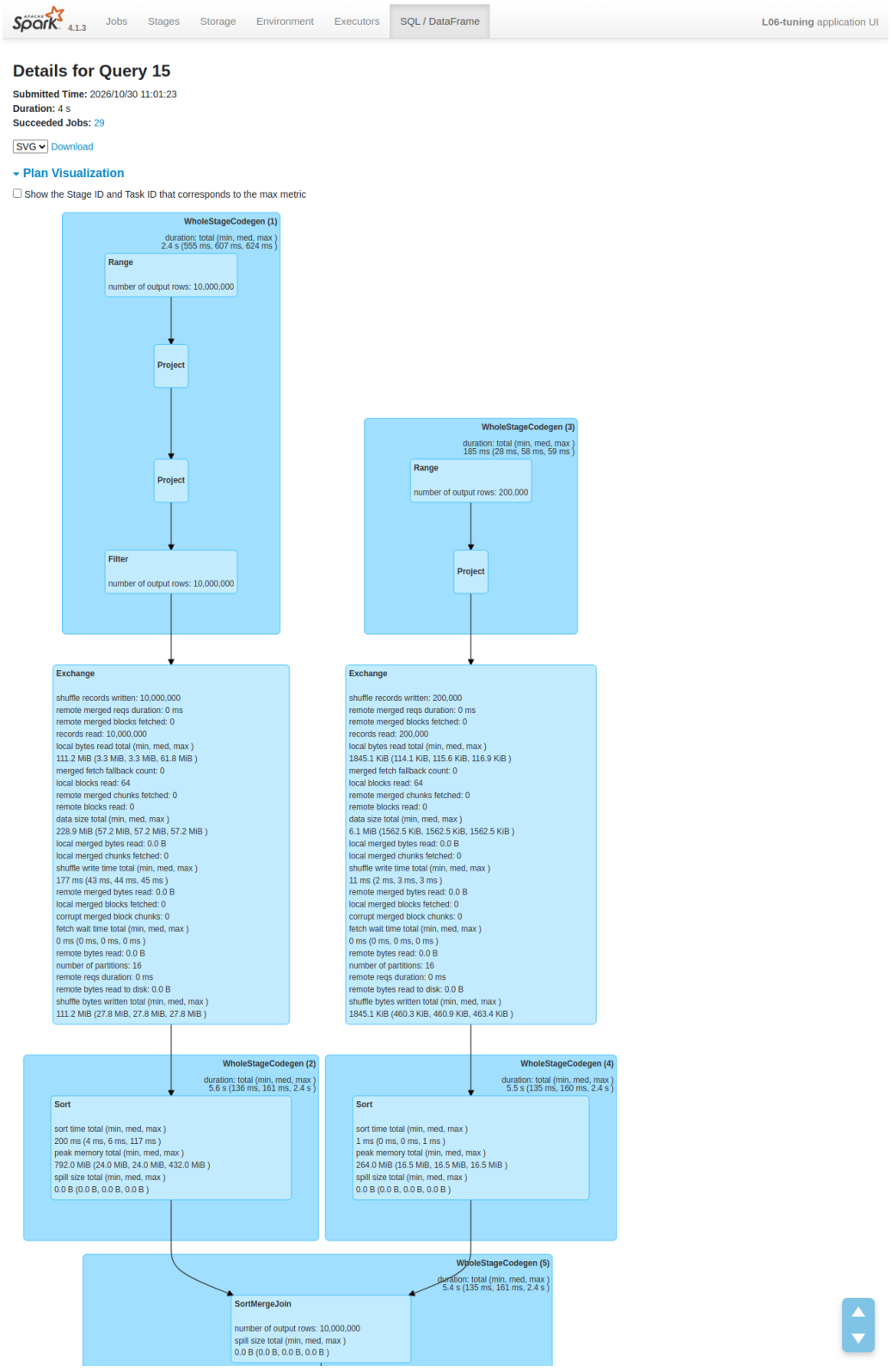
Stage (biến thể)	Thời gian task (ms)	Bản ghi shuffle đọc / task
42: baseline, AQE off	140 / 157 / 165 / 379 / 2,366	260,166 / 261,225 / 262,619 / 264,416 / 6,264,438
46: salting 64	308 / 484 / 590 / 901 / 1,197	359,285 / 543,450 / 638,133 / 823,338 / 917,267
52: AQE skew join	129 / 144 / 186 / 199 / 682	260,166 / 261,225 / 262,677 / 265,121 / 1,576,365

```
E3: task time and shuffle records per task

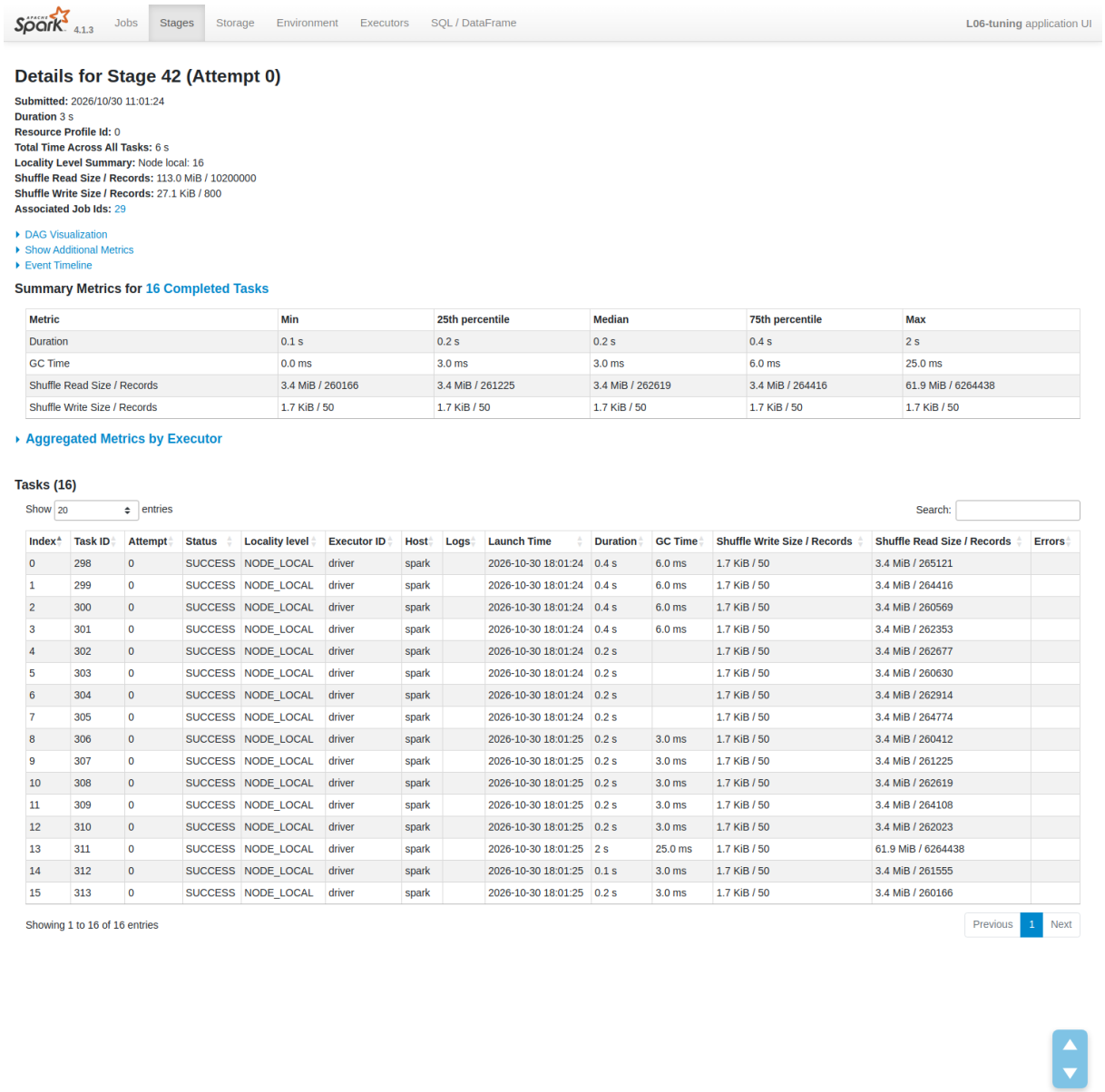
# task quantiles [min, p25, median, p75, max] from /api/v1/.../taskSummary
stage 42: runTime [140.0, 157.0, 165.0, 379.0, 2366.0] shuffleReadRecords [260166.0, 261225.0, 262619.0, 264416.0, 6264438.0]
stage 46: runTime [308.0, 484.0, 590.0, 901.0, 1197.0] shuffleReadRecords [359285.0, 543450.0, 638133.0, 823338.0, 917267.0]
stage 52: runTime [129.0, 144.0, 186.0, 199.0, 682.0] shuffleReadRecords [260166.0, 261225.0, 262677.0, 265121.0, 1576365.0]
```

Hình 6.8. Phân vị thời gian task và số bản ghi shuffle mỗi task của ba stage join E3, lấy từ Spark REST API.

Baseline. Kế hoạch là `Exchange` + `SortMergeJoin` với 16 partition (Hình 6.9). Ở stage 42, task chậm nhất lâu hơn median khoảng 14 lần. Theo Summary Metrics (Hình 6.10), task này đọc 61.9 MiB, trong khi các task khác chỉ khoảng 3.4 MiB. Cả stage đọc 113.0 MiB / 10,200,000 bản ghi qua 16 task. Trong lúc task đó chạy, ba core còn lại không có việc gì làm.

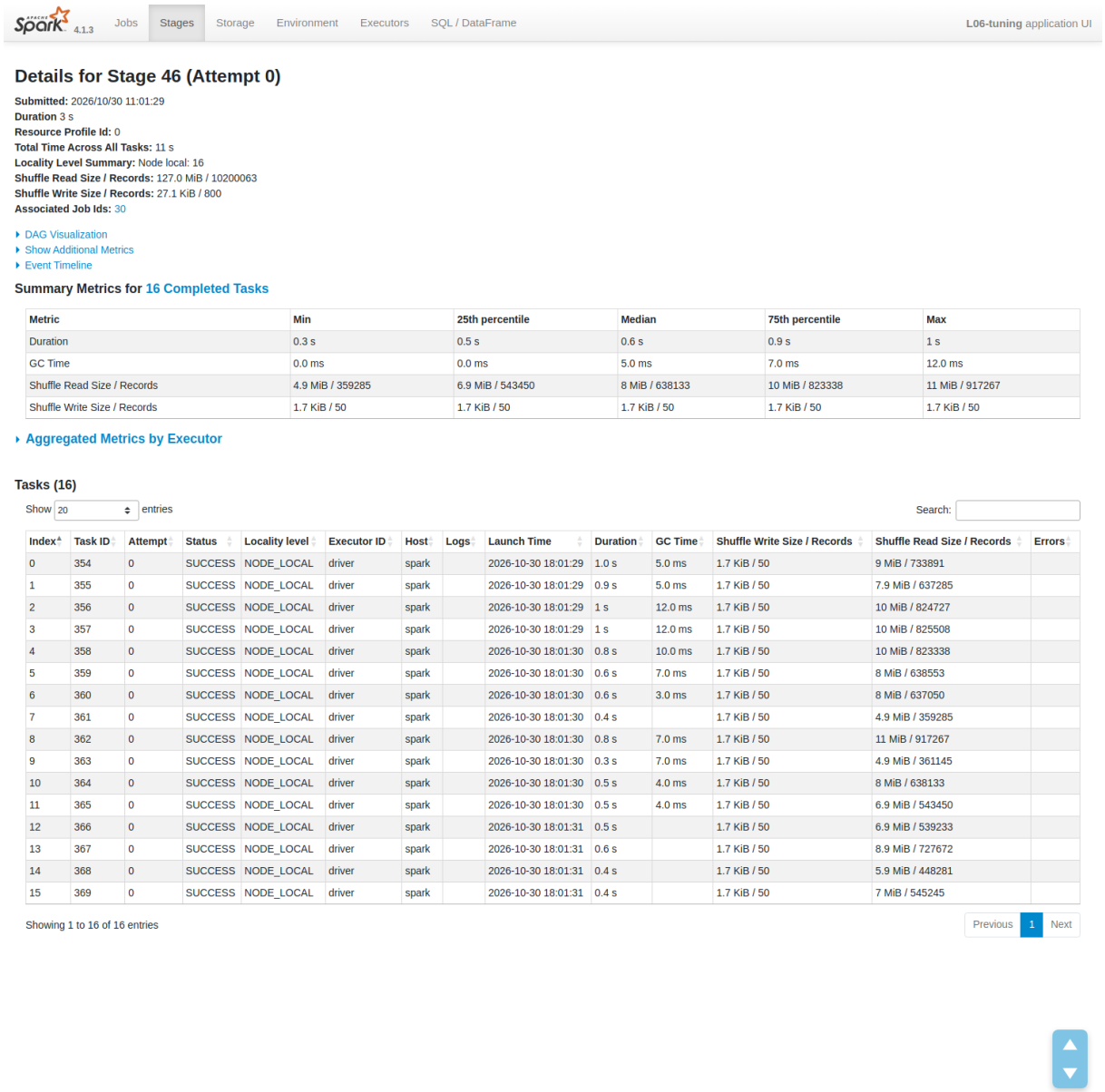


Hình 6.9. Kế hoạch E3 baseline (AQE tắt): sort-merge join trên dữ liệu lệch.



Hình 6.10. Summary Metrics của stage 42 (baseline): max 61.9 MiB / 6,264,438 bản ghi so với median 3.4 MiB / 262,619 bản ghi; task index 13 mất 2 s.

Salting. Rải hot key ra 64 giá trị salt làm task max giảm gần một nửa và số bản ghi lớn nhất mỗi task chỉ còn chưa tới một triệu (Hình 6.11). Bù lại, median tăng lên 590 ms vì partition nào cũng nhận thêm một phần hot key, lại thêm chi phí `rand()`, cột join phụ và 64 bản sao dòng dim. Kết quả là thời gian tổng gần như đứng yên (4.01 s so với 4.10 s). Tổng `amount` vẫn đúng (500061629), nghĩa là salting không làm sai kết quả.



Spark 4.1.3 Jobs Stages Storage Environment Executors SQL / DataFrame L06-tuning application UI

Details for Stage 46 (Attempt 0)

Submitted: 2026/10/30 11:01:29
Duration 3 s
Resource Profile Id: 0
Total Time Across All Tasks: 11 s
Locality Level Summary: Node local: 16
Shuffle Read Size / Records: 127.0 MiB / 10200063
Shuffle Write Size / Records: 27.1 KiB / 800
Associated Job Ids: 30

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 16 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.3 s	0.5 s	0.6 s	0.9 s	1 s
GC Time	0.0 ms	0.0 ms	5.0 ms	7.0 ms	12.0 ms
Shuffle Read Size / Records	4.9 MiB / 359285	6.9 MiB / 543450	8 MiB / 638133	10 MiB / 823338	11 MiB / 917267
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

▸ Aggregated Metrics by Executor

Tasks (16)

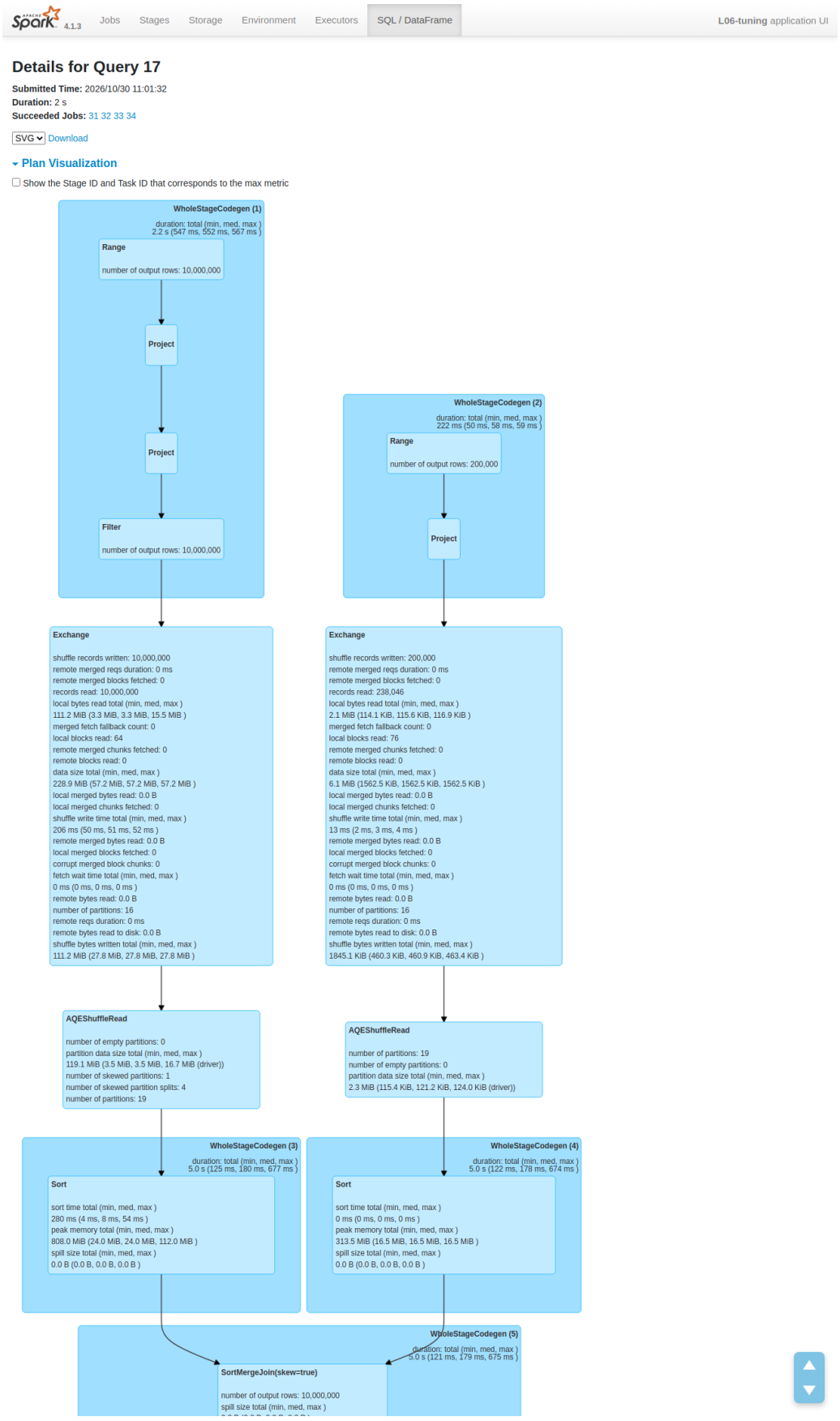
Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	354	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1.0 s	5.0 ms	1.7 KiB / 50	9 MiB / 733891	
1	355	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	0.9 s	5.0 ms	1.7 KiB / 50	7.9 MiB / 637285	
2	356	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1 s	12.0 ms	1.7 KiB / 50	10 MiB / 824727	
3	357	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1 s	12.0 ms	1.7 KiB / 50	10 MiB / 825508	
4	358	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.8 s	10.0 ms	1.7 KiB / 50	10 MiB / 823338	
5	359	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.6 s	7.0 ms	1.7 KiB / 50	8 MiB / 638553	
6	360	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.6 s	3.0 ms	1.7 KiB / 50	8 MiB / 637050	
7	361	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.4 s		1.7 KiB / 50	4.9 MiB / 359285	
8	362	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.8 s	7.0 ms	1.7 KiB / 50	11 MiB / 917267	
9	363	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.3 s	7.0 ms	1.7 KiB / 50	4.9 MiB / 361145	
10	364	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.5 s	4.0 ms	1.7 KiB / 50	8 MiB / 638133	
11	365	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.5 s	4.0 ms	1.7 KiB / 50	6.9 MiB / 543450	
12	366	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.5 s		1.7 KiB / 50	6.9 MiB / 539233	
13	367	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.6 s		1.7 KiB / 50	8.9 MiB / 727672	
14	368	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.4 s		1.7 KiB / 50	5.9 MiB / 448281	
15	369	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.4 s		1.7 KiB / 50	7 MiB / 545245	

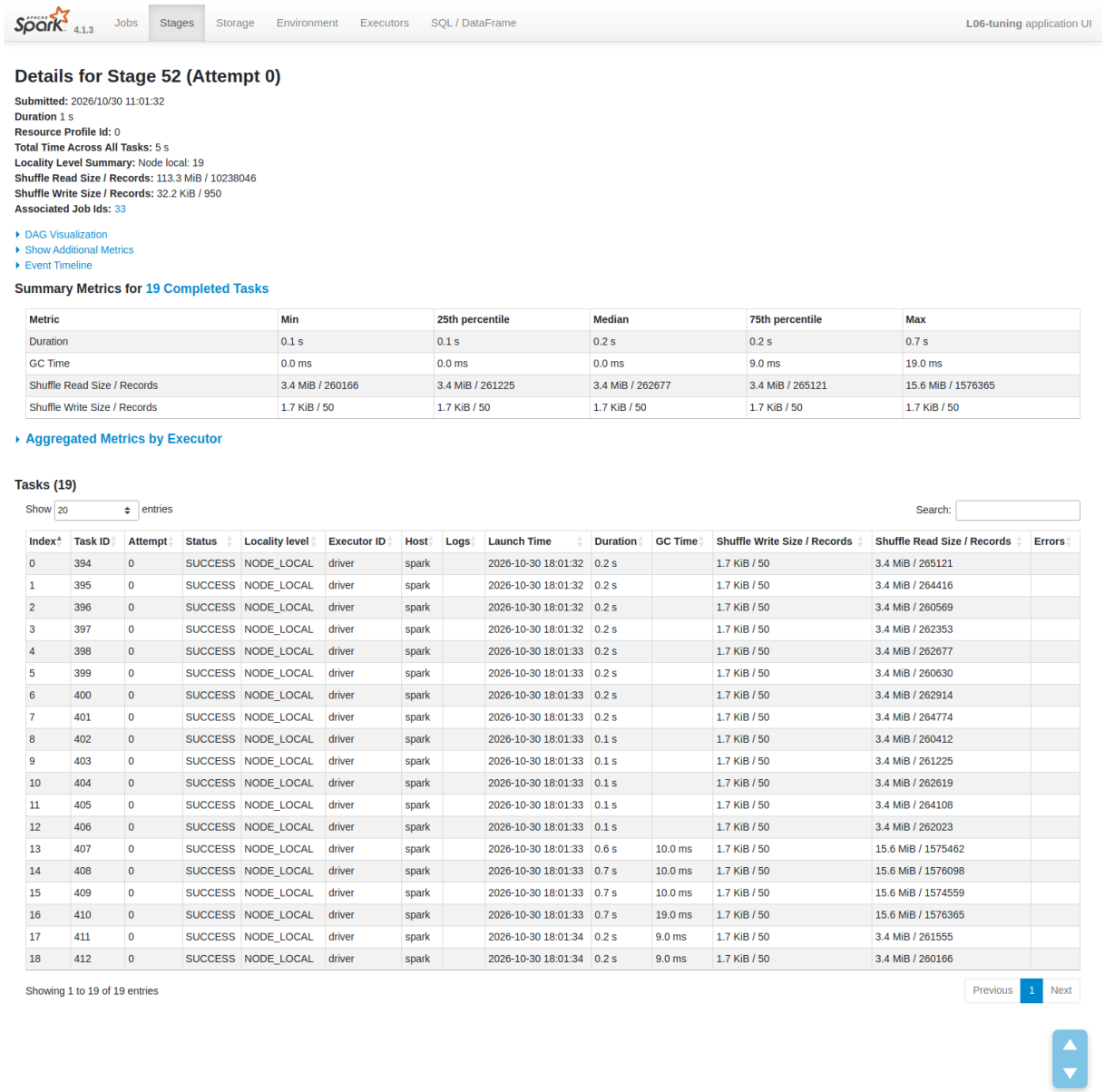
Showing 1 to 16 of 16 entries Previous 1 Next

Hình 6.11. Summary Metrics của stage 46 (salting 64): phân bố bản ghi đều hơn, max 917,267 bản ghi, nhưng median thời gian task tăng lên 590 ms.

AQE skew join. Kế hoạch cuối có `SortMergeJoin(skew=true)`, và `AQEShuffleRead` báo *number of skewed partitions: 1, skewed partition splits: 4, number of partitions: 19* (Hình 6.12). Phía fact ghi 10,000,000 bản ghi shuffle (111.2 MiB); sau khi đọc, partition có median 3.5 MiB và max 16.7 MiB. Ở stage 52 (Hình 6.13), số bản ghi lớn nhất mỗi task xấp xỉ một phần tư của baseline, đúng với 4 split. Thời gian tổng còn **2.22 s, nhanh gần gấp đôi baseline**, và không phải sửa dòng truy vấn nào. Lúc đầu chúng tôi để ngưỡng mặc định thì AQE không tách gì, vì partition nóng chỉ có 61.9 MiB, thấp hơn nhiều so với mức 256 MB mặc định. Vì vậy phải hạ ngưỡng như đã nêu ở Mục 6.3.



Hình 6.12. Kế hoạch E3 với AQE skew join: `AQEShuffleRead` báo 1 partition lệch được tách thành 4 split (16 thành 19 partition), sau đó là `SortMergeJoin(skew=true)`.



Spark 4.1.3 Jobs Stages Storage Environment Executors SQL / DataFrame L06-tuning application UI

Details for Stage 52 (Attempt 0)

Submitted: 2026/10/30 11:01:32
Duration 1 s
Resource Profile Id: 0
Total Time Across All Tasks: 5 s
Locality Level Summary: Node local: 19
Shuffle Read Size / Records: 113.3 MiB / 10238046
Shuffle Write Size / Records: 32.2 KiB / 950
Associated Job Ids: 33

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 19 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.1 s	0.1 s	0.2 s	0.2 s	0.7 s
GC Time	0.0 ms	0.0 ms	0.0 ms	9.0 ms	19.0 ms
Shuffle Read Size / Records	3.4 MiB / 260166	3.4 MiB / 261225	3.4 MiB / 262677	3.4 MiB / 265121	15.6 MiB / 1576365
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

▸ Aggregated Metrics by Executor

Tasks (19)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	394	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 265121	
1	395	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 264416	
2	396	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 260569	
3	397	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 262353	
4	398	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 262677	
5	399	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 260630	
6	400	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 262914	
7	401	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 264774	
8	402	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 260412	
9	403	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 261225	
10	404	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 262619	
11	405	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 264108	
12	406	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 262023	
13	407	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.6 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1575462	
14	408	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1576098	
15	409	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1574559	
16	410	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	19.0 ms	1.7 KiB / 50	15.6 MiB / 1576365	
17	411	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:34	0.2 s	9.0 ms	1.7 KiB / 50	3.4 MiB / 261555	
18	412	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:34	0.2 s	9.0 ms	1.7 KiB / 50	3.4 MiB / 260166	

Showing 1 to 19 of 19 entries Previous 1 Next

Hình 6.13. Summary Metrics của stage 52 (AQE skew join): max 682 ms và 1,576,365 bản ghi mỗi task.

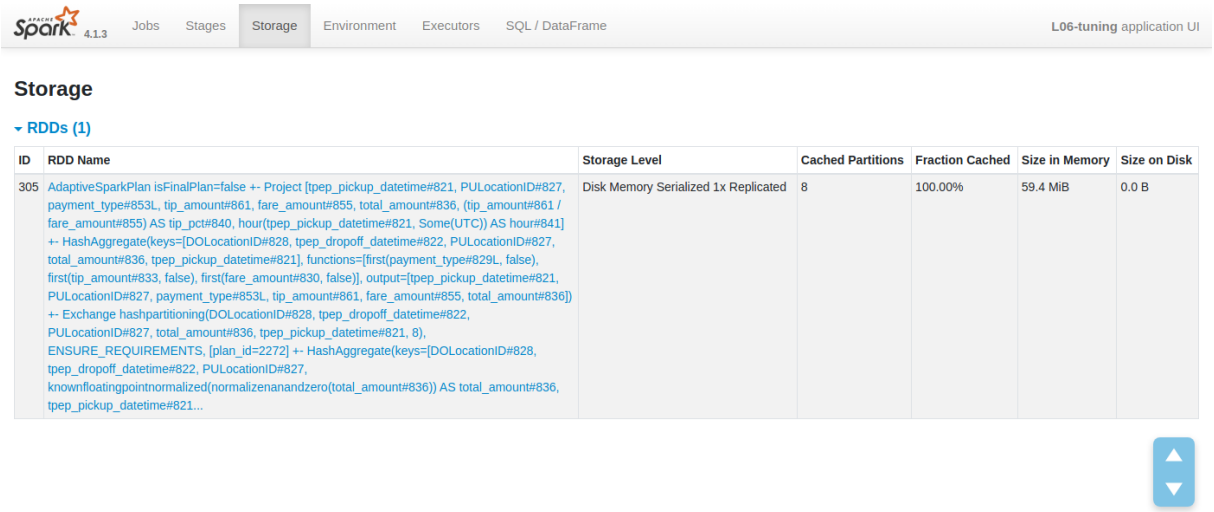
6.4.5 E4: repartition và coalesce trước khi ghi

`repartition(64)` chậm nhất vì vừa có một shuffle đầy đủ vừa ghi 64 file nhỏ. `coalesce(4)` không shuffle nên nhanh nhất, nhưng chỉ ra 3 file chứ không phải 4. Ban đầu chúng tôi nghĩ có lỗi; xem lại thì file Parquet đầu vào chỉ có 3 row group nên một trong 4 partition đầu vào rỗng. `coalesce(1)` dồn cả bước đọc lẫn bước ghi vào một task duy nhất và mất hết song song. `repartition(4)` cho 4 file đều nhau, đổi lại phải trả thêm một shuffle (Bảng 6.3).

6.4.6 E5: cache

DataFrame "cleaned" gồm một phép lọc và `dropDuplicates` trên năm cột, tức là shuffle toàn bộ khoảng 3 triệu dòng. Nó được dùng cho ba phép tổng hợp: theo giờ, theo `PULocationID` và theo

`payment_type`. Không cache thì dedup chạy lại ba lần. Có cache thì `count()` để materialize mất 3.42 s và trả về 2,869,714 dòng, sau đó ba truy vấn chỉ còn **0.99 s**. Tab Storage (Hình 6.14) cho thấy 8/8 partition đã được cache (100 %), chiếm 59.4 MiB bộ nhớ và 0.0 B đĩa, mức `Disk Memory Serialized 1x Replicated`; REST API báo `memoryUsed` = 62,240,847 byte. Tính cả thời gian materialize, cache vẫn lời (khoảng 4.4 s so với 6.4 s), và càng dùng lại nhiều lần thì càng lời.



Hình 6.14. Tab Storage của Spark UI: DataFrame `cleaned` được cache 8/8 partition, 100 %, 59.4 MiB trong bộ nhớ.

6.4.7 So sánh với kết quả tham chiếu

Đề bài kèm một bảng tham chiếu đo trên máy Apple M-series, Docker 4 GB, `local[4]` (Bảng 6.6). Thời gian trên máy chúng tôi cao hơn khoảng 1.3 đến 1.9 lần, có thể do khác phần cứng và có thêm lớp WSL2. Mọi xu hướng thì khớp: trong E1, 200 partition chậm nhất và AQE bắt kịp bản chỉnh tay; trong E2, gợi ý broadcast nhanh nhất; baseline E3 có tỉ lệ max/median lớn, salting không rút được thời gian tổng còn AQE skew join nhanh nhất; và `coalesce(4)` cũng ra 3 file. Chỗ khác là số task của AQE skew join (27 so với 43), vì số partition sau khi tách phụ thuộc kích thước partition thực tế lúc chạy.

Bảng 6.6. So sánh với bảng kết quả tham chiếu của bài lab.

Chỉ số	Tham chiếu	Lần 2
E1, 200 partition, AQE off	1.29 s	2.20 s
E1, AQE on	0.21 s	0.53 s
E2, broadcast hint	0.24 s	0.40 s
E3 baseline (median / max task)	110 / 1,263 ms	165 / 2,366 ms
E3 salting	2.97 s	4.01 s

Chỉ số	Tham chiếu	Lần 2
E3 AQE skew join (thời gian, task)	1.73 s, 43 task	2.22 s, 27 task
E4 coalesce(4)	1.85 s, 3 file	3.12 s, 3 file
E5 không cache / có cache	4.11 / 0.70 s	6.40 / 0.99 s

6.5 Thảo luận

6.5.1 Vì sao 200 partition chậm, và vì sao không nên đặt 1 partition dù nó chạy nhanh?

Kết quả `groupBy(PU, DO)` chỉ có khoảng 20 nghìn nhóm, dữ liệu shuffle chưa tới 1.2 MiB. Chia nhỏ chừng đó ra 200 phần thì mỗi task reduce chỉ làm việc khoảng 6 ms, còn chi phí lập lịch, serialize và khởi tạo của 204 task lớn hơn phần việc thật. Vì vậy bản này chậm hơn bản 8 partition khoảng 6.7 lần (Bảng 6.3). Một partition chạy nhanh ở đây chỉ vì shuffle quá nhỏ. Với dữ liệu thật, một reducer phải ôm toàn bộ dữ liệu shuffle: không có song song, dễ spill ra đĩa và dễ OOM. Thực tế nên nhắm khoảng 100–200 MB mỗi partition shuffle, hoặc để AQE tự quyết.

6.5.2 `AQEShuffleRead coalesced` làm gì, và AQE biết gì mà planner tĩnh không biết?

Khi stage map của shuffle chạy xong, AQE có kích thước thật của từng partition shuffle. `AQEShuffleRead coalesced` cho một reducer đọc gộp nhiều partition nhỏ liền kề cho tới khi đạt `advisoryPartitionSizeInBytes`. Trong E1, 200 partition chỉ còn một task reduce và tổng số task giảm từ 204 xuống 5 (Hình 6.5). Planner tĩnh chỉ có ước lượng dựa trên kích thước file, và ước lượng này hay sai sau filter, join hoặc aggregate.

6.5.3 Con số nào cho thấy skew, vì sao salting không rút ngắn thời gian tổng trên 4 core, và khi nào salting thắng?

Bằng chứng là chênh lệch giữa task max và median ở stage join: **2,366 ms so với 165 ms**, và task chậm nhất đọc 6,264,438 bản ghi trong khi median là 262,619 (Bảng 6.5, Hình 6.10).

Salting cắt task max xuống 1,197 ms nhưng thời gian tổng gần như không đổi. Một phần là vì salting thêm việc ở mọi chỗ: `rand()` cho từng dòng, thêm cột join, nhân dòng dim lên 64 lần. Phần lớn hơn là khoảng 6 triệu dòng hot vẫn phải chia trên 16 partition chạy trên 4 core. Tổng khối lượng việc không giảm, chỉ được chia lại, nên median tăng từ 165 lên 590 ms. Lần chạy đầu cũng vậy: salting chỉ nhanh hơn baseline một chút (4.28 s so với 4.62 s), còn bảng tham chiếu thậm chí ghi salting chậm hơn (2.97 s so với 2.45 s). Salting sẽ thắng rõ khi có nhiều core và nhiều partition, vì lúc đó một straggler giữ hàng trăm executor rảnh. Nó cũng có lợi khi mỗi

dòng xử lý nặng (dòng rộng, join many-to-many, có spill) hoặc khi skew nặng hơn, ví dụ 90 % hot.

6.5.4 AQE nhận ra partition lệch thế nào, xử lý ra sao, và kém salting ở đâu?

Một partition bị coi là lệch khi nó vừa lớn hơn `skewedPartitionFactor` lần median, vừa lớn hơn `skewedPartitionThresholdInBytes`. Mặc định là 5 lần và 256 MB; bài lab hạ xuống 3 lần và 16 MB cho vừa dữ liệu nhỏ. Partition hot ở đây là 61.9 MiB so với median 3.4 MiB, thỏa cả hai điều kiện. AQE tách partition đó theo các khoảng map output (4 split, nên 16 partition thành 19), rồi nhân bản partition tương ứng ở phía còn lại; kế hoạch hiện `SortMergeJoin(skew=true)` (Hình 6.12). Số bản ghi lớn nhất mỗi task còn 1,576,365, xấp xỉ 6,264,438 / 4.

AQE có mấy giới hạn mà salting không có. Nó chỉ áp dụng cho join có shuffle ở cả hai phía (sort-merge hoặc shuffled hash join), và không tách được phía phải giữ nguyên của outer join. Ngưỡng phải hợp với dữ liệu, nếu không nó sẽ không kích hoạt; chúng tôi đã gặp đúng chuyện này ở E3. Quan trọng nhất là AQE không xử lý skew trong `groupBy` hay skew phát sinh sau join, trong khi salting hoặc aggregate hai pha làm được.

6.5.5 Khi nào không nên broadcast dù một phía "nhỏ"?

Không nên broadcast trong các trường hợp sau:

- bảng vẫn lớn khi nằm trong bộ nhớ: driver phải collect rồi gửi bản sao tới mọi executor, dễ OOM ở driver hoặc broadcast timeout;
- thống kê đánh giá thấp kích thước thật, ví dụ sau filter hoặc UDF;
- join là full outer join (không hỗ trợ), hoặc phía cần broadcast lại là phía được giữ của outer join;
- join chạy lặp lại trên dữ liệu thay đổi, nên mỗi lần phải dựng lại broadcast.

Bảng zones trong E2 chỉ có 265 dòng và không rơi vào trường hợp nào ở trên, nên broadcast là đúng (Hình 6.6).

6.5.6 `repartition(n)` hay `coalesce(n)`: cái nào shuffle, cái nào tạo file lệch, cái nào làm giảm song song stage trước?

`repartition(n)` luôn shuffle. Nhờ vậy partition cân bằng (4 file đều nhau) và stage trước vẫn giữ mức song song, cái giá là thêm một shuffle. `coalesce(n)` gộp partition có sẵn mà không shuffle nên rẻ hơn, nhưng file ra có thể lệch: ở E4 chỉ có 3 file vì đầu vào chỉ có 3 row group. Vì `coalesce` là narrow dependency, nó kéo luôn stage trước về cùng số task. `coalesce(1)` đọc và ghi trong một task duy nhất, mất 6,800 ms (Bảng 6.3).

6.5.7 Vì sao cache có ích trong E5, còn cache một DataFrame rẻ thì thường không?

DataFrame được cache trong E5 là kết quả của một bước đắt (`dropDuplicates`, shuffle toàn bộ khoảng 3 triệu dòng) và được dùng lại ba lần. Cache nên bước này chỉ chạy một lần thay vì ba (Hình 6.14). Một phép quét Parquet chỉ lấy vài cột thì vốn đã rẻ nhờ đọc vectorized, column pruning và predicate pushdown; cache nó chỉ tốn thêm bộ nhớ và thời gian materialize. Nguyên tắc là cache cái gì vừa đắt vừa được dùng lại, xong thì gọi `unpersist()`.

6.5.8 Nên tinh chỉnh gì trước khi thêm phần cứng?

1. Đọc kế hoạch và Spark UI: tìm stage chậm nhất, spill, thời gian GC, và tỉ lệ task max/median để phát hiện skew (như Bảng 6.5).
2. Lọc và chọn cột càng sớm càng tốt: partition pruning, predicate/column pushdown, dùng định dạng cột (Parquet/ORC).
3. Để partition shuffle có kích thước hợp lý: bật AQE (coalesce và skew join), nhắm khoảng 100–200 MB mỗi partition.
4. Chọn đúng kiểu join: broadcast bảng nhỏ; nếu có skew thì dùng AQE skew join, salting, hoặc tách hot key.
5. Tránh UDF Python (dùng hàm built-in hoặc pandas UDF), chỉ cache khi cần, và tránh sinh file nhỏ khi ghi (repartition hoặc compaction).

6.5.9 Hạn chế của thực nghiệm

Mọi thứ chạy local trên một máy 4 core, không có chi phí mạng, nên chênh lệch giữa các chiến lược nhỏ hơn nhiều so với trên cluster. Vì thế cột max/median đáng tin hơn thời gian tuyệt đối. Dữ liệu nhỏ (một tháng taxi, 10 triệu dòng tổng hợp) buộc phải hạ ngưỡng skew của AQE. Chỉ có hai lần chạy, và các biến thể dưới 1 s dao động khá mạnh (Bảng 6.4). Lớp Docker/WSL2 cũng làm I/O chậm hơn so với máy tham chiếu. Các phần mở rộng (25 triệu dòng, `HOT_SHARE = 0.9`, bucketing, shuffled hash join) chưa được làm.

6.6 Kết luận

Ngay trên một máy, các tham số tinh chỉnh đã cho khác biệt đo được. Số partition shuffle không hợp làm truy vấn chậm khoảng 6.7 lần, và AQE tự sửa được việc này. Skew lộ ra rõ qua tỉ lệ task max/median khoảng 14 lần. AQE skew join giảm gần một nửa thời gian tổng mà không phải sửa mã, còn salting chỉ chia lại việc khi số core có hạn. `coalesce` rẻ nhưng có thể sinh file lệch và kéo stage trước về một task. Cache chỉ có lợi khi kết quả đắt và được dùng lại.

Với một job production trên cluster 20 node, chúng tôi đề xuất:

1. Luôn bật AQE (mặc định từ Spark 3.2 và trong Spark 4). Đặt `shuffle.partitions` ban

đầu cao, khoảng 2–3 lần tổng số core (400–800), và để `coalescePartitions` gộp về khoảng 64–128 MB mỗi partition (`advisoryPartitionSizeInBytes`). Không đặt cứng 1 hay 8.

2. Broadcast các bảng dimension nhỏ. Có thể nâng `autoBroadcastJoinThreshold` lên khoảng 50–100 MB nếu executor đủ bộ nhớ, và dùng `broadcast()` khi thống kê sai. Không broadcast bảng vài trăm MB vì driver phải collect trước, dễ OOM.
3. Với skew, để AQE skew join chạy với ngưỡng mặc định (5 lần, 256 MB). Với hot key đã biết, hoặc skew trong `groupBy`/outer join mà AQE không xử lý được, dùng salting hoặc tách hot key ra xử lý riêng. Theo dõi tỉ lệ task max/median trong UI.
4. Khi ghi output, dùng `repartition(n)` hoặc `repartition(col)` để mỗi file khoảng 128 MB–1 GB, tránh sinh hàng nghìn file nhỏ. Tránh `coalesce(1)` trên dữ liệu lớn.
5. Chỉ `persist(MEMORY_AND_DISK)` những kết quả đắt và được dùng lại ít nhất hai lần, rồi `unpersist()` ngay. Không cache phép quét Parquet rẻ.

Nếu có thêm thời gian, chúng tôi muốn chạy lại E3 với 25 triệu dòng và 90 % hot để xem ba chiến lược thay đổi thế nào khi dữ liệu lớn lên, thử shuffled hash join với dimension lớn, dùng bucketing để bỏ `Exchange`, và lặp lại toàn bộ trên một cluster nhiều node thật.

Tài liệu tham khảo

[1] M. Zaharia *et al.*, “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” in *Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.

[2] M. Armbrust *et al.*, “Spark SQL: Relational Data Processing in Spark,” in *Proc. ACM SIGMOD International Conference on Management of Data*, 2015.

[3] The Apache Software Foundation, “Performance Tuning – Spark SQL, DataFrames and Datasets,” Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-performance-tuning.html

[4] B. Chambers and M. Zaharia, *Spark: The Definitive Guide*. Sebastopol, CA: O’Reilly Media, 2018.

[5] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O’Reilly Media, 2017.

[6] The Apache Software Foundation, “Monitoring and Instrumentation (REST API),” Apache Spark Documentation. https://spark.apache.org/docs/latest/monitoring.html

[7] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” in *Proc. 6th Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.

Phụ lục A: Danh mục tệp nộp kèm

Các tệp nộp kèm báo cáo được liệt kê ở Bảng 6.7.

Bảng 6.7. Danh mục tệp nộp kèm.

Tệp	Nội dung
`l06_experiments.py`	Script thí nghiệm: E1 bật AQE, E2 gợi ý `broadcast()`, E3 salting và AQE skew join
`results.csv`	34 dòng (2 lần chạy × 17 biến thể)
`RESULTS.md`	Bảng lần chạy thứ hai, nhận xét E1–E5, khuyến nghị cho cluster 20 node
`01_run1.txt`, `02_run2.txt`	Log console của hai lần chạy
`03_e3_task_quantiles.txt`	Phân vị thời gian task và số bản ghi shuffle mỗi task của ba stage join E3 (REST API)
`img/`	Ảnh chụp console và Spark UI

Phụ lục B: Trích đoạn mã nguồn

Trong `l06_experiments.py`, ở E1 chúng tôi bật lại AQE nhưng giữ 200 partition để xem AQE tự gộp:

```
set_conf(spark__sql__adaptive__enabled=True,
 ↪  spark__sql__shuffle__partitions=200)
run("E1", "AQE on,  shuffle.partitions=200", q(), "AQE coalesces small
 ↪  shuffle partitions")
```

Ở E2, ngưỡng được để ở -1 nhưng gắn gợi ý broadcast cho bảng zones:

```
set_conf(spark__sql__adaptive__enabled=True,
 ↪  spark__sql__autoBroadcastJoinThreshold=-1)
run("E2", "AQE on,  threshold -1 + broadcast() hint",
    trips_df().join(F.broadcast(zones_df()), F.col("PULocationID") ==
 ↪  F.col("LocationID"))
    .groupBy("Borough").agg(F.count("*").alias("trips")),
    "an explicit hint beats the threshold")
```

Salting ở E3 chỉ gắn salt cho hot key; phía dim nhân dòng hot lên `SALT` = 64 bản:

```
hot = F.col("key") == 0
fact_s = fact.withColumn(
    "salt", F.when(hot, (F.rand(seed=5) *
     ↪  SALT).cast("int")).otherwise(F.lit(0)))
```

```
salts = spark.range(SALT).select(F.col("id").cast("int").alias("salt"))
dim_s = (dim.filter(~hot).withColumn("salt", F.lit(0))
         .unionByName(dim.filter(hot).crossJoin(salts)))
salted = (fact_s.join(dim_s, ["key", "salt"])
          .groupBy("segment").agg(F.sum("amount").alias("amount")))
run("E3", f"salted SMJ (salt={SALT}), AQE off", salted)
```

Biến thể cuối của E3 bật AQE skew join và hạ ngưỡng cho vừa dữ liệu cỡ lab, đồng thời tắt coalesce để thấy rõ phần tách:

```
set_conf(spark__sql__adaptive__enabled=True,
         spark__sql__adaptive__skewJoin__enabled=True,
         spark__sql__adaptive__coalescePartitions__enabled=False,
         spark__sql__adaptive__advisoryPartitionSizeInBytes="4m",
 ↪  spark__sql__adaptive__skewJoin__skewedPartitionThresholdInBytes="16m",
         spark__sql__adaptive__skewJoin__skewedPartitionFactor=3)
run("E3", "skewed SMJ, AQE skewJoin on",
    fact.join(dim, "key").groupBy("segment").agg(F.sum("amount")),
    "look for AQEShuffleRead (skewed) in the SQL tab")
```